Miguel Sebastian Carlos <miguel_carlos@dlsu.edu.ph>
to Gregory

2:02 PM (46 minutes ago)

Good afternoon, Sir Cu,

We would like to request your signature for our thesis requirement. Attached are the files that need your signature. Thank you!

One attachment • Scanned by Gmail ⓘ Add to Drive



PDF NIS16-Full_Thesis...

Gregory Cu
to me

2:32 PM (16 minutes ago)

I am approving the document for submission

Greg

•••

On Sat, 24 Oct 2026, 2:02 pm Miguel Sebastian Carlos, <miguel_carlos@dlsu.edu.ph> wrote:

> Good afternoon, Sir Cu,
>
> We would like to request your signature for our thesis requirement. Attached are the files that need your signature. Thank you!



Cross-Layer Dataset Design and Exploratory Analysis of ESP32-Based ESP-WIFI-MESH Network

A Thesis Proposal Presented to

The Faculty of the College of Computer Studies

De La Salle University

In Partial Fulfillment

of the Requirements for the Degree of

Bachelor of Science in Computer Science

by

Calpoporo, Jose Angelo C.

Carlos, Miguel Sebastian C.

Ong, Kyle Edward M.

Reinante, Christian Victor G.

Mr. Gregory G. Cu

Faculty Adviser

October 24, 2026

Abstract

Wireless Mesh Networks (WMNs) are widely used in Internet of Things (IoT) systems due to their decentralized, multi-hop communication capabilities. The ESP32 microcontroller, with built-in Wi-Fi support and ESP-WIFI-MESH functionality, enables low-cost deployment of infrastructure-less mesh networks. However, most existing intrusion detection research relies on simulation-based datasets that do not accurately capture the cross-layer interactions and hardware-level behaviors observed in real ESP32 mesh deployments.

This study addresses this gap by designing and generating a hardware-derived cross-layer dataset from an ESP32-based ESP-WIFI-MESH testbed deployed in a controlled indoor environment. A network of 8 nodes is configured to collect telemetry under normal operation and controlled attack scenarios: blackhole and wormhole behaviour, both emulated at the application layer, so neither attack falsifies the mesh's routing. Metrics are gathered across the Physical (PHY), Medium Access Control (MAC), and Network layers, including RSSI values, counts of failed send attempts (application-layer counters; 802.11 retransmissions are read only from a passive sniffer, in sniffer runs), hop transitions, forwarding ratios, and parent-switching events.

Rather than implementing a real-time intrusion detection system, the study focuses on documenting cross-layer behavioral patterns and applying exploratory data analysis and unsupervised clustering to examine separability between normal and attack conditions. The resulting dataset provides a structured foundation for future research on ESP32-based wireless mesh security.

TABLE OF CONTENTS

1. Introduction 1
1.1. Overview 1
1.2. Problem Statement 2
1.3. Research Objectives 3
1.3.1. General Objective 3
1.3.2. Specific Objectives 3
1.4. Project Scope and Limitation 3
1.4.1. Limitations of the Study 4
1.5. Significance of the Study 5
1.6. Resources 6
1.6.1. Hardware 6
1.6.2. Software 7
2. Review of Related References 9
2.1. Wireless Mesh Networks (WMNs) 9
2.2. Security Challenges and Common Attacks in WMNs 10
2.3. Infrastructure-less and Decentralized Network Environments 12
2.4. Intrusion Detection Systems (IDS) for WMNs 13
2.5. Machine Learning and Anomaly Detection in IDS 14
2.6. Cross Layer 15
2.7. ESP32 Technical Literature 16
2.8. Existing Wireless Network Datasets 18
2.9. Dataset Design and Feature Engineering 20
2.10. Exploratory Data Analysis and Clustering in Network Telemetry 22
3. Theoretical Framework 26
3.1. Wireless Mesh Network Fundamentals 26
3.1.1. Wireless Mesh Network Characteristics 26
3.1.2. ESP-WIFI-MESH Architecture and Node Roles 28

3.1.3. Control Plane vs. Data Plane Operations 29
3.1.3.1. Control Plane 29
3.1.3.2. Data Plane 30
3.2. Topology Formation and Physical Influences 31
3.2.1. RSSI Thresholds and Parent Selection 31
3.2.1.1. RSSI Thresholding Mechanism 31
3.2.1.2. Parent Selection Criteria 32
3.2.2. Environmental Effects on Topology Formation 33
3.2.2.1. Impact of Physical Obstacles 33
3.2.2.2. Multi-path and Interference Effects 33
3.2.2.3. Chain Formation in Linear Deployments 34
3.2.3. Logical vs. Physical Topology Relationships 34
3.2.3.1. Hop Count and RSSI Relationship 34
3.2.3.2. RSSI Reference Levels in ESP-WIFI-MESH 35
3.2.3.3. Topology Formation Principles 35
3.3. Routing-Based Attack Theory 36
3.3.1. Blackhole Attack: Conceptual Foundation 36
3.3.1.1. Theoretical Characterization 36
3.3.1.2. Adaptation to ESP-WIFI-MESH Context 37
3.3.1.3. Theoretical Expectations for Observable Behavior 37
3.3.2. Wormhole Attack: Conceptual Foundation 38
3.3.2.1. Theoretical Characterization 38
3.3.2.2. Adaptation to ESP-WIFI-MESH Context 39
3.3.2.3. Wireless Topology Distortion 40
3.3.2.4. Cross-Layer Effects of Wormhole Distortion 40
3.4. Cross-Layer Telemetry Perspective 41
3.4.1. Why Routing Violations Affect Multiple Protocol Layers 41
3.4.1.1. Physical Layer Effects 41
3.4.1.2. MAC Layer Effects 42

3.4.1.3. Network Layer Effects 42
3.4.2. Collected Metrics and Data Sources 43
3.4.3. Temporal Aggregation of Cross-Layer Telemetry 43
3.4.4. Expected Observable Inconsistencies 44
3.4.4.1. Expected Blackhole Observables 44
3.4.4.2. Expected Wormhole Observables 45
3.4.4.3. Timing and Latency Inconsistencies 46
3.4.5. Expected Observable Inconsistencies 46
3.4.5.1. Observational Objectives 46
3.4.5.2. Methodological Principles 46
3.4.5.3. Distinction from Detection System 47
3.5. Conceptual Framework of the Study 48
3.5.1. Normal Behavior as Consistent Relationships 48
3.5.2. Misbehavior as Broken Relationships 48
3.5.2.1. Blackhole: Broken Reception-Forwarding Relationship 49
3.5.2.2. Wormhole: Broken Physical-Logical Topology Relationship 49
3.5.3. Experimental Emulation of Routing Manipulations 49
3.5.4. Secondary Broken Relationships 51
3.5.5. These Inconsistencies Motivate Dataset Collection 51
3.5.5.1. Theoretical Justification for Dataset Requirements 51
3.5.6. Exploratory Behavioral Analysis and Pattern Discovery 52
3.5.7. Expected Contribution 53
3.5.8. Framework Summary 54
4. Research Design 56
4.1. Architectural Design 56
4.2. Experimental Topology and Controlled Behavior Framework 57
4.2.1. Attack Implementation: Wormhole and Blackhole Manipulations 58
4.2.1.1. Common Experiment Control Infrastructure 58
4.2.1.2. Forwarding Suppression (Blackhole) Implementation 62

4.2.1.3. Topology Distortion (Wormhole-Inspired) Implementation 72
4.2.1.4. Implementation Summary 87
4.2.2. Physical Deployment Topologies 89
4.2.2.1. Star Topology 90
4.2.2.2. Tree Topology 93
4.2.2.3. Linear Chain Topology 95
4.2.2.4. Partial Mesh Topology 96
4.2.2.5. Deployment Sites and Scenario 98
4.2.3. Data Collection 100
4.2.3.1. Victim Node Data Collection 101
A. Physical Layer (PHY) Metrics 101
B. Medium Access Control (MAC) Layer Metrics 101
C. Network Layer Metrics 102
4.2.3.1.4. Probe Generation 103
4.2.3.2. Attack Node Data Collection 103
4.2.3.2.1. Attack Node Types 103
4.2.3.2.2. Forwarding Suppression (Blackhole) Node Metrics 104
4.2.3.2.3. Topology Distortion (Wormhole-Inspired) Node Metrics 104
4.2.3.3. Data Acquisition and Logging Architecture 105
4.2.3.3.1. Sampling Loop 105
4.2.3.3.2. Forwarding Counter Implementation 106
C. Tunnel Metric Sampling (Wormhole-Inspired Nodes) 107
4.2.3.4. Phase Labeling and Synchronization 107
A. Phase Control Algorithm 108
4.2.3.4.2. Phase Broadcast Mechanism 108
4.2.3.5. Experimental Label Encoding and Assignment 109
4.2.3.6. Local Storage and Post-Experiment Extraction 110
A. Local Storage Architecture 110
B. Post-Experiment Log Extraction 110

4.2.3.6.3. Data Integrity During Manipulation 111
4.2.3.7. Dataset Completeness Verification 113
4.2.4. Feature Extraction 114
4.2.4.1 Preprocessing 115
4.2.4.2 Feature Computation 118
4.2.4.2.1. Forwarding Behavior Features 118
4.2.4.2.2. Link Reliability Features 119
4.2.4.2.3. Topology Stability Features 120
4.2.4.2.4. Physical Layer Features 121
4.2.4.2.5. Cross-Layer Inconsistency Features 122
4.2.4.2.6. Auxiliary Tunnel Activity Features (Experiment-Level) ... 123
4.2.4.3 Feature Summary 124
4.2.4.4. Dataset Record Structure (Windowed Per-Node Schema) 125
4.2.4.5 Feature Engineering Design Principles 128
4.2.4.6 Feature Selection and Dimensionality Analysis 129
4.2.5. Data Clustering 130
4.2.5.1. Clustering Algorithms 130
4.2.5.1.1. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 130
4.2.5.1.2. K-Means (Comparative Baseline) 136
4.2.5.1.3. Hierarchical Clustering 142
4.2.5.1.4. Gaussian Mixture Model (GMM) 149
4.2.5.1.5. Spectral Clustering 155
4.2.5.2. Cluster Evaluation Metrics 160
4.2.5.2.1. Silhouette Score 160
4.2.5.2.2. Davies–Bouldin Index 161
4.2.5.2.3. Cluster Purity (Post-Analysis Only) 161
4.2.5.2.4. Adjusted Rand Index (Post-Analysis Only) 161
4.2.5.3. Cluster Characterization Using Ground Truth 162

4.2.5.4. Expected Outcomes 162
4.2.6. Exploratory Data Analysis (EDA) 163
4.2.7. Attack Validation 164
4.2.7.1. Ground-Truth and Statistical Check 165
4.2.7.2. Corroboration with Published Studies 166
4.2.7.3. Single-Feature Leakage Audit 167
4.2.7.4. Predictions Tested and Not Observed 167
4.2.7.5. Limits 168
4.2.7.6. Reference Exhibits from the Validation Papers 168
4.2.7.7. Verification of Each Run 173
4.2.7.7.1. Attack Signature 175
4.2.7.7.2. Topology 176
4.2.7.7.3. Dataset 177
4.2.7.7.4. Scenario 178
4.2.7.7.5. Outcome and Grade 179
4.3. Theoretical Analysis 180
4.3.1. Theoretical Justification of Selected Features 180
4.3.1.1. Forwarding Behavior Features (Blackhole Theory) 181
4.3.1.2. Link Reliability Features (Indirect Propagation Theory) 181
4.3.1.3. Topology Stability Features (Routing Adaptation Theory) 182
4.3.1.4. Physical Layer Features (Signal Consistency Theory) 182
4.3.1.5. Cross-Layer Inconsistency Theory 183
4.3.1.6. Cross-Layer Feature Relationship Preservation 183
4.3.2. Theoretical Basis for Data Clustering 184
4.3.2.1. Behavioral State Hypothesis 184
4.3.2.2. Absence of Pre-Defined Signatures 184
4.3.2.3. Density-Based Justification (DBSCAN) 184
4.3.2.4. Partition-Based Justification (K-Means) 185
4.3.2.5. Justification for the Other Algorithms 185

4.3.2.6. Validation Theory 185
4.3.2.7. Alignment with Research Objectives 185
4.3.2.8. Theoretical Expectation 186
4.4. Cluster Visualization and Interpretation 186
5. Methodological Deviations 189
5.1. Overview of the Changes 189
5.2. Data Collection and Sampling 193
5.3. Features and Measurements 196
5.4. Implementation of the Attacks 201
5.5. Experimental Design 208
5.6. Run Control and Validation 213
5.7. What the Changes Mean for the Results 219
6. Results and Discussion 220
6.1. Dataset Collected 221
6.2. Attack Validation Results 230
6.2.1. Blackhole: Forwarding Suppression 230
6.2.2. Wormhole: Tunnel Replay 234
6.2.3. Single-Feature Audit 238
6.2.4. Predictions Tested and Not Observed 244
6.2.5. Cells That Could Not Be Fully Tested 246
6.2.6. Benign Runs: No Attack Signature 247
6.3. Exploratory Data Analysis 248
6.3.1. Blackhole Runs 248
6.3.2. Wormhole Runs 252
6.3.3. Benign Runs 254
6.4. Clustering Results 260
6.4.1. Setup 260
6.4.2. Quality and Agreement with the Label 262
6.4.3. What Limits the Agreement 266

6.4.4. Robustness across Sites, Topologies and Labelling.....269
6.4.5. Projections.....270
6.4.6. Wormhole Runs.....271
6.4.7. Pooled Runs with Three Labels, with and without the Benign Runs..275
6.5. Discussion.....283
6.5.1. In Relation to the Research Objectives.....283
6.5.2. In Relation to the Published Studies.....285
6.5.3. What an Emulation Can and Cannot Show.....289
6.5.4. Limits.....290
7. Conclusion.....293
7.1. Summary of the Study.....293
7.2. The Dataset.....293
7.3. Evidence That the Dataset Is Valid.....294
7.4. Answers to the Research Objectives.....295
7.5. Contribution and Final Remarks.....296
7.6. Recommendations.....297
7.7. Final Conclusion.....299
Appendix A. Bibliography.....300
Appendix B. Research Ethics Forms.....309
Appendix C. Deviations Log.....315

TABLE OF TABLES

Table 2.1 Summary of Overview of Wireless Mesh Networks.....10
Table 2.2 Summary of Security Challenges and Common Attacks in WMNs.....12
Table 2.3 Summary of Literature on Infrastructure-less/Decentralized Environments13
Table 2.4 Summary of Intrusion Detection Systems (IDS) for WMNs.....13
Table 2.5 Summary of ML/Anomaly-Based IDS.....14
Table 2.6 Summary of Cross-Layer.....16
Table 2.7 Summary of ESP32 Technical Literature.....18

Table 2.8 Summary of Existing Wireless Network Datasets 20
Table 2.9 Summary of Dataset Design and Feature Engineering 22
Table 2.10 Summary of EDA and Clustering in Network Topology 25
Table 3.1 ESP-WIFI-MESH Node Roles and Responsibilities 30
Table 3.2 ESP-WIFI-MESH Default RSSI Thresholds 31
Table 3.3 Expected RSSI Ranges by Node Position 35
Table 3.4 Expected Observable Inconsistencies Under Blackhole Behavior 44
Table 3.5 Expected Observable Inconsistencies Under Wormhole Behavior 45
Table 3.6 Distinction Between Cross-Layer Observation and Intrusion Detection 47
Table 3.7 Consistent Relationships in Normal ESP-WIFI-MESH Operation 48
Table 3.8 Theoretical Basis for Dataset Design Requirements 51
Table 4.1 Phase Definition and Labeling 58
Table 4.2 Blackhole Attacker State Variables and Counters 66
Table 4.3 Wormhole Attacker State Variables and Counters 78
Table 4.4 Blackhole Conformance to Airehrour et al. [70] 88
Table 4.5 Wormhole Conformance to Hu, Perrig, and Johnson [46] 88
Table 4.6 Data Collection Sites 99
Table 4.7 Run Scenarios and Run Status, 80 Runs (Final) 99
Table 4.8 Physical Layer (PHY) Metrics 101
Table 4.9 Medium Access Control (MAC) Layer Metrics and Application Counters (D-15) 101
Table 4.10 Network Layer Metrics 102
Table 4.11 Forwarding Suppression (Blackhole) Node Metrics 104
Table 4.12 Topology Distortion (Wormhole-Inspired) Node Metrics 104
Table 4.13 Experimental Phase Label Encoding 109
Table 4.14 Local Storage Architecture 110
Table 4.15 Capture and Verification Tools 114
Table 4.16 Window Aggregation 117
Table 4.17 Feature Summary 124

Table 4.18 Windowed Per-Node Dataset Schema 127
Table 4.19 DBSCAN Parameter Selection 131
Table 4.20 Attack Validation Checks 164
Table 4.21 Testbed Parameters. Adapted from Airehrour et al. [70, Table 2] 168
Table 4.22 Performance of the ESP-MESH Protocol. Adapted from Khan et al. [22, Table 6] 171
Table 4.23 Route Changes and Comparative Traffic Analysis under the Wormhole Attack. Adapted from Zhukabayeva et al. [71, Table 7] 172
Table 4.24 Rules for the Outcome and the Grade of a Run 179
Table 5.1 Groups of Changes and Where Their Effect Is Reported 189
Table 5.2 Index of the Deviation Identifiers D-1 to D-37 190
Table 5.3 Changes to Data Collection and Sampling 193
Table 5.4 Changes to Feature Definitions and Measurements 197
Table 5.5 Changes to the Implementation of the Attacks 201
Table 5.6 Changes to the Experimental Design 208
Table 5.7 Changes to Run Control and Validation 214
Table 6.1 Runs Captured against the Plan, by Topology, Site and Scenario 221
Table 6.2 Windows in the Dataset by Phase and Kind of Run 222
Table 6.3 The 8 Benign Runs 222
Table 6.4 Runs and Boards by Topology 223
Table 6.5 Earlier Captures Replaced by a Newer Capture 224
Table 6.6 Runs Carrying a Note 225
Table 6.7 Features, Their Units and Their Use in This Chapter 226
Table 6.8 Windows with a Value for Each Feature, by Kind of Board (%) 227
Table 6.9 Attack Checks of the Verifier over the 36 Blackhole Runs 230
Table 6.10 Collapse and Recovery of Delivery in the Victim Boards of the 36 Blackhole Runs 232
Table 6.11 Attack Checks of the Verifier over the 36 Wormhole Runs 234
Table 6.12 Evidence of the Tunnel by Topology and Scenario, over the 36 Wormhole Runs 235

Table 6.13 Best Single-Threshold Accuracy of Each Feature in the Blackhole Runs (%) 238

Table 6.14 Best Single-Threshold Accuracy of Each Feature in the Wormhole Runs (%) 240

Table 6.15 Features with Missing Values in the Single-Feature Audit 242

Table 6.16 Predicted Indicators of the Wormhole (Table 3.5) and What Was Measured in the 36 Wormhole Runs 245

Table 6.17 Verifier Result for the 8 Benign Runs 247

Table 6.18 Mean of Each Feature in the Baseline and in the Attack Phase, and Cohen's d, for the Victims and the Bystanders of the Blackhole Runs 248

Table 6.19 Mean of Each Feature in the Baseline and in the Attack Phase, and Cohen's d, for the Two Tunnel Ends and the Non-Attacker Nodes of the Wormhole Runs 252

Table 6.20 Delivery, Signal and Mesh Stability in Each of the 8 Benign Runs 255

Table 6.21 Normal Windows of the Benign Runs and of the Attack Runs, by Scenario 256

Table 6.22 Normal Windows of the Benign Runs against the Normal Windows of the Blackhole and the Wormhole Runs 257

Table 6.23 Baseline of Each Benign Run against the Baseline of the Attack Run of the Same Topology, Site and Scenario 259

Table 6.24 Feature Sets Used for Clustering 261

Table 6.25 Clustering Quality and Agreement with the Label, Means over Seeds 42, 7 and 123 262

Table 6.26 K-Means Clusters at k = 2 on the REDUCED Set (All Non-attacker Nodes, Seed 42) 268

Table 6.27 K-Means (k = 2) ARI inside Each Site and Topology, All Non-attacker Nodes 269

Table 6.28 Agreement of the Clusters with the Attack Label in the Wormhole Runs (ARI, Means over Seeds 42, 7 and 123) 272

Table 6.29 K-Means Clusters of the Entry End B with TunnelIntensity (Seed 42) .. 273

Table 6.30 Groups of Windows in the Pooled Clustering of the 80 Runs 276

Table 6.31 Clustering Quality and Agreement with the Three Labels in the Pooled Runs, Means over Seeds 42, 7 and 123 (k = 3) 276

Table 6.32 Windows of Each Group that Fall in the Cluster of the Victims (K-Means, k = 2, REDUCED Set, Fitted on a Sample of 6,000 Windows) 281

Table 6.33 What Each Kind of Run Leaves in the Data 285

Table 6.34 Measured Values Compared with Published Studies 286
Table 6.35 Branching of the Mesh by Topology and Kind of Run 291
Table C.1 Deviations from the Approved Proposal 315

TABLE OF FIGURES

Figure 3.1 Architecture of ESP32-WFI-MESH 29
Figure 3.2 Conceptual Framework of the Study 54
Figure 4.1 Architecture Design 56
Figure 4.2 Root Phase Controller 60
Figure 4.3: Node Phase Monitor 61
Figure 4.4 Blackhole Topology with Attacker as Intentional Relay 66
Figure 4.5 Blackhole Attacker Relay Loop 67
Figure 4.6 Blackhole Attacker Decision Flowchart 69
Figure 4.7 Baseline vs Suppression Sequence Diagram 71
Figure 4.8 Wormhole Endpoint Setup with Auxiliary Tunnel 77
Figure 4.9 Wormhole Tunnel Packet Structure 78
Figure 4.10 Attacker B – Relay and Tunnel 80
Figure 4.11 Attacker B (Leaf-side) Process Flowchart 81
Figure 4.12 Attacker A – Tunnel Receive and Inject 81
Figure 4.13 Attacker A (Root-side) Process Flowchart 83
Figure 4.14 Wormhole Shortcut Relay Mechanism 85
Figure 4.15 Traffic Redistribution During Distortion Window 86
Figure 4.16 Wormhole Attack on Star Topology 91
Figure 4.17 Blackhole Attack on Star Topology 92
Figure 4.18 Wormhole Attack on Tree Topology 93
Figure 4.19 Blackhole Attack on Tree Topology 94
Figure 4.20 Wormhole Attack on Linear Chain Topology 95

Figure 4.21 Blackhole Attack on Linear Chain Topology 96
Figure 4.22 Wormhole Attack on Partial Mesh Topology 97
Figure 4.23 Blackhole Attack on Partial Mesh Topology 98
Figure 4.24 Victim Node Telemetry Sampling Loop 105
Figure 4.25 Forwarding Counter Update (Attack Node) 106
Figure 4.26 Tunnel Metric Sampling 107
Figure 4.27 Experimental Phase Control (Root Node) 108
Figure 4.28 Log Retrieval Routes 111
Figure 4.29 Core, Border, and Noise Points in DBSCAN 131
Figure 4.30 DBSCAN Cluster Formation with Noise 132
Figure 4.31 DBSCAN Density-Based Cluster Detection 134
Figure 4.32 Density Reachability in DBSCAN 135
Figure 4.33 K-Means Iterative Cluster Formation in Three Stages 137
Figure 4.34 K-Means Algorithm Flowchart. Adapted from Lloyd (1982). 138
Figure 4.35 Step-by-Step K-Means Cluster Formation in Six Stages. Adapted from Bishop (2006). 139
Figure 4.36 Example of K-Means Clustering 141
Figure 4.37 Example Dendrogram 143
Figure 4.38 Hierarchical Clustering Dendrogram (Ward Method) 145
Figure 4.39 Agglomerative vs Divisive Hierarchical Clustering 146
Figure 4.40 Agglomerative Hierarchical Clustering Tree 148
Figure 4.41 Covariance Types in Gaussian Mixture Models 150
Figure 4.42 GMM Probability Density Contours 151
Figure 4.43 Gaussian Mixture Model Clustering Example 153
Figure 4.44 Cluster Fitting Using Gaussian Mixture Model 154
Figure 4.45 Spectral Clustering Algorithm Overview 156
Figure 4.46 Eigenvector Components Used in Spectral Clustering 158
Figure 4.47 Spectral Clustering Result on Non-Linear Data 159
Figure 4.48 Packet Loss under Blackhole Attacks. Adapted from Airehrour et al. [70, Figure 10] 169

Figure 4.49 Frequency of Mote Rank Changes under Blackhole Attacks. Adapted from Airehrour et al. [70, Figure 8] 169

Figure 4.50 Throughput under Blackhole Attacks. Adapted from Airehrour et al. [70, Figure 9] 170

Figure 4.51 Received Packet Delivery, Packet Loss and Package Fault Rate per Node. Adapted from Khan et al. [22, Figure 20] 171

Figure 4.52 Hop Count under the Wormhole Attack. Adapted from Zhukabayeva et al. [71, Figure 4] 172

Figure 4.53 Network Behaviour before and after the Attack. Adapted from Zhukabayeva et al. [71, Figure 19] 173

Figure 4.54 Wormhole Attack with a Malicious Node. Adapted from Ramírez Gómez et al. [72, Figure 2] 173

Figure 4.55 Order of the Checks That the Verifier Applies to Every Run 175

Figure 4.56 Attack Signature Check on Two Runs: (a) ForwardingRatio of the Blackhole Attacker, Linear Topology, G402, Stationary; (b) TunnelIntensity of Wormhole End B, Tree Topology, G402, Stationary 176

Figure 4.57 Layouts Rebuilt by the Topology Check at the Start of the Attack (Six Runs at G402) 177

Figure 4.58 Dataset Check: Share of Each Phase Covered by Windows, per Board (Blackhole, Partial Mesh, G402, Highload) 178

Figure 4.59 Scenario Check: Probes Sent per Child Board per Second in Three Runs (Blackhole, Star, G402) 179

Figure 4.60 t-SNE Cluster Visualization (Illustrative Example) 186

Figure 6.1 Mean PDR of the Victim Boards in the Attack Phase, for Each of the 36 Blackhole Runs 232

Figure 6.2 PDR of the Victim and Bystander Boards over the Run, by Topology 233

Figure 6.3 Forwarding Ratio of the Attacker over the Run (Left) and in Each One-Second Window (Right) 233

Figure 6.4 Hops Skipped by the Tunnel and Median Spread between the Two Copies of a Probe, in Each of the 36 Wormhole Runs 236

Figure 6.5 Tunnel Traffic, Duplicates and Spread over the Run, Averaged over the Runs of Each Scenario....237
Figure 6.6 Spread between the Two Copies of a Probe, by Topology (Left) and against the Hops Skipped (Right)....237
Figure 6.7 Best Single-Threshold Accuracy of Each Feature in the Blackhole Runs, for Three Groups of Boards....240
Figure 6.8 Best Single-Threshold Accuracy of Each Feature in the Wormhole Runs, for Three Groups of Boards....242
Figure 6.9 Distributions of Ten Features by Phase for the Bystanders and the Victims of the Blackhole Runs....250
Figure 6.10 Correlations between the Features of the Child Nodes of the Blackhole Runs 251
Figure 6.11 PDR of the Non-Attacker Nodes of the Wormhole Runs and of the Victims of the Blackhole Runs, by Topology....253
Figure 6.12 Distributions of Ten Features by Phase for the Non-Attacker Nodes and the Two Ends of the Wormhole Runs....254
Figure 6.13 Delivery in the 8 Benign Runs over Time and the Lowest PDR of Each Board....255
Figure 6.14 Mean PDR, Median LatencyHopRatio and Probe Rate of the Benign Runs and of the Normal Windows of the Attack Runs, by Scenario....257
Figure 6.15 Normal Windows of the Child Boards: Benign Runs against the Blackhole and the Wormhole Runs....258
Figure 6.16 Baseline RSSI_mean of Each Benign Run against That of the Attack Run of the Same Cell....259
Figure 6.17 DBSCAN k-Distance Graphs for the Three Feature Sets (All Non-attacker Nodes, Seed 42)....262
Figure 6.18 Adjusted Rand Index against the Number of Clusters, by Feature Set and Population (Mean and Spread over Three Seeds)....264
Figure 6.19 Silhouette Score against the Number of Clusters, by Feature Set and Population....265
Figure 6.20 Davies–Bouldin Index against the Number of Clusters, by Feature Set and Population....266

Figure 6.21 Composition of Each K-Means Cluster at k = 3, by Feature Set (All Non-attacker Nodes, Seed 42) 268
Figure 6.22 PCA and t-SNE Projections of the FULL Feature Set (Blackhole Runs) 270
Figure 6.23 PCA and t-SNE Projections with the Delivery Ratio Removed (Blackhole Runs) 271
Figure 6.24 Adjusted Rand Index against the Number of Clusters in the Wormhole Runs (Mean and Spread over Three Seeds) 274
Figure 6.25 PCA and t-SNE Projections of the Wormhole Runs, Coloured by Phase 275
Figure 6.26 Adjusted Rand Index against the Three Labels, by Feature Set and Population (Mean and Spread over Three Seeds) 278
Figure 6.27 Composition and Feature Means of the K-Means Clusters at k = 2 in the Pooled 80 Runs (Seed 42) 279
Figure 6.28 Composition and Feature Means of the K-Means Clusters at k = 3 in the Pooled 80 Runs (Seed 42) 280
Figure 6.29 Feature Values inside Each K-Means Cluster at k = 3 (REDUCED Set, Pooled 80 Runs, Seed 42) 280
Figure 6.30 PCA and t-SNE Projections of the Pooled Windows of the 80 Runs, FULL Set 282
Figure 6.31 PCA and t-SNE Projections of the Pooled Windows of the 80 Runs, without PDR 282

TABLE OF EQUATIONS

Equation 4.1 115
Equation 4.2 119
Equation 4.3 119
Equation 4.4 120
Equation 4.5 120
Equation 4.6 120
Equation 4.7 120

Equation 4.8 121
Equation 4.9 121
Equation 4.10 121
Equation 4.11 122
Equation 4.12 122
Equation 4.13 122
Equation 4.14 122
Equation 4.15 123
Equation 4.16 123
Equation 4.17 124
Equation 4.20 155
Equation 4.21 160
Equation 4.22 161
Equation 4.23 161
Equation 4.24 161

1. Introduction

1.1. Overview

Wireless Mesh Networks (WMNs) have become a critical communication architecture for Internet of Things (IoT) systems due to their self-organizing, infrastructure-less, and multi-hop characteristics [1]. These networks are widely used in smart cities, environmental monitoring, disaster recovery, and rural connectivity, where centralized infrastructure is impractical or unavailable [2]. By allowing each node to function as both a host and a router, WMNs provide scalability and fault tolerance in dynamic environments [1].

Despite these advantages, the decentralized nature of WMNs introduces significant security and reliability challenges [5]. Routing-based misbehavior, such as blackhole and wormhole attacks, can severely degrade network performance, disrupt communication paths, and reduce data delivery reliability [3, 4, 18, 20]. These behaviors are difficult to analyze because malicious nodes often exploit normal routing mechanisms and remain protocol-compliant at the physical and link layers [3, 4].

The ESP32 microcontroller is widely adopted in low-cost IoT deployments and academic research due to its integrated Wi-Fi capabilities, low power consumption, and support for ESP-WIFI-MESH networking [21]. ESP32-based mesh networks have been used for indoor and outdoor air-quality monitoring [22] and for wireless communication between laboratory test instruments [23]. However, empirical studies examining routing behavior and attack-induced anomalies in real ESP32 mesh hardware remain limited.

The study is set in a smart-campus scenario: indoor environmental and building monitoring at DLSU Manila, where low-cost ESP32 nodes send periodic sensor readings through a mesh to a root node. Khan et al. [22] deployed an ESP-MESH network of ESP32 sensor nodes across a college campus for indoor and outdoor air-quality monitoring and reported a packet delivery rate above 97% at every node except one (node H), which the authors suspect was affected by its outdoor location. This suggests that the platform can support this kind of monitoring. A node that drops or redirects readings in such a network would silently corrupt the monitoring data, so the dataset records how the two chosen attacks show up across the PHY, MAC and network layers. The testbed stays an isolated mesh, and the scenario describes the type of traffic and place, not a live campus system.

Most existing network security datasets are generated from simulations or traditional IP-based networks and fail to capture the cross-layer behavior, routing dynamics, and hardware-level

characteristics of microcontroller-based wireless mesh systems [29, 30, 31]. As noted by Sommer and Paxson [30], evaluating detection methods on simulated activity or small laboratory networks often lacks realism and relevance, and in this domain there are rarely standardized test sets or readily available representative data. Similarly, Ring et al. [31] surveyed network-based intrusion detection datasets and compared them on 15 properties, including whether the traffic is real, emulated or synthetic. Their survey is limited to packet-based and flow-based network traffic and does not cover cross-layer or microcontroller-mesh data, which is the gap this study addresses. As a result, there is limited understanding of how routing-based misbehavior manifests in real ESP32 mesh deployments.

To address this gap, this research focuses on the design and generation of a cross-layer dataset collected from physical ESP32-based wireless mesh networks. The dataset captures observable network behavior under both normal operation and controlled attack scenarios, implemented as application-layer emulations of blackhole and wormhole behavior [25, 26]. Using this dataset, the study performs exploratory data analysis and unsupervised clustering to examine whether distinct behavioral patterns emerge and whether normal and malicious behaviors are separable based on observable network features [38, 41].

1.2. Problem Statement

Wireless Mesh Networks (WMNs) built using ESP32 microcontrollers are increasingly deployed in real-world IoT systems. However, their decentralized and multi-hop nature makes them vulnerable to routing-based misbehavior such as blackhole and wormhole attacks, which can significantly impact communication reliability and topology integrity.

While prior research has explored attack detection in simulated environments, there is a lack of hardware-generated datasets that capture how such behaviors manifest in real ESP32 wireless mesh networks. Existing datasets are largely simulation-based or designed for conventional networks, and therefore do not represent the cross-layer interactions, routing dynamics, and wireless characteristics inherent to ESP32-based mesh systems.

This lack of realistic datasets limits empirical analysis, reproducibility, and understanding of routing-based misbehavior in microcontroller-based mesh networks. There is a need for a dedicated, hardware-derived dataset that enables exploratory analysis and unsupervised examination of network behavior under normal and adversarial conditions.

1.3. Research Objectives

1.3.1. General Objective

To design, generate, and analyze a comprehensive cross-layer dataset derived from an ESP32-based wireless mesh network to explore and evaluate behavioral differences between normal operation and controlled attack scenarios.

1.3.2. Specific Objectives

Specifically, this study aims to:

- To establish an ESP32-based wireless mesh testbed capable of generating realistic multi-hop network traffic under both normal operation and controlled attack scenarios.
- To collect and structure cross-layer features from the Physical (PHY), Medium Access Control (MAC), and Network layers into a labeled and well-documented dataset that captures observable mesh behavior.
- To evaluate the usability and quality of the generated dataset through exploratory data analysis (EDA) and unsupervised or baseline offline machine-learning techniques, to assess whether normal and attack-induced behaviors are separable based on the collected features.

1.4. Project Scope and Limitation

This study focuses on the design, generation, and analysis of a realistic, hardware-based dataset derived from physical ESP32 wireless mesh networks using the ESP-WIFI-MESH protocol. The dataset captures observable network behavior, including signal strength variations, packet-forwarding dynamics, and routing-related inconsistencies that naturally arise in decentralized, multi-hop mesh networks.

Data collection is restricted to two primary network states: normal operation and controlled attack scenarios. Specifically, the study considers two attack types, emulated at the application layer on real ESP32 hardware: blackhole (forwarding suppression) and wormhole (out-of-band tunneling of traffic), as these represent fundamental and distinct categories of routing-layer threats—one targeting

forwarding compliance and the other targeting topology integrity. A cross-layer data collection strategy is employed to gather indicators from the Physical (PHY), Medium Access Control (MAC), and Network layers, including Received Signal Strength Indication (RSSI), counts of failed send attempts, packet forwarding ratios, changes in hop count, and parent-switching events.

The collected features are intended to support exploratory analysis, clustering, and behavioral separability studies, rather than the development or deployment of any intrusion detection system. The scope of the study is limited to observing and documenting how routing-based disruptions manifest across protocol layers in a real ESP32 mesh network.

1.4.1. Limitations of the Study

Several limitations are acknowledged regarding the dataset's scope and applicability. First, the experimental testbed is limited to a small-scale deployment consisting of 8 ESP32 nodes. As a result, the dataset may not fully represent the behavior of large-scale or high-density mesh networks. In early seven-node highload captures, made before the arrival logging of the root was changed (Section 4.2.3.6), delivery collapsed partway through the run and the mesh did not recover during cooldown, and a diagnostic run without any attack collapsed in the same way. This was not seen in the final dataset: in every highload run of the campaign, the delivery of the victims returned within 3 s of the end of the attack (Table 6.10). Highload is still the scenario closest to the limits of the testbed (Section 4.2.7.5). The microSD logging adds three limits. The fsync change (D-13) compiles, and it has not been validated by resetting a board in the middle of a run. The card clock is capped at 4 MHz because jumper wires cannot carry the default speed. Captures made before the fsync change that ended badly may have a card file with 0 rows, and those runs are treated as lost and not as empty logs (Section 4.2.7.5). None of the 637 telemetry files of the 80 runs has 0 rows. One board has no file: B768 in the blackhole run on the tree topology at G402 with highload, and that run is analysed without it (Section 6.1). In all 19 partial-mesh runs (9 blackhole, 9 wormhole and 1 benign), no board was seen under a second parent in the labelled phases, so these runs behave as plain trees in which no board has more than two children, and the partial-mesh results carry that limit (Section 4.2.7.5).

Second, the threat model is strictly limited to two specific attack behaviors: blackhole (forwarding suppression) and wormhole-inspired (tunnel-based) attacks. Both are implemented in the application layer on real ESP32 hardware, as emulations of what each attack does to traffic, and neither changes how ESP-WIFI-MESH chooses parents or builds

routes. The mesh organizes itself into a tree, and its documentation describes a way to read a node's routing table but none to write one [21,75]. The dataset therefore records the effect of each attack on the traffic it touches and not the route-attraction stage of a classical blackhole or wormhole, and for the same reason the position of the attacker in the mesh is an experimental variable. The dataset does not include other routing-layer threats (such as grayhole, Sybil, or selective forwarding), attacks on application payloads, cryptographic exploits, physical tampering, firmware compromise, or adversarial radio interference beyond these two controlled behaviors.

Third, data collection took place at three indoor sites on the DLSU campus: a room in the Gokongwei College of Computer Studies (GCCS) G402, a small part of the DLSU Library, and part of the Yuchengco Building. The sites were chosen so that the dataset contains different levels of Wi-Fi interference. The Library and Yuchengco Building areas are public spaces, and the study records only the telemetry of its own boards and the radio frames of its own mesh, not the people or the other networks around them. The mesh itself stays isolated: the testbed forms its own ESP-WIFI-MESH network and connects to no campus or production network. The results therefore describe controlled attack insertion on an isolated mesh in indoor campus spaces, not attacks on a live campus network.

Fourth, the study emphasizes exploratory and unsupervised analysis, rather than predictive performance or real-time deployment. Consequently, the dataset is not evaluated for detection accuracy, online response time, or operational security guarantees.

Finally, all experiments are conducted indoors on an isolated mesh to ensure repeatability and consistent labeling. While this improves experimental reliability, it may not capture environmental variability such as outdoor interference, changes in node placement during a run, or weather-related signal effects.

1.5. Significance of the Study

This study contributes to the growing body of research on wireless mesh networking using low-cost microcontroller platforms by providing a structured, hardware-derived dataset that captures cross-layer behavior in ESP32-based ESP-WIFI-MESH systems.

For researchers and students, the study offers a practical reference for building and instrumenting real ESP32 mesh testbeds, demonstrating how observable routing behavior can be captured beyond purely

theoretical models or simulation-based environments. The dataset supports exploratory investigations into how normal and disrupted mesh behaviors differ across protocol layers.

More broadly, the study highlights methodological considerations in cross-layer data collection, including synchronization, logging constraints, and interpretation challenges when working with decentralized mesh protocols. Rather than presenting definitive security solutions, the findings aim to support future analytical and methodological research on wireless mesh behavior.

In alignment with United Nations Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure), this work supports the advancement of resilient, low-cost communication infrastructure by improving the empirical understanding of wireless mesh networks deployed in resource-constrained environments.

1.6. Resources

1.6.1. Hardware

- **ESP32 Development Boards (8 units)**
    - The ESP32 microcontroller serves as the core hardware platform for constructing the experimental ESP-WIFI-MESH testbed and generating a realistic, hardware-based dataset. Its integrated Wi-Fi capability supports multi-hop mesh communication via the ESP-MESH protocol, enabling the observation of routing behavior, packet-forwarding dynamics, and cross-layer interactions under both normal and adversarial conditions. A network of 8 ESP32 nodes (the root and seven children) forms small-scale mesh topologies for controlled experimentation, traffic generation, and network metric logging.

      The ESP32 devices are used solely for data generation and behavior observation; no real-time intrusion detection is deployed on the nodes in this study.
- **ESP32 Sniffer Board and Laptop**
    - A spare ESP32 board flashed with the project's sniffer firmware listens passively on the mesh channel during selected runs and streams the 802.11 frames it hears to a laptop, which writes them to a Wireshark capture file (tools/sniff.py). The sniffer decodes only 20 MHz channel widths, so the radio of every node is fixed to 20 MHz (D-14). The mesh traffic is encrypted, so the

sniffer records which node transmitted, when, and whether a frame was retried, but not the contents of the probes.

- **Laptop/Workstation (2–3 units)**
    - High-performance laptops equipped with multicore processors and at least 16 GB of RAM serve as the primary environment for log aggregation, dataset consolidation, exploratory data analysis (EDA), and baseline offline machine learning experiments. These systems are also used for experiment orchestration, visualization, and result documentation.

- **MicroSD card reader modules and microSD Cards (one per ESP32 board)**
    - Each board mirrors its telemetry and probe-arrival logs to a FAT-formatted microSD card through an SPI card reader. The card removes the need to connect every board to a laptop after each run, and it lets a board be deployed with only a wall charger or a power bank. Internal flash remains the first log, so a missing or failed card never blocks a run.

- **Monitoring and Diagnostic Tools**
    - Serial monitors and diagnostic logging interfaces are used to observe mesh events, node state transitions, packet counters, and timing behavior during experimental runs. These tools support validation of network stability, attack execution timing, and ground-truth labeling rather than real-time detection. Delay is not read from the serial monitors; it is computed offline from the probe timestamps in the logs.

1.6.2. Software

This section describes the software environment used to implement, monitor, and validate the ESP32-based ESP-WIFI-MESH testbed. The selected tools support firmware development, cross-layer telemetry extraction, controlled experimentation with routing behavior, and dataset preprocessing for subsequent exploratory analysis. The software stack is designed to balance detailed metric collection with the processing constraints of resource-limited ESP32 devices.

The mesh network is implemented using the ESP-IDF framework (version 5.5.4, and 5.3.5 on one build laptop; each run log records the version that built its firmware) with ESP-WIFI-

MESH functionality enabled. Core mesh functions such as initialization, packet transmission, packet reception, and event monitoring are integrated into the application layer. Logging routines are refined to capture selected cross-layer metrics—such as forwarding behavior, RSSI observations, and routing events—while minimizing processing overhead to preserve normal mesh operation.

Additional tools include Wireshark for packet-level verification and traffic validation, and Python for preprocessing, feature extraction, and exploratory statistical analysis of the generated dataset.

- **ESP-IDF SDK**
    - The ESP-IDF SDK (versions 5.5.4 and 5.3.5) is used to develop and deploy firmware for ESP32 nodes implementing ESP-WIFI-MESH functionality, traffic generation, controlled attack behaviors, and cross-layer metric logging. It is also used for accessing low-level mesh APIs, event handlers, and diagnostic counters required for dataset generation.

- **Wireshark**
    - Wireshark is used on the sniffer laptop as a supplemental packet-capture and verification tool to observe wireless traffic patterns, confirm packet-forwarding behavior, and support validation of attack effects. Because the mesh traffic is encrypted, it shows which node transmitted to which, when, and whether a frame was retried, but not the contents of the probes. It is employed for traffic inspection and ground-truth verification rather than for evaluating detection accuracy.

- **C / Python**
    - C is used for implementing embedded logic on ESP32 nodes, including mesh operation, attack behavior control, and metric logging. Python 3 scripts are used for dataset preprocessing, feature extraction, correlation analysis, and statistical evaluation during EDA and baseline offline ML experiments, using pandas, NumPy, SciPy, scikit-learn, Matplotlib and seaborn; the minimum versions are listed in the repository's requirements file [79].

2. Review of Related References

2.1. Wireless Mesh Networks (WMNs)

Wireless Mesh Networks (WMNs) represent an important evolution in wireless communication, designed to provide high adaptability, reliability, and scalability in dynamic environments. Akyildiz et al. [1] describe WMNs as a self-organizing and self-configuring system where nodes automatically establish and maintain mesh connectivity with one another. This decentralized architecture eliminates the need for a central authority, allowing the network to operate autonomously. Mesh routers within WMNs form a self-healing web of connections, enabling the system to maintain communication even when certain nodes or links fail. This self-healing property ensures fault tolerance, as data can automatically reroute through alternate paths whenever disruptions occur.

WMNs are characterized as multi-hop networks in which information can travel through multiple intermediate nodes before reaching its destination. This feature allows them to cover large geographic areas with minimal infrastructure, reducing deployment costs while maintaining flexibility and scalability. As noted by Akyildiz et al. [1] and Cilfone et al. [2], this multi-hop capability makes WMNs suitable for applications requiring robust and dynamic connectivity, such as Internet of Things (IoT) systems like smart cities and smart agriculture [2], and community and metropolitan area networks [1]. Because they can dynamically reconfigure, WMNs are ideal for environments where reliable connectivity and rapid deployment are critical, including smart cities, industrial IoT, and remote monitoring systems.

According to Cilfone et al. [2], WMNs have become a key enabler of IoT due to their scalability, cost-effectiveness, and flexibility. Each node can act as both a host and a router, allowing seamless multi-hop communication without fixed infrastructure. These networks can self-configure, self-heal, and adapt to changing topologies, maintaining connectivity even when nodes or links fail. Such capabilities make WMNs an attractive communication backbone for large-scale, heterogeneous IoT environments.

Despite their advantages, WMNs face significant challenges, particularly in security and network management [1]. The open and distributed nature of WMNs exposes them to security vulnerabilities, as the lack of centralized control and reliance on shared wireless channels make them susceptible to data interception, unauthorized access, and routing attacks. Cilfone et al. [2] add that interoperability between devices from different vendors, limited node resources, and the heterogeneity of devices and protocols remain open issues in IoT mesh deployments. Akyildiz et al. [1] further noted that while WMNs exhibit strong self-organization and self-healing properties, achieving full automation in real-world deployments remains difficult due to constraints in routing algorithms, scalability, and resource management.

Given these limitations, there is a clear need for security mechanisms that protect WMNs from malicious activity. Intrusion Detection Systems (IDSs) play a crucial role in detecting, analyzing, and responding to network intrusions, thereby enhancing the trustworthiness and resilience of these decentralized, multi-hop systems.

Author and Title	Description	Approach	Contribution

Akyildiz, Wang & Wang (2005). Wireless mesh networks: A survey (Computer Networks)	**Comprehensive survey of WMN architectures, routing, scalability, and fundamental design challenges.**	**Literature review and conceptual analysis of WMN architecture, self-healing features, and multi-hop capabilities**	**Defines WMN architecture & roles (mesh routers/clients), and characterizes multi-hop/self-healing properties.**
Cilfone, Davoli, Belli & Ferrari (2019). Wireless mesh networking: An IoT-oriented perspective, Survey on relevant technologies (Future Internet)	**Updated IoT-oriented survey covering WMN use in heterogeneous IoT deployments: protocols, interoperability, and application scenarios.**	**Survey of IEEE 802.11, Bluetooth, IEEE 802.15.4 and LoRa technologies for mesh networking**	**Positions WMNs as enablers for IoT and smart-city applications, also highlighting interoperability, resource-constraint, and heterogeneity issues in IoT contexts.**

Table 2.1 Summary of Overview of Wireless Mesh Networks

2.2. Security Challenges and Common Attacks in WMNs

Wireless Mesh Networks (WMNs) are widely used for decentralized communication but remain vulnerable to various routing-layer attacks such as blackhole, grayhole, Sybil, and wormhole. In these attacks, malicious nodes manipulate routing paths by absorbing, selectively dropping, or tunneling packets to disrupt data transmission. They are particularly difficult to detect because they often mimic normal node behavior, rendering traditional signature- or rule-based defenses unreliable. This challenge highlights the need for adaptive, learning-based intrusion detection systems (IDSs) capable of dynamically analyzing network patterns.

Blackhole attacks represent a fundamental routing-layer threat in which a malicious node first positions itself on active data paths by advertising favorable routing metrics, such as low hop count or strong signal quality, to attract traffic from neighboring nodes. Once established as an intermediate forwarder, the node silently drops all received packets rather than relaying them to their destinations. Bhonsle et al. [20] demonstrated that deep learning approaches using Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks can effectively detect blackhole attacks in WMNs by analyzing network traffic patterns, achieving significant improvements in detection rates compared to traditional rule-based systems. Their work on the NSL-KDD dataset showed reduced false positives and adaptable detection capabilities suitable for dynamic mesh environments. NSL-KDD is a general-purpose intrusion dataset and contains no mesh-network traffic. More recently, Vanlalhruaia et al. [18] proposed a detection mechanism for AODV-based WMNs that utilizes destination sequence number comparison and hop count verification to identify inconsistencies caused by blackhole nodes, demonstrating effective mitigation of attack impacts through simulation-based analysis. In the testbed of this study, the blackhole is emulated in the application layer: the attacker drops the probes it relays but does not advertise false routes (D-20).

Wormhole attacks pose an equally serious threat through a distinct mechanism: two colluding malicious nodes establish a low-latency out-of-band tunnel to relay packets between distant parts of the network, creating the illusion that physically remote nodes are immediate neighbors. Hanif et al. [3] conducted a systematic review of AI-based wormhole detection methods in Wireless Sensor Networks and concluded that AI- and ML-based techniques are better than state-of-the-art techniques in detection accuracy. Their review also cites a scheme that was accurate in simulation but reached only up to 75% accuracy in a real-time system, which illustrates the gap between simulated and real-world performance. Alshehri [13] proposed two separate models, an SVM and a DNN run at the base station and tested in NS-3.37, for IoT and WSN environments that detect wormhole attacks without special hardware and, in

simulation, show lower end-to-end latency than the compared methods; adaptive learning is left as future work. Bhatti et al. [4] introduced a behavior-based wormhole detection method that targets the actions of an attacker after a wormhole forms, such as packet dropping, TTL tampering, replay, and looping, rather than the tunnel itself. In ns-2 simulations with 200 nodes, the distributed method reached a 98–99% detection rate with a detection and isolation time of about 0.67 s, at an overhead of about 22%. In the testbed of this study, the wormhole is emulated with a wired tunnel between two nodes that the mesh never adopts as a route (D-20).

Recent advances in cross-layer defense mechanisms show promise for simultaneously addressing both attack types. Srinivasan [19] proposed an innovative cross-layer framework that integrates knowledge from the physical, MAC, and network layers to effectively mitigate wormhole and blackhole attacks in wireless ad hoc networks. Their Enhanced Support Vector Machine (E-SVM) algorithm, implemented in NS3, demonstrated superior performance across multiple metrics, including packet delivery ratio (89.29%), dropping ratio (12.5%), and energy consumption, outperforming traditional SVM techniques across various network sizes. Collectively, these studies underline that machine learning, behavioral analysis, and cross-layer approaches offer promising solutions for detecting complex routing-layer attacks in WMNs. By enabling nodes to learn and adapt, these approaches pave the way for intelligent, energy-efficient, and resilient network defense systems. However, the persistent reliance on simulated datasets underscores the need for hardware-generated data from real mesh deployments.

Author and Title	Description	Approach	Contribution
Hanif et al. (2022). AI-Based Wormhole Attack Detection Techniques in WSNs (Electronics)	**Analyzing low-powered network detection methods**	**Systematic review of AI/ML wormhole detection schemes**	**Shows Machine Learning, which generally outperforms traditional methods**
Alshehri (2024). Wormhole detection and mitigation model for IoT/Wireless Sensor Network using Machine Learning	**Two machine learning models (SVM and DNN) that spot tunneling behaviors**	**Separate SVM and DNN models at the base station (NS-3.37)**	**No special hardware; lower end-to-end latency in simulation; adaptive learning left as future work**
Bhatti et al. (2024). Detection & isolation of wormhole nodes based on post-attack actions	**Detects the behavior of malicious relays after the attack**	**Distributed based on behavior features**	**98–99% detection in ns-2 simulation; about 22% overhead**
Bhonsle, M., et al. (2025). Deep Learning-Based Defense Mechanisms against black hole attacks in wireless mesh networks	**Investigates deep learning approaches for detecting blackhole attacks through network traffic pattern analysis**	**RNN and LSTM networks evaluated on the NSL-KDD dataset**	**Demonstrates significant improvements in detection rates compared to traditional rule-based systems; reduces false positives in dynamic mesh environments.**

Vanlalhruaia et al. (2025). Securing AODV against black hole attack for wireless mesh network	**Proposes a detection mechanism specifically for AODV-based WMNs**	**Destination sequence number comparison and hop count verification to identify routing inconsistencies**	**Demonstrates effective mitigation of blackhole attack impacts through simulation-based analysis**
Srinivasan (2025). Innovative cross-layer defense mechanisms for blackhole and wormhole attacks in wireless ad-hoc networks	**Proposes a cross-layer framework integrating knowledge from physical, MAC, and network layers**	**Enhanced Support Vector Machine (E-SVM) algorithm implemented in NS3**	**Achieves superior performance across delivery ratio (89.29%), dropping ratio (12.5%), and energy consumption; outperforms traditional SVM across various network sizes.**

Table 2.2 Summary of Security Challenges and Common Attacks in WMNs

2.3. Infrastructure-less and Decentralized Network Environments

According to Al-Anzi [5], such autonomy and flexibility make WMNs highly adaptable for real-time applications such as disaster recovery, vehicular networks, and remote monitoring. However, the same characteristics that promote flexibility also introduce significant security challenges. Without a central authority to validate identities or enforce policies, malicious nodes can more easily infiltrate the network, manipulate routing information, or disrupt data transmission.

The decentralized nature of these environments complicates the implementation of traditional Intrusion Detection Systems (IDS). Conventional IDS models often depend on centralized monitoring nodes that aggregate traffic data and analyze it for malicious behavior. As Al-Anzi [5] notes, this centralized approach creates bottlenecks and single points of failure, which are unsuitable for large-scale, dynamically changing WMNs. To overcome these limitations, security mechanisms must be distributed and cooperative, where individual nodes participate in both monitoring and decision-making.

Ioulianou et al. [6] addressed this need with a trust-based IDS for IoT networks that use the IPv6 Routing Protocol for Low-Power and Lossy Networks (RPL), aimed at a combination of Rank and Blackhole attacks. Dedicated detector nodes observe their neighbors and a root node makes the final decision about which nodes are malicious. In simulation, the packet overhead of the scheme in attack scenarios was below 2%, and the monitored devices spent no extra energy on sniffing.

Pedroso et al. [39] proposed a collaborative network IDS (C-NIDS) for IoT integration in which signature-based NIDS devices communicate directly and share alarms, detection rules, and attack signatures instead of relying on a central entity.

Collectively, these studies underscore a paradigm shift in how security is maintained in infrastructure-less environments. Instead of relying on centralized servers, emerging approaches distribute intelligence and evaluation across participating nodes. Such designs ensure scalability, fault tolerance, and adaptability—core requirements for WMNs and IoT systems operating under limited energy and computational resources.

Author and Title	Description	Approach	Contribution

Al-Anzi (2022). Design & analysis of IDS for WMNs	**Wireless mesh networks security & IDS constraints**	**IDS analysis focused on Signatures**	**Identifies resource centralization limits**
Pedroso et al. (2024). A direct collaborative network intrusion detection system for IoT network integration	**Cooperative detection across multiple nodes**	**Direct exchange of alarms, rules and signatures between signature-based NIDS**	**99% detection rate; up to 50% better mitigation**

Table 2.3 Summary of Literature on Infrastructure-less/Decentralized Environments

2.4. Intrusion Detection Systems (IDS) for WMNs

The development of Intrusion Detection Systems (IDS) has been a primary focus for securing Wireless Mesh Networks (WMNs) against routing-based attacks. However, the effectiveness of any IDS—whether signature-based, anomaly-based, or distributed—is fundamentally constrained by the quality and realism of the data used to build and validate it.

Al-Anzi [5] provides a foundational analysis of IDS design in WMNs, demonstrating that while signature-based detection achieves high accuracy against known threats, it exhibits critical limitations. Specifically, such systems fail to detect novel or adaptive routing misbehavior, including variations of blackhole and selective forwarding attacks, because they rely on pre-defined attack databases. Signature-based IDS therefore cannot recognize attacks that are not already in the database, such as zero-day exploits in dynamic mesh environments.

Furthermore, Al-Anzi [5] highlights that resource constraints inherent to mesh nodes—limited processing power, memory, and energy—complicate the deployment of comprehensive monitoring. Centralized IDS architectures introduce single points of failure and detection latency, while distributed collaborative designs are more resilient but require nodes to exchange information with one another [6, 39].

These findings point to a data gap as well as an architectural one. Detecting novel attacks and designing distributed monitoring both require knowing what normal and attacked behavior look like on the target hardware, yet most of the studies reviewed above evaluate their designs in simulation or on general-purpose datasets such as NSL-KDD [20]. Hardware-derived datasets that record how routing-layer attacks appear across the PHY, MAC and network layers of an ESP32 mesh would give such work an empirical starting point. This study does not design an IDS. By generating such a dataset under controlled normal and attack conditions, it aims to provide that starting point for future IDS research, whether signature-based, anomaly-based, or distributed.

Author and Title	Description	Approach	Contribution
Al-Anzi (2022). Design & analysis of IDS for WMNs	**Baseline analysis of Wireless Mesh Network threats and Intrusion Detection System**	**Signature-based, centralized learning**	**Makes things clear on the accuracy of known attacks, also exposes zero-day blindness and the risks of centralization**

Table 2.4 Summary of Intrusion Detection Systems (IDS) for WMNs

2.5. Machine Learning and Anomaly Detection in IDS

The integration of machine learning (ML) into Intrusion Detection Systems (IDS) has transformed how modern networks detect and respond to cyber threats. Traditional IDS often relies on static, signature-based methods that fail to recognize new or zero-day attacks. In contrast, ML models such as autoencoders and deep neural networks can learn patterns of normal network behavior and flag deviations as potential intrusions. This anomaly-based detection approach allows systems to identify unknown threats, adapt to evolving attack strategies, and minimize false positives. Rassam [41] applied an autoencoder-based neural network to anomaly detection in Wireless Body Area Networks (WBANs), reporting an F1-score of about 0.96 on physiological data from the PhysioNet database (two ICU patients). The work presents the autoencoder as an unsupervised approach for settings where normal or labeled data are scarce, but it was evaluated on patient signals rather than network traffic and reports no measurements on resource-constrained devices.

Expanding on this concept, Hernandez-Ramos et al. [40] conducted a systematic review of federated learning (FL) in IDS covering literature from 2018 to 2022, emphasizing privacy-preserving collaboration across distributed devices. Instead of transmitting raw data, FL lets multiple IDS nodes train models locally and share only model updates, which the review links to privacy, lower latency, and scalability. It identifies open challenges in communication overhead, security and privacy, non-IID data and system heterogeneity, and the lack of standardized evaluation metrics. Collaboration is not limited to learning-based systems: Pedroso et al. [39] showed that signature-based NIDS devices that share detection rules directly reached a 99% attack detection rate and improved attack mitigation by up to 50% compared with non-collaborative scenarios.

Together, the autoencoder and federated learning studies [40, 41] affirm that machine learning–driven anomaly detection, particularly through autoencoders, neural networks, and federated learning, enables IDS to evolve from static monitoring tools into adaptive, intelligent, and distributed defense systems. By learning normal behavior and securely sharing knowledge across devices, these models provide a robust framework for detecting both known and unknown attacks in complex network environments, such as IoT and Wireless Mesh Networks.

Author and Title	Description	Approach	Contribution
Rassam (2024). Autoencoder-Based Neural Network Model for Anomaly Detection in Wireless Body Area Networks	**Autoencoder anomaly detection on WBAN physiological data**	**Autoencoders**	**F1 about 0.96 on PhysioNet data**
Hernandez-Ramos et al. (2025). Intrusion detection based on federated learning: A systematic review	**Privacy-centric collaborative IDS**	**Systematic review of federated learning in IDS**	**Privacy, latency and scalability benefits; open challenges in communication overhead, non-IID data, heterogeneity, and metrics**

Table 2.5 Summary of ML/Anomaly-Based IDS

2.6. Cross Layer

Cross-layer intrusion detection systems (IDSs) have emerged as a solution to the limitations of single-layer IDS models, which typically monitor only one layer of the network stack—such as the physical, MAC, or network layer—leaving other attack vectors undetected. Full cross-layer IDS frameworks aggregate parameters from multiple layers to improve detection coverage, enabling the system to observe correlations across events in different layers of the network stack. For example, Al-Anzi [5] developed a full cross-layer IDS for Wireless Mesh Networks (WMNs) that collected features including node forwarding rate, routing table changes, packet drop count at the network layer, and RSSI variance at the physical layer to detect attacks such as flooding and battery exhaustion. While this approach increased the ability to detect attacks across multiple stages, Al-Anzi [5] acknowledged several limitations: processing overhead and synchronization delays significantly impacted performance, and early-stage attacks were often missed. Specifically, during a data-link layer flooding attack, detection was delayed because inter-layer parameters needed to be fully exchanged before the system could flag an anomaly. Likewise, the first three battery-exhaustion attacks were undetected because physical-layer metrics could not be promptly transmitted to the network layer. Al-Anzi's results show that although full cross-layer IDSs enhance threat visibility compared to single-layer systems, they add latency and processing overhead.

To mitigate these inefficiencies, recent research has emphasized optimized cross-layer feature selection, focusing only on the most relevant metrics rather than collecting all possible parameters from every layer. Amouri et al. [8] proposed a two-stage, anomaly-based cross-layer IDS for WSN and MANET environments. In their design, the first level uses dedicated sniffers in promiscuous mode that count MAC-layer and network-layer packets and classify them with a decision-tree-based classifier, and the second level sends the results to a supernode that applies linear regression and an accumulated measure of fluctuation (AMoF). The scheme was tested at two node speeds and two power levels and performed well in most cases, but its performance declined sharply at the highest speed and lowest power. Similarly, Singh et al. [9] proposed an optimized multi-feature selection algorithm that picks efficient and useful features from a cross-layer architecture for intrusion detection in wireless sensor networks.

Li et al. [42] further explored the trade-offs between feature selection and feature extraction in IoT intrusion detection. Using the TON-IoT dataset, they found that feature extraction, which creates new derived features from existing data, could improve detection accuracy when the total number of features was extremely limited. However, feature selection, which identifies the most relevant raw features, produced faster inference times and lower computational overhead, making it more suitable for devices with limited memory and processing power. This work underscores the importance of balancing detection performance with system efficiency, particularly in resource-constrained environments. Across these studies, the indicators used include MAC-layer and network-layer packet counts [8] and, in Al-Anzi's IDS, forwarding rate, routing table changes, packet drop count and RSSI variance [5]. By focusing on a small set of such parameters, systems can retain most of the benefits of cross-layer monitoring while avoiding unnecessary computational and communication overhead.

Collectively, these studies suggest that full cross-layer IDSs give broad detection coverage, but their processing time, synchronization delays, and communication overhead can make them hard to run on low-power IoT and WMN devices [5]. Selecting a smaller set of relevant cross-layer features is a pragmatic alternative [8, 9, 42]. This study does not build an IDS. Instead, this insight guides what the testbed records: a compact set of features from the PHY, MAC and network layers, such as RSSI, forwarding ratio, failed send counts, hop count changes and parent-switching events, so that the dataset can support later feature selection and detection work.

Author and Title	Description	Approach	Contribution

Al-Anzi, F. S. (2022). Design and analysis of intrusion detection systems for wireless mesh networks	**Proposed a full cross-layer IDS for WMNs that integrates parameters from multiple layers of the network stack**	**Collected physical, MAC, and network-layer metrics such as RSSI variance, routing table changes, and packet drop rate**	**Enhanced visibility of multi-layer attacks, and identified synchronization delays and processing overhead as major constraints in resource-limited nodes**
Amouri, A., et al. (2018). A Cross-Layer, Anomaly-Based IDS for WSN and MANET.	**Introduced a two-stage anomaly-based cross-layer IDS for WSN and MANET**	**Sniffers count MAC-layer and network-layer packets and a decision-tree classifier labels them; a supernode applies linear regression with AMoF**	**Performed well in most tested settings; performance declined sharply at the highest node speed and lowest power**
Singh, G., et al. (2022). A Cross-Layer Based Optimized Feature Selection Scheme for IDS in WSNs	**Presented an optimized multi-feature selection algorithm for cross-layer features in WSN intrusion detection**	**Optimized multi-feature selection over cross-layer features**	**Selects efficient and useful features from a cross-layer architecture for intrusion detection**
Li, J., et al. (2024). Optimizing IoT Intrusion Detection System: Feature Selection versus Feature Extraction	**Compared the performance of feature selection and feature extraction strategies in IoT IDS**	**Evaluated both methods using the TON-IoT dataset to assess accuracy, inference time, and resource use**	**Provided an empirical basis for lightweight IDS feature optimization, and found that feature selection offers faster inference and lower overhead**

Table 2.6 Summary of Cross-Layer

2.7. ESP32 Technical Literature

In recent research, the ESP32 microcontroller platform and its native mesh networking protocol (ESP-WIFI-MESH) have been increasingly adopted in wireless IoT systems due to their self-organizing, self-healing, and multi-hop communication capabilities. The ESP-WIFI-MESH protocol, provided as part of the official Espressif ESP-IDF framework, enables ESP32 devices to form a tree-based mesh topology in which nodes simultaneously act as both stations and access points, facilitating upstream and downstream links without requiring traditional Wi-Fi infrastructure [21].

Empirical evaluations in open-access literature demonstrate the applicability of ESP-WIFI-MESH in real deployment scenarios. For example, Khan et al. [22] deployed an ESP-MESH network of seven ESP32 nodes on a college campus for indoor and outdoor air-quality monitoring. They reported a packet delivery rate above 97% and packet error and loss rates below 1.8% at every node except

one (node H), which they suspect was affected by its outdoor location. In a short self-healing test, the network recovered in about 4.2 s after a sink-node failure and about 3.5 s after an intermediate-node failure.

Beyond environmental sensing, recent open-access works also integrate ESP-WIFI-MESH in other IoT applications, illustrating its adaptability and ease of implementation. For instance, Qian et al. [23] developed a wireless communication platform for laboratory test instruments (GPIB, RS232, RS485 and CAN) on ESP32-S2-based modules using ESP-WIFI-MESH, reporting a networking time under 10 s, self-repair in about 4.5 s after a child-node failure and about 10 s after a root-node failure, and no communication failures in 12 h of continuous testing.

The ESP32 platform offers distinct advantages for experimental network research and dataset generation, thanks to its programmability, built-in Wi-Fi, and support for detailed runtime logging. Khan et al. [22] demonstrated, through extensive performance evaluation, that ESP-MESH networks can reliably report packet delivery ratios, loss rates, and error rates, metrics essential for behavioral analysis. Their work confirms that ESP32-based testbeds can produce repeatable, quantifiable network data suitable for subsequent analysis. Similarly, Qian et al. [23] measured network establishment and self-repair times on a multi-hop ESP-WIFI-MESH network.

Collectively, these studies establish that ESP-WIFI-MESH on the ESP32 platform provides a low-cost, scalable, and well-characterized foundation for building experimental mesh network testbeds. The platform's combination of self-organizing protocol behavior, accessible hardware, and documented performance metrics makes it highly suitable for systematic dataset generation and behavioral analysis in wireless intrusion detection research. By enabling controlled observation of both normal and anomalous routing behaviors, ESP32-based testbeds can address the gap described in Section 2.8: the absence of hardware-generated, cross-layer datasets from real mesh deployments.

Author and Title	Description	Approach	Contribution
Espressif Systems (n.d.). ESP-WIFI-MESH Programming Guide	**Official programming guide and API documentation for the ESP-WIFI-MESH protocol**	**Technical documentation of mesh architecture, node roles (root/intermediate/leaf), control and data plane operations, parent selection criteria, and RSSI thresholding mechanisms**	**Defines the protocol behavior, forwarding requirements, and configuration parameters that form the baseline for understanding normal mesh operation and potential exploitation vectors**
Khan, A. et al. (2022). An Efficient Wireless Sensor Network Based on the ESP-MESH Protocol for Indoor and Outdoor Air Quality Monitoring.	**Empirical evaluation of ESP-MESH performance using ESP32 nodes for environmental monitoring applications**	**Deployed a multi-hop ESP-MESH network; measured packet delivery ratio, packet loss rate, packet error rate, and self-healing capabilities under real indoor/outdoor conditions**	**Reports a packet delivery ratio above 97% and loss/error rates below 1.8% at all nodes except one outdoor node (H); network recovery in about 3.5–4.2 s in a short self-healing test**

Qian, L. et al. (2024). A Wireless Ad Hoc Network Communication Platform and Data Transmission Strategies for Multi-Bus Instruments	**Implementation of ESP-WIFI-MESH for multi-bus instrument communication in laboratory instrument test systems**	**Developed robust multi-hop ad hoc wireless network; measured network establishment time, self-repair capabilities, and transmission reliability under controlled conditions**	**Reports network formation in under 10 s, self-repair in about 4.5 s (child node) and about 10 s (root node), and no communication failures in 12 h of continuous testing**

Table 2.7 Summary of ESP32 Technical Literature

2.8. Existing Wireless Network Datasets

Several public datasets have supported the development of intrusion detection systems for wireless networks, yet most remain unsuitable for studying routing-based misbehavior in microcontroller mesh deployments. Datasets such as NSL-KDD [29], which is derived from the synthetic KDD Cup 99 traffic, and UNSW-NB15 [27], which was generated in a laboratory testbed with a traffic generator, have been widely used as benchmarks for network intrusion detection. However, these datasets lack the radio-frequency dynamics, hardware-specific constraints, and cross-layer interactions inherent to physical wireless mesh networks. They cannot capture phenomena such as received signal strength variation, retransmission behavior, or parent switching dynamics that arise from real environmental conditions and device-level resource limitations. Despite these limitations, feature extraction approaches from these datasets inform the present study. Specifically, Das et al. [26] built UNR-IDD from the port statistics of an emulated software-defined network, a design that supports the detection of attacks including blackhole and provides a basis for selecting forwarding-related features such as packet counters and traffic asymmetry indicators. Similarly, Avina and Geetha [24] used a ResNeXt and deep stacked autoencoder framework for wormhole detection in a simulated wireless sensor network and on a public wormhole dataset. Their interpretability analysis found that packet- and flow-based features contributed most, and hop-count and neighbor features least, which suggests that topology-related features such as hop count consistency should be examined rather than assumed to be informative.

Hardware-derived datasets offer greater realism but remain limited to specific protocols and attack scenarios. Boiano et al. [25] provide a Zigbee mesh dataset collected from 21 commercial devices deployed in two network topologies, supporting IoT network forensic analysis. However, the authors note that models trained on one topology degrade significantly when tested on others, underscoring the need for topology-aware dataset design. Miekkala et al. [28] present cross-border QoS measurements in 5G and beyond networks, capturing physical and network-layer metrics, but in cellular infrastructure contexts rather than decentralized mesh architectures. Neither dataset addresses the ESP-WIFI-MESH protocol, routing-based attacks such as blackhole and wormhole, or the resource constraints of microcontroller-class devices.

Collectively, existing datasets fail to provide (1) ESP-WIFI-MESH protocol behavior, (2) cross-layer metrics spanning PHY, MAC, and network layers from resource-constrained nodes, (3) documented blackhole and wormhole attack scenarios on real hardware, and (4) multiple topology configurations for comparative analysis. The present study directly addresses these gaps through a hardware-derived ESP32 mesh dataset, in which both attacks are emulated at the application layer (Section 1.4.1), with feature selection informed by the indicators used in Das et al. [26] and Avina and Geetha [24].

Author and Title	Description	Approach	Contribution

Avina, A., & Geetha, M. R. (2025). ResNeXt and Deep Stacked Autoencoder-based framework for wormhole attack detection on network control system	**Framework for wormhole attack detection in network control systems using deep learning architectures**	**ResNeXt and deep stacked autoencoder; simulated WSN (LEACH) and a public wormhole dataset**	**Packet- and flow-based features dominate in their data; hop and neighbor features contribute least**
Boiano, A., et al. (2026, February 3). Analyzing Zigbee Traffic: Datasets, classification, and storage trade-offs	**Introduction of ZIOTP2025 Zigbee mesh dataset collected from 21 commercial devices for IoT network forensic analysis**	**Collection of Zigbee traffic in mesh topology configurations; analysis of classification performance and storage requirements**	**Reveals that models trained on one topology degrade significantly when tested on others; underscores the need for topology-aware dataset design in mesh network research**
Das, T., et al. (2023, January). Unr-idd: Intrusion detection dataset using network port statistics	**Development of an intrusion detection dataset specifically focused on network port statistics for attack detection**	**Collection of network port statistics under normal and attack conditions; emphasis on statistical features rather than packet payload inspection**	**Built from the port statistics of an emulated software-defined network and supports detection of attacks including blackhole; provides a basis for selecting forwarding-related features such as packet counters and traffic asymmetry indicators in the current study**
Moustafa, Nour & Slay, Jill. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)	**Creation of a modern network intrusion dataset capturing contemporary attack patterns and normal traffic behaviors**	**Hybrid of realistic synthetic traffic generation and real network captures; includes nine families of attacks with associated features**	**Addresses limitations of older datasets (KDD'99, NSL-KDD) by including modern attack types and more representative network traffic; provides a comprehensive feature set for IDS evaluation**

Miekkala, T., et al. (2024). NordicDat: A Cross-Border Predictive QoS Dataset	**Cross-border QoS dataset for 5G and beyond networks, capturing physical and network-layer metrics**	**Collection of QoS measurements across Nordic country borders; includes physical layer metrics and network performance indicators**	**Provides valuable cross-layer metrics but in cellular infrastructure contexts; demonstrates the feasibility of hardware-derived QoS datasets while highlighting the absence of decentralized mesh architecture data**
Tavallaee, M., et al. (2009). A detailed analysis of the KDD CUP 99 data set	**Critical analysis and improvement of the original KDD'99 dataset for intrusion detection benchmarking**	**Statistical analysis of dataset redundancies and difficulties; proposed NSL-KDD as a refined version addressing the inherent problems of the original KDD'99**	**Provides a widely-used benchmark dataset with reduced redundancy and balanced record distribution; enables fairer evaluation of intrusion detection systems; widely cited as a baseline for network intrusion detection research**

Table 2.8 Summary of Existing Wireless Network Datasets

2.9. Dataset Design and Feature Engineering

Constructing meaningful datasets for network behavior analysis requires careful attention to experimental design, data collection, and feature representation. Poorly designed datasets introduce biases and fail to capture real-world dynamics, which undermines subsequent analysis [29, 30]. Sommer and Paxson [30] note that evaluating detection methods on simulated activity or small laboratory networks often lacks realism and relevance, and that standardized test sets and representative data are rarely available in this domain. This supports the present study's decision to generate hardware-based ESP32 mesh data rather than rely only on simulation.

Tavallaee et al. [29] analyzed the weaknesses of the KDD Cup 99 dataset, including redundancy, imbalance, and unrealistic traffic representation. They proposed NSL-KDD, which removes the redundant records, and showed that such flaws bias evaluation, which underscores the need to inspect the structure of a dataset before modelling it. Ring et al. [31] surveyed network-based intrusion detection datasets and compared them on 15 properties, including whether the traffic is real, emulated or synthetic. Their survey is limited to packet-based and flow-based network traffic and does not cover cross-layer or microcontroller-mesh data, which is the gap this study addresses. García-Teodoro et al. [32] emphasized that effective anomaly modeling depends on well-structured datasets and meaningful feature representations, which support the cross-layer metric selection adopted here.

Feature engineering is equally critical for capturing network behavior. Akyildiz et al. [1] describe hop count, link quality and routing metrics as basic characteristics of mesh routing, which motivates topology-based features such as hop count changes and parent-switching events. Cilfone et al. [2] survey mesh technologies for IoT and discuss link-quality estimation and packet delivery ratio as performance measures, which motivates recording RSSI and packet delivery ratio without heavy payload inspection. Tsai et al. [33] reviewed machine-learning techniques for intrusion detection, including the datasets used to evaluate them, which reinforces the need

to examine a dataset's features before modelling. Chandola et al. [34] survey anomaly detection techniques and group them into classification-based, clustering-based, nearest-neighbor, statistical and information-theoretic approaches, which situates the clustering-based, unsupervised analysis used in this study. Building on this, the principles of feature selection, specifically the identification and removal of redundant or low-variance attributes to improve model performance and interpretability, are well-established in data mining literature [36, 37]. The present study applies these principles by incorporating a systematic feature relevance analysis, including variance thresholding and correlation matrices, to ensure the dataset is optimized for subsequent unsupervised clustering.

Collectively, these studies establish that dataset design must balance completeness with resource constraints, cross-layer feature selection enables behavioral characterization without excessive overhead, and ground-truth labeling is essential for validating dataset utility. The present study applies these principles to construct a hardware-derived ESP32 mesh dataset capturing routing behavior under normal and adversarial conditions, with feature selection informed by detection-relevant indicators from the literature.

Author and Title	Description	Approach	Contribution
Tavallaee, M., et al. (2009). A detailed analysis of the KDD CUP 99 data set	**Evaluation of the KDD Cup 99 dataset weaknesses, including redundancy, imbalance, and unrealistic traffic**	**Statistical analysis of dataset characteristics; proposed NSL-KDD, which removes redundant records**	**Shows that such flaws bias evaluation, which reinforces the need to inspect a dataset's structure (EDA) before clustering**
García-Teodoro et al. (2009). Anomaly-Based Network Intrusion Detection	**Survey of anomaly-based IDS techniques in network security**	**Statistical profiling and anomaly modeling**	**Supports the use of well-structured datasets and meaningful feature representations for anomaly modeling**
Tsai, C., et al. (2009). Intrusion detection by machine learning: A review	**Review of machine-learning techniques for intrusion detection**	**Literature review of ML-based IDS techniques and the datasets used to evaluate them**	**Reinforces the need to examine a dataset's features before modelling**
Ring, M., et al. (2019). A survey of network-based intrusion detection data sets	**Survey of network-based intrusion detection data sets, compared on 15 properties**	**Comparison of datasets by properties such as whether the traffic is real, emulated or synthetic**	**Covers only packet-based and flow-based traffic, so cross-layer and microcontroller-mesh data remain uncovered; situates the ESP32 cross-layer dataset within that gap**

Chandola, V., et al. (2009). Anomaly Detection: A Survey.	**Comprehensive survey of anomaly detection methodologies**	**Classification-based, clustering-based, nearest-neighbor, statistical and information-theoretic approaches**	**Situates the clustering-based, unsupervised analysis used in this study among established anomaly detection approaches**
Akyildiz, I. F., et al. (2005). Wireless Mesh Networks: A Survey	**Foundational survey of mesh network architecture, routing behavior, and multi-hop dynamics**	**Literature review and conceptual analysis of mesh network characteristics**	**Supports selection of topology-based features (hop count dynamics, parent switching frequency) grounded in mesh network theory**
Cilfone, A., et al. (2019). Wireless Mesh Networking: An IoT-Oriented Perspective	**Survey of mesh networking technologies for IoT, including link-quality estimation and packet delivery ratio**	**Survey of IEEE 802.11, Bluetooth, IEEE 802.15.4 and LoRa mesh technologies**	**Supports recording RSSI and packet delivery ratio without heavy payload inspection**
Sommer & Paxson (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. Proceedings	**Examines why anomaly detection research rarely reaches operational use, including the difficulty of obtaining realistic evaluation data**	**Critical analysis of why machine learning is rarely used for network intrusion detection in practice**	**Supports the decision to generate hardware-based ESP32 mesh data rather than rely only on simulation**

Table 2.9 Summary of Dataset Design and Feature Engineering

2.10. Exploratory Data Analysis and Clustering in Network Telemetry

Exploratory Data Analysis (EDA) provides a foundational approach to understanding the structure of a dataset before formal modeling or hypothesis testing. Tukey [35] formally introduced EDA as a philosophy focused on discovering underlying structure, detecting anomalies, and testing assumptions through visualization and summary statistics rather than confirmatory techniques. This framework is particularly relevant to network telemetry analysis, where distributed system behavior is often non-linear, cross-layer, and context-dependent. Tukey's work supports the present study's decision to conduct a dedicated EDA stage to examine cross-layer mesh metrics before clustering, ensuring that the behavioral characteristics of the ESP-WIFI-MESH network are understood before structural assumptions are imposed.

Building on EDA principles, Aggarwal [36] discusses essential preprocessing steps before clustering or classification, including data cleaning, feature transformation, normalization, and correlation analysis. These steps are critical in network telemetry contexts where distributed node logs may suffer from timestamp drift, inconsistent sampling intervals, or partial packet records. Aggarwal's emphasis on understanding feature relationships before algorithmic application directly supports this study's workflow of log aggregation, timestamp alignment, feature scaling, and cross-layer correlation analysis, all performed before applying unsupervised methods.

Similarly, Han et al. [37] emphasize descriptive statistics, distribution analysis, and visualization to understand dataset structure, thereby justifying the inclusion of topology-dependent visualization and correlation analysis in the exploratory phase of the present research.

In networked and IoT environments, EDA is often complemented by unsupervised anomaly detection techniques because labeled attack datasets are scarce. Chatterjee and Ahmed [48] survey 64 IoT anomaly detection studies published between January 2019 and July 2021 and identify the scarcity of ground-truth data as an open gap, which motivates methods that do not depend on labeled attacks. Similarly, broader anomaly detection literature emphasizes deviation-based modeling in high-dimensional systems where attack signatures may not be predefined [34]. These perspectives support the study's reliance on behavioral baselining rather than supervised intrusion classification.

Further strengthening this direction, Guerrero-Higueras et al. [49] provide a systematic literature review of unsupervised anomaly detection in network flow analysis. Their PRISMA-based review documents the use of density-based methods, isolation-based techniques, reconstruction models such as autoencoders, and one-class learning approaches for identifying deviations without labeled supervision. The findings of [49], together with surveys of network-based intrusion detection datasets and techniques [31–33], reinforce the methodological decision to explore structural deviations in feature space rather than claim real-time detection performance.

Clustering methodologies naturally extend EDA by enabling the identification of natural groupings within unlabeled telemetry data. Jain [38] reviews 50 years of clustering research, stressing that clustering success depends critically on understanding data structure, feature relevance, and distance metrics before selecting an algorithm. Jain emphasizes that clustering is inherently exploratory rather than confirmatory and that it requires careful interpretation and validation. This aligns with the present study's objective of using unsupervised clustering to examine whether normal and manipulated network conditions exhibit separable patterns in feature space, without asserting operational intrusion detection capability.

Common clustering techniques applied in network telemetry include centroid-based approaches such as K-Means, density-based algorithms such as DBSCAN, and hierarchical clustering methods. These approaches are frequently used to identify operational modes, transitional behaviors, or structural regime changes in network flow and wireless telemetry datasets [38, 49]. In mesh network contexts, clustering provides a means to evaluate whether baseline configurations and controlled manipulation phases occupy distinct regions in feature space.

High-dimensional telemetry datasets often require dimensionality reduction to improve interpretability and assess clusterability. Jolliffe [52] presents Principal Component Analysis (PCA) as a variance-maximizing linear projection technique useful for identifying dominant sources of variance and detecting feature redundancy. PCA supports preliminary feature validation by revealing which engineered metrics contribute most strongly to overall variability, consistent with preprocessing principles discussed in [36, 37].

For nonlinear visualization, van der Maaten and Hinton [50] introduce t-Distributed Stochastic Neighbor Embedding (t-SNE), which preserves local neighborhood relationships in reduced-dimensional space. t-SNE is widely used for exploratory visualization of potential separability in high-dimensional datasets. Similarly, McInnes, Healy, and Melville [51] propose Uniform Manifold Approximation and Projection (UMAP), a scalable manifold-learning method that preserves both local and global structural properties. UMAP is particularly useful for visualizing topology-dependent grouping tendencies in network telemetry and for examining clusterability in complex embeddings. As emphasized in the visualization literature [50, 51], these techniques primarily serve as exploratory tools and do not, on their own, validate clustering performance without complementary quantitative measures.

Collectively, these works establish EDA [35–37], unsupervised anomaly detection [48, 49], clustering [38], and dimensionality reduction techniques [50–52] as complementary methodologies for understanding complex telemetry datasets. In the context of this study, EDA establishes statistical baselines and cross-layer feature relationships; dimensionality reduction evaluates variance structure and potential separability; and clustering explores whether baseline and manipulated mesh conditions form distinguishable behavioral

groupings. Together, these methods form the analytical backbone of the research, supporting dataset validation and structural behavior discovery rather than real-time intrusion detection deployment.

Author and Title	Description	Approach	Contribution
Tukey, J. W. (1977). Exploratory Data Analysis	**Foundational introduction to EDA for discovering structure and detecting anomalies**	**Philosophy emphasizing visualization over confirmatory techniques**	**Supports a dedicated EDA stage for examining cross-layer mesh metrics before clustering**
Aggarwal, C. C. (2015). Data Mining: The Textbook	**Discussion of preprocessing, transformation, and correlation analysis before clustering**	**Systematic coverage of data preparation methodologies**	**Supports log aggregation, timestamp alignment, and feature scaling before unsupervised methods**
Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques	**Emphasis on descriptive statistics and visualization for understanding the dataset structure**	**Comprehensive treatment of data exploration methodologies**	**Justifies distribution analysis, topology visualization, and cross-layer correlation examination**
Jain, A. K. (2010). Data Clustering: 50 Years Beyond K-Means	**Review of clustering methodologies, stressing understanding data structure before algorithm selection**	**Historical and methodological analysis of clustering techniques**	**Supports exploratory clustering; reinforces careful feature examination before unsupervised methods**
Chatterjee & Ahmed (2022). IoT Anomaly Detection Methods and Applications: A Survey	**Summarizes IoT anomaly detection methods and application domains**	**Survey of 64 IoT anomaly detection studies published between January 2019 and July 2021**	**Identifies the scarcity of ground-truth data as an open gap, which motivates unsupervised baselining when labeled attack data are limited in IoT telemetry environments**

Guerrero-Higueras et al. (2026). Unsupervised Anomalous Traffic Detection Based on Flows (SLR)	**Reviews unsupervised algorithms for anomaly detection in network flow data**	**PRISMA-style systematic literature review of flow-based anomaly detection techniques**	**Supports the use of unsupervised methods (e.g., autoencoders, density-based, and outlier detection techniques) to detect deviations without supervised labels**
van der Maaten, L., & Hinton, G. (2008). Visualizing Data Using t-SNE	**Introduces t-SNE for nonlinear dimensionality reduction and visualization of high-dimensional datasets**	**Manifold-based embedding focused on preserving local neighborhood structure**	**Useful for inspecting whether telemetry windows form separable visual groupings; supports exploratory cluster visualization rather than confirmatory validation**
McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection	**Presents UMAP as a scalable nonlinear dimensionality reduction technique**	**Topological manifold learning for preserving both local and global structure**	**Useful for exploring global and local structure in mesh telemetry and examining topology-dependent grouping tendencies**
Jolliffe, I. T. (2002). Principal Component Analysis (2nd ed.)	**Foundational work on PCA and variance-based dimensionality reduction**	**Linear orthogonal projection maximizing variance explained**	**Supports feature redundancy checks and identification of dominant variance drivers in engineered cross-layer mesh features**

Table 2.10 Summary of EDA and Clustering in Network Topology

3. Theoretical Framework

This chapter establishes the theoretical foundations that inform the design, generation, and subsequent analysis of a cross-layer dataset from ESP32-based wireless mesh networks. The theoretical framework integrates concepts from wireless mesh networking, routing protocol behavior, attack theory, and cross-layer observation to provide a basis for understanding how routing-based misbehavior manifests in observable network telemetry.

The chapter is organized into five main sections. Section 3.1 presents wireless mesh networks, characterizing the ESP-WIFI-MESH architecture and its operational principles. Section 3.2 examines topology formation and physical influences, detailing how environmental factors mediate the relationship between physical node placement and logical network structure. Section 3.3 provides a theoretical analysis of routing-based misbehavior, adapting canonical blackhole and wormhole attack concepts to the ESP-WIFI-MESH context. Section 3.4 develops the cross-layer telemetry perspective, explaining why routing violations affect multiple protocol layers and identifying expected observable inconsistencies. Finally, Section 3.5 synthesizes these elements into a conceptual framework that positions normal behavior as consistent relationships and misbehavior as broken relationships, explains why the attacks are reproduced by application-layer emulation, and provides the theoretical motivation for dataset collection.

Throughout this chapter, the emphasis is on theoretical characterization rather than detection—the goal is to establish what can be observed under different network conditions, not to prescribe how to identify attacks in real time.

Two terms are used in a fixed way throughout the thesis. First, ESP-IDF numbers the root node as layer 1 (MESH_ROOT_LAYER) [76]. This thesis reports hop count = layer − 1, so the root is at hop 0 and its direct children are at hop 1; the word “layer” is used for the ESP-IDF value only when it is stated as such. Second, “network layer” refers to the mesh routing and forwarding layer of ESP-WIFI-MESH, which sits between the application and the IEEE 802.11 MAC/PHY; it does not mean the IP layer.

3.1. Wireless Mesh Network Fundamentals

3.1.1. Wireless Mesh Network Characteristics

Wireless Mesh Networks (WMNs) represent a decentralized communication paradigm characterized by self-organization, minimal reliance on fixed infrastructure, and multi-hop communication. Unlike traditional centralized networks, a mesh node can act as both a host and a router, forwarding packets

for other nodes to maintain network connectivity [1, 2]. This flexibility makes WMNs suitable for Internet of Things (IoT) deployments such as smart cities and other monitoring scenarios where fixed infrastructure is unavailable or impractical [2].

The ESP-WIFI-MESH protocol, implemented on ESP32 microcontrollers, organizes nodes into a hierarchical tree topology [21]. The architecture comprises three node roles: the Root Node, which sits at the top of the tree and coordinates the mesh (and serves as the gateway to an external network when a router is used); Intermediate Nodes, which have child nodes and therefore operate in dual Access Point/Station (AP/STA) mode, relaying packets between their parent and their children; and Leaf Nodes, which are not permitted to have child nodes, so they can only transmit and receive their own packets and cannot forward the packets of other nodes [21]. A node's role follows from whether it has children, not from a fixed layer number.

The protocol's operation involves two logical traffic types: the control plane, which handles node discovery, parent selection, and layer assignment (parent selection favors the shallowest layer and, among candidates in the same layer, the fewest children, with RSSI acting as a threshold rather than a ranking value; see Section 3.2.1); and the data plane, which carries application payloads hop by hop through the tree, mainly from the nodes toward the root, using best-effort delivery. This design enables simple, self-organizing connectivity but introduces security vulnerabilities, because forwarding behavior is not verified at the protocol level: a node that accepts packets on behalf of others is trusted to pass them on.

From a theoretical standpoint, routing decisions in ESP-WIFI-MESH assume that a neighbor's advertised layer and the observed signal strength reflect its real position in the network, and that a node accepting packets will forward them. When these assumptions hold, the network exhibits stable packet propagation. In the deployment reported by Khan et al. [22], ESP-MESH nodes achieved packet delivery ratios above 97% with packet loss and packet fault rates below 1.8%, except for one node (node H). Because this is the result of a single reported deployment, it is used here only as an indicative baseline and not as a guarantee; this study measures its own baseline from the baseline phase of every run (D-5), and the 8 matched benign runs serve as a stand-alone check (Section 6.2.6). When a metric is manipulated or a relay does not forward, the absence of centralized verification lets network behavior degrade without any violation of protocol syntax, which is why routing-based misbehavior is difficult to see at the protocol level.

3.1.2. ESP-WIFI-MESH Architecture and Node Roles

ESP-WIFI-MESH organizes nodes into a tree topology [21]. At the top is the Root Node (hop 0), which coordinates the mesh and may act as a gateway to an external network. Below it are Intermediate Nodes, which operate in dual AP + STA mode: they connect upward to a parent while simultaneously accepting connections from child nodes. At the end of each branch are Leaf Nodes, which are not permitted to have children and therefore need only the station (STA) interface. The ESP-IDF API numbers the root as layer 1 [76]; this thesis reports hop = layer − 1, so the root is hop 0 and its direct children are hop 1.

The node roles are defined as follows:

- **Root Node (hop 0):** Coordinates the mesh and may act as a gateway to an external network. In this testbed no router is used, so the root does not connect to an external network; it acts as the sink for upstream probe traffic.
- **Intermediate Nodes (hop 1 and deeper, with children):** Operate in dual AP + STA mode; connect upward to a parent while simultaneously accepting connections from child nodes.
- **Leaf Nodes (no children):** Need only the STA interface. They transmit and receive their own data but cannot forward the packets of other nodes.

The right side of Figure 3.1 shows the ESP32 network stack, where ESP-WIFI-MESH operates below the application layer and above the IEEE 802.11 MAC/PHY. Routing and forwarding are handled inside the mesh stack: the application sends and receives packets through the mesh API, and the stack selects the next hop. The application can read the node's routing table through esp_mesh_get_routing_table() [75], but it takes no part in routing decisions or in the forwarding of packets that are not addressed to it.

This design separates functionality into two logical components:

- **Control Plane (dashed arrows):** Handles node discovery, parent selection, and layer assignment. Parent selection favors the shallowest layer and the fewest children, with RSSI used through thresholds.
- **Data Plane (solid arrows):** Carries application payloads hop by hop through the tree, mainly from the nodes toward the root, using best-effort delivery.

This enables simple, self-organizing connectivity, but also means forwarding behavior is not verified at the protocol level.

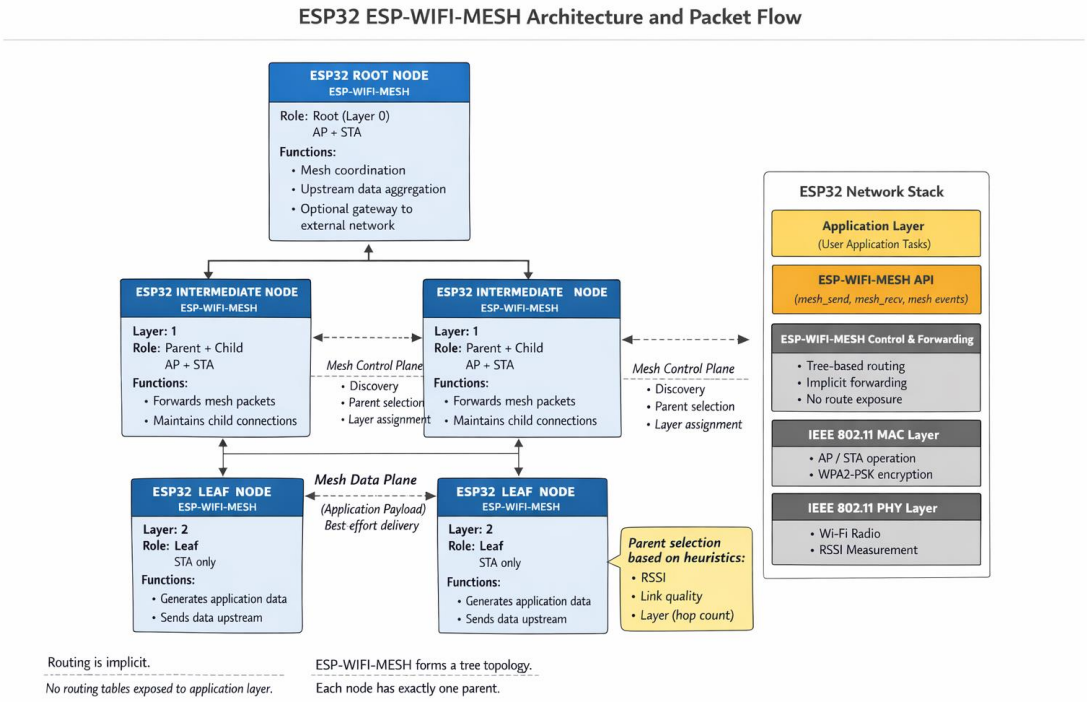


Figure 3.1 Architecture of ESP32-WFI-MESH

Figure 3.1 illustrates how ESP32 devices form a mesh network using ESP-WIFI-MESH and how data flows through it.

3.1.3. Control Plane vs. Data Plane Operations

3.1.3.1. Control Plane

The control plane handles network management and topology maintenance functions, including:

- **Node Discovery:** A node that is not yet part of the mesh scans for the beacons of existing mesh members to identify potential parents [21].
- **Parent Selection:** Nodes evaluate candidate parents by the layer of the candidate first and by the number of existing child connections second. RSSI enters the decision through thresholds rather than as a value to be maximized (Section 3.2.1) [21].
- **Layer Assignment:** Based on parent selection, a node's layer is its parent's layer plus one, so each hop from the root increases the layer number by one (the root is layer 1 in ESP-IDF and hop 0 in this thesis).
- **Topology Maintenance:** The mesh stack monitors the link to the parent and keeps the routing table up to date. Node failures or link degradation trigger parent re-selection, which lets the mesh heal itself [21].

3.1.3.2. Data Plane

The data plane carries application payloads through the mesh using best-effort delivery. Key characteristics include:

- **Hop-by-Hop Forwarding:** Packets traverse from source to destination through intermediate nodes, with each node forwarding toward the root (upstream) or away from the root (downstream) [21].
- **Forwarding Discipline:** When an application sends a packet to a destination several hops away, the mesh stack handles the forwarding entirely on the mesh layer [21], below the application, and the application is handed only the packets addressed to the node itself (esp_mesh_recv() receives a packet "targeted to self") [75]. The application therefore has no per-hop view of whether a relay passed a packet on. To make forwarding observable, the testbed relays probes explicitly at the application layer on every node: each node sends its probes to its own parent using esp_mesh_send() with the MESH_DATA_P2P flag, and a node that receives a probe from a child with esp_mesh_recv() forwards it one hop upstream, counting the packets received, forwarded, and dropped [75, 76]. This explicit relay requirement establishes the behavioral baseline against which deviations can be observed.
- **Queue Management:** A node must also read the packets addressed to it regularly with esp_mesh_recv(), because the mesh RX queue should be checked regularly to avoid running out of memory [76].

From a theoretical standpoint, routing decisions in ESP-WIFI-MESH assume that locally observed metrics accurately reflect physical proximity and forwarding reliability. When these assumptions hold, the network exhibits stable packet propagation and predictable performance. In the single deployment reported by Khan et al. (2022), ESP-MESH nodes achieved packet delivery ratios above 97%, with packet loss and packet fault rates below 1.8% at every node except one; this is an indicative figure from one deployment, not a property of every ESP-MESH network.

Table 3.1 ESP-WIFI-MESH Node Roles and Responsibilities

Node Type	Layer	Mode	Forwarding Responsibility	Primary Function
Root	0	STA + AP (to router)	Yes	Gateway to external networks
Intermediate	1+	AP/STA dual	Yes	Relay packets + generate traffic
Leaf	2+	STA only	No	Generate traffic only

This design enables simple, self-organizing connectivity but introduces inherent trust assumptions: forwarding nodes are expected to comply with protocol requirements, and locally observed metrics are trusted for routing decisions. When these assumptions are violated, the absence of centralized verification creates systemic vulnerabilities that can distort network behavior without violating protocol syntax.

3.2. Topology Formation and Physical Influences

3.2.1. RSSI Thresholds and Parent Selection

Parent selection in ESP-WIFI-MESH is governed by the layer of each candidate parent and by the number of children it already has [21]. RSSI is not a ranking metric: it enters the decision through the thresholds described below, which group candidates into signal ranges and flag weak links, but among acceptable candidates the shallowest layer wins and the child count breaks ties. RSSI therefore reflects physical proximity only indirectly, since a distant or obstructed candidate is more likely to have a weak link, without determining which parent is chosen.

3.2.1.1. RSSI Thresholding Mechanism

ESP-WIFI-MESH implements RSSI thresholding to prevent nodes from forming weak connections that could degrade network performance (Espressif Systems, 2023). The protocol defines three default RSSI thresholds:

Table 3.2 ESP-WIFI-MESH Default RSSI Thresholds

Threshold	Value	Significance
High	-78 dBm	Minimum acceptable RSSI for preferred parent relationships
Medium	-82 dBm	Intermediate threshold for parent evaluation
Low	-85 dBm	Threshold below which parent switching is triggered

These thresholds are defined in the ESP-WIFI-MESH internal header file (esp_mesh_internal.h) within the mesh_rssi_threshold_t structure [76]. Parent switching is controlled by separate parameters (the mesh_switch_parent_t structure, including its switch_rssi field) [76]. The firmware leaves the RSSI thresholds of Table 3.2 at their defaults. In tree runs it does change the parent-switch parameters: the keep and switch thresholds of mesh_switch_parent_t are lowered by 6 dB and the weak-link watch lasts three times longer, so a child keeps its parent unless the link degrades substantially (D-26, Section 4.2.2.2). Tree runs captured before this change used the defaults.

3.2.1.2. Parent Selection Criteria

According to official ESP-WIFI-MESH documentation, the preferred parent node is determined based on two primary criteria (Espressif Systems, 2023):

1. **Layer Position:** The layer of the candidate parent node—parents must reside at a lower layer than the connecting node.
2. **Child Count:** The number of downstream connections the candidate parent node currently has.

When multiple candidates exist on the same layer, “the candidate with the fewest child nodes will be preferred” [21]. This load-balancing mechanism spreads connections across the available parents. It also means that parent choice depends on advertised values (layer and child count) that the joining node cannot verify, which is the kind of trust assumption that canonical routing attacks exploit.

The mesh_switch_parent_t structure further defines the switch_rssi parameter, which determines when a node will "disassociate with current parent and switch to a new parent when the RSSI is

greater than this set threshold" (Espressif, 2023b). This mechanism enables dynamic topology adaptation but also provides an exploitation vector for a node that appears with a stronger signal than its position justifies, for example by transmitting at a higher power (RSSI is measured by the receiver; it is not advertised).

3.2.2. Environmental Effects on Topology Formation

Physical environment characteristics significantly influence observed RSSI values and, consequently, the logical topology formed by ES-WIFI-MESH. Understanding these effects is essential for interpreting network behavior and designing controlled experiments.

3.2.2.1. Impact of Physical Obstacles

RF shielding objects—including walls, metal structures, furniture, and building materials—can dramatically attenuate wireless signals, reducing RSSI even when nodes are physically proximate [21]. This attenuation creates several important phenomena:

- A physically close node may be rejected as a parent candidate due to low RSSI caused by obstacles
- A physically distant node with a clear line-of-sight may become the preferred parent despite greater physical separation
- The resulting logical topology may differ significantly from the physical node layout

As illustrated in the Espressif documentation, an RF shielding object can cause nodes to disregard physically adjacent parent candidates and instead connect to more distant nodes with stronger signals [21]. This decoupling of physical and logical topology has important implications for experimental design and behavior interpretation.

3.2.2.2. Multi-path and Interference Effects

Indoor environments typically exhibit complex RF characteristics, including:

- Multi-path propagation causes signal fading and reinforcement
- Co-channel interference from other Wi-Fi networks
- Transient signal variations due to environmental changes

These factors introduce variability in RSSI measurements, which must be accounted for in the experimental design. Indoor environments with limited external interference improve repeatability and make ground-truth labeling easier. Because interference differs from site to site, the site of every run is recorded so that site-dependent effects can be examined during analysis.

3.2.2.3. Chain Formation in Linear Deployments

When nodes are arranged linearly—for example, along a hallway or corridor—the network can form a chain topology. In this testbed, the linear, partial, and star topologies are obtained by combining node placement with limits on the number of children and layers each node may accept. This configuration has distinct characteristics compared to branched tree topologies, including:

- Longer end-to-end paths with higher hop counts
- Single points of failure, where the failure of an intermediate node disconnects the nodes behind it
- Cumulative latency and packet loss effects

Understanding these environmental influences guides experimental testbed design and informs interpretation of observed network behaviors.

3.2.3. Logical vs. Physical Topology Relationships

Parent selection mediates the relationship between physical node placement and logical network topology: RSSI thresholds flag weak links, and the layer and child-count rules then choose among the remaining candidates (Section 3.2.1). This mediation creates several theoretically important relationships.

3.2.3.1. Hop Count and RSSI Relationship

Two ideas are easily confused and are kept apart in this thesis. First, the RSSI that a node reads with esp_wifi_sta_get_rssi() describes only the link between that node and its current parent. It is a per-link value; it is not accumulated along the path, so it does not add up with hop count, and there is no fixed attenuation per hop. Second, a node that is deeper in the tree is usually, but not always, physically farther from the root, and a longer link tends to have a weaker RSSI.

The relationship that can reasonably be assumed under normal conditions is therefore a loose, statistical one: nodes deeper in the tree tend to have per-link RSSI values that are no stronger than those of nodes closer to the root, and every link stays above the low threshold (-85 dBm) at which the

stack reports a weak link (Table 3.2). In this thesis the hop count–RSSI relationship is treated as a tendency to be examined in the collected data, not as a fixed rule.

3.2.3.2. RSSI Reference Levels in ESP-WIFI-MESH

Rather than assuming fixed RSSI values for each position, this study reads the RSSI of every node's parent link from the collected data. The thresholds in Table 3.2 are the only reference levels that the mesh stack itself defines; Table 3.3 shows the RSSI ranges they create.

Table 3.3 Expected RSSI Ranges by Node Position

Node Position	Expected RSSI Range	Relationship to Thresholds
Proximal to the router	≈ -30 dBm	Far above high threshold (-78 dBm)
Normal parent node (Layer 1)	≈ -50 to -78 dBm	Above or near the high threshold
Leaf node (Layer 2-3)	≈ -78 to -85 dBm	Between high and low thresholds
Unstable connection	< -85 dBm	Below the low threshold—triggers parent switching

3.2.3.3. Topology Formation Principles

Based on these relationships, several principles govern topology formation:

1. **Layer Progression:** Each node must connect to a parent at a strictly lower layer, ensuring hierarchical structure.
2. **Signal-Based Selection:** RSSI only decides which candidates are acceptable, through the thresholds of Table 3.2; it does not rank them (Section 3.2.1).
3. **Load Balancing:** Among acceptable candidates on the same layer, those with fewer children are preferred.
4. **Stability Maintenance:** Nodes remain with their current parents unless a superior alternative appears or the current parent's RSSI degrades below thresholds.

These principles establish the theoretical baseline for normal network behavior against which attack-induced deviations can be observed. When malicious nodes manipulate RSSI advertisements or child count presentations, they can distort logical topology in ways that violate the expected relationships between physical placement and network structure.

3.3. Routing-Based Attack Theory

3.3.1. Blackhole Attack: Conceptual Foundation

Blackhole attacks are theoretically classified as routing-layer denial-of-service attacks in which a malicious node attracts network traffic by advertising favorable routing metrics and then drops the data packets it receives instead of forwarding them [43]. In the basic form the node drops all received data packets; in a selective variant it drops only some of them. The defining characteristic of a blackhole attack is not the transmission of malformed packets or explicit protocol violations, but the non-forwarding of data after a route through the attacker has been established.

Unlike attacks that disrupt routing syntax or corrupt packet structure, blackhole behavior exploits cooperative routing assumptions. The attacker participates in control-plane exchanges, thereby appearing legitimate to neighboring nodes while discarding data traffic.

3.3.1.1. Theoretical Characterization

Baadache and Belmehdi [44] likewise treat the blackhole as a node that attracts traffic and then refuses to forward it, creating a logical “hole” in the network. In the canonical form, the attacker takes part in route discovery, typically while advertising a falsely attractive route [43, 44].

The attack exploits a core assumption of multi-hop wireless mesh routing: that intermediate nodes will forward packets received on behalf of others. When this assumption is violated, end-to-end communication reliability degrades without immediate structural collapse of the routing topology.

The operational stages of a canonical blackhole attack can be conceptualized as:

- **Attractor Phase:** The malicious node advertises favorable routing metrics—such as lower hop count, stronger signal quality, or superior positioning—causing legitimate nodes to select it as a next-hop forwarder.
- **Establishment Phase:** The attacker becomes integrated into active routing paths and begins receiving packets from nodes while maintaining protocol-compliant control-plane participation.

- **Execution Phase:** The attacker discards received data packets instead of forwarding them, while continuing to maintain normal neighboring control-plane behavior to avoid detection.

Importantly, the routing structure may appear stable even while data forwarding behavior is compromised.

In the emulation used in this study, the attractor phase is replaced by placement: the blackhole is a legitimate mesh node positioned so that probe traffic passes through it (Section 3.5.3). The establishment and execution phases are reproduced as described.

3.3.1.2. Adaptation to ESP-WIFI-MESH Context

In ESP-WIFI-MESH, the mesh stack relays packets along the tree on its own, so a relay does not have to do anything for forwarding to happen. Because forwarding is not cryptographically enforced or verified, the protocol assumes that relay nodes comply cooperatively. In this study, forwarding is made explicit at the application layer (Section 3.1.3.2): every node relays probes hop by hop, and the blackhole node simply does not relay the probes it receives.

In this application-layer relay, a blackhole node therefore:

- Associates with the mesh through the normal process and acts as a parent node,
- Participates in mesh control exchanges,
- Receives packets from child nodes,
- Fails to forward those packets toward the root.

Since MAC-layer acknowledgments may still occur at the link level, upstream nodes may observe successful transmission even though end-to-end delivery fails. This decoupling of link-layer success from network-layer forwarding creates observable behavioral inconsistencies.

3.3.1.3. Theoretical Expectations for Observable Behavior

From a cross-layer observational perspective, blackhole behavior produces measurable deviations from normal forwarding relationships.

Network Layer

- Forwarding ratio significantly below unity (packets received > packets forwarded).
- Positive ingress–egress delta at the malicious node.

- Decreased packet delivery ratio for nodes downstream of the attacker.
- End-to-end packet loss without a corresponding topology change.

MAC Layer

- Successful link-layer acknowledgments despite end-to-end delivery failure.
- A possible increase in retransmission attempts by upstream nodes. Because the attacker still acknowledges the frames it receives, any such increase is expected to be small, and it is treated as a hypothesis to be tested (Table 3.4).

Physical Layer (PHY)

- RSSI patterns remain consistent with physical placement (unlike wormhole).
- No necessary distortion in signal strength or in the hop–layer relationship.

These observable effects form the theoretical basis for cross-layer telemetry collection in this study. The objective is not to define real-time detection rules, but to document how forwarding suppression manifests in measurable system behavior under controlled experimental conditions. This characterization supports subsequent exploratory data analysis and unsupervised clustering of behavioral patterns.

3.3.2. Wormhole Attack: Conceptual Foundation

Wormhole attacks are theoretically distinct from blackhole attacks in that they do not primarily rely on packet dropping, but on topology distortion. In its canonical form, a wormhole attack involves two colluding malicious nodes that establish a low-latency out-of-band communication channel and relay packets between distant parts of a wireless mesh network, thereby creating the illusion that physically remote nodes are direct wireless neighbors [46].

Unlike attacks that modify packet contents or violate protocol syntax, wormhole attacks exploit routing trust assumptions. By capturing packets at one location and replaying them at another with minimal delay, attackers artificially shorten perceived routing paths. As a result, nodes may believe that distant nodes are only one hop away, even though they remain separated by multiple legitimate intermediate nodes.

3.3.2.1. Theoretical Characterization

As described in the literature, during a wormhole attack a malicious node captures packets from one location in the network and tunnels them to another malicious node at a distant point, which replays

them locally [45]. This tunnel may be implemented using an out-of-band channel, encapsulation, or high-speed relaying. The key requirement is that tunneled packets arrive faster or with fewer perceived hops than they would through legitimate multi-hop routing.

This process creates the illusion that the two endpoints of the tunnel are geographically close to each other. In routing protocols that rely on metrics such as hop count, signal strength (RSSI), or timing characteristics, this illusion significantly distorts the network's logical topology.

Key theoretical properties of wormhole attacks include:

- **Collusion Requirement:** The attack requires at least two cooperating malicious nodes to establish the tunnel.
- **Topology Distortion:** The perceived logical network structure is altered without necessarily modifying packet contents.
- **Stealth Operation:** Attackers may remain absent from routing tables while still influencing or controlling traffic flows [45].
- **Protocol Independence:** Wormhole attacks can be launched without compromising cryptographic keys or altering legitimate nodes [45, 46].

3.3.2.2. Adaptation to ESP-WIFI-MESH Context

In ESP-WIFI-MESH, parent selection relies on the layer and child count of candidate parents, with RSSI acting through thresholds (Section 3.2.1). A canonical wormhole would therefore attract routing decisions toward a tunnel endpoint if the tunnel made that endpoint appear closer to the root or better connected than it is. Nodes might switch parents toward a malicious endpoint because it appears to offer:

- A shallower layer (closer to the root),
- A link strong enough to pass the RSSI thresholds,
- More attractive routing characteristics, such as a lower child count.

Importantly, this distortion occurs without overt protocol violations. Control-plane exchanges remain syntactically valid, and the routing state appears internally consistent from the perspective of individual nodes. The inconsistency only becomes evident when comparing physical deployment constraints with observed routing relationships.

In this study, the wormhole is emulated rather than implemented at the protocol level. Two testbed nodes are connected by a wired UART link that serves as the out-of-band tunnel: the entry node

(Node B) copies each of its own probes into the tunnel, and the exit node (Node A), which is the shallower of the two nodes in the mesh, re-injects the copies into the mesh near the root. The root keeps both copies, so the same probe arrives twice, once through the legitimate multi-hop path and once through the tunnel, and the two arrivals differ in latency. This duplicate-arrival and latency-mismatch pattern is the signature that the emulation produces and that the telemetry is designed to capture. The roles of the two ends are assigned at run time according to node depth, so that the tunnel is a shortcut toward the root, and no node is designated as a victim. The mesh stack does not use the tunnel for routing, so it causes no parent switches or hop-count changes. The topology effects described in Section 3.3.2.3 are therefore the theoretical expectation for a canonical wormhole, and Chapter 6 reports which of them were observed (Section 6.2.4).

3.3.2.3. Wireless Topology Distortion

From a routing-theoretic perspective, a canonical wormhole that the mesh adopts produces structural inconsistencies in the mesh:

- Nodes may form parent–child relationships that bypass legitimate intermediate layers.
- Hop counts may decrease abruptly without corresponding changes in physical node placement.
- Routing tables may reflect physically implausible neighbor relationships.
- Logical proximity may no longer correspond to physical proximity.

This results in a decoupling of the physical and logical topologies. Under normal conditions, physical signal propagation mediates the routing structure. Under canonical wormhole conditions, the artificial tunnel overrides these physical constraints. Because the emulated tunnel is not adopted by the mesh, these structural effects are not expected in the collected runs; the analysis nevertheless checks for them.

3.3.2.4. Cross-Layer Effects of Wormhole Distortion

Although wormhole attacks originate at the routing level, their effects propagate across protocol layers. Theoretical expectations for a canonical wormhole include the following; of these, the emulated tunnel is designed to reproduce the duplicate-arrival and timing effects.

Network Layer

- Unexpected reductions in hop count relative to physical layout.
- Parents switch events toward physically distant nodes.
- Routing entries are inconsistent with deployment constraints.

- Apparent one-hop relationships between nodes separated by multiple legitimate hops.

Physical Layer (PHY)

- RSSI values are inconsistent with expected distance-based attenuation.
- Violations of the typical relationship between hop count and signal strength.
- Abrupt signal-strength changes without physical repositioning.

MAC Layer

- Abnormal frame arrival timing due to tunnel relay.
- Potential duplicate or out-of-order frame patterns.
- Local retransmission anomalies near tunnel endpoints.

These cross-layer inconsistencies form the theoretical basis for telemetry collection in this study. The objective is not to prescribe real-time detection mechanisms, but to document how the effects of a tunnel manifest in measurable system behavior under controlled experimental conditions. This observational characterization supports subsequent exploratory analysis and unsupervised clustering of behavioral patterns in later chapters.

3.4. Cross-Layer Telemetry Perspective

3.4.1. Why Routing Violations Affect Multiple Protocol Layers

Routing-based misbehavior in wireless mesh networks does not remain confined to the network layer where it originates. Instead, routing violations can produce correlated effects across multiple protocol layers due to the interdependent nature of wireless communication. Understanding these cross-layer effects is essential for designing telemetry that captures the behavioral signature of network anomalies.

3.4.1.1. Physical Layer Effects

Routing manipulations influence physical layer observables through several mechanisms:

- **RSSI–Hop Correlation Disruption:** Under normal operation, a node's per-link RSSI tends to be weaker for nodes that are deeper in the tree (Section 3.2.3). If a wormhole made the mesh adopt an artificial shortcut, a node could report a hop count that is inconsistent with its link RSSI.
- **Signal Consistency Patterns:** Each link along a legitimate multi-hop path has its own RSSI, which follows the physical layout. Attack-induced topology distortion would add abrupt, unexplained changes in a node's link RSSI when it re-attaches elsewhere.

- **Transmission Power Artifacts:** A malicious node could transmit at a different power level than legitimate nodes, creating inconsistencies in received signal strength relative to distance. All nodes in this study use the same default transmit power, so this mechanism is not exercised.

3.4.1.2. MAC Layer Effects

Link-layer behavior may respond to routing anomalies through:

- **Retransmission Dynamics:** 802.11 retransmissions occur when a link-layer acknowledgment is missing. Packets that are dropped above the MAC (blackhole) are still acknowledged, so a large rise in retransmissions is not necessarily expected, and retransmissions are expected to reflect channel conditions as well. In this study, retransmission evidence comes from the Retry bit of the 802.11 frames captured by the sniffer (reported as MacRetryRate). The node-side retry_count column is an application-layer counter of failed send attempts and is not an 802.11 retransmission count.
- **Acknowledgment Patterns:** MAC-layer acknowledgments may succeed even when network-layer forwarding fails, creating an observable dissociation between link-level and end-to-end delivery success.
- **Channel Access Contention:** Extra traffic created by an attack, such as the duplicated probes of a wormhole tunnel, could in principle raise channel contention. The ESP32 does not expose contention counters, so this effect is not measured directly.

3.4.1.3. Network Layer Effects

The network layer directly manifests routing anomalies through:

- **Forwarding Ratio Deviations**: The relationship between packets received and packets forwarded provides direct visibility into forwarding compliance.
- **Topology Consistency**: Parent-child relationships, hop counts, and routing table entries reflect the logical network structure, which may diverge from physical reality under attack.
- **End-to-End Delivery Metrics**: Packet delivery ratio, loss rate, and latency patterns capture the ultimate effect of routing manipulation on application-layer communication.

3.4.2. Collected Metrics and Data Sources

Received Signal Strength Indicator (RSSI)

RSSI values are collected using the ESP32 Wi-Fi driver API (esp_wifi_sta_get_rssi()). Measurements are sampled at fixed intervals (10 times per second in this study) during each experimental run and logged with timestamps. Each reading describes the link between the node and its parent; it is a per-link value, not a cumulative path value.

RSSI Variation Over Time

Temporal changes in RSSI are not computed on-device; instead, they are derived during dataset preprocessing by comparing successive samples within fixed observation windows.

Transmit Power Configuration (Metadata)

All nodes use the same default transmit power (the ESP-IDF default maximum of 20 dBm), and no run changes it. It is documented as configuration metadata to ensure consistent radio conditions across experimental trials, and it supports reproducibility and contextual interpretation of signal behavior. Physical-layer metrics provide context for interpreting routing and forwarding behavior by distinguishing between environmental wireless variability and structural network effects.

3.4.3. Temporal Aggregation of Cross-Layer Telemetry

Wireless network telemetry, particularly at the physical and MAC layers, exhibits rapid fluctuations due to environmental factors, transient interference, and normal protocol dynamics. Individual 10 Hz samples of RSSI, retransmission counts, or parent state may reflect momentary conditions rather than stable behavioral patterns. Therefore, for meaningful analysis, raw telemetry must be aggregated over appropriate time windows.

This study adopts a temporal aggregation approach in which raw 10 Hz samples are grouped into fixed-duration windows (1 second in this study). Within each window, raw counters are converted to deltas, and continuous metrics are summarized using statistical descriptors (mean, variance, minimum, maximum). This aggregation serves several purposes:

1. **Noise Reduction**: Aggregation smooths out transient fluctuations, revealing underlying behavioral trends.
2. **Pattern Capture**: Many attack-induced effects (e.g., increased parent switching, forwarding ratio degradation) manifest over periods of seconds rather than individual packet events.
3. **Statistical Robustness**: Derived features such as forwarding ratio require sufficient samples to produce stable estimates; windowing ensures this.

4. **Cross-Layer Alignment**: Windowing enables correlation of metrics from different layers within the same temporal context.

The choice of window duration (1 second) balances the need for sufficient samples with the desire to capture attack-induced behavioral changes that may persist for minutes.

3.4.4. Expected Observable Inconsistencies

Based on analyses of blackhole and wormhole behavior, specific cross-layer inconsistencies are expected to manifest for each attack type. Tables 3.4 and 3.5 record these expectations as they were formulated before data collection, except for the last two rows of Table 3.5 (see the note below that table). They are predictions to be tested, not findings; Chapter 6 reports which of them were observed in the collected runs and which were not (Section 6.2.4).

3.4.4.1. Expected Blackhole Observables

Table 3.4 Expected Observable Inconsistencies Under Blackhole Behavior

Layer	Metric	Expected Deviation
Network	Forwarding Ratio	< 1.0 at attacker node (packets received but not forwarded)
Network	Ingress/Egress Delta	> 0 at attacker node (positive difference indicating packets absorbed)
Network	Packet Delivery Ratio	Decrease for victims downstream of the attacker
MAC	Retransmission Count	Increase for victim nodes (repeated attempts with no acknowledgment)
PHY	RSSI	Stable at attacker (positioning unchanged throughout)

3.4.4.2. Expected Wormhole Observables

Table 3.5 Expected Observable Inconsistencies Under Wormhole Behavior

Layer	Metric	Expected Deviation
Network	Hop Count Consistency	Nodes report unexpectedly low hop counts given physical placement
Network	Parent-Child Validity	Relationships violate physical constraints (e.g., Layer 4 child of Layer 0)
Network	Routing Table Entries	Self-referential entries or impossible neighbor relationships
PHY	RSSI-Hop Correlation	High RSSI observed from physically distant nodes
PHY	RSSI Variance	Abnormally high RSSI values are inconsistent with the expected path loss
MAC	Frame Arrival Pattern	Duplicates or out-of-order frames due to tunneling
MAC	Retransmission Count	May increase near tunnel endpoints due to timing issues
Network	Duplicate Arrivals	The same probe reaches the root twice, through the multi-hop path and through the tunnel
Network	Latency Mismatch (TunnelLatency)	The two arrivals of one probe differ in latency

3.4.4.3. Timing and Latency Inconsistencies

Both attack types may produce timing anomalies:

- **Blackhole:** End-to-end latency for successful packets may remain normal, but packet loss creates apparent "gaps" in traffic flow.
- **Wormhole:** Packets may arrive faster than physically possible, given the claimed hop count, or exhibit abnormal inter-arrival patterns due to tunnel latency variations. In the emulated tunnel, this appears as the latency difference between the two arrivals of the same probe.

3.4.5. Expected Observable Inconsistencies

The cross-layer telemetry perspective adopted in this study serves a specific purpose: characterizing how routing misbehavior manifests across protocol layers in a real ESP32 mesh network. This observational framework is explicitly distinguished from intrusion detection—the goal is not to identify attacks in real time, but to document behavioral patterns for subsequent exploratory analysis.

3.4.5.1. Observational Objectives

- **Baseline Characterization:** Establish empirical baselines for cross-layer metrics under normal operating conditions across multiple topology configurations.
- **Deviation Documentation:** Record how metrics change when the controlled attack behaviors are introduced, capturing the observable signature of each misbehavior type across layers.
- **Pattern Identification:** Enable exploratory analysis to identify which metrics consistently differentiate normal from manipulated behavior.
- **Feature Selection Guidance:** Provide an empirical basis for selecting features most informative for behavioral characterization, supporting future research without pre-committing to specific detection approaches.

3.4.5.2. Methodological Principles

The cross-layer observation framework follows several methodological principles:

- **Non-intervention:** The measurement does not attempt to correct anomalies or to change the mesh's routing decisions; the attack behaviors are the only intended intervention.

- **Comprehensive Coverage:** Metrics are collected from the PHY, MAC, and network layers to capture correlated effects.
- **Ground Truth Alignment:** Observations are time-stamped and labeled according to controlled experimental conditions.
- **Reproducibility Focus:** Collection methods are documented to enable replication and extension.

3.4.5.3. Distinction from Detection System

This observational approach differs fundamentally from intrusion detection systems:

Table 3.6 Distinction Between Cross-Layer Observation and Intrusion Detection

Aspect	Cross-Layer Observation (This Study)	Intrusion Detection System
Purpose	Characterize behavior	Identify and alert on attacks
Timing	Offline analysis	Real-time or near-real-time
Decision Making	None (data collection only)	Automated response or alerting
Performance Metrics	Data quality, completeness	Detection accuracy, latency
Output	Labeled dataset	Alerts, classifications

By maintaining this distinction, the study focuses on its core contribution: a hardware-derived, cross-layer dataset documenting ESP-WIFI-MESH behavior under normal and manipulated conditions, suitable for subsequent exploratory analysis by the research community.

3.5. Conceptual Framework of the Study

3.5.1. Normal Behavior as Consistent Relationships

Based on the theoretical foundations established in preceding sections, normal ESP-WIFI-MESH operation is characterized by consistent, predictable relationships among observable network metrics. These relationships arise from the protocol's design principles and the physical constraints of wireless communication.

Table 3.7 Consistent Relationships in Normal ESP-WIFI-MESH Operation

Relationship	Theoretical Basis	Expected Consistency
Hop Count ↔ RSSI	Monotonic signal degradation with distance	RSSI decreases predictably as hop count increases
Reception ↔ Forwarding	Protocol forwarding requirement	Packets received ≈ packets forwarded (ratio ≈ 1.0)
Parent ↔ Child Layer	Hierarchical structure	Parent layer < child layer
Physical ↔ Logical Topology	RSSI-based parent selection	Logical topology reflects physical proximity (mediated by obstacles)
Transmission ↔ Acknowledgment	MAC-layer reliability	Successful transmissions receive timely acknowledgments

These relationships serve as the baseline against which deviations can be observed. When the network operates normally, these relationships hold consistently across time and topology configurations.

3.5.2. Misbehavior as Broken Relationships

Routing-based misbehavior fundamentally disrupts the relationships that characterize normal operation. Both blackhole and wormhole attacks break specific relationships in theoretically predictable ways.

3.5.2.1. Blackhole: Broken Reception-Forwarding Relationship

The defining characteristic of blackhole behavior is the decoupling of packet reception from packet forwarding:

- **Normal:** reception → forwarding (causal link preserved)
- **Blackhole**: reception → NO forwarding (causal link broken)

This broken relationship manifests in multiple observable indicators:

- Forwarding ratio deviates from unity
- Ingress/egress delta becomes positive
- Victims experience packet loss despite successful reception at the attacker

3.5.2.2. Wormhole: Broken Physical-Logical Topology Relationship

Wormhole behavior disrupts the expected correspondence between physical node placement and logical network topology:

- **Normal**: physical proximity → strong RSSI → parent selection
- **Wormhole**: forged advertisements → strong RSSI perception → parent selection WITHOUT physical proximity

This broken relationship manifests through:

- Violation of monotonic RSSI-hop correlation
- Physically impossible parent-child relationships
- Routing tables are inconsistent with node placement

3.5.3. Experimental Emulation of Routing Manipulations

In controlled experimental research on wireless mesh networks, it is not always feasible or necessary to implement attacks exactly as they would occur in the wild. Full protocol-level exploitation—such as raw 802.11 frame capture and replay for wormhole attacks, or falsified routing advertisements—often requires low-level driver access and firmware modifications that may be unavailable or impractical on commercial microcontroller platforms like the ESP32. Furthermore, such low-level implementations introduce significant complexity and may raise safety or ethical concerns.

Therefore, this study adopts an experimental emulation approach to routing-based misbehavior. Rather than replicating the exact low-level mechanics of blackhole and wormhole attacks, the

methodology focuses on reproducing their observable forwarding and timing effects within a controlled testbed. This approach is justified on several grounds:

1. **Platform Constraints:** The ESP32's closed-source Wi-Fi firmware and limited access to raw frame injection make true protocol-layer attacks difficult to implement reliably across firmware versions. Application-layer emulation ensures reproducibility and platform compatibility.
2. **Behavioral Focus:** The primary research objective is to generate a dataset of cross-layer telemetry under both normal and manipulated conditions. The specific mechanism by which manipulation is achieved is secondary to the resulting behavioral patterns. As long as the emulation produces the same observable inconsistencies (e.g., forwarding ratio degradation, loss of delivery, duplicate arrivals and the latency difference between the two copies of a probe), the dataset remains valid for exploratory analysis.
3. **Controlled Activation:** Emulation allows precise temporal control over manipulation windows, enabling clean ground-truth labeling that would be difficult to achieve with true protocol exploits that may have variable activation timing.
4. **Ethical Considerations:** Application-layer emulation operates entirely within the normal bounds of mesh communication, avoiding any risk of unintended network disruption or security compromise beyond the controlled testbed.

The blackhole emulation reproduces forwarding suppression: a legitimate mesh node drops the probes that it is asked to relay (Section 3.3.1.2). Traffic reaches it through its position in the topology, as a relay on the path or as the hub of a star, and not through any manipulation of mesh routing. The wormhole emulation reproduces the tunnel itself: a wired UART link carries copies of one node's probes to a second node near the root, which re-injects them into the mesh (Section 3.3.2.2), and the mesh does not use this link for routing. Neither emulation advertises a false route or alters the mesh stack or the 802.11 frames, so both keep the protocol compliant. Some setups do fix parent choices before the baseline, and these belong to the setup, not to the attack: in the star attack runs every child joins the hub (D-16, D-28), the two wormhole ends choose their parents by firmware rules and join as leaf nodes (D-30, D-36), and tree runs use stickier parent switching (D-26). These rules are the same in the baseline, attack and cooldown phases. As a result, the emulations reproduce the forwarding and timing inconsistencies described in this chapter, but not the routing-state inconsistencies of a wormhole that the mesh adopts.

3.5.4. Secondary Broken Relationships

Both attack types may induce secondary relationship disruptions:

- **MAC-Network Layer Consistency:** MAC-layer success may persist despite network-layer failure (blackhole) or abnormal timing (wormhole)
- **End-to-End Reliability Expectations:** Packet delivery falls below the baseline phase measured in each run
- **Topology Stability:** With a canonical attack, parent-switching frequency may increase as nodes react to abnormal conditions. The emulations used here are not expected to cause parent switches, and parent changes are recorded to verify this.

3.5.5. These Inconsistencies Motivate Dataset Collection

The theoretical framework developed in this chapter establishes a clear rationale for dataset collection: documenting how routing-based misbehavior disrupts the consistent relationships that characterize normal ESP-WIFI-MESH operation.

3.5.5.1. Theoretical Justification for Dataset Requirements

Table 3.8 Theoretical Basis for Dataset Design Requirements

Requirement	Theoretical Basis
Cross-layer feature collection	Attacks affect multiple protocol layers simultaneously.
Hardware-based generation	Real RF dynamics and protocol timing cannot be simulated accurately.
Multiple topology configurations	Relationship consistency varies with network structure.

Controlled attack scenarios	Enables observation of specific relationship disruptions
Ground truth labeling	Essential for identifying which relationships break under which conditions

3.5.6. Exploratory Behavioral Analysis and Pattern Discovery

Given that the precise manifestations of routing misbehavior in real ESP32 hardware are not fully characterized a priori, this study adopts an exploratory analytical approach. Rather than assuming known attack signatures, the methodology employs unsupervised techniques to examine whether normal and manipulated network states form separable patterns in the feature space.

This approach is justified on several grounds:

1. **Unknown Behavioral Signatures:** While theoretical expectations exist (e.g., forwarding ratio degradation for blackhole, RSSI-hop mismatch for wormhole), how these manifest in real hardware under controlled conditions is an empirical question. Unsupervised discovery allows patterns to emerge without imposing pre-conceived classifications.
2. **Natural Separability Hypothesis:** If the theoretical framework is correct—if forwarding suppression and topology distortion indeed produce distinct, measurable cross-layer inconsistencies—then normal and manipulated behavioral states should form separable clusters in the multidimensional feature space. Clustering serves as a validation mechanism for this theoretical prediction.
3. **Anomaly Discovery Orientation:** The DBSCAN algorithm's ability to identify noise points aligns with the possibility that some experimental windows may exhibit transitional or mixed behaviors not cleanly belonging to any single category. These points can be examined separately to understand boundary conditions between behavioral states.
4. **Avoidance of Overfitting:** Supervised classification would require partitioning the limited dataset into training and test sets, potentially overfitting to specific experimental conditions. Unsupervised clustering uses all available data to discover structure, making no assumptions about which windows correspond to which labels until after clustering is complete.
5. **Alignment with Research Objectives:** The study's primary objective is to generate a dataset and explore its structure, not to build a deployable classifier. Clustering directly

supports this objective by revealing whether the collected features capture meaningful distinctions between behavioral states.

6. **Multivariate Nature of Anomalie**s: As established in Section 3.4, routing attacks create correlated inconsistencies across multiple protocol layers. Relying on a single metric (e.g., a low forwarding ratio) could lead to false positives or miss subtle, sophisticated manipulations. Therefore, the analysis must consider the combined, multivariate state of features to capture the full behavioral signature of an attack, a principle that motivates the subsequent feature selection and clustering methodology.

Thus, the conceptual framework positions normal behavior as consistent relationships and misbehavior as broken relationships, with the expectation that these broken relationships will manifest as separable behavioral states in the feature space.

3.5.7. Expected Contribution

The dataset generated under this theoretical framework will enable:

1. **Empirical Validation:** Testing whether theoretically expected relationship disruptions actually manifest in real hardware.
2. **Pattern Discovery:** Identifying which metrics most consistently indicate specific types of misbehavior.
3. **Baseline Establishment:** Documenting normal relationship ranges across topologies for future comparison.
4. **Feature Prioritization:** Determining which cross-layer features are most informative for behavioral characterization.

3.5.8. Framework Summary

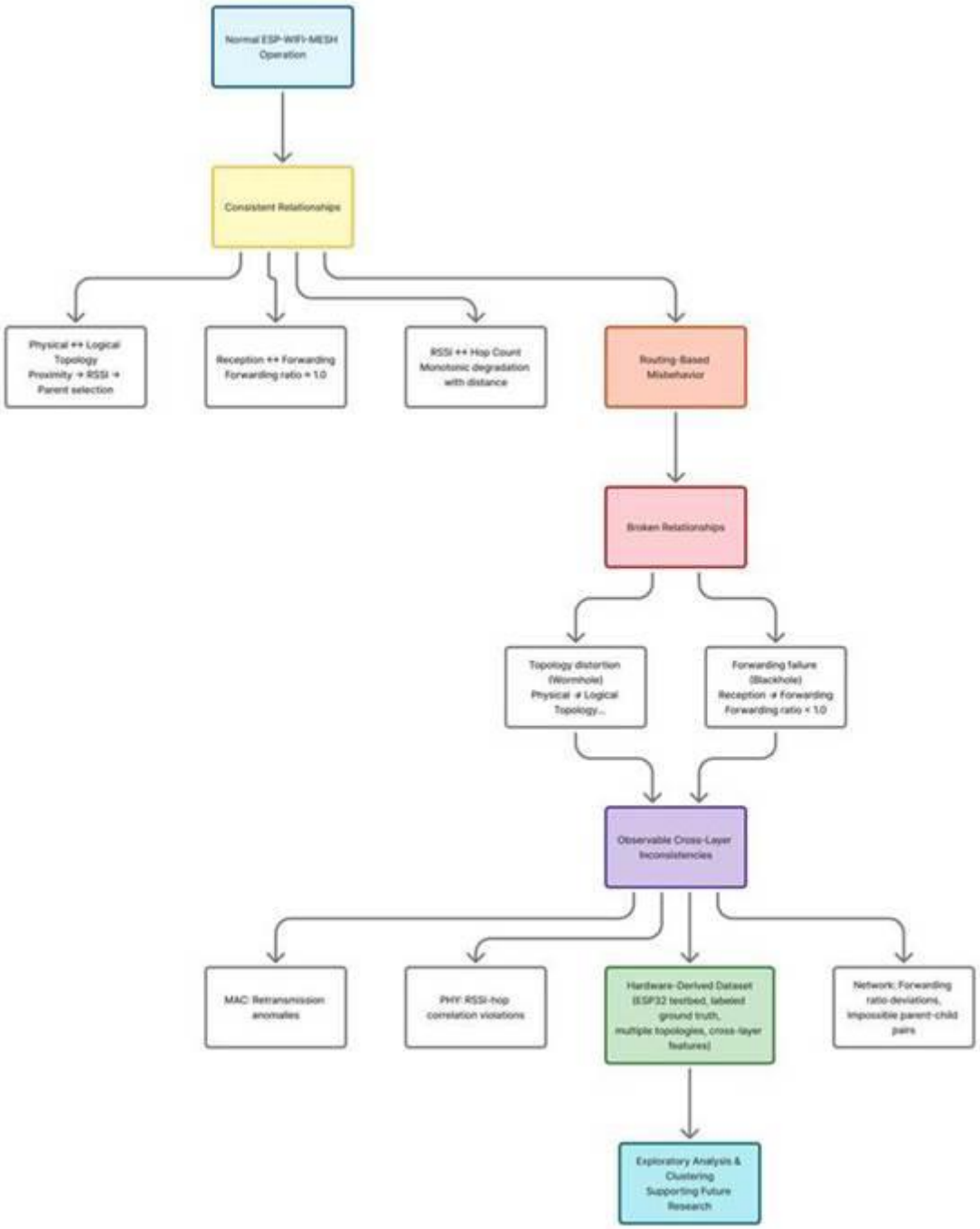


Figure 3.2 Conceptual Framework of the Study

Figure 3.2 presents the conceptual framework of this study, illustrating the progression from wireless mesh networking theory to the generation of an empirical dataset and exploratory analysis. Wireless mesh theory establishes the operational properties of ESP-WIFI-MESH and the baseline relationships expected during normal operation (e.g., the tendency of per-link RSSI to weaken with depth, hierarchical parent–child layering, and receive–forward consistency at relay nodes). Routing-based

misbehavior is then described as the disruption of these expected relationships, where forwarding suppression alters packet propagation patterns and a wormhole tunnel alters packet arrival timing (and, for a canonical wormhole that the mesh adopts, the correspondence between physical signal conditions and logical routing structure).

Building on these foundations, the study adopts a cross-layer telemetry perspective to justify why disruptions at the routing level may produce observable inconsistencies across the PHY, MAC, and network layers. Importantly, this framework is observational rather than detection-oriented: it does not define real-time decision rules or claim operational attack identification. Instead, it motivates collecting cross-layer indicators and ground-truth labels so that subsequent EDA and unsupervised clustering can examine whether normal and manipulated runs exhibit separable behavioral structure in the feature space. In this way, the framework directly informs feature selection and experimental design in Chapter 4 and provides the interpretive lens for the exploratory analyses in later chapters.

The theoretical relationships and observable deviations described in this chapter guide the experimental design and dataset generation procedures presented in Chapter 4. By grounding the dataset in established wireless mesh theory and attack theory, the study aims to collect telemetry that captures behaviorally meaningful patterns and can support future research on intrusion detection and network analysis.

4. Research Design

This study follows an experimental system-development approach focused on the construction and observation of an ESP32-based ESP-WIFI-MESH network. The methodology describes how the testbed is built, how controlled network behaviors are introduced, and how cross-layer data are collected to support anomaly analysis. The testbed and both controlled behaviors are implemented, and the data-capture campaign was completed. This chapter therefore describes the workflow as it was implemented, and the items that still depend on the final pooled analysis are marked. The study aims to observe how mesh networking behavior evolves under both baseline and adversarial conditions.

The methodology is organized into three working stages:

1. Mesh network configuration using ESP32 devices.
2. Implementation of controlled network behaviors that resemble known routing anomalies.
3. Capture of cross-layer network metrics, followed by offline preprocessing, feature extraction and analysis.

The stages overlap. While the remaining runs are captured, the offline analysis tools are applied to the runs that already exist, and the capture procedure is adjusted whenever a verification step shows a problem. The capture and analysis tools are kept in the project repository [79].

4.1. Architectural Design

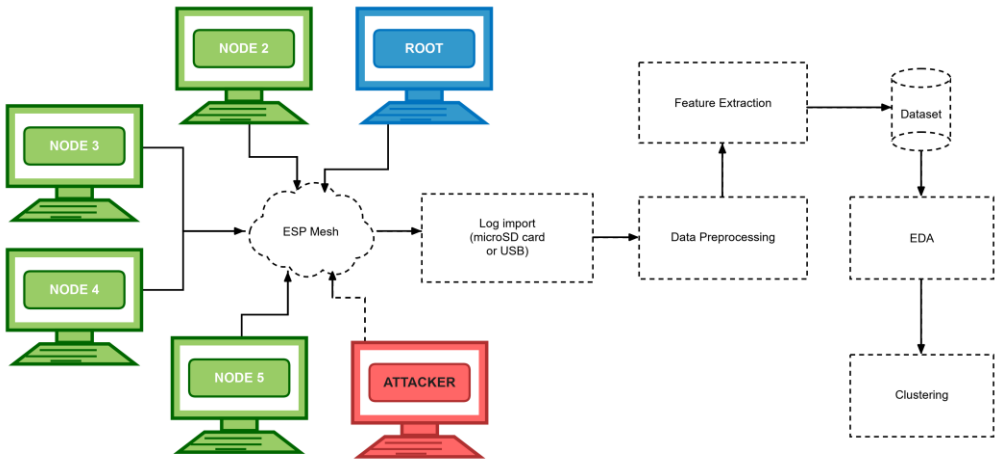


Figure 4.1 Architecture Design

The architectural design in *Figure 4.1* illustrates the complete experimental workflow of the study, showing how data flows from the physical ESP32 mesh network through a structured processing pipeline to produce analytical results. The ESP-WIFI-MESH testbed consists of multiple ESP32 nodes, including a root node, several regular nodes, and attacker nodes, which together form a wireless mesh network that generates telemetry data during experimental runs.

Each node samples its cross-layer measurements at 10 Hz and writes them to its own log, on internal flash and on a microSD card. After a run, the logs are brought to a laptop, either by removing the cards and reading them together in a card reader, or by pulling each log over USB serial. The Data Collector step of *Figure 4.1* is therefore a file import and not a live collection over the mesh. The imported files are placed in one folder tree by attack, topology, site and scenario, where the Data Preprocessing stage reads them. Preprocessing aligns the timestamps and aggregates the measurements into one-second windows.
A laptop that is not part of the mesh runs an 802.11 sniffer during the captures that use one. It only listens, and it gives an independent record of the frames on the channel, so the evidence for an attack does not rest only on the attacker counting its own drops.

Next, Feature Extraction transforms the preprocessed windows into meaningful behavioral metrics such as forwarding ratios, MAC retry rates observed by the sniffer, hop stability indicators, and RSSI–hop inconsistency values. These computed metrics form the structured Dataset, which contains engineered features together with the ground-truth phase labels corresponding to baseline operation, blackhole manipulation, and wormhole manipulation.

Finally, Exploratory Data Analysis (EDA) and Clustering are applied to this dataset. EDA is used to examine the statistical properties of the data and visualize behavioral patterns, while clustering is performed to determine whether normal operation, blackhole behavior, and wormhole-induced anomalies form separable groups within the feature space.

4.2. Experimental Topology and Controlled Behavior Framework

This section describes the experimental implementation of two routing manipulations—forwarding suppression (blackhole emulation) and topology distortion (wormhole-inspired emulation)—designed to generate labeled telemetry data in the ESP-WIFI-MESH testbed. Both attacks are implemented at the application layer without modifying the underlying ESP-WIFI-MESH firmware. Manipulations are activated only during controlled experiment windows, with the root node

broadcasting phase transitions to all mesh participants. All telemetry is logged locally on each node and retrieved post-experiment for offline analysis.

4.2.1. Attack Implementation: Wormhole and Blackhole Manipulations

This section describes the experimental implementation of two routing manipulations—forwarding suppression (blackhole emulation) and topology distortion (wormhole-inspired emulation)—designed to generate labeled telemetry data in the ESP-WIFI-MESH testbed. Both attacks are implemented at the application layer without modifying the underlying ESP-WIFI-MESH firmware. Manipulations are activated only during controlled experiment windows, with the root node broadcasting phase transitions to all mesh participants. All telemetry is logged locally on each node and retrieved post-experiment for offline analysis.

4.2.1.1. Common Experiment Control Infrastructure

All experiments share a common control infrastructure managed by the root node. This infrastructure ensures consistent phase timing, synchronized manipulation activation, and reliable ground-truth labeling across all network participants.

A. Phase Definition and Labeling

Each experimental run follows a fixed temporal structure with distinct phases. Two parallel numbering schemes are maintained:

- Phase ID: Used for experiment orchestration and broadcast messaging (0 to 4). A further announcement, 254, is sent while the mesh forms and is not a phase.
- Ground-Truth Label: Embedded in telemetry records for offline analysis (0, 1 or 2). A row that was logged before the first phase was heard carries 255, meaning no ground truth.

Table 4.1 Phase Definition and Labeling

Phase ID	Ground-Truth Label	Duration	Phase Name	Description
254	255	60s or Longer	Formation	The mesh forms. The root announces PREPARE every 5 s, and the nodes log their rows without a label. These rows are excluded from the analysis.

0	0	300 s	Baseline	Normal mesh operation; no manipulation active
1	1	180 s	Blackhole Manipulation	Forwarding suppression enabled
2	2	180 s	Wormhole Manipulation	Tunnel-based shortcut enabled
3	0	120 s	Cooldown	Manipulation disabled; network stabilization
4	—	—	Terminate	Experiment end; logs ready for extraction

Note: The root broadcasts the Phase ID, together with a sequence number, a timestamp and a session identifier. Each node maintains a local mapping from Phase ID to Ground-Truth Label, which is appended to every telemetry record for offline analysis. The formation period lasts at least 60 s and continues until every expected child is present in the root's routing table and until the pre-run gates pass and the operator types START (D-22). A failed pre-run check also locks START in the operator's tool; an override needs a typed reason, is written to the run log and is reported by the verifier (D-24). In a benign stationary or highload run the attack window is skipped. A benign burst run keeps a window of attack length, so that its burst fires at the same offset as in the matched attack run.

B. Root Phase Controller Implementation

As shown in *Figure 4.2*, the root node executes a controller that manages the sequence of experiment phases, including formation, baseline, manipulation (blackhole or wormhole), cooldown, and termination. Each phase update is sent to every node in the root's routing table as an individual peer-to-peer message, with one additional copy sent to the root itself. A transition is sent in five rounds, 100 ms apart, that carry the same sequence number, so a node that receives more than one copy applies the transition once. The root also repeats the last phase unchanged every 10 s, so that a node

that rejoins the mesh during a phase adopts it. A session identifier drawn when the root boots lets a node tell a restarted root from a repeated message.

Figure 4.2 Root Phase Controller

```
procedure EXPERIMENT_CONTROLLER(experiment_type):
                    /**
            * Controls the timeline of a single experimental run.
            *
            * Input:
            *   experiment_type: "none" (benign), "blackhole" or "wormhole"
            *
            * Behavior:
            *   - Announces PREPARE while the mesh forms and waits for every child
            *   - Broadcasts each phase transition (repeatedly) to all mesh nodes
            *   - Logs phase start/end events with timestamps
            *   - Activates the manipulation through the phase broadcast
            */

            // Formation: 60 s, rows are logged unlabelled (255)
            for second = 0 to 59:
                        if second mod 5 == 0:
        broadcast_prepare()    // phase 254
      sleep_until_next_second()
   wait_until_all_children_in_routing_table()

            // Phase 0: Baseline (5 minutes)
            current_phase ← 0
   broadcast_phase_repeated(current_phase, 5)   // 5 rounds
   log_event("PHASE_START", current_phase, get_time())
            sleep(300)
   log_event("PHASE_END", current_phase, get_time())

            // Phase 1 or 2: Manipulation (3 minutes)
            if experiment_type == "blackhole":
                        current_phase ← 1
            else if experiment_type == "wormhole":
                        current_phase ← 2
            if experiment_type != "none":
      broadcast_phase_repeated(current_phase, 5)
      log_event("PHASE_START", current_phase, get_time())
                        sleep(180)
      log_event("PHASE_END", current_phase, get_time())

            // Phase 3: Cooldown (2 minutes)
            current_phase ← 3
   broadcast_phase_repeated(current_phase, 5)
   log_event("PHASE_START", current_phase, get_time())
            sleep(120)
   log_event("PHASE_END", current_phase, get_time())

            // Phase 4: Terminate, repeated for 60 s
            current_phase ← 4
   broadcast_phase_repeated(current_phase, 5)
            for t = 5 to 60 step 5:
                        sleep(5)
      broadcast_phase_repeated(current_phase, 5)
   log_event("EXPERIMENT_END", current_phase, get_time())
end procedure

procedure broadcast_phase_repeated(phase, rounds):
            /**
            * Sends the current phase to every node, several times.
```

```
        * Message format: { magic, phase_id, seq_num, timestamp, session_id }.
        * All rounds of one call share the same seq_num.
        */
        message ← pack(phase, next_seq(), get_time(), session_id)
        for i = 1 to rounds:
    esp_mesh_send(NULL, message, MESH_DATA_FROMDS)    // copy for the root itself
                for each node in esp_mesh_get_routing_table():
      esp_mesh_send(node, message, MESH_DATA_P2P)          // one unicast per node
                end for
                sleep(100)   // short delay between rounds
        end for
end procedure
```

C. Node-Side Phase Monitoring

Every mesh node runs a background monitoring task that continuously listens for phase messages from the root node, as implemented in *Figure 4.3*. When a message with a newer sequence number is received, the node updates its local phase state and corresponding ground-truth label. Copies with an older or equal sequence number are ignored. A message from a different session identifier is treated as a new run: the node forgets the old sequence numbers and its phase until the new root announces one. This mechanism ensures that all nodes remain synchronized with the experiment timeline while maintaining consistent labeling of the collected dataset.

.

Figure 4.3: Node Phase Monitor

```
// Global state maintained by each node
int     current_phase_id   = 255          // 255 = no phase heard yet
int     ground_truth_label = 255
uint32_t last_seq          = 0
uint32_t session_id        = 0
mutex    phase_mutex

procedure PHASE_MONITOR_TASK():
        while experiment_active:
                message ← esp_mesh_recv(portMAX_DELAY)

                if is_phase_broadcast(message):
      (received_phase, seq, session) ← unpack(message)

                if session != session_id:          // the root restarted: a new run
                session_id ← session
                last_seq ← 0
        current_phase_id ← 255
        ground_truth_label ← 255
                end if

                if seq > last_seq:
                last_seq ← seq

                if received_phase == 254:     // PREPARE: start logging, keep rows unlabelled
          open_log()
                else:
          lock(phase_mutex)
          current_phase_id ← received_phase

                // Map phase ID to ground-truth label
                if received_phase in {0, 3, 4}:
```

```
                ground_truth_label ← 0  // cooldown and terminate are labeled 0
                    else:
                ground_truth_label ← received_phase   // 1 = blackhole, 2 = wormhole
            unlock(phase_mutex)

            log_event("PHASE_UPDATE", received_phase, ground_truth_label, get_time())
                    end if
                    end if
                    end if
        end while
end procedure

function get_current_label():
        lock(phase_mutex)
        label ← ground_truth_label
        unlock(phase_mutex)
        return label
end function
```

4.2.1.2. Forwarding Suppression (Blackhole) Implementation

The blackhole attacker operates as an application-layer relay. Every node sends its own application-layer probe packets to its parent, using the peer-to-peer data flag of the ESP-IDF mesh API [75], [76], and relays the probes it receives one hop upstream, keeping counters for packets received, forwarded and dropped. The blackhole attacker runs the same relay as every other node, with one switch that turns forwarding off during the manipulation window. This follows Section 3.1.3.2 and *Table 4.2*. It also means that the forwarding features exist on every relaying node and not only on the attacker. The attacker remains a legitimate mesh participant, maintaining its parent relationship and responding to control messages. Because it is placed in the forwarding path, the position of the attacker decides which traffic it intercepts. Captures made before hop-by-hop relaying was introduced are not comparable with later ones and are excluded from pooled analysis.

In the star topology the attacker is the hub: the root sits one hop behind it, and every victim connects to the hub (Section 4.2.2.1).

The attack keeps the name Forwarding Suppression. The attacker is placed in the forwarding path and does not lure traffic with a falsely advertised route. The route-advertisement half of a classical blackhole, in which the attacker advertises a false low rank so that nodes choose it as parent [70], is therefore not reproduced.

The blackhole is an application-layer emulation. ESP-WIFI-MESH forms a tree by itself: each node selects its parent from the candidates it can hear, preferring the one on the shallowest layer and then the one with the fewest children, and each node keeps its own routing table [21]. The documentation describes a way to read the routing table but none to write it [21], [75]. An application can switch off

self-organization and choose its own parent or its own selection rule [21], but that decides only the node's own connection. A blackhole that draws traffic by advertising a false route would have to be built below the application layer, for example by forging the mesh's own beacons, which this study does not attempt. The attacker instead sits in the forwarding path, where the topology puts it, and drops what it receives. The dataset therefore shows what such a node does to delivery, and not how traffic comes to be sent through it.

A. Design Scope and Implementation Boundaries

The implementation described in this section is conducted exclusively within a controlled laboratory testbed composed of researcher-owned ESP32 development boards. The purpose of detailing the manipulation model is to ensure reproducibility of dataset generation for academic evaluation, not to enable deployment against external or production networks. All experiments are performed in an isolated laboratory network with no connection to external or production infrastructure and under controlled conditions with explicit activation windows and ground-truth labeling.

Canonical blackhole attacks described in wireless networking literature are typically implemented at the routing protocol level, often involving the advertisement of favorable routing metrics to attract traffic before dropping packets. Such implementations generally require manipulation of routing control messages and protocol state. However, these capabilities are not reliably exposed within the ESP-WIFI-MESH framework on ESP32 devices, where parent selection and forwarding decisions are handled internally by the protocol implementation.

ESP32 microcontrollers operate using a closed-source Wi-Fi firmware stack beneath the ESP-IDF application layer. Core networking operations—including parent selection, forwarding decisions, and mesh control-plane management—are handled internally by the ESP-WIFI-MESH protocol implementation. As a result, direct manipulation of routing advertisements or forwarding tables at the firmware level is not accessible through standard ESP-IDF APIs.

Accordingly, this study does not attempt to implement a protocol-level routing advertisement attack. Instead, the experiment adopts a blackhole-inspired forwarding suppression emulation model, designed to reproduce the observable behavioral effects of a blackhole attack rather than replicating its exact protocol-level mechanics.

B. Design Principles

The objective of this research is not to exploit or compromise the ESP-WIFI-MESH protocol, but rather to generate a reproducible hardware-derived dataset capturing cross-layer behavioral anomalies associated with forwarding suppression. To achieve this, the manipulation model is designed according to the following principles:

- Operate entirely within supported ESP-IDF and ESP-WIFI-MESH APIs
- Maintain full protocol compliance and mesh membership at all times
- Introduce controlled forwarding suppression that produces packet loss effects
- Induce measurable cross-layer inconsistencies consistent with blackhole theory
- Enable precise activation windows and reliable ground-truth labeling for dataset generation

This design ensures that the resulting dataset remains:

- Reproducible across multiple experimental runs
- Stable across ESP-IDF software versions
- Ethically contained within a controlled laboratory environment
- Suitable for exploratory analysis and offline intrusion-detection research

By focusing on observable cross-layer deviations—such as forwarding ratio degradation and end-to-end packet loss—the study preserves the theoretical characteristics of blackhole-induced forwarding suppression while maintaining practical feasibility within the constraints of the ESP32 platform.

The resulting manipulation is therefore best described as behavioral blackhole emulation, rather than low-level protocol manipulation. This distinction is important for interpreting the collected dataset and for ensuring that future researchers clearly understand the experimental conditions under which the forwarding suppression traces were generated.

C. Attacker Role and Traffic Model

To reproduce the forwarding suppression effects associated with blackhole attacks, the experiment deploys a single attacker node that serves as an intentional relay for the probe traffic of the nodes below it. The traffic model follows a simple design:

Attacker Node. This node is placed so that other nodes select it as their parent, or reach the root through it. During baseline operation, the attacker forwards the probes it receives to its own parent, which leads toward the root. During the suppression window, it drops them instead.

Victim Nodes. These are the nodes whose path to the root passes through the attacker. Like every other node, they send their application-layer probe packets to their parents. The tree, and not a configured address, sends their traffic through the attacker, so the number of victims in a run depends on where the attacker is placed. A node that does not pass through the attacker is a bystander, and its delivery is expected not to change.

Root Node. The root receives the probe packets that reach it and records their arrival times as part of the measurement framework. During suppression phases, the root observes packet loss from every victim, because the attacker drops their relay traffic.

The attacker joins the mesh network as a legitimate participant and maintains normal protocol behavior throughout the experiment. No mesh control-plane messages are altered, and all communication with the network continues to use standard ESP-WIFI-MESH APIs.

Through this intentional relay mechanism, the attacker introduces controlled packet loss that distorts the expected forwarding behavior—thereby reproducing the cross-layer behavioral signatures associated with blackhole-induced forwarding suppression.

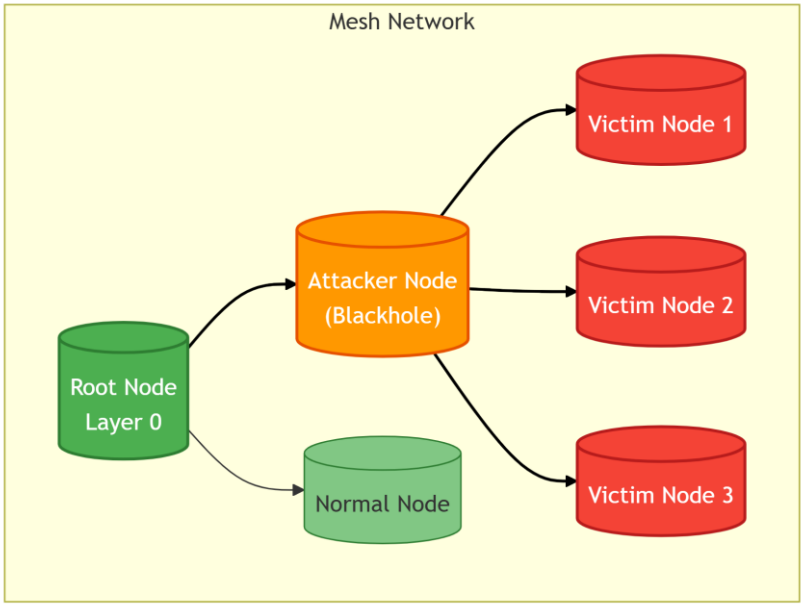


Figure 4.4 Blackhole Topology with Attacker as Intentional Relay

Figure 4.4 shows the blackhole topology with the attacker as an intentional relay. Victims (red) send their probe packets to the attacker (orange), which forwards them to the root (green) during baseline. During suppression, the attacker drops packets instead of forwarding them.

D. Attacker State Management

The blackhole attacker maintains counters and a state flag, as shown in *Table 4.2*. Each counter is written by the relay task and read by the logging task, and it counts a packet exactly once, so that the number received equals the number forwarded plus the number dropped.

Table 4.2 Blackhole Attacker State Variables and Counters

Variable	Type	Initial Value	Description	Update Trigger
suppression_active	bool	false	Manipulation state flag	Set true when phase ID = 1 received; cleared when phase ID = 3 received
recv_counter	uint32	0	Total application packets received	Incremented on every successful *esp_mesh_recv()*
forward_counter	uint32	0	Total packets forwarded to root	Incremented after successful *esp_mesh_send()* to root
drop_counter	uint32	0	Total packets intentionally dropped	Incremented when a packet is dropped during suppression
self_packet_counter	uint32	0	Failed esp_mesh_send() calls	ncremented on a failed end; logged as etry_count

E. Baseline Stabilization

Before Phase 0 (Baseline) begins, the mesh is allowed to converge to a stable topology for a stabilization interval of at least 60 seconds that lasts until the pre-run gates pass and the operator types START (D-22). The topology check accepts a board that is attached when the baseline begins and does not re-route during it (D-27). During this interval, the following baseline properties are recorded:

- Each node's mesh layer (hop distance), parent identity, and parent switching events (if any)
- Per-node RSSI statistics (mean/variance) under stable topology
- Baseline end-to-end latency and packet delivery ratio (PDR) from victim nodes to the root
- Verification that the attacker has at least one child, so that probes pass through it. In the tree topology, which is not forced, the root waits for this before it starts the baseline.

This baseline serves as the reference distribution against which suppression effects are evaluated.

F. Packet Processing Logic

Every node runs the same relay. The attacker's relay receives probes from the nodes below it and, based on the current phase, either forwards them to its parent or drops them during the attack phase. This behavior is illustrated in *Figure 4.5*.

Figure 4.5 Blackhole Attacker Relay Loop

```
/**
* Probe relay task. Every node runs it; the blackhole attacker installs
* one extra decision hook.
*
* Behavior:
*   - Outside phase 1: forward every received probe to the parent.
*   - During phase 1 (suppression_active): drop every received probe.
*/
atomic_uint32_t recv_counter = 0
atomic_uint32_t forward_counter = 0
atomic_uint32_t drop_counter = 0
atomic_uint32_t send_fail_counter = 0
queue_t relay_queue              // 128 entries

function forward_decision(probe):        // installed on the attacker only
            return get_current_phase() != 1             // FALSE means drop
end function

procedure ON_PROBE_RECEIVED(packet):  // called by the mesh receive task
            if packet.magic is PROBE or PROBE_WORMHOLE:
      atomic_increment(recv_counter)    // counted before the decision
                        if not queue_push(relay_queue, packet):
         atomic_increment(drop_counter)   // overflow counts as a drop
                        end if
            end if
end procedure
```

```
procedure ATTACKER_RELAY_TASK():
        while experiment_active:
                // Blocks until a probe is queued
                probe ← queue_pop(relay_queue)

                if forward_decision(probe) == FALSE:
                // Suppression active: drop the probe
      atomic_increment(drop_counter)
                // No forwarding call: the probe is silently discarded
                else:
                // Normal relay: forward to the parent
                result ← esp_mesh_send(parent_address,
                probe,
                   MESH_DATA_P2P)
                if result == ESP_OK:
        atomic_increment(forward_counter)
                else:
        atomic_increment(drop_counter)
        atomic_increment(send_fail_counter)
        log_error("FORWARD_FAIL", result)
                end if
                end if
        end while
end procedure
```

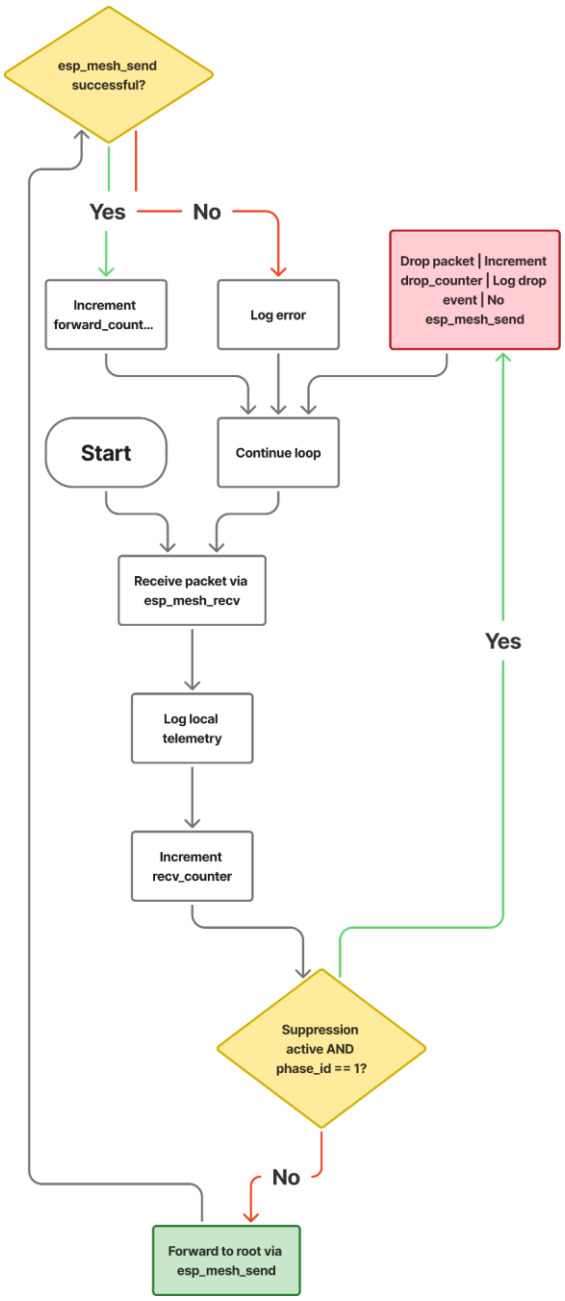


Figure 4.6 Blackhole Attacker Decision Flowchart

Figure 4.6 shows the blackhole attacker packet processing flowchart. The attacker receives a probe, logs telemetry, and increments the receive counter. If suppression is active during phase 1, the probe is dropped and the drop counter is incremented. Otherwise, the probe is forwarded to the parent, and success or failure is tracked.

G. Explanation of Key Behaviors

The blackhole-inspired implementation creates a forwarding suppression effect through several coordinated behaviors:

1. **Intentional Relay:** Every node sends its probes to its own parent, and every node relays what it receives. The attacker therefore legitimately receives the probes of the nodes below

it through esp_mesh_recv(), and no promiscuous capture or transit traffic monitoring is required.

2. **Normal Forwarding:** During baseline (suppression_active = false), the attacker forwards each received probe to its parent using esp_mesh_send(). The path is Victim → Attacker → Root, with any further nodes in between, and the root receives the probe as usual. This establishes the baseline forwarding behavior and latency.
3. **Suppression Mechanism:** During the manipulation window (suppression_active = true and phase ID = 1), the attacker simply omits the *esp_mesh_send()* call for received packets. The packet buffer is freed, and no forwarding occurs. This creates controlled packet loss while the attacker continues normal mesh participation.
4. **Protocol Compliance:** The attacker never modifies packet contents, never injects malformed frames, and never disrupts control-plane exchanges (beacons, parent maintenance, etc.). Only application-layer relay behavior is altered. The attacker remains a legitimate mesh node in all other respects.
5. **Counter Updates:** Atomic counters track the exact number of received, forwarded, and dropped packets. The recv_counter increments for every packet, while forward_counter and drop_counter track the outcomes of forwarding decisions. These provide raw data for subsequent feature extraction.
6. **Normal Traffic Preservation:** The attacker continues to participate in all control-plane exchanges and maintains parent relationships throughout all phases. This ensures that the mesh protocol's behavior is minimally perturbed, allowing clean observation of the suppression effect.

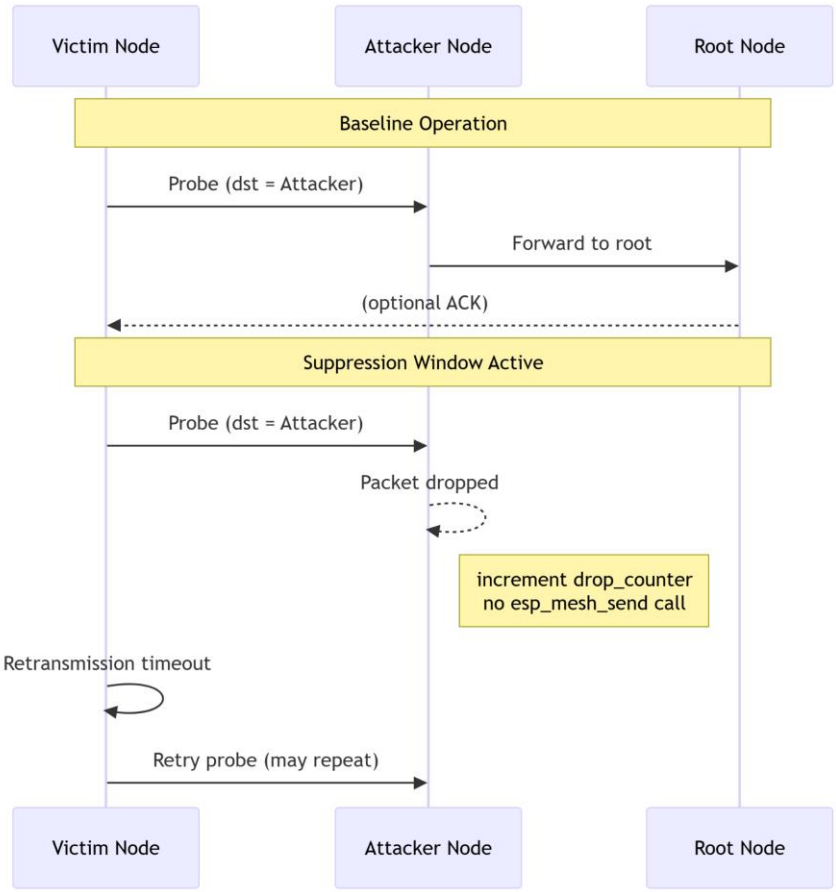


Figure 4.7 Baseline vs Suppression Sequence Diagram

Figure 4.7 is the comparison of baseline and suppressed operation. During baseline (top), probes flow Victim → Attacker → Root. During suppression (bottom), the attacker drops the probes, so they never reach the root. The attacker still acknowledges every frame at the link layer, so the victim's own radio does not see a failure.

H. Observable Effects and Dataset Contributions

The forwarding suppression mechanism produces several measurable deviations that are captured in the unified dataset:

- **Forwarding Ratio Degradation**: The attacker's forwarding ratio (packets forwarded / packets received) drops significantly during the suppression window, approaching zero if all transit packets are dropped. This is the primary indicator of blackhole behavior.
- **Retransmissions (tested, not assumed):** Chapter 3 predicts that victim nodes increase their retransmission attempts when probes are dropped. The attacker keeps acknowledging frames at the link layer, so the victims' own send counters need not

change. Whether MAC-layer retries rise is checked in the sniffer record and reported in Section 4.2.7.4.

- **Reduced Packet Delivery Ratio**: End-to-end packet delivery from victims to the root decreases during suppression, as probes are dropped at the attacker.
- **Stable RSSI at Attacker**: Unlike wormhole distortion, the attacker's RSSI remains consistent with its physical placement, providing a baseline for comparison.
- **Parent Switching (Secondary Effect)**: Not expected. The attacker keeps acknowledging every frame, so a victim sees no failure and has no reason to leave its parent; parent changes are still recorded to confirm this (Section 6.2.4).

I. Implementation Summary for Blackhole

The blackhole-inspired implementation is designed around several key principles:

- **Intentional Relay:** The attacker is a placed relay. The nodes below it deliver their probes to it through the normal parent chain, which removes any need for passive packet capture or transit traffic visibility.
- **Suppression by Omission:** Forwarding suppression is achieved by omitting the *esp_mesh_send()* call, not by modifying control traffic or injecting malformed packets.
- **Protocol Compliance:** The attacker remains a legitimate mesh participant throughout all phases, maintaining parent relationships and control-plane exchanges.
- **Counter-Based Telemetry:** Dedicated atomic counters track received, forwarded, and dropped packets, providing the raw data for subsequent feature extraction.
- **Phase-Controlled Activation:** The suppression_active flag is tied directly to experiment phase broadcasts, ensuring precise temporal alignment with ground-truth labels.
- **Normal Traffic Preservation:** The attacker continues forwarding all original packets during baseline, maintaining realistic mesh behavior during non-suppression windows.

4.2.1.3. Topology Distortion (Wormhole-Inspired) Implementation

The wormhole-inspired manipulation uses two colluding attacker endpoints connected by an out-of-band tunnel. No raw 802.11 frames are captured or injected; only the application-layer probe is relayed through the tunnel, as its own fields: sequence number, source address and send timestamp. This approach maintains full protocol compliance while generating measurable cross-layer inconsistencies. Both attackers remain legitimate mesh participants throughout the experiment.

A. Design Scope and Implementation Boundaries

The implementation described in this section is conducted exclusively within a controlled laboratory testbed composed of researcher-owned ESP32 development boards. The purpose of detailing the manipulation model is to ensure reproducibility of dataset generation for academic evaluation, not to enable deployment against external or production networks. All experiments are performed in an isolated laboratory network with no connection to external or production infrastructure and under controlled conditions with explicit activation windows and ground-truth labeling

Canonical wormhole attacks described in wireless networking literature are typically implemented at the IEEE 802.11 control-plane level, often involving the capture, replay, or manipulation of management frames and topology-related information elements. Such implementations generally require low-level driver access and raw frame injection capabilities. However, these capabilities are not reliably exposed within the ESP-WIFI-MESH framework on ESP32 devices.

ESP32 microcontrollers operate using a closed-source Wi-Fi firmware stack beneath the ESP-IDF application layer. Core networking operations—including parent selection, beacon generation, association handling, and mesh control-plane management—are handled internally by the ESP-WIFI-MESH protocol implementation. As a result, direct manipulation of IEEE 802.11 management frames, vendor-specific information elements, or mesh discovery advertisements at the firmware level is not accessible through standard ESP-IDF APIs.

Accordingly, this study does not attempt to implement a firmware-level management-frame replay wormhole. Instead, the experiment adopts a wormhole-inspired topology distortion emulation model, designed to reproduce the observable behavioral effects of a wormhole rather than replicating its exact physical-layer mechanics.

B. Design Principles

The objective of this research is not to exploit or compromise the ESP-WIFI-MESH protocol, but rather to generate a reproducible hardware-derived dataset capturing cross-layer behavioral anomalies associated with routing-based topology distortion. To achieve this, the manipulation model is designed according to the following principles:

- Operate entirely within supported ESP-IDF and ESP-WIFI-MESH APIs
- Maintain full protocol compliance and mesh membership at all times
- Introduce a controlled auxiliary tunnel that produces artificial shortcut effects
- Induce measurable cross-layer inconsistencies consistent with wormhole theory

- Enable precise activation windows and reliable ground-truth labeling for dataset generation

This design ensures that the resulting dataset remains:

- Reproducible across multiple experimental runs
- Stable across ESP-IDF software versions
- Ethically contained within a controlled laboratory environment
- Suitable for exploratory analysis and offline intrusion-detection research

By focusing on observable cross-layer deviations—such as duplicate arrivals of the same probe at the root and the latency difference between the two copies (the tunnel leaves the routing tree unchanged)—the study preserves the theoretical characteristics of wormhole-induced topology distortion while maintaining practical feasibility within the constraints of the ESP32 platform.

The resulting manipulation is therefore best described as behavioral wormhole emulation, rather than low-level protocol manipulation. This distinction is important for interpreting the collected dataset and for ensuring that future researchers clearly understand the experimental conditions under which the topology distortion traces were generated.

C. Attacker Roles and Tunnel Architecture

To reproduce the shortcut effects associated with wormhole attacks, the experiment deploys two cooperating attacker nodes connected by an auxiliary out-of-band tunnel. Each attacker performs a distinct role within the topology distortion model.

Two attacker endpoints are deployed with the following responsibilities:

- **Attacker A (Root-Side Endpoint).**

  This node is positioned near the root region of the mesh network. It receives the probes that Attacker B writes to the tunnel and re-injects a copy of each toward the root using standard ESP-WIFI-MESH messaging, so each of Node B's probes reaches the root a second time, by a shorter path.

- **Attacker B (Leaf-Side Endpoint).**

  This node sits deeper in the mesh, farther from the root. It generates its own probes and sends each one through the mesh toward the root in every phase. During distortion phases it also writes a copy of each of its own probes to the out-of-band tunnel to Attacker A. Only Node

B's own probes are tunnelled; the probes of other nodes are relayed normally, so a wormhole run has no victims.

Both attackers join the mesh network as legitimate participants and maintain normal protocol behavior throughout the experiment. No mesh control-plane messages are altered, and all communication with the network continues to use standard ESP-WIFI-MESH APIs.

Through this coordinated relay mechanism, the tunnel gives each of Node B's probes a second, shorter path, so the delay of one copy no longer matches Node B's hop count—thereby reproducing the cross-layer behavioral signatures associated with wormhole-induced topology distortion.

The two attacker boards are joined by a physical UART wire, which plays the role of the out-of-band tunnel that Zhukabayeva et al. [71] built with a GSM/GPRS link. The two ends exchange a HELLO handshake over the wire every second to decide which is Node A and which is Node B. The root also holds the baseline until both ends report UART OK and Node A sits shallower than Node B (D-22); for runs captured before this gate existed, the same conditions were checked afterwards from the data. An optional numbered-ping test can also check the wire. Every tunnel message carries a CRC32, and the firmware prints a warning if the tunnel carried nothing.

The title Wormhole-Inspired is kept. The tunnel runs above the routing layer of the mesh, so it reproduces duplicate arrivals of the same packet. Ramírez Gómez et al. [72] propose duplicated routing packets as a signature of a wormhole in ZigBee, which is related to this effect, but our tunnel does not change the topology. In the linear capture at the DLSU Library, all 181 probes that Node B sent in the attack phase reached the root twice, about 2.8 ms apart, with no duplicates in baseline or cooldown and no parent switches.

The wormhole is also an application-layer emulation (D-20). ESP-WIFI-MESH does not take the wire into its routing, so no node treats the two ends as neighbours and the tree does not change. What the wire produces is the packet-level signature of a wormhole: a copy of a packet arrives at the root by a second path, with a different delay. The false-neighbour effect that defines a wormhole in the literature [46] is therefore not reproduced (Table 4.5).

The two tunnel ends are assigned when the run starts and not when the boards are flashed (D-17). Both boards exchange a checked HELLO message over the tunnel wire, and the board deeper in the mesh becomes Node B (entry) while the shallower one becomes Node A (exit), so the tunnel is always a shortcut toward the root. The roles lock at the first phase, never change during the run, and are written to the role column of every telemetry row. Only Node B’s own probes are tunnelled, so a

wormhole run has no victims, and every other node is labelled not tunnelled. This replaces the blackhole rule that all nodes downstream of the attacker are victims. In the first run with this assignment, Node A sat at layer 5 and Node B at layer 7, so the wire skipped two hops. In tree and partial-mesh wormhole runs, the board built as Node B never takes the root as its parent and must sit deeper than Node A, and Node A takes the shallowest open parent it hears, so the tunnel always skips at least one hop. The two ends change parent only when the link is lost or the parent is full, while every other board self-heals as before (D-30). Earlier captures set the depth of the ends by placement only. In tree and partial-mesh runs both ends also join as leaf nodes, so no other board can use a tunnel end as its parent (D-36). The rule held on hardware: in all 36 wormhole runs of the final dataset, Node A sat shallower than Node B, and the tunnel skipped two to five hops in the linear topology, one hop in the star and one or two hops in the tree and the partial mesh (Section 6.2.2).

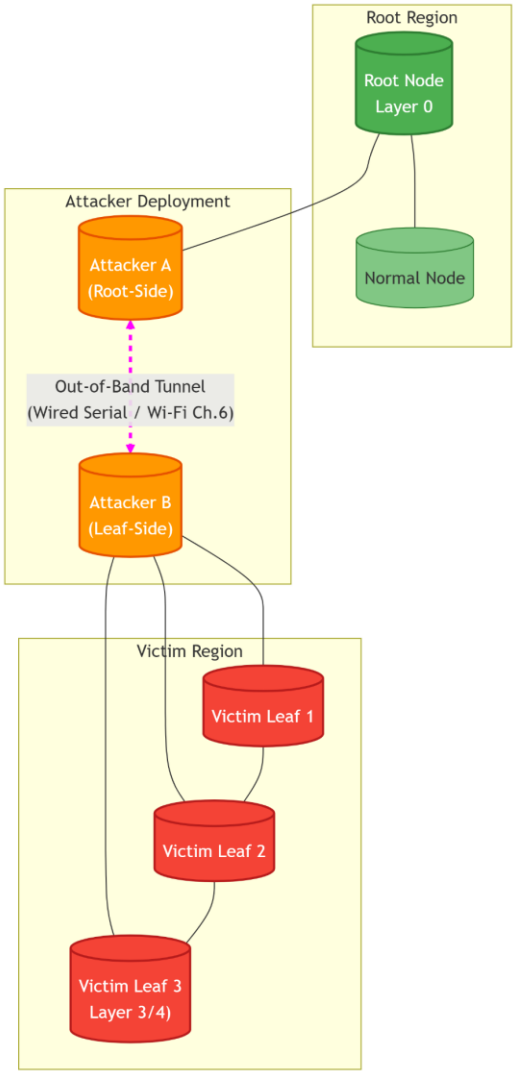


Figure 4.8 Wormhole Endpoint Setup with Auxiliary Tunnel

Figure 4.8 is the wormhole endpoint setup with an auxiliary tunnel. Attacker A (orange) is positioned shallower in the mesh, nearer the root (green), and Attacker B (orange) is positioned deeper, toward the leaf nodes. The out-of-band tunnel (magenta dashed line) connects the two attackers independently of the mesh network, enabling the shortcut effect without interfering with normal mesh routing. .

D. Tunnel Implementation

The out-of-band tunnel is a wired serial link, independent of the mesh network, shown in *Figure 4.9*. The two boards are connected by UART1 at 115200 baud, with the transmit pin (GPIO 17) of each board wired to the receive pin (GPIO 16) of the other. The same link carries two frame types of identical size: the tunnel frame and the HELLO frame used to assign the ends. A wire was chosen

because it cannot interfere with the radio channel that the mesh and the sniffer share. Two other options, a host relay through a laptop and a second Wi-Fi interface on another channel, were not used.

Figure 4.9 Wormhole Tunnel Packet Structure

```
struct tunnel_packet {          // 30 bytes, packed
        uint32_t magic;                         // 0x544E4C31 ("TNL1")
        struct probe {                  // the captured probe, copied verbatim (22 bytes)
                uint32_t magic;                 // 0x50524F42 ("PROB")
                uint32_t seq_num;               // probe sequence number
                int64_t  send_ts_us;            // send timestamp of the original probe
                uint8_t  src_mac[6];            // originator, preserved across every hop
        } probe;
        uint32_t crc;                   // CRC32 over magic and probe
};
```

The HELLO frame has the same length. It carries the sender's MAC address, mesh layer, current end, a flag that tells whether the end is locked, a flag that tells whether the sender hears its peer, and a CRC32. A receiver reads the wire in fixed-size windows and, when a window fails the magic or CRC check, slides forward by one byte until it locks on a valid frame again.

The tunnel is configured to exhibit:

- Stable connectivity throughout baseline and manipulation windows
- Lower and more consistent latency than the equivalent multi-hop mesh route under baseline conditions
- No dependence on ESP-MESH routing decisions

E. Attacker State Management

Both attackers maintain counters to track tunnel activity and reinjection operations. Table 4.3 lists all state variables. All counters are implemented as atomic variables to ensure thread safety, as they are accessed by both the packet processing task and the logging task.

Table 4.3 Wormhole Attacker State Variables and Counters

Variable	Type	Initial Value	Description	Update Trigger
distortion_active	bool	false	Manipulation flag	Set true when phase ID = 2 received; cleared when phase ID = 3 received
role, role_locked	enum, bool	A, false	The end this board plays, and whether it is fixed	Set from the HELLO exchange; locked at the first phase

probes_generated	uint32	0	Probes created by Node B	Incremented on each probe generated
probes_to_root	uint32	0	Probes Node B sent into the mesh toward the root	Incremented after a successful send
probes_tunneled	uint32	0	Probes written to the tunnel (Node B)	Incremented on each complete tunnel frame written during phase 2
tunnel_received	uint32	0	Valid tunnel frames received (Node A)	Incremented on each frame that passes the magic and CRC check
probes_reinjected	uint32	0	Probes re-injected toward the root (Node A)	Incremented after a successful send of a re-injected probe
tunnel_fail, reinject_fail	uint32	0	Failed sends (Node B) and failed re-injections (Node A)	Incremented on each failed send

F. Baseline Stabilization

Before Phase 0 (Baseline) begins, the mesh is allowed to converge to a stable topology for a stabilization interval of at least 60 seconds that lasts until the pre-run gates pass and the operator types START (D-22). The topology check accepts a board that is attached when the baseline begins and does not re-route during it (D-27). During this interval, the following baseline properties are recorded:

- Each node's mesh layer (hop distance), parent identity, and parent switching events (if any)
- Per-node RSSI statistics (mean/variance) under stable topology
- Baseline end-to-end latency and packet delivery ratio (PDR) from the nodes below the attacker region to the root

This baseline serves as the reference distribution against which distortion effects are evaluated.

G. Attacker B (Leaf-side) Implementation

Attacker B, as shown in *Figure 4.10*, generates its own probes. In every phase it sends each probe to its parent, so that the root receives the normal copy. During the wormhole phase it also writes the same probe to the tunnel, after adding a CRC32.

Figure 4.10 Attacker B – Relay and Tunnel

```
/**
* Attacker B: entry endpoint of the wormhole manipulation (the deeper board).
*
* Responsibilities:
*   - Joins the mesh normally and relays the probes of nodes below it (end A is the hub in the star), except in tree and partial-mesh runs, where it joins as a leaf (D-36)
*   - Generates its own probes once its role is locked
*   - Every phase: sends each probe to its parent (the normal mesh copy)
*   - Wormhole phase: also writes the probe to the tunnel with a CRC32
*/
queue_t probe_queue
atomic_uint32_t probes_generated = 0
atomic_uint32_t probes_to_root = 0
atomic_uint32_t probes_tunneled = 0
atomic_uint32_t tunnel_fail = 0

procedure PROBE_GENERATOR_TASK():
        while experiment_active:
                if role == B and role_locked:
                probe ← make_probe(next_seq(), get_time(), own_mac)
        queue_push(probe_queue, probe)
        atomic_increment(probes_generated)
                end if
    sleep(PROBE_INTERVAL)                    // 1000 ms
        end while
end procedure

procedure TUNNEL_FORWARDER_TASK():
        while experiment_active:
                probe ← queue_pop(probe_queue)        // blocks until a probe is queued

                // Step 1: always send the probe up the mesh (normal copy)
                result ← esp_mesh_send(parent_address, probe, MESH_DATA_P2P)
                if result == ESP_OK:
        atomic_increment(probes_to_root)
                else:
        atomic_increment(tunnel_fail)
                end if

                // Step 2: during the wormhole phase, also write the probe to the tunnel
                if get_current_phase() == 2:
                frame.magic ← TUNNEL_MAGIC
                frame.probe ← probe
                frame.crc   ← crc32(frame.magic, frame.probe)
                written ← uart_write(frame, sizeof(frame))
                if written == sizeof(frame):
        atomic_increment(probes_tunneled)
                else:
        atomic_increment(tunnel_fail)
                end if
                end if
        end while
end procedure
```

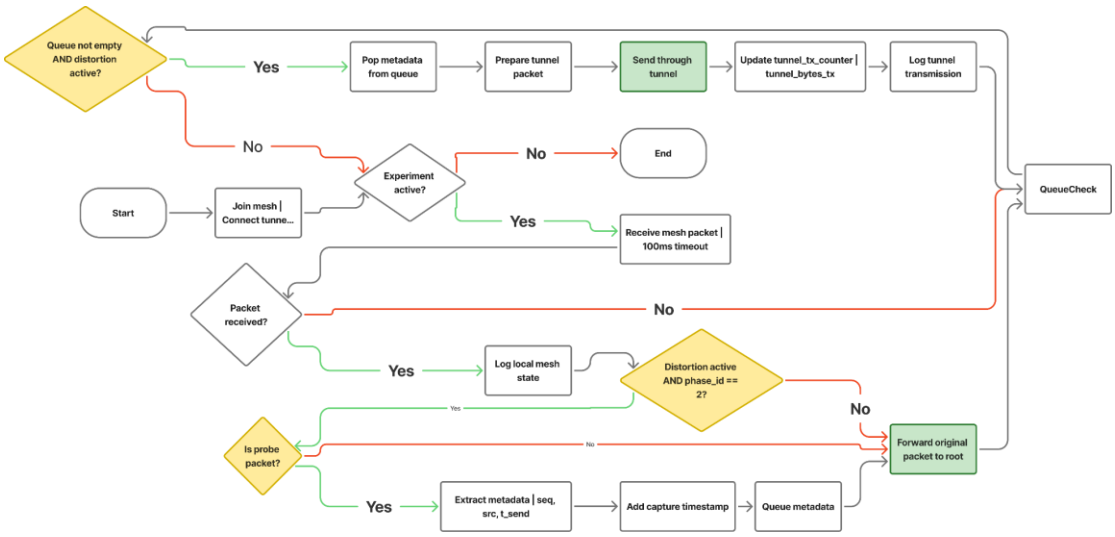


Figure 4.11 Attacker B (Leaf-side) Process Flowchart

Figure 4.11 shows the Attacker B (entry endpoint) process flowchart. The attacker generates a probe and sends it to its parent. During the wormhole phase the probe is also wrapped in a tunnel frame, protected by a CRC32, and written to the tunnel, with the counters updated accordingly.

H. Attacker A (Root-side) Implementation

Attacker A reads the tunnel, checks each frame, and re-injects the original probe toward the root, as shown in *Figure 4.12*. The re-injected copy keeps the sequence number, source address and send timestamp of Node B's original probe, and carries a different magic value so that the root records it as a second arrival and does not discard it as a duplicate. Every node between Attacker A and the root relays this magic as well

Figure 4.12 Attacker A – Tunnel Receive and Inject

```
/**
* Attacker A: exit endpoint of the wormhole manipulation (the shallower board).
*
* Responsibilities:
*   - Joins the mesh normally and relays the probes of nodes below it (end A is the hub in the star), except in tree and partial-mesh runs, where it joins as a leaf (D-36)
*   - Reads the tunnel and discards frames that fail the magic or CRC check
*   - Re-injects each valid probe toward the root through its parent
*/
atomic_uint32_t tunnel_received = 0
atomic_uint32_t probes_reinjected = 0
atomic_uint32_t reinject_fail = 0
queue_t reinject_queue

procedure TUNNEL_RECEIVE_TASK():
        while experiment_active:
                frame ← uart_read_window(sizeof(tunnel_packet))
                if frame.magic == HELLO_MAGIC and crc_ok(frame):
    update_peer_state(frame)          // used to assign the ends
                else if frame.magic == TUNNEL_MAGIC and crc_ok(frame) and role == A:
    atomic_increment(tunnel_received)
    queue_push(reinject_queue, frame.probe)
    log_event("TUNNEL_RX", frame.probe.seq_num, get_time())
```

```
                else:
        slide_window_by_one_byte()          // resynchronise on the next valid frame
                end if
          end while
end procedure

procedure REINJECT_TASK():
          while experiment_active:
                probe ← queue_pop(reinject_queue)   // blocks until a frame arrives

                // Tag the copy so that the root does not treat it as a duplicate to discard
                probe.magic ← PROBE_WORMHOLE_MAGIC                // "PROW"
                // seq_num, send_ts_us and src_mac stay those of Node B's original probe

                result ← esp_mesh_send(parent_address, probe, MESH_DATA_P2P)
                if result == ESP_OK:
        atomic_increment(probes_reinjected)
        log_event("INJECT_SUCCESS", probe.seq_num, get_time())
                else:
        atomic_increment(reinject_fail)
        log_event("INJECT_FAIL", probe.seq_num, result)
                end if
          end while
end procedure
```

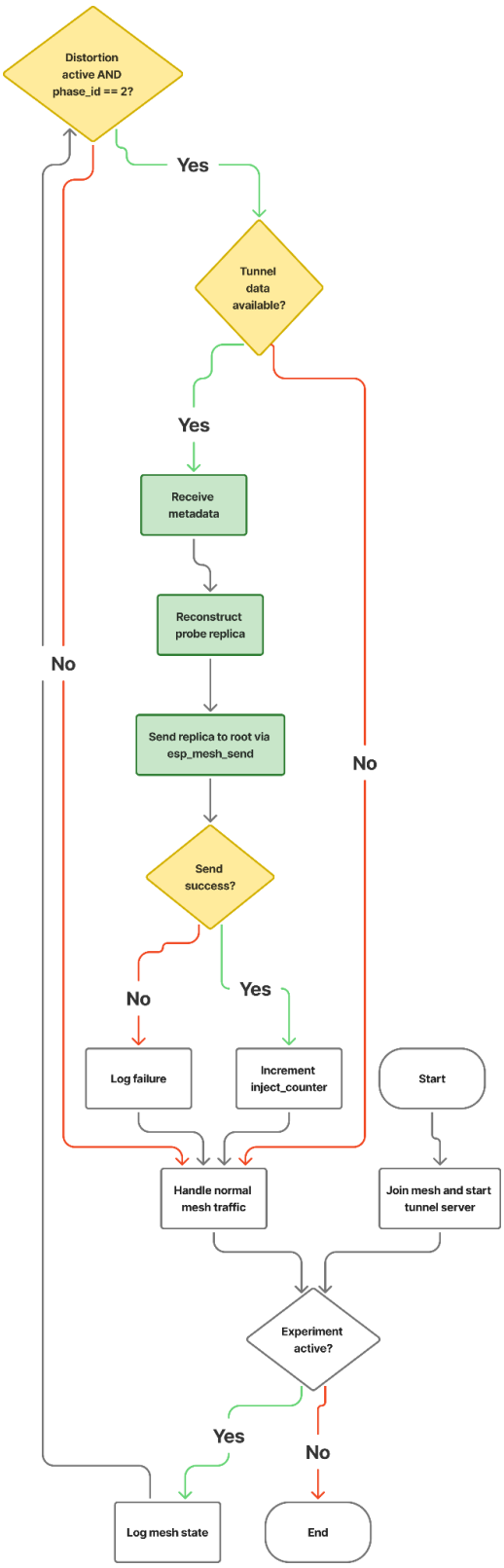


Figure 4.13 Attacker A (Root-side) Process Flowchart

In *Figure 4.13,* the Attacker A (exit endpoint) process flowchart, the attacker reads the tunnel, validates each frame, and re-injects a tagged copy of the probe toward the root through the mesh. Normal mesh traffic is relayed concurrently.

I. Explanation of Key Behaviors

The wormhole-inspired implementation creates an artificial shortcut effect through several coordinated behaviors:

1. **Own-Probe Source:** Attacker B generates its own probes. Its probes reach the root through the mesh in every phase, so no raw frame capture or promiscuous mode is required.
2. **Normal Relay:** During baseline, B sends each probe to its parent, which relays it hop by hop to the root. The root receives the probe as usual, and this establishes the baseline latency for the path.
3. **Distortion Relay:** During the manipulation window, B also writes the probe to the out-of-band tunnel for A. The tunnel frame carries only the probe fields needed to reproduce timing and identity (sequence number, source, send timestamp), which keeps the tunnel overhead small.
4. **Probe Replication:** A checks each frame and re-sends the original probe, with the same sequence number, source and send timestamp, toward the root.
5. **Mesh Re-injection:** A re-injects the probe through its own parent. The root therefore receives two copies of the probe: one through the normal multi-hop path from B, and one through the tunnel and A. The two copies arrive at different times, which creates the duplicate-arrival and delay anomalies that the features measure, without modifying the mesh control plane.
6. **Normal Traffic Preservation:** Where other nodes hang below them (linear runs, and Node A as the star hub), both attackers relay the probes of those nodes normally; in tree and partial-mesh runs both ends join as leaf nodes and relay nothing (D-36). This ensures that the mesh protocol's behavior is minimally perturbed.
7. **Tunnel Independence:** The tunnel is a separate physical medium and does not interfere with mesh communication. Its timing differs from that of the mesh path, and that difference is what creates the observable shortcut effect.

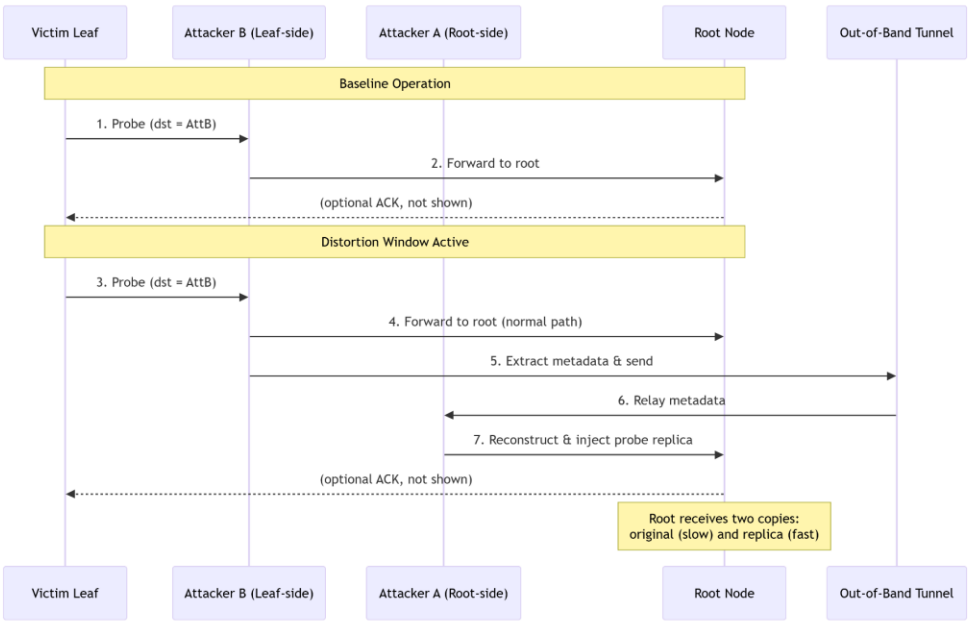


Figure 4.14 Wormhole Shortcut Relay Mechanism

Figure 4.14 is the wormhole shortcut relay mechanism. During baseline (top), Attacker B sends its probes to the root through the mesh. During distortion (bottom), Attacker B also sends each probe through the out-of-band tunnel to Attacker A, which re-injects it toward the root. The root thus receives two copies of the same probe at different times.

J. Observable Effects and Dataset Contributions

The topology distortion mechanism produces several measurable deviations that are captured in the unified dataset:

- **Delay Inconsistency:** The two copies of a probe reach the root by different paths and with different delays, so the delay of a probe is no longer consistent with the hop count of its sender.
- **Duplicate Probe Reception:** The root receives two copies of the same probe (identical sequence numbers) within a short time window, which does not occur during normal operation.
- **Traffic Concentration Shifts:** The second path may alter traffic distribution patterns, potentially concentrating forwarding activity on paths adjacent to the attackers. This is monitored as a secondary effect.
- **RSSI-Hop Inconsistency:** Not expected, because the tunnel adds no radio link and changes no parent, so the RSSI of each parent link stays as it was; it is monitored as a check.

- **Topology Stability (checked, not expected to change):** The tunnel does not enter the mesh's routing, so a change of topology is not expected. Parent switching is monitored to confirm this, together with natural RSSI variation and traffic load changes.

All these effects are captured through the cross-layer metrics defined in Section 4.2.3 and contribute to the feature set used for exploratory analysis and clustering.

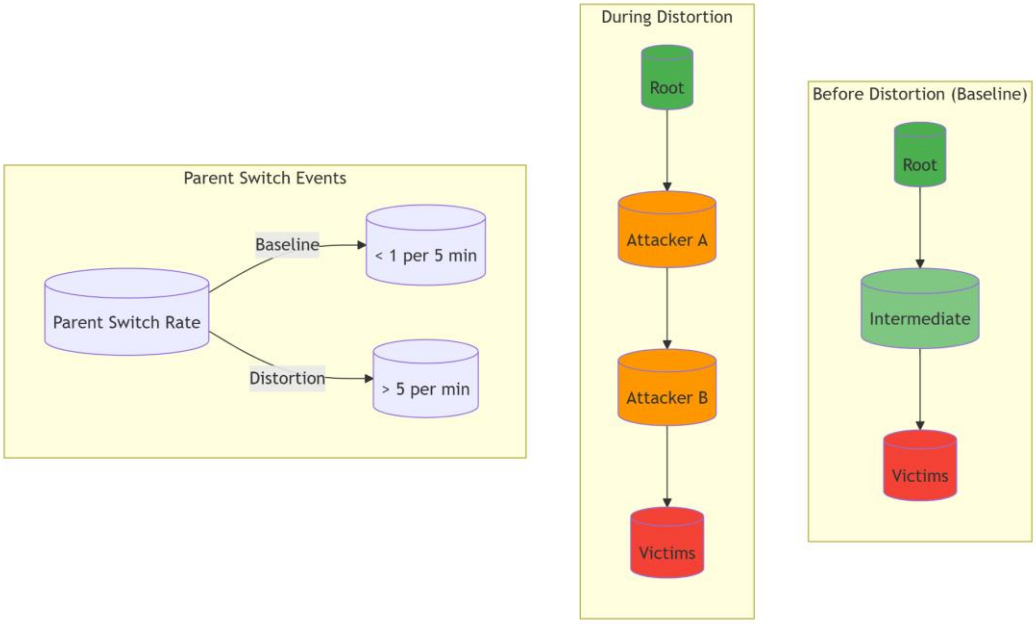


Figure 4.15 Traffic Redistribution During Distortion Window

Figure 4.15 illustrates the prediction from Chapter 3 that is tested against the captured data. Left: baseline traffic flows through legitimate intermediate nodes. Center: during distortion, traffic concentrates on attacker-adjacent paths (orange). Right: the prediction that parent switching becomes more frequent during the distortion window than in baseline. The captured runs show no parent switches during the wormhole phase (Section 4.2.7.4).

K. Implementation Summary for Wormhole

The wormhole-inspired implementation is designed around several key principles:

- **Own-Probe Tunneling:** Attacker B generates its own probes and tunnels copies of them, which removes any need for passive packet capture.
- **Probe-Only Tunneling:** Only the application-layer probe (sequence number, source, send timestamp) is tunneled, not raw frames, ensuring application-layer compliance and minimal overhead.
- **Probe Duplication, Not Modification**: Attackers never modify original packets; they only create additional replicas, preserving all original traffic for baseline comparison.

- **Independent Tunnel**: The out-of-band channel operates completely separately from the mesh network, preventing interference with normal protocol operation.
- **Normal Traffic Preservation**: Both attackers continue forwarding original packets, maintaining realistic mesh behavior during the distortion window.
- **Counter-Based Telemetry**: Dedicated atomic counters track tunnel activity and reinjection success, providing the raw data for subsequent feature extraction.
- **Phase-Controlled Activation:** The tunnel is written only while phase ID 2 is current, ensuring precise temporal alignment with ground-truth labels.

4.2.1.4. Implementation Summary

The implementation described in this section provides a complete, reproducible framework for generating labeled datasets containing both normal and manipulated mesh behavior. Key characteristics include:

1. **Application-layer only:** All manipulations are implemented in the testbed's own application firmware using standard mesh APIs; the ESP-WIFI-MESH library is not modified and no raw frames are injected.
2. **Phase-controlled activation:** Manipulations are activated only during designated time windows via root broadcasts, ensuring precise temporal alignment with ground-truth labels.
3. **Protocol compliance:** Attackers remain legitimate mesh participants throughout all phases, maintaining parent relationships and control-plane exchanges. Only the specific relay behavior (blackhole) or tunnel relay (wormhole) is modified.
4. **Comprehensive state tracking:** Each attacker maintains dedicated atomic counters that capture the intensity and outcome of manipulations. These counters, together with event-driven logs (parent changes, phase transitions, send/recv outcomes), provide rich raw data for subsequent feature extraction.
5. **Reproducible procedures:** The documented algorithms and state machines can be implemented identically across multiple experimental runs and topology configurations.

Telemetry logging, data storage, and post-experiment extraction are described separately in Section 4.2.3. This separation keeps the implementation logic focused on manipulation behavior, while data collection is handled by a unified framework common to all node types.

The implementations are checked against the published definitions criterion by criterion, including the criteria that fail. Table 4.4 covers the blackhole against Airehrour et al. [70], and Table 4.5 covers the wormhole against Hu, Perrig, and Johnson [46].

Table 4.4 Blackhole Conformance to Airehrour et al. [70]

Criterion	Met	Evidence
Adversary sits on the forwarding path	Yes	Victims address the attacker, which relays to the root
Receives the packets rather than jamming	Yes	The relay queue receives before the forward or drop decision
Drops instead of forwarding	Yes	During the blackhole phase the drop counter rises and nothing is sent on
Stays protocol-compliant, so the node looks alive	Yes	It remains a mesh participant and keeps logging telemetry
Delivery collapses while the node stays reachable	Yes	In the G402 linear stationary run, attacker ForwardingRatio falls from 1.000 to 0.001 and the PDR of the six nodes behind it from 1.000 to 0.000
Attracts traffic by advertising a false route	No	The attacker is a placed relay, so no routing decision is fooled

Table 4.5 Wormhole Conformance to Hu, Perrig, and Johnson [46]

Criterion	Met	Evidence
Two colluding endpoints	Yes	Node A (exit) and Node B (entry)
Out-of-band channel outside the normal path	Yes	Physical UART link between the two boards
Packets captured at one end and replayed at the other	Yes	181 of 181 probes from Node B arrived twice in the attack phase, and none did in baseline or cooldown (linear, DLSU Library, stationary)
No compromise of keys or hosts	Yes	Application layer only, no key material touched
Creates a false neighbour relationship	No	No parent switches or layer changes during the wormhole phase

The wormhole evidence was checked in all 36 wormhole runs, and the verifier rated 21 of them strong and 15 good. The tunnel skipped 1 to 5 hops: 2 to 5 on the linear topology (median 4), 1 to 2 on the partial mesh and on the tree, and 1 on the star (Section 6.2.2). Of the 12,465 probes of Node B in the attack phases, 12,448 (99.9%) reached the root twice, once through the mesh and once through the tunnel, and none of the 67,462 probes of the other boards did. No probe of Node B arrived twice in a baseline phase. No window of any wormhole run shows a change of parent or of hop count, which is the evidence for the row “Creates a false neighbour relationship” of the table above. In every run Node B is deeper than Node A, so the tunnel gave a shortcut in every run. An earlier tree capture at G402 had both ends at the same depth, so its tunnel gave no shortcut (D-30), and in none of the 36 wormhole runs of the dataset are the two ends at the same depth. The captures that were replaced by a newer capture are listed in Chapter 6 (Section 6.1).

4.2.2. Physical Deployment Topologies

In ESP-WIFI-MESH, the network topology is not manually configured as in traditional networks, such as bus, ring, or star. Instead, nodes automatically organize themselves into a hierarchical tree structure via the protocol's parent-selection mechanism, in which each node selects an upstream parent from the candidates it can hear, preferring the one on the shallowest layer and then the one with the fewest children [21]. This self-organizing behavior allows the mesh network to construct routing paths dynamically, without predefined topology assignments.

However, in controlled experimental environments such as this study, different logical layouts can be created by physically positioning ESP32 nodes in specific spatial arrangements. While these layouts resemble traditional network topologies (e.g., star or linear chain), the underlying routing behavior still follows the ESP-WIFI-MESH hierarchical tree architecture. The resulting logical topology therefore emerges from the interaction between node placement, RSSI relationships, and the mesh protocol's parent-selection algorithm.

The physical placement of nodes plays a critical role in shaping the resulting logical network structure. Because a node can only choose among the parents it can hear, node spacing and relative positioning must be carefully arranged to encourage the desired routing relationships. Proper spacing helps ensure that nodes preferentially connect to intended neighbors rather than forming unintended long-range links. This stabilization of RSSI relationships is essential for maintaining consistent topology behavior across experimental runs and for ensuring that attack effects can be clearly observed in the collected telemetry data.

Placement is supported by the topology options of the ESP-IDF mesh API [75]. The star caps the depth of the mesh at two layers, or three when the hub is the blackhole attacker or wormhole end A (D-16, D-28). The linear chain selects the chain topology and lets each node accept one child. The partial mesh narrows the number of children each node accepts, so that the nodes do not all attach to the root. The tree uses the default self-organization, with stickier parent switching (D-26). Every board in a run is flashed with the same topology option, and the structure that actually forms is checked after every run from the parent addresses and layers in the logs (*Table 4.15*). Throughout this chapter the position of a node is given in hops, where the hop count is the mesh layer minus one, so that the root is at hop 0. ESP-WIFI-MESH numbers the root as layer 1 (the constant MESH_ROOT_LAYER in esp_mesh.h).

Node placement is particularly important when deploying routing-based attacks such as wormhole and blackhole attacks. The positioning requirements differ fundamentally between the two attack types:

For blackhole attacks, the attacker must be positioned so that the traffic of other nodes reaches the root through it. ESP-WIFI-MESH does not let a node advertise a favorable route, so the position is arranged through the layout and the topology option, and not through signal strength alone. Once other nodes are attached below the attacker, it receives their traffic but intentionally drops it during the manipulation window. A run in which nothing attaches below the attacker looks benign, so the tree runs are not started until the attacker has at least one child.

For wormhole attacks, two attacker nodes are deployed at different depths of the network, one shallower and one deeper. Under normal operation, a probe from the deeper end would have to traverse several intermediate nodes through the mesh hierarchy. The wormhole attack introduces an out-of-band tunnel between the two attackers, allowing the probes of the deeper end to be copied directly across distant network segments. This creates a second path for the same probe, with a different delay, and distorts the expected relationship between hop count and latency.

To investigate how these attack behaviors manifest under different routing conditions, the experiments employ several physical deployment layouts, each designed to emphasize different aspects of mesh routing behavior. These layouts are physical node arrangements combined with the topology options above, and the mesh still builds its routing tree itself. The exceptions are fixed in firmware before the baseline: in the star attack runs every child joins the hub (D-16, D-28), and the two wormhole ends choose their parents by rule (D-30, D-36). Figures 4.16 to 4.23 show example layouts; the position of the attacker varied from run to run and is recorded for every run. The following deployment layouts are implemented in the experimental environment:

- Star topology
- Tree topology
- Linear chain topology
- Partial mesh topology

4.2.2.1. Star Topology

In the **star topology**, a central ESP32 node acts as the communication hub, while surrounding nodes transmit traffic directly to it. This setup provides a baseline environment where routing complexity is minimized, and direct RSSI relationships can be observed. In the benign runs the hub is the root. In the blackhole runs the hub is the attacker (D-16), and in the wormhole runs it is wormhole end A (D-28); in both attack cases the root sits one hop behind the hub.

.

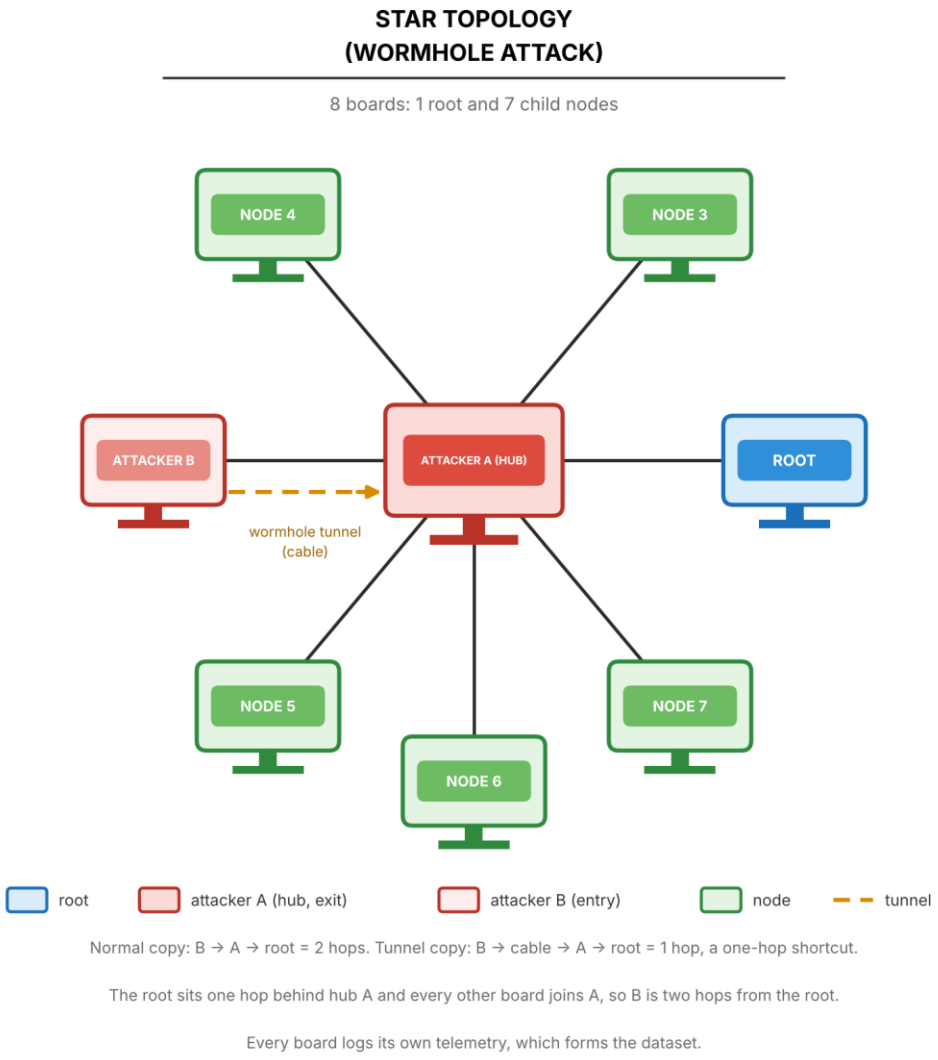


Figure 4.16 Wormhole Attack on Star Topology

For wormhole experiments in *Figure 4.16*, two attacker endpoints are deployed. In a flat star every node is one hop from the root, so the two ends would sit at the same depth and the tunnel could not skip a hop. The star is therefore built one hop down, as for the blackhole (D-28): Node A is the hub of the star, the root sits one hop behind it as in the blackhole star (D-16), and every other board, Node B included, joins the hub. Node B's normal copy then takes two hops to the root and its tunnelled copy one, a one-hop shortcut, and the root's setup gate can pass. When the distortion window is activated, Node B writes its probes to the tunnel as well as sending them through the mesh, and Node A re-injects them toward the root, so the root receives each probe twice. Node A also relays the normal copy of Node B, and the control boards are two hops from the root. Figure 4.16 still shows the earlier flat star and is to be redrawn. The first hub-star captures are not yet usable: in the G402 stationary capture three boards never attached to the root, and in the highload attempts the hub board

browned out and reset. The hub-star design still needs the adviser's approval. All nodes log continuous telemetry, with the tunnel endpoints recording additional counters for the probes tunnelled and re-injected.

.

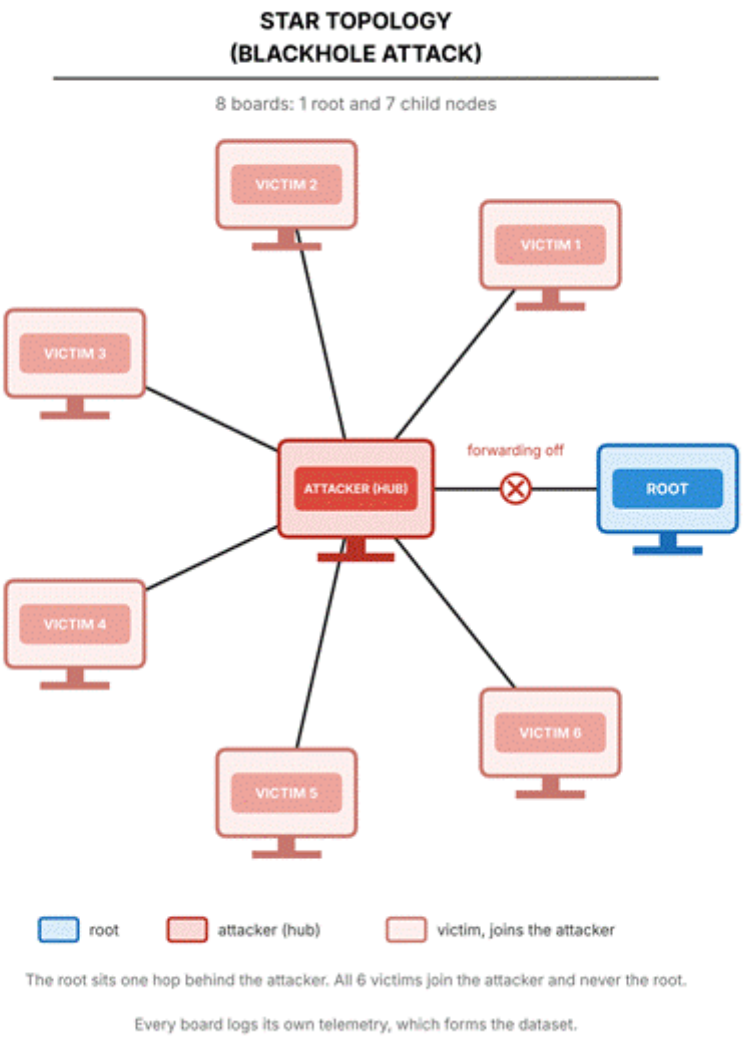


Figure 4.17 Blackhole Attack on Star Topology

For blackhole experiments in *Figure 4.17*, the attacker is the hub. The root sits one hop behind it, about one metre away, and the victims form a ring around the attacker. The victims are configured to join the attacker directly and never join the root, so that the star structure and the attacker position are fixed. During the baseline phase, the victims send their probes to the hub, which forwards them to the root. When the blackhole manipulation is activated, the attacker continues to receive packets from the victims but intentionally drops them rather than forwarding them. The root is the destination and the referee of the experiment, which is why it is not used as the attacker. Throughout the experiment, all nodes log telemetry, including timestamps, node IDs, phase labels, layer information, parent MAC addresses, RSSI values, and role-specific counters. The dataset captures the contrast between normal

forwarding behavior during baseline and the packet-dropping behavior during the manipulation window. Figure 4.17 must be redrawn with the attacker as the hub and the root behind it.

4.2.2.2. Tree Topology

The **tree topology** reflects the native ESP-WIFI-MESH structure, where nodes are layered beneath a root node. Intermediate nodes connect upward to a parent while accepting downstream child connections, enabling multi-hop packet forwarding within the hierarchical mesh architecture. The tree children keep their parent unless the link degrades substantially: ESP-WIFI-MESH's own parent-switch and weak-link thresholds are moved 6 dB lower and its weak-link watch lasts three times longer, while the first choice of parent is unchanged and the loss of a parent still makes its children move (D-26). Tree runs captured before this change used the stock thresholds, and this change is not yet confirmed on hardware.

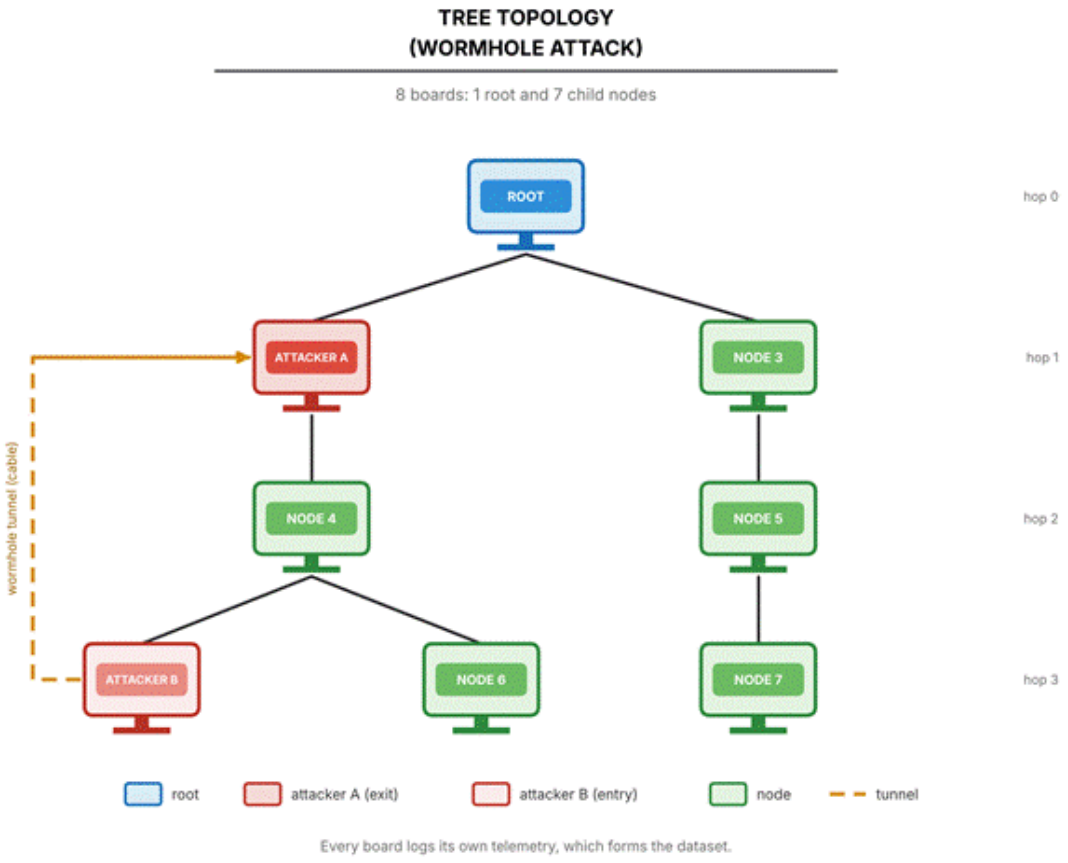


Figure 4.18 Wormhole Attack on Tree Topology

For wormhole experiments in *Figure 4.18*, Attacker A is positioned at hop 1 near the root, while Attacker B is positioned at hop 3 near the leaf nodes. The physical separation ensures that baseline communication between these regions requires traversing multiple layers through legitimate

intermediate nodes. During the baseline phase, probes from Attacker B follow the natural multi-hop path up the tree. When the distortion window is activated, Attacker B also writes its probes to the out-of-band tunnel, and Attacker A re-injects them toward the root. This creates a second, shorter path for the same probe, potentially observable as a different delay and as an inconsistency between delay and hop count. Tunnel endpoints log their counters, while all nodes record standard mesh telemetry for comparison between baseline and distortion windows.

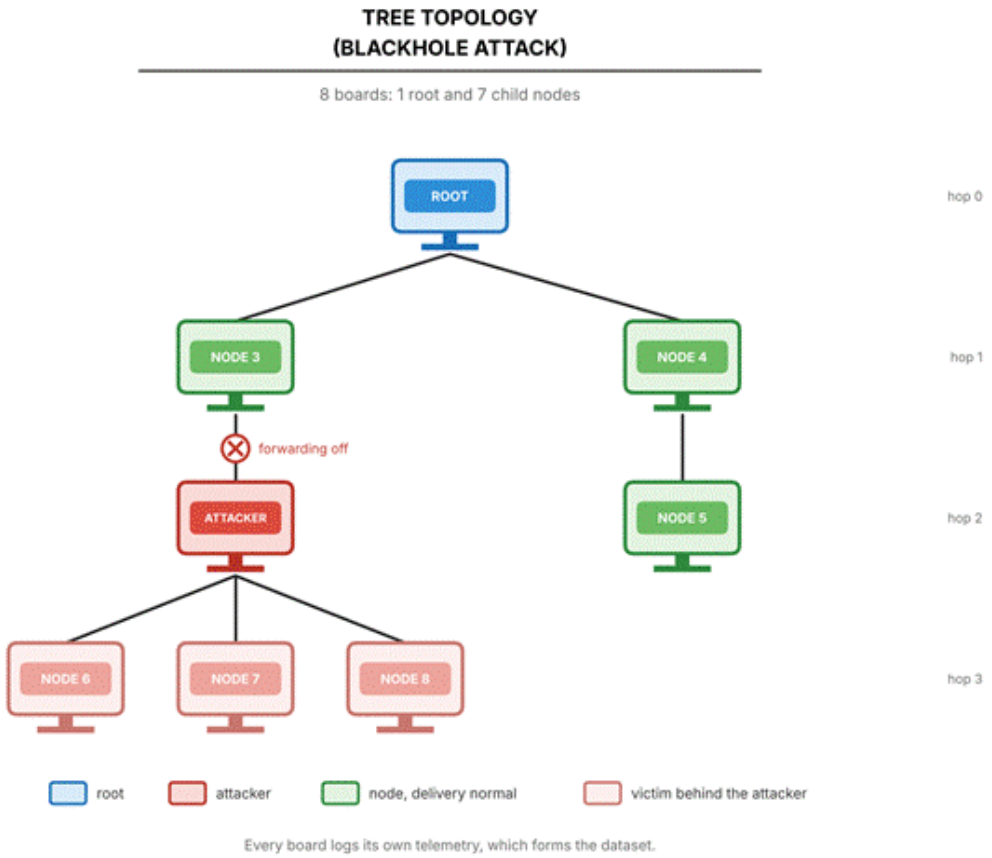


Figure 4.19 Blackhole Attack on Tree Topology

For blackhole experiments in *Figure 4.19*, the attacker is positioned at hop 2 as an intermediate node that serves as the parent for multiple downstream leaf nodes. The root occupies hop 0, with legitimate intermediate nodes at hop 1. During baseline operation, victims at hop 3 forward their probes upward through the attacker, which relays them toward the root. When the blackhole manipulation activates, the attacker continues to receive transit packets from its victims but intentionally drops them instead of forwarding them. The dataset captures the attacker's forwarding ratio degradation, the loss of delivery from its victims, and any parent-switching events that occur during the manipulation window. All nodes log layer information, parent MAC addresses, RSSI values, and packet counters throughout all phases.

4.2.2.3. Linear Chain Topology

A linear chain arrangement of ESP32 nodes is used to observe sequential packet propagation and localized forwarding dependencies within the ESP-WIFI-MESH network. Nodes are positioned in a straight line so that each device primarily communicates with its immediate neighbor, forming a chain-like multi-hop routing path from the farthest node toward the root. This configuration constrains routing paths and enables controlled observation of how packets propagate hop-by-hop through intermediate nodes..

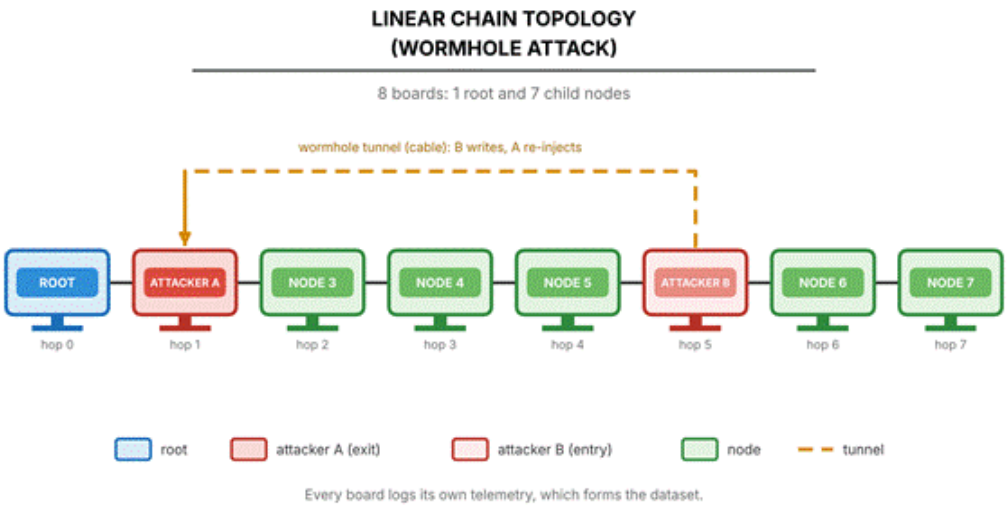


Figure 4.20 Wormhole Attack on Linear Chain Topology

For wormhole experiments in *Figure 4.20*, Attacker A is positioned toward the root end of the chain, while Attacker B is placed farther along the chain, with the ends assigned at run time by depth. The physical separation ensures that baseline communication between these regions must traverse all intermediate nodes sequentially, resulting in latency proportional to the number of hops. An out-of-band tunnel connects the two attackers independently of the mesh network. During the distortion window, Attacker B writes its own probes to the tunnel, and Attacker A re-injects them toward the root. This creates an apparent path compression effect, as the second copy of each probe arrives by a path that skips the hops between the two ends. The dataset captures duplicate arrivals, latency anomalies, hop-count stability, and tunnel activity metrics alongside standard mesh telemetry.

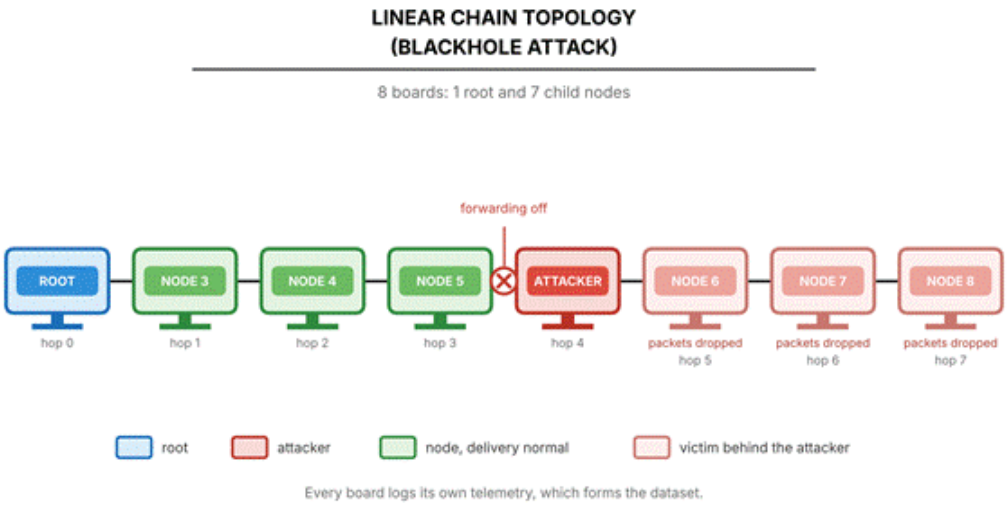


Figure 4.21 Blackhole Attack on Linear Chain Topology

For blackhole experiments in *Figure 4.21*, the attacker is positioned as an intermediate forwarding node within the chain, such that all traffic between the far end and the root must pass through it. Nodes are spaced several meters apart (typically 3 to 5 m in the laboratory setup) so that the strongest RSSI links occur between adjacent devices, encouraging sequential parent selection along the linear arrangement. During baseline operation, probes from the far end travel toward the root and traverse each intermediate node in sequence, with the attacker acting as a normal relay. When the blackhole manipulation activates, the attacker continues to receive probes from the nodes beyond it but intentionally drops them instead of forwarding them toward the root, which cuts those nodes off from the root. Nodes beyond the attacker experience complete loss of delivery, while nodes on the root side of the attacker continue to operate normally. The dataset captures the cumulative impact of forwarding suppression, including the complete delivery failure of the nodes beyond the attacker and the unchanged delivery of the nodes before it.

4.2.2.4. Partial Mesh Topology

The **partial mesh topology** represents the primary experimental environment, in which nodes are connected to a subset of other nodes rather than all of them. Nodes select parents by layer and then child count, among the candidates whose signal passes the RSSI thresholds (Section 3.2.1). This semi-structured topology still allows observation of network self-organization and adaptive parent switching, but connectivity is limited compared to a full mesh. The partial mesh is defined by how it is built: the boards are placed so that each can hear several candidate parents, and each node accepts at most two children, which forces branching and depth. Switching to a second parent is allowed but was not observed in the nine processed runs, so these runs behave as trees (D-29, to be confirmed with the adviser).

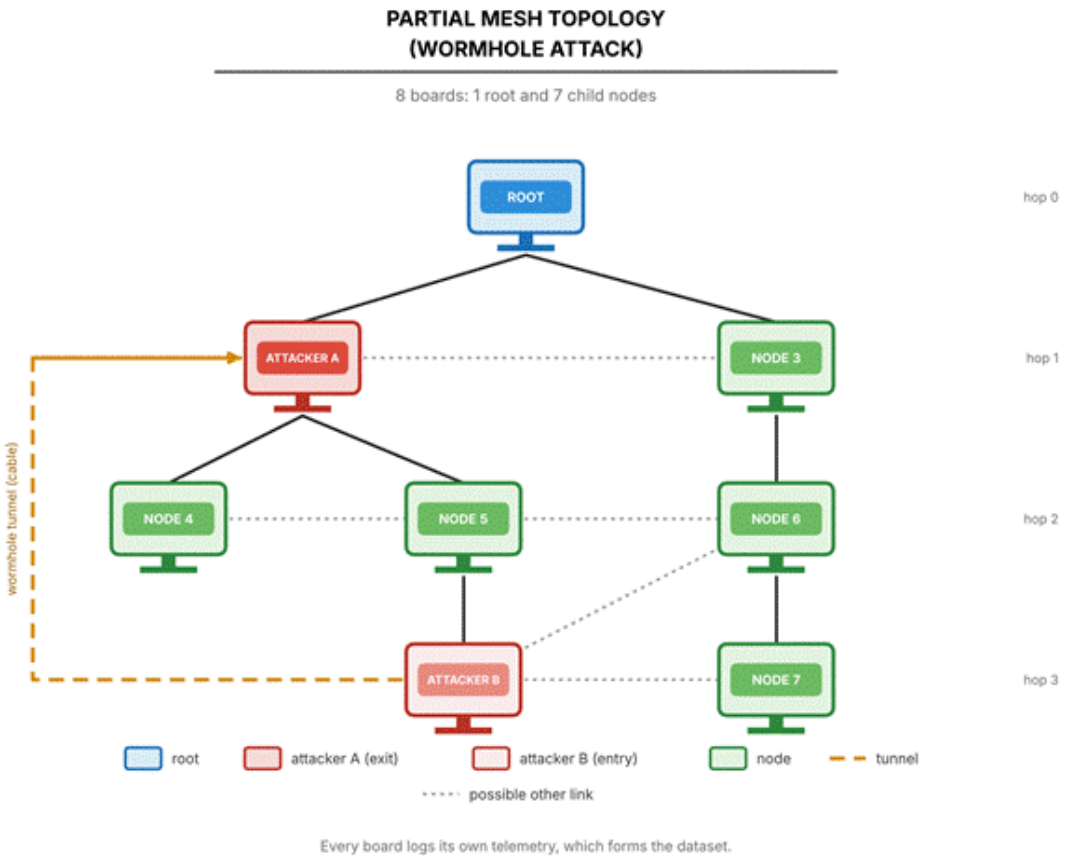


Figure 4.22 Wormhole Attack on Partial Mesh Topology

For wormhole experiments in *Figure 4.22*, Attacker A is positioned toward the root region, and Attacker B is positioned near a cluster of nodes deeper in the mesh. The connectivity matrix is arranged so that there are several candidate paths between the regions, and the baseline routes are determined by the protocol's parent selection. An out-of-band tunnel connects the two attackers independently of the mesh. During the distortion window, Attacker B writes its own probes to the tunnel, and Attacker A re-injects them toward the root. This creates apparent path compression, and the dataset captures changes in end-to-end latency, duplicate arrivals, hop-count stability, parent switching frequency, and RSSI-hop correlation across baseline and distortion windows.

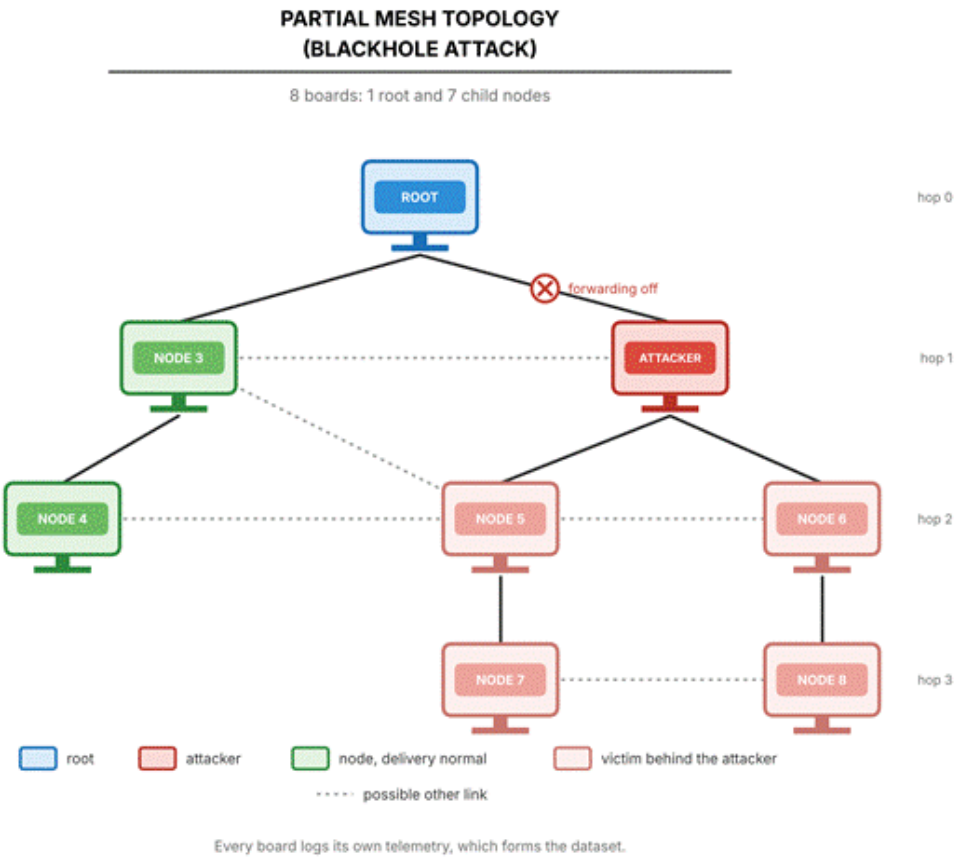


Figure 4.23 Blackhole Attack on Partial Mesh Topology

For blackhole experiments in *Figure 4.23*, the attacker is positioned as a high-betweenness relay node, central in the cluster, through which the paths of several victim nodes to the root pass. During baseline operation, traffic from the victims flows through the attacker toward the root. When the blackhole manipulation activates, the attacker continues to receive transit packets but omits forwarding them, causing a loss of delivery from its victims. The attacker keeps acknowledging the victims' frames, so the victims see no failure and are not expected to re-route; parent-switching events are recorded to confirm this. The dataset captures the direct effects of forwarding suppression and any parent change that does occur.

4.2.2.5. Deployment Sites and Scenario

The deployment is framed as a smart-campus scenario: indoor environmental and building monitoring at DLSU Manila, where each node sends a periodic sensor-style probe toward the root. This follows Khan et al. [22], who deployed an ESP-MESH network across a campus and reported packet delivery above 97% at every node except one outdoor node. The four topologies are logical layouts of the nodes. They are not tied to particular kinds of indoor place, and each is set up at every site.

Data are collected at three sites, listed in Table 4.6. Each site has a different amount of Wi-Fi interference, which is deliberate variance in the dataset.

Table 4.6 Data Collection Sites

Site	Description	Status
Room G402 (GCCS)	Controlled classroom	All runs captured: 24 of 24 attack runs and 4 of 4 benign runs.
DLSU Library	Small part of a busy library with heavy Wi-Fi traffic	All runs captured: 24 of 24 attack runs and 2 of 2 benign runs.
Yuchengco Building (DLSU)	Area within the building:	All runs captured: 24 of 24 attack runs and 2 of 2 benign runs.

Each run also follows one of the run scenarios in Table 4.7. The scenarios add controlled variation in traffic, so that traffic volume alone cannot indicate an attack.

Table 4.7 Run Scenarios and Run Status, 80 Runs (Final)

Scenario	Variation introduced	Attack runs captured, of 24 (blackhole + wormhole)	Benign runs captured
Stationary	None. The reference run.	24 (12 + 12)	4
Burst	One child sends 300 probes back to back 60 s into the attack window	24 (12 + 12)	1
Highload	Every child probes at 250 ms instead of 1000 ms for the whole run	24 (12 + 12)	3
All three		72 (36 + 36) of 72	8

All 80 planned runs were captured: 72 attack runs (36 blackhole and 36 wormhole) and 8 benign runs (Table 4.7). By topology there are 21 linear, 19 partial-mesh, 21 star and 19 tree runs, and by scenario 28 stationary, 25 burst and 27 highload runs. Fifteen earlier captures of 11 cells were replaced by a newer capture, and one of these cells is the blackhole run on the linear topology at the Yuchengco Building with stationary traffic, which had been held out earlier because one of its nodes logged under 95% of its expected samples (Section 6.1). The export folders also hold 12 captures that are not part of the final plan: 5 of the jitter scenario of the earlier design and 7 recorded at home, which served as tests.

Each of the 24 combinations of attack, topology and scenario has 3 runs, one at each site, so the 72 attack runs are complete. The 8 benign runs are, at G402, a linear run with highload, a star run with burst, a tree run with stationary traffic and a partial-mesh run with highload; at the DLSU Library, a linear and a star run, both stationary; and at the Yuchengco Building, a linear run with stationary traffic and a star run with highload (Section 6.1). Every run follows the same fixed schedule, so the attack starts at the same time in every run and the position in the run could reveal the phase (Section 4.2.7.3).

The burst scenario is run as a matched pair: one burst inside a baseline run and one inside an attack run, at the same offset. The pair tests whether a legitimate burst can be told apart from an attack. Every run records its site, room, topology, attack, repeat, attacker position, start time, interference notes and board map, so each run describes itself.

The design crosses 4 topologies, 2 attacks, 3 sites and 3 scenarios, which gives 72 attack runs, one for each combination. It adds 8 benign runs with no attack, 4 at G402, 2 at the DLSU Library and 2 at the Yuchengco Building, which is the plan of D-18 as revised by D-35 (Section 5.5). Four of the 8 are stationary, 3 are highload and 1 is burst, so the scenarios are not balanced among them, and the benign runs serve as a control and not as a counterpart of every attack run (Section 6.5.4). The campaign has 80 runs in total (Table 4.7). Because each combination is run once, the three sites act as the repeats, so site differences and run-to-run noise cannot be separated. The campaign plan file lists exactly the 72 attack runs of this design, and every captured run of the 80 is a run of the plan.

The Library and Yuchengco Building areas are public spaces. Other people may be present during a run, and the study records only the telemetry of its own boards and the radio frames of its own mesh. The sniffer only listens. A note added to the ethics statement in Appendix B covers these sites.

4.2.3. Data Collection

This section describes the systematic approach employed to capture cross-layer telemetry from all ESP32 nodes participating in the experimental ESP-WIFI-MESH testbed. The data collection architecture is designed to operate independently of routing availability, ensuring that controlled manipulations, particularly forwarding suppression (blackhole), do not compromise dataset integrity. All telemetry is recorded locally on each node and retrieved after the experiment on a laptop workstation that consolidates the files. The telemetry is never streamed through the laptop. In the

captures that use a sniffer, a laptop also listens passively to the channel, and it takes no part in the mesh.

The data collection framework is organized into four functional components: (1) per-node metric definitions for victim and attack nodes, (2) data acquisition and logging architecture, (3) phase labeling and synchronization, and (4) local storage and post-experiment extraction. The following subsections detail each component, with emphasis on the raw telemetry fields captured during experimentation. Derived features (e.g., forwarding ratio, RSSI-hop consistency) are computed offline during the feature extraction stage described in Section 4.2.4.

4.2.3.1. Victim Node Data Collection

Victim nodes are standard mesh participants that generate periodic application-layer probe traffic, send it to their parent, relay the probes of the nodes below them, and record raw cross-layer telemetry. In the logs they carry the role child, because whether a node is a victim depends on where the attacker sits in that run. They are typically positioned at hop 1 or deeper, and they serve as the primary sources of observable network behavior during both baseline and manipulation phases.

A. Physical Layer (PHY) Metrics

Table 4.8 Physical Layer (PHY) Metrics

Metric	Description	Collection Method	Sampling
rssi	Received Signal Strength Indicator	esp_wifi_sta_get_rssi()	10 Hz
timestamp	Microsecond-resolution timestamp	esp_timer_get_time()	Per sample

B. Medium Access Control (MAC) Layer Metrics

Table 4.9 Medium Access Control (MAC) Layer Metrics and Application Counters (D-15)

Metric	Description	Collection Method	Sampling

retry_count	Cumulative failed sends (application-layer counter, not MAC retransmissions; D-15)	Application send routine	10 Hz
tx_count	Cumulative frames accepted for sending (application-layer counter)	Application send routine	10 Hz
MacRetryRate	Fraction of the frames from this node to its parent that carry the 802.11 Retry bit, per one-second window. Available only in sniffer captures	Sniffer record, read offline with Wireshark [78]	1 s window

Note: ESP-IDF does not expose a per-link MAC retransmission counter to the application. Its statistics call (esp_wifi_statis_dump) only prints Wi-Fi statistics to the console. The columns retry_count and tx_count are therefore application-layer counters, and 802.11 retransmissions are measured from the Retry bit of the frames that the sniffer records. Where the sniffer did not cover a node, MacRetryRate is missing and is never recorded as zero.

C. Network Layer Metrics

Table 4.10 Network Layer Metrics

Metric	Description	Collection Method	Sampling
layer	Current mesh layer. The root is layer 1, and the hop count is the layer minus one	esp_mesh_get_layer()	10 Hz
parent_mac	Address of the upstream parent; all zero when there is no parent	*esp_mesh_get_parent_bssid()*	10 Hz
probes_count	Application-layer probe counter: own probes accepted by the mesh stack	Incremented on each probe transmission	10 Hz

recv_count, forward_count, drop_count	Probes this node received for relay, passed on, and did not pass on	Incremented by the relay task	10 Hz

Parent switches are not stored as separate events. They are derived offline from changes of parent_mac between consecutive samples.

4.2.3.1.4. Probe Generation

Victim nodes generate periodic probe packets that travel hop by hop toward the root. A probe is sent every 1000 ms, or every 250 ms in the highload scenario. Each probe contains:

- Sequence number
- Transmission timestamp
- Node identifier
- Magic value that identifies the packet as a probe

The root logs one row for each probe that arrives, with the originating node, the sequence number, the send timestamp, the arrival timestamp and the phase at arrival. These probes enable relative one-way delay measurement and delivery ratio calculation during offline analysis. Delivery is computed by joining the root's arrivals to a node's probe counters on the sequence number, because the boards share no clock.

4.2.3.2. Attack Node Data Collection

Attack nodes are responsible for executing controlled behavioral manipulations while simultaneously logging telemetry. These nodes record all metrics collected by victim nodes, plus additional counters specific to their manipulation roles.

4.2.3.2.1. Attack Node Types

Two distinct attack node configurations are implemented:

- Forwarding Suppression (Blackhole) Node: A single attacker that drops the probes it relays during the manipulation window. All manipulation occurs at the application layer; no lower-layer protocol violations are introduced.
- **Topology Distortion (Wormhole-Inspired) Nodes:** Two colluding attackers that relay information through an out-of-band tunnel to create a shortcut effect. No raw IEEE 802.11 frame capture, modification, or replay is performed. Wormhole behavior is emulated at the application/flow and telemetry level via an auxiliary tunnel, preserving full protocol compliance.

4.2.3.2.2. Forwarding Suppression (Blackhole) Node Metrics

In addition to the victim node metrics, the blackhole attacker records:

Table 4.11 Forwarding Suppression (Blackhole) Node Metrics

Metric	Description	Collection Method
recv_count (probes_count)	Probes received for relay	*Incremented when a probe is accepted for relay*
forward_count (tx_count)	Probes forwarded to the parent	*Incremented after a successful esp_mesh_send()*
drop_count	Probes intentionally dropped, plus probes that could not be sent	Incremented when a probe is not passed on; received equals forwarded plus dropped
phase_id, gt_label	Manipulation state: suppression is active while the phase ID is 1	Logged on every sample

4.2.3.2.3. Topology Distortion (Wormhole-Inspired) Node Metrics

Both tunnel endpoints record the following metrics to document tunnel activity and its effects:

Table 4.12 Topology Distortion (Wormhole-Inspired) Node Metrics

Metric	Description	Collection Method
tunnel_tx_count	Probes written to the tunnel (Node B)	Logged in retry_count on Node B; incremented on each tunnel frame written
tunnel_rx_count	Valid tunnel frames received (Node A)	Logged in probes_count on Node A; incremented on each valid frame
shortcut_inject_count	Probes re-injected toward the root (Node A)	Logged in tx_count on Node A; incremented after each re-injection
inject_fail_count	Failed re-injections (Node A)	Logged in retry_count on Node A
direct_send_count, generated_count	Probes Node B sent through the mesh, and probes it created (Node B)	Logged in tx_count and probes_count on Node B

role	wormhole_a or wormhole_b, assigned at run time	Logged on every sample

These metrics document the intensity and effect of the manipulation without requiring low-level frame capture. All wormhole-inspired effects are achieved via the application-layer tunnel and do not involve modifying mesh control frames. The tunnel is not timed with echo messages. Its effect on delay is derived offline from the difference between the arrival times of the two copies of the same probe at the root.

4.2.3.3. Data Acquisition and Logging Architecture

The data acquisition system implements a decentralized logging framework embedded within each node's firmware. Logging operates as a background task that does not interfere with mesh responsibilities.

4.2.3.3.1. Sampling Loop

All nodes execute a continuous telemetry sampling loop with a 100 ms interval (10 Hz). *Figure 4.24* illustrates the victim node implementation; attack nodes extend this with manipulation-specific counters.

Figure 4.24 Victim Node Telemetry Sampling Loop

```
Constants:
SAMPLING_INTERVAL ← 100 milliseconds

Variables:
node_id : unique identifier
current_phase, ground_truth_label : updated via root broadcasts

procedure CHILD_TELEMETRY_LOOP():
        initialize_storage()
        next_sample ← now()

        while experiment_active == TRUE do
                // Capture timestamp
                timestamp ← get_high_resolution_time()

                // Physical layer
                rssi ← esp_wifi_sta_get_rssi()

                // Network layer
                layer ← esp_mesh_get_layer()
                parent_mac ← esp_mesh_get_parent_bssid()

                // Application layer (cumulative counters)
                retry_count ← failed_send_counter()
                tx_count ← accepted_probe_counter()
                probes_count ← accepted_probe_counter()
                recv_count, forward_count, drop_count ← relay_counters()

                // Format CSV record
                log_entry ← CSV_FORMAT(
                timestamp, node_id, "child", layer, parent_mac, rssi,
                retry_count, tx_count, probes_count,
```

```
                    current_phase, ground_truth_label,
                    recv_count, forward_count, drop_count
                    )

                    // Write to internal flash and to the microSD card
    append_to_log(log_entry)

                    // Wait for the next sampling instant (fixed period)
                    next_sample ← next_sample + SAMPLING_INTERVAL
    sleep_until(next_sample)
        end while
end procedure
```

4.2.3.3.2. Forwarding Counter Implementation

Accurate tracking of receive and forward events is essential for subsequent computation of the forwarding ratio. Attack nodes maintain cumulative counters updated on each mesh API call.

Figure 4.25 Forwarding Counter Update (Attack Node)

```
// Counters kept on every relaying node
atomic uint32_t recv_counter = 0
atomic uint32_t forward_counter = 0
atomic uint32_t drop_counter = 0

// Called when a probe arrives for relay
on_event probe_received(probe):
atomic_increment(recv_counter)          // counted before any decision
queue_push(relay_queue, probe)
end

// Called by the relay task for each queued probe
procedure forward_packet(probe, destination):
        if should_forward(probe):
                result = esp_mesh_send(destination, probe, MESH_DATA_P2P)
                if result == ESP_OK:
        atomic_increment(forward_counter)
                else:
        atomic_increment(drop_counter)
                end if
        else:
    atomic_increment(drop_counter)
        end if
end procedure

// Forwarding suppression decision (attacker only; honest nodes always forward)
procedure should_forward(probe):
        if suppression_active:
                return FALSE
        end if
        return TRUE
end procedure
```

The algorithm in *Figure 4.25* uses counters that are written by the relay task and read by the logging task. When a probe arrives, recv_counter is incremented. The relay task then calls should_forward(). If suppression is active, drop_counter is incremented and the probe is discarded; otherwise forward_counter is incremented after a successful esp_mesh_send(). Every probe is counted exactly once, so the number received equals the number forwarded plus the number dropped. This provides the raw counts needed to compute the forwarding ratio offline.

C. Tunnel Metric Sampling (Wormhole-Inspired Nodes)

For topology distortion experiments, tunnel endpoints record activity metrics using the sampling loop in *Figure 4.26*.

Figure 4.26 Tunnel Metric Sampling

```
procedure TUNNEL_METRIC_SAMPLING():
        next_sample ← now()
        while experiment_active == TRUE do
                timestamp ← get_high_resolution_time()

                // Snapshot the tunnel counters; the three role-dependent columns depend on the end
                if role == B:
                retry_count ← read_probes_tunneled()
                tx_count ← read_probes_to_root()
                probes_count ← read_probes_generated()
                else:
                retry_count ← read_reinject_fail()
                tx_count ← read_probes_reinjected()
                probes_count ← read_tunnel_received()
                end if
                recv_count, forward_count, drop_count ← relay_counters()

                log_entry ← CSV_FORMAT(
                timestamp, node_id, role, layer, parent_mac, rssi,
                retry_count, tx_count, probes_count,
                current_phase, ground_truth_label,
                recv_count, forward_count, drop_count
                )

    append_to_log(log_entry)
                next_sample ← next_sample + SAMPLING_INTERVAL
    sleep_until(next_sample)
        end while
end procedure
```

This loop snapshots the current tunnel counters at 100 ms intervals. The counters track the number of probes written to the tunnel and re-injected, which gives a direct measure of manipulation intensity. The relay counters stay separate from the tunnel counters, so that recv_count, forward_count and drop_count mean the same thing on every role.

Each board samples at 10 Hz, and the analysis keeps all ten samples in each one-second window. The radio is forced to 20 MHz (HT20) instead of the ESP32 default of 40 MHz, so that an independent sniffer can decode the data frames. Earlier captures used 40 MHz and are not pooled with later captures. A sniffer on a laptop records 802.11 frames from outside the mesh, so the evidence does not rest only on the attacker counting its own drops.

4.2.3.4. Phase Labeling and Synchronization

Precise ground-truth labeling is achieved through coordinated phase transitions broadcast by the root node. Each node embeds the current phase label in every telemetry record.

A. Phase Control Algorithm

Figure 4.27 Experimental Phase Control (Root Node)

```
Input: formation_duration, baseline_duration, manipulation_duration, post_duration

procedure CONTROL_EXPERIMENT_PHASES():
        // Formation: the mesh forms, rows are logged unlabelled
  announce_prepare_every(5 seconds, for formation_duration)
  wait_until_all_children_present()

        // Baseline phase: normal operation, no manipulation
        phase ← BASELINE
        broadcast_phase(phase)
  log_event("PHASE_START", phase, current_time())
  sleep(baseline_duration)

        // Manipulation phase: activate attack behavior
        phase ← MANIPULATION
  activate_manipulation()     // Enable forwarding suppression or tunnel
        broadcast_phase(phase)
  sleep(manipulation_duration)

        // Post-condition phase: disable manipulation, observe recovery
        phase ← POST_CONDITION
  deactivate_manipulation()  // Disable all attack behaviors
        broadcast_phase(phase)
        sleep(post_duration)

        // Termination: end of experiment
        phase ← TERMINATE
        broadcast_phase(phase)        // repeated every 5 s for 60 s
end procedure
```

In *Figure 4.27*, the root node controls the experiment timeline by broadcasting phase change messages to all mesh participants. Each phase has a fixed duration, and the root logs the start and end times for later validation. The broadcast_phase() function does not rely on a reliable broadcast service. It repeats each transition in five rounds with the same sequence number, as described in Section 4.2.1.1, so that every node adopts the transition once.

4.2.3.4.2. Phase Broadcast Mechanism

Phase transitions are disseminated as individual peer-to-peer messages to every node in the root's routing table. All nodes update their local current_phase variable upon receipt, ensuring temporal alignment across the network. Nodes also log each phase transition event locally, providing a secondary check during dataset consolidation.

Each run follows a timeline of at least 60 s of mesh formation (it lasts until the pre-run gates pass and the operator types START, D-22), 300 s of baseline, 180 s of attack and 120 s of cooldown. The formation period carries the label 255 and is excluded from analysis, as *Table 4.1* shows. The benign control of an attack run is its own phase 0 instead of a separate baseline run. In addition, 8 matched runs with no attack were captured (D-35): one at each site for the linear and the star topologies, and one at G402 for the tree and the partial mesh, so every topology also has a stand-alone benign capture.

Because the attack starts at the same time in every run, position in the run could reveal the phase, so it is kept out of the model inputs (Section 4.2.7.5).

4.2.3.5. Experimental Label Encoding and Assignment

Ground-truth labels are assigned to every telemetry record based on the phase broadcasts from the root node, as described in Section 4.2.3.4. Each node maintains a local current_phase variable updated upon receipt of phase transition messages; this value is appended to every logged telemetry sample. The mapping between the experimental phase and numerical label is defined in *Table 4.13*.

.

Table 4.13 Experimental Phase Label Encoding

Label	Experimental State
0	Baseline (Normal Operation)
1	Controlled Forwarding Suppression (Blackhole)
2	Controlled Topology Distortion (Wormhole-Inspired)
255	No ground truth: formation, or rows logged before the first phase was heard. Excluded from the dataset

This labeling approach ensures ground-truth integrity for the following reasons:

- **Independence from feature values:** Labels are derived from experiment control logs and broadcast messages, not from any observed metric threshold.
- **No implicit detection logic:** The labeling process does not embed assumptions about what constitutes an attack; it only records which manipulation was active at a given time.
- **Consistency across all nodes and windows:** Every node records the same phase label during each interval, enabling reliable temporal alignment during dataset consolidation.

4.2.3.6. Local Storage and Post-Experiment Extraction

A. Local Storage Architecture

Each node maintains its own telemetry log using the on-chip flash file system (SPIFFS). Logs are written in CSV format with the following characteristics:

Table 4.14 Local Storage Architecture

Parameter	Value
File system	SPIFFS on internal flash, and FAT on a microSD card that mirrors the same rows
Log format	CSV, with the same columns on flash and on the card
File naming (flash)	nodeID_runID_YYYYMMDD_HHMMSS.csv by the laptop at export
File naming (card)	attack/topology/location/role_nodeID_rRUN_bBOOT_kind.csv, with a runs.csv manifest in each folder that records the rows, the clean or unclean ending and the build of each boot
Buffer and flush	Flushed every 10 records. The card file is also fsync'ed about every 5 s (D-13)
Maximum log size	About 500 KB per run on flash. On the card the limit is the card's capacity
Probe arrivals at the root, highload runs	Written to the card only, in 4 KB batches flushed every 100 rows
Card interface	SPI card reader on GPIO 5, 18, 19 and 23, powered from VIN (5 V), clock capped at 4 MHz

B. Post-Experiment Log Extraction

After the run, the logs are imported to a laptop by one of two routes. In the main route the microSD cards are removed from the boards and read together in a card reader, and an import script files each card's logs into the dataset folder for the right attack, topology, site and scenario. The card holds the attack, topology, site, role and node, so the script only asks for the scenario and the repeat number, and it takes the date at the time of import. A card often holds several boots, so the script selects the boots that belong to the run and skips boots that never ended cleanly unless told otherwise. In the second route, a board is connected by USB and its logs are streamed to the laptop with the EXPORT_LOGS command, as in the original design. The script can also read a card that is still inside the board, over USB.

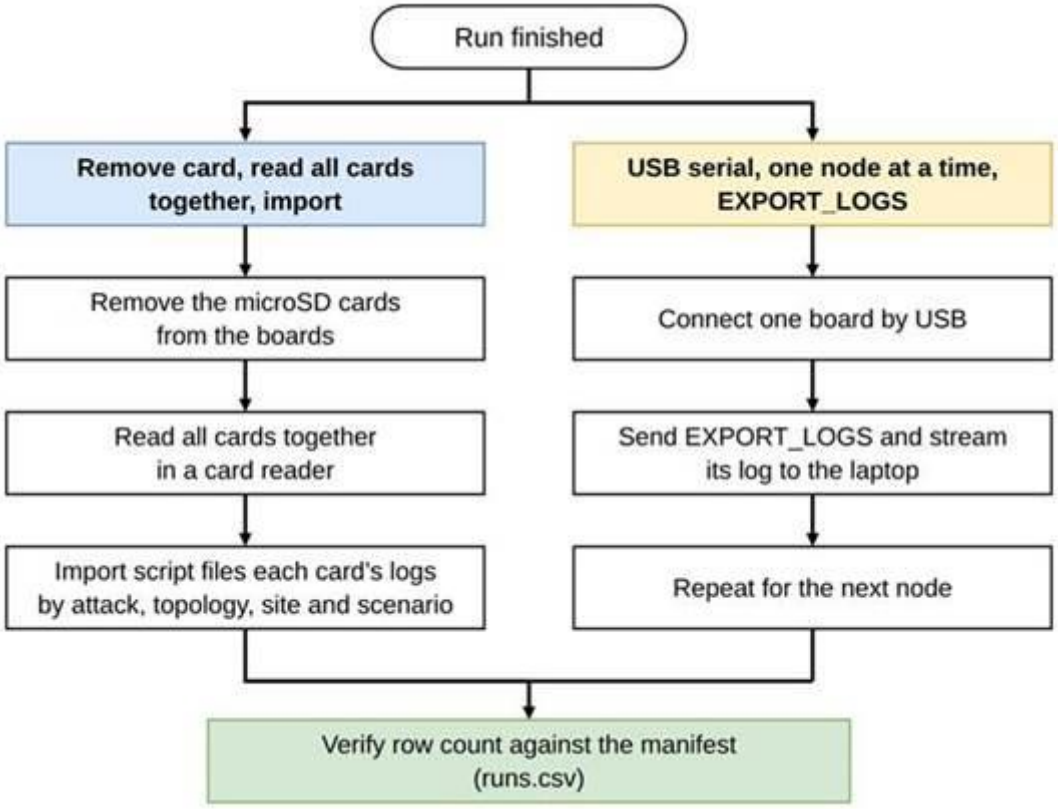


Figure 4.28 Log Retrieval Routes

Each imported file is checked against the runs.csv manifest on the card. If the number of rows received differs from the number the board logged, the script prints a warning, which is the sign of a capture that was cut short. A source file is deleted only if the person asks for that, after its copy has been verified. By default, files that are replaced on a card are archived and not deleted.

4.2.3.6.3. Data Integrity During Manipulation

The local storage architecture ensures dataset completeness even during active attacks:

- Logging continues regardless of routing success – The logging task runs independently of mesh operations.
- All transmission attempts and failures are recorded – Counters capture every packet, regardless of outcome.
- Logs are retained until the copy is verified. Nothing is deleted from a board or a card until the imported file has been checked against the manifest, and deletion is optional.
- Telemetry logs are not streamed over the mesh during the experiment. Only experiment traffic (probes and control messages) crosses the mesh. Logging data stays on the board's flash and card.

This design keeps the record independent of the attacks being studied: an attack cannot stop a board from logging, and the ground-truth labels are embedded in the data stream. It does not make every capture complete. A board that crashes, browns out or loses its card can leave a gap, which the checks of Section 4.2.3.7 detect and the verifier grades by the board's role (D-32).

The raw telemetry collected through this framework serves as input to the feature extraction stage described in Section 4.2.4, where derived behavioral indicators (e.g., forwarding ratio, RSSI-hop consistency, parent switching frequency) are computed for exploratory analysis and clustering.

Each board still logs to its internal flash, and a microSD card now mirrors every telemetry and arrivals row. Data are exported by pulling the card and reading it on a laptop. If no card is present the board falls back to flash only, and the run is never blocked. The card reader is wired by SPI on GPIO 5, 18, 19 and 23, its power pin goes to VIN (5 V) and not to 3V3, and the SPI clock is capped at 4 MHz because jumper wires cannot carry the default speed.

Rows are flushed to the card with fsync() about every 5 s, so an unclean ending loses at most the last few seconds (D-13). Before this change a run that ended badly could leave a card file with 0 rows, and such a file is treated as a lost capture and not as an empty log. The change compiles, and it has not been validated by resetting a board in the middle of a run. None of the 637 telemetry files of the 80 runs has 0 rows (Section 4.2.3.7). A card that fails during a run is also retried: the board reopens the file and then remounts the card every 5 s, stops after six failed attempts in a row, and records the rows it could not write in the card’s runs.csv, so the import flags an incomplete card copy. Internal flash keeps every row. This recovery is not yet tested on hardware.

On highload runs the root writes probe arrivals to the card only, in 4 KB batches flushed every 100 rows, instead of to flash and the card every 10 rows. The double write took about 400 ms per row and lost 3,151 rows at about 28 rows a second in an earlier highload run. The columns and values are unchanged, and only where and when the rows are written differs. The first highload run with the change lost no arrival rows, and highload captures made before it are being recaptured.

4.2.3.7. Dataset Completeness Verification

To ensure that the final dataset accurately reflects the intended experimental conditions and to identify potential data loss due to high traffic or transient logging failures, a completeness verification procedure is performed during preprocessing.

- **Expected Sample Count:** During each experimental phase, nodes are configured to log telemetry at 10 Hz. For a 5-minute baseline phase (300 seconds), the expected number of samples per node is 3,000. For manipulation phases (3 minutes), the expected count is 1,800. Coverage is measured over the experiment phases only (baseline, attack and cooldown), and every board must reach 95% of its expected samples; formation rows are excluded.

- **Completeness Check**: After import, the number of samples in each one-second window is counted. Windows with fewer than 8 of the 10 expected samples are discarded, and gaps of 1 or 2 samples inside a kept window are filled (D-9). The proportion of discarded windows per run is reported as a dataset quality measure.
- **Row count against the manifest.** The number of rows imported for each boot is compared with the number recorded in the card's runs.csv. A difference marks the capture as cut short. A zero-row card file from before the fsync change (D-13) is treated as lost data, not as a node that logged nothing.
- **Run inventory.** The inventory script compares every captured run with the campaign plan and lists the cells that are missing, extra or due for a recapture. Against the final plan, all 80 runs were captured and none is missing. Twelve captures lie outside the plan (5 of the jitter scenario and 7 recorded at home) and are not part of the 80 runs. Fifteen earlier captures of 11 cells were replaced by a newer capture, and only the newest capture of a cell is analysed (Section 6.1).
- **Integrity and topology checks.** validate_integrity.py and verify_topology.py are run on every capture. **The verifier's check of the setup of each run found no failure in the 80 runs. Its topology check gave a warning in 19 of the 72 attack runs: the 18 partial-mesh runs, in which every board was seen under a single parent, and the blackhole run on the tree topology at G402 with stationary traffic, which is also the one run of the 80 in which a board was seen under a second parent in the labelled phases. validate_integrity.py read the 637 telemetry files of the 80 runs: 600 passed, 30 gave a warning and 7 failed, and 66 of the 80 runs have no warning**

or failure in any file. Each of the 7 failures is a file in which a timestamp is smaller than the one before it, one file in each of 7 runs. No run was excluded: the verifier confirmed all 72 attack runs and all 8 benign runs. A capture that could not be used was captured again (Section 6.1).

- **Card files. None of the 637 telemetry files of the 80 runs has 0 rows, so no run is treated as lost data. One board has no file: B768 in the blackhole run on the tree topology at G402 with highload, and that run is analysed without it (Section 6.2.5).**
- **Discarded windows. The preprocessing formed 457,568 one-second windows in the 80 runs and discarded 1,372 of them (0.30%) because fewer than 8 of the 10 expected samples were present. Per run the share lies between 0.16% and 0.89% (median 0.27%). The highest share is in the blackhole run, partial-mesh topology, the DLSU Library, highload, and no run loses more than 1% of its windows. A window with at least 8 samples can miss at most 2 samples, so the second rule of the code, which rejects a gap of more than 2 consecutive samples, did not discard any window.**

Capture and checking are done with the tools in Table 4.15, and an inventory script checks every captured run against the campaign plan. The tools are kept in the project repository [79].

Table 4.15 Capture and Verification Tools

Tool	Purpose
Run wizard and menu	Flash boards, export logs, save presets, identify boards by MAC address [77]
validate_integrity.py	Checks that the captured files are complete
verify_topology.py	Checks that the mesh formed the intended topology
verify_attack.py	Tests the attack against the baseline (Section 4.2.7)
Sniffer capture	Independent 802.11 frame record, read with Wireshark [78]

4.2.4. Feature Extraction

This section describes the computational procedures for transforming raw telemetry logs into structured behavioral features suitable for exploratory analysis and clustering. All derivations are

performed offline during dataset preprocessing, after log extraction and timestamp alignment. The resulting feature vectors capture cross-layer behavioral patterns across the PHY, MAC, and network layers, with auxiliary tunnel activity features recorded only during topology-distortion runs. In this chapter the network layer is the layer of the ESP-WIFI-MESH tree, which runs below IP, and not OSI layer 3.

4.2.4.1. Preprocessing

Raw telemetry logs collected at 10 Hz are first processed to ensure temporal consistency and to convert cumulative counters into interpretable metrics.

Timestamp Alignment. The clocks of the boards are not synchronized, and each node records a local timestamp with esp_timer_get_time() that counts from its own boot. A raw difference between the timestamps of two boards therefore reflects mostly the difference in their boot times and is not used. Instead, the timeline of each node is aligned to that node's own first exit from phase 0, which is the one event that every node observes through the root's phase broadcast. Time is expressed relative to that anchor, and the last 300 s of phase 0 before it is labelled baseline. Earlier rows are excluded from the labelled dataset.

Counter Delta Conversion. Cumulative counters (e.g., retry_count, tx_count, recv_count, forward_count) are recorded as monotonically increasing values. For analysis, these are converted into per-window deltas:

Equation 4.1

$$\Delta counter = counter(t_{end}) - counter$$

This conversion captures the activity intensity within each window independent of prior history.

Application-Layer Counters. ESP-IDF does not expose MAC-layer retransmission counters, so retry_count and tx_count are application-layer counters and are logged as such. A window in which a node made no send attempt has no RetryRate and is left empty and not set to zero. MAC retransmissions come only from the sniffer, as MacRetryRate.

Handling Missing Values

Raw telemetry logs may occasionally contain missing samples due to transient logging failures, buffer overflows, or temporary node unavailability. Several strategies exist for dealing with such missing data:

- **Linear interpolation** – Estimate missing values by averaging the nearest valid samples before and after the gap. Suitable for continuous metrics that change smoothly (e.g., RSSI).
- **Forward/backward fill** – Propagate the last observed value forward (or the next observed value backward) to fill short gaps. Appropriate for cumulative counters that should not decrease.
- **Deletion of incomplete windows** – Discard any aggregation window that contains missing samples beyond a tolerable threshold. This preserves data integrity but may reduce dataset size.
- **Flagging and separate handling** – Retain incomplete windows but mark them with a special flag, allowing downstream analysis to decide whether to include or exclude them.

The choice among these strategies depends on the nature of the metric, the extent of missing data, and the tolerance for estimation error. A hybrid approach is therefore used that balances data completeness with reliability, applying different rules depending on the type of metric and the duration of the gap.

1. **For continuous metrics (e.g., RSSI):**
    - If a gap of 1-2 consecutive missing samples occurs within a window, linear interpolation is applied using the nearest valid samples.
    - If more than two consecutive samples are missing (0.2 s at 10 Hz), the entire one-second window is discarded from further analysis. This threshold ensures that estimated values do not introduce excessive uncertainty.
2. **For cumulative counters (e.g., forward_count, retry_count):**
    - Counters are monotonic; therefore, if one or two consecutive samples are missing, the counter value is assumed unchanged from the previous logged sample (i.e., forward fill).
    - Longer gaps cause the window to be discarded because we cannot reliably determine the counter increment over the interval.
3. Counters that ESP-IDF does not expose:
    - retry_count and tx_count are application-layer counters. RetryRate is kept as a dataset column only, and it is excluded from verification, exploratory analysis and model inputs.

- A window with no send attempt has no RetryRate (NaN). MAC-layer retries are taken from the sniffer, as MacRetryRate and MacFramesHeard, in sniffer runs only.

4. **Window completeness threshold:**
    - Regardless of the imputation method, any window that contains fewer than 8 valid 10 Hz samples (out of the expected 10) is discarded. This guarantees that summary statistics are based on a representative subset of the window's duration.

All interpolation and flagging decisions are logged per node and per window, ensuring full transparency. The fraction of discarded windows is reported as a dataset quality metric, allowing future users to assess the potential impact of missing data.

Window Aggregation. To reduce sensitivity to transient fluctuations and to capture behavioral trends, all telemetry is aggregated into non-overlapping one-second windows:

Table 4.16 Window Aggregation

Parameter	Value
Window Duration	1 second
Sampling Grid	10 Hz
Window Type	Non-overlapping
Minimum Valid Samples	8 of 10
Continuous Metrics	Mean, variance, min, max per window
Cumulative Counters	Delta per window

Event Counts	Total events per window (e.g., parent switches)

Windowed aggregation balances granularity with statistical robustness. Telemetry is now aggregated into non-overlapping one-second windows on a 10 Hz grid (D-9). A window with fewer than 8 of its 10 expected samples is discarded, which keeps the 80% completeness rule of the approved text. Each node therefore yields about 60 windows per minute. The fraction of discarded windows and the number of captures excluded for integrity are reported as dataset quality figures.

4.2.4.2. Feature Computation

The following features are computed for each node per observation window. Features are organized by the behavioral dimension they capture, with formulas and interpretations provided. Where relevant, we distinguish between transit packets (those received for forwarding) and locally generated traffic.

The hop count is used instead of the layer: the hop count equals the layer minus 1, so the root is at 0 hops, and the count of depth changes is named HopChangeCount. LatencyHopRatio (Equation 4.14) uses the relative one-way delay divided by the hop count, because the probes have no response leg. TunnelLatency (Equation 4.18) is the gap between the two arrivals of one duplicated probe at the root.

A window receives a PDR (Equation 4.5) only if the node sent probes in it, was attached to the mesh, and the root was logging. A window with nothing sent is left blank and is not set to 0. The forwarding features are computed for every relaying node, not only the attacker.

The counters retry_count and tx_count in *Table 4.9* are application-layer counters: failed sends and frames accepted for sending. ESP-IDF exposes no 802.11 MAC statistics, so these are not MAC retransmissions [75]. RetryRate is therefore kept as a dataset column only, and it is excluded from verification, exploratory analysis and model inputs. Real MAC retries come only from the independent sniffer, as the features MacRetryRate and MacFramesHeard.

4.2.4.2.1. Forwarding Behavior Features

Forwarding Ratio (Blackhole Indicator)

Measures the proportion of received transit packets that are forwarded by a node. This feature is the primary indicator of forwarding suppression. The counters recv_count and forward_count refer only to transit packets; locally generated packets are excluded.

Equation 4.2

$$\text{Forwarding Ratio} = \frac{\Delta forward_cc}{\Delta recv_count}$$

Where:

- $\Delta forward_count$ = forward_count at window end − forward_count at window start
- $\Delta recv_count$ = recv_count at window end − recv_count at window start
- ϵ is a small constant (10^{-6}) to prevent division by zero

Interpretation:

- ≈ 1.0 → Normal forwarding behavior (all received transit packets forwarded)
- ≈ 0.0 → Complete forwarding suppression (blackhole active)
- 0.0 < ratio < 1.0 → Partial suppression or forwarding instability
- Undefined (no packets received) → Node not handling transit traffic

Ingress-Egress Delta

Absolute difference between transit packets received and forwarded, indicating packet absorption.

Equation 4.3

$$\text{Ingress-Egress Delta} = \Delta recv_count - \Delta forwar$$

Interpretation:

- ≈ 0 → Balanced forwarding (normal operation)
- > 0 → Packets absorbed (potential blackhole)
- < 0 (rare) → May indicate local packet generation or forwarding of previously queued packets

4.2.4.2.2. Link Reliability Features

Retry Rate (application counters)

Proportion of send attempts that failed, computed from the application counters retry_count and tx_count. These are not 802.11 MAC statistics [75] (D-15), so RetryRate is kept as a dataset column only and is not used in verification, exploratory analysis or model inputs. MAC-layer retries are taken from the independent sniffer as MacRetryRate.

Equation 4.4

$$\text{Retry Rate} = \frac{\Delta retry_cou}{\Delta tx_count}$$

- Kept as a dataset column only. No cited study supports it as an attack signature, and it measured flat in the captured runs.

Packet Delivery Ratio (PDR)

Proportion of a node's probes that are received at the root, computed per child node per window. The probes of a window are identified by their sequence numbers, and the root's arrival log is joined to the node's counters on the sequence number.

Equation 4.5

$$\text{PDR} = \frac{\#\text{ unique probe sequence numbers receive}}{\#\text{ probes sent by victim}}$$

Interpretation:

- ≥ 0.97 → Normal operation (Khan et al. [22] report above 0.97; the baselines in this study average about 0.998, Section 6.2)
- Decreased values → End-to-end delivery degradation, potentially due to blackhole or routing disruption

4.2.4.2.3. Topology Stability Features

Parent Switch Rate

Frequency of parent changes within the observation window, indicating topology instability.

Equation 4.6

$$\text{Parent Switch Rate} = \frac{\text{Number of parent change}}{\text{Window Duration (secc}}$$

Interpretation:

- Very low (≈ 0) → Stable topology
- Moderate (> 0 but small) → Normal adaptive adjustments
- High values → Persistent instability or distortion pressure

Layer Change Count

Number of times the node's mesh layer (hop count) changed during the window.

Equation 4.7

$$\text{Layer Change Count} = \text{Count of layer chan}$$

Interpretation:

- 0 → Stable layer assignment

- > 0 → Node moving up/down in hierarchy, indicating topology adaptation or instability

Hop Stability Duration

Longest continuous interval within the window during which the node maintained the same layer and parent. Computed by scanning event logs and measuring the maximum duration where (layer, parent_mac) remained constant.

$$\text{Hop Stability Duration} = \max(\text{stable in}$$

Equation 4.8

Interpretation:

- Long durations → Stable topology
- Short durations → Frequent reconfiguration, potential distortion effects

4.2.4.2.4. Physical Layer Features

RSSI Mean

Average received signal strength within the window.

$$\text{RSSI Mean} = \frac{1}{n}\sum_{i=1}^{n} RSSI_i$$

Equation 4.9

Interpretation:

- Values reflect link quality and physical proximity to the parent
- Under normal operation, deeper nodes tend to have parent links that are no stronger than those of shallower nodes; this is a loose tendency, not a rule (Section 3.2.3.1)

RSSI Variance

Statistical variance of RSSI samples within the window, indicating signal stability.

$$\text{RSSI Variance} = \frac{1}{n}\sum_{i=1}^{n}(RSSI_i - \bar{}$$

Equation 4.10

Interpretation:

- Low variance → Stable link conditions
- High variance → Environmental interference, mobility, or topology instability

RSSI Stability Duration

Longest continuous interval within the window where RSSI remains within ±3 dBm of the window mean.

$$\text{RSSI Stability Duration} = \max(\text{intervals with } |RSSI_i - \quad \textbf{Equation 4.11}$$

Interpretation:

- Long duration → Stable signal conditions
- Short duration → Fluctuating signal, potentially due to interference or parent switching

4.2.4.2.5. Cross-Layer Inconsistency Features

RSSI–Hop Inconsistency

Deviation between observed RSSI and the value expected based on the node's reported layer. This feature captures physical-logical topology mismatches characteristic of wormhole distortion.

The expected RSSI for a given layer is not a fixed global table; instead, it is derived empirically from the baseline windows of the same experimental run. For each stable baseline segment, the median RSSI per layer is computed. During distortion windows, the observed RSSI is compared against the baseline median for the reported layer:

$$\text{RSSI-Hop Diff} = |\text{Observed RSSI} - \text{Expected_RS} \quad \textbf{Equation 4.12}$$

Where

$$\text{Expected_RSSI(layer)} = \text{Median}\{RSSI \mid \text{layer, baseli} \quad \textbf{Equation 4.13}$$

This approach makes the metric environment-specific and robust to testbed variations.

Interpretation:

- Small diff (< 5 dB) → RSSI consistent with reported layer
- Large diff (> 10 dB) → Potential physical-logical mismatch; node may be receiving packets from distant sources with unusually strong signal (wormhole effect)

Latency–Hop Inconsistency

Ratio of end-to-end latency to hop count, indicating whether path length matches expected delay.

$$\text{Latency-Hop Ratio} = \frac{\text{Mean RTT (ms}}{\text{Layer}} \quad \textbf{Equation 4.14}$$

where the delay is the relative one-way delay of each probe to the root: its arrival time minus its send timestamp, minus the smallest such value for that sender in the run, which removes the constant offset between the two unsynchronised clocks. The probes have no response leg, so no round trip exists (D-2).

Interpretation:

- Consistent ratio → Normal multi-hop forwarding (measured baseline medians are about 2 to 7 ms per hop)
- Abnormally low ratio → Path appears shorter than reported layer (potential shortcut effect)
- Abnormally high ratio → Congestion or indirect routing

Forwarding Consistency Score

Deviation of the forwarding ratio from the ideal value of 1.0 provides a unified measure of forwarding compliance.

$$\text{Consistency Score} = |1.0 - \text{Forwarding}$$ **Equation 4.15**

Interpretation:

- ≈ 0 → Perfect forwarding compliance (Forwarding Ratio ≈ 1.0)
- > 0 → Increasing deviation from expected behavior
- Closer to 1 → Severe forwarding suppression

4.2.4.2.6. Auxiliary Tunnel Activity Features (Experiment-Level)

These metrics are recorded only on nodes participating in topology distortion experiments. They are not part of the standard PHY/MAC/Network layer set, but are included to document manipulation intensity.

Tunnel Intensity

Combined rate of tunnel activity, indicating the intensity of the wormhole-inspired manipulation.

$$\text{Tunnel Intensity} = \frac{\Delta tunnel_tx_count + \Delta tunnel_}{\text{Window Duration}}$$ **Equation 4.16**

where:

- Δtunnel_tx_count = tunnel transmissions within the window
- Δtunnel_rx_count = tunnel receptions within the window
- Window Duration = 1 second (in this study)

Interpretation:

- 0 → No tunnel activity (baseline or tunnel inactive)
- > 0 → Tunnel active; higher values indicate more frequent relaying

Tunnel Bytes Transferred

Total bytes transmitted through the tunnel per window, indicating data volume.

$$\text{Tunnel Bytes} = \Delta tunnel_bytes_tx + \Delta tunnel$$ **Equation 4.17**

Interpretation:

- Correlates with manipulation intensity; higher values may indicate a more significant shortcut

Tunnel Latency

The gap between the arrival times of the two copies of one duplicated probe at the root (D-3). Both copies carry the same sender timestamp and arrive on the same root clock, so the clock offset cancels exactly. The tunnel is a one-way write and no echo message is sent.

Interpretation:

- Consistent low latency → Tunnel operating as designed
- Variable latency → May affect shortcut effectiveness

4.2.4.3. Feature Summary

The complete feature set per node per window comprises the following 16 attributes, categorized by the layer they primarily represent:

Table 4.17 Feature Summary

Feature	Layer	Behavioral Dimension
Forwarding Ratio	Network	Forwarding compliance
Ingress-Egress Delta	Network	Packet absorption
Retry Rate (application counters)	Application	Link stress (dataset column only)
MacRetryRate	MAC	Link stress (sniffer capture)
Packet Delivery Ratio	Network	End-to-end reliability

Parent Switch Rate	Network	Topology stability
Layer Change Count	Network	Topology stability
Hop Stability Duration	Network	Topology stability
RSSI Mean	PHY	Signal strength
RSSI Variance	PHY	Signal stability
RSSI Stability Duration	PHY	Signal stability
RSSI-Hop Inconsistency	Cross-layer	Physical-logical mismatch
Latency-Hop Inconsistency	Cross-layer	Path length mismatch
Forwarding Consistency Score	Cross-layer	Forwarding deviation
Tunnel Intensity	Auxiliary	Manipulation intensity
Tunnel Bytes	Auxiliary	Manipulation volume
Tunnel Latency	Auxiliary	Arrival gap between the two copies of a probe

These features form the input space for clustering analysis, capturing the cross-layer behavioral signatures of normal operation, forwarding suppression, and topology distortion.

4.2.4.4. Dataset Record Structure (Windowed Per-Node Schema)

Dataset Scope and Observation Granularity

To clarify the structure of the dataset used in this study, telemetry observations are collected from individual ESP32 nodes and subsequently merged into a unified dataset representing the overall experimental mesh network environment. Rather than representing the network as a single aggregated measurement, the dataset is constructed at the node-level observation granularity, where each record corresponds to the behavior of a specific node during a defined observation window.

During each experimental phase, all nodes participating in the ESP-WiFi-Mesh testbed generate telemetry logs containing cross-layer indicators related to routing behavior, link conditions, packet forwarding activity, and topology participation. These logs are retrieved after each experiment and synchronized using timestamp alignment to compensate for asynchronous device clocks. Once synchronized, logs from all nodes are merged into a single dataset while preserving the identity of each node and the corresponding observation window.

Each dataset entry, therefore, represents a node–window observation, consisting of:

- Node identifier
- Observation window or timestamp interval
- Experimental phase label (baseline, blackhole attack, or wormhole attack)
- Extracted behavioral features derived from telemetry metrics

After preprocessing and feature computation, each one-second observation window produces one record per node. The complete dataset is structured as a flat table, where rows correspond to node–window observations and columns contain the features defined in Section 4.2.4.2, together with metadata and experimental labels. *Table 4.18* summarizes the dataset schema.
This design preserves node-level behavioral characteristics, which are essential for identifying routing anomalies. For example, a blackhole attack typically manifests as a decrease in forwarding activity at the attacker node, while the nodes behind it lose delivery to the root. If the dataset were aggregated at the network level, these localized behavioral changes would be obscured.

The final dataset combines observations from all experimental runs and all topology configurations (star, tree, linear chain, and partial mesh) into a single unified table. Additional metadata fields, such as attack, topology, location, scenario, run_repeat and run_id, are included to allow researchers to filter or stratify the dataset during analysis. For instance, analyses may be restricted to a specific topology to examine localized attack behavior, or the entire dataset may be used to evaluate clustering performance across multiple network configurations.

Table 4.18 Windowed Per-Node Dataset Schema

Column	Description
window_start	Start timestamp of the one-second aggregation window
node_id	Unique node identifier
node_role	Node role as logged (root, child, blackhole, wormhole_a, wormhole_b); whether a child is a victim is derived per run from its position
RSSI_mean	Average RSSI within window (dBm)
RSSI_var	RSSI variance
RSSI_stability	Longest stable RSSI interval (seconds)
RetryRate	Share of send attempts that failed (application counters, not MAC; D-15)
PDR	Packet delivery ratio (every child node that sent probes in the window)
ForwardingRatio	Transit packet forwarding ratio
IngressEgressDelta	Packet absorption count
ConsistencyScore	Forwarding deviation metric
ParentSwitchRate	Parent changes per second

LayerChangeCount	Number of layer transitions
HopStabilityDuration	Longest stable (layer, parent) interval
RSSI_Hop_Diff	Physical-logical mismatch (dB)
LatencyHopRatio	Latency per hop (ms)
TunnelIntensity*	Tunnel messages per second
TunnelBytes*	Total tunnel bytes transferred
TunnelLatency*	Gap between the two arrivals of a duplicated probe at the root (ms)
Label	Ground-truth phase (0,1,2)

*Tunnel features are defined only in wormhole runs. TunnelIntensity and TunnelBytes are computed for the two tunnel endpoints, and TunnelLatency for the node whose probes were duplicated. Elsewhere these fields are empty or close to zero. †MacRetryRate and MacFramesHeard (columns of the table not listed above): sniffer runs only, and blackhole runs only in the current analysis; empty where the sniffer did not cover the node.

Clustering analysis is performed on these windowed records; each window is treated as an independent behavioral observation. No time-series or sequential models are applied, ensuring that clustering results reflect feature-space separation rather than temporal dependencies.

4.2.4.5. Feature Engineering Design Principles

The feature set defined in this section follows four methodological principles that guide selection, computation, and documentation:

- **Observability:** All features are derived from standard ESP-IDF APIs (*esp_wifi_sta_get_rssi()*, mesh events), the testbed's own application-layer probe and

relay counters (Section 4.2.3), and, for MacRetryRate, the passive sniffer. No change to the mesh protocol is required.

- **Low Overhead:** Feature computation is performed offline during dataset preprocessing; on-node logging captures only raw counters and timestamps. This ensures that the logging process does not interfere with mesh operations or manipulation effects.
- **Relationship Preservation:** Features are designed to preserve cross-layer relationships (e.g., receive–forward consistency, RSSI-hop correlation) that theoretical analysis identifies as attack-sensitive. They encode structural expectations rather than attack-specific thresholds.
- Reproducibility: All derived metrics are explicitly defined with fixed window durations (1 second), delta conversion rules, and expected value baselines computed empirically from stable windows. This ensures that the feature extraction process can be replicated exactly by other researchers.

4.2.4.6. Feature Selection and Dimensionality Analysis

While the feature set defined in **Section 4.2.4.3** captures a comprehensive range of cross-layer behaviors, not all features may contribute equally to distinguishing between normal and malicious network states. To address the scenario where a single feature might dominate or where features are redundant, a feature selection and dimensionality analysis phase is integrated into the preprocessing workflow. The primary goals are to identify and potentially remove features that are highly correlated or have low variance, ensuring the subsequent clustering analysis is robust and not biased by irrelevant data.

First, a low-variance filter will be applied. Features that exhibit near-constant values across the vast majority of observation windows (e.g., >90% of samples have the same value) provide little discriminative power and can distort distance-based clustering. Such features will be flagged for removal. For example, if RSSI_Hop_Diff is zero for nearly all baseline windows, its presence might add noise rather than signal.

Second, a correlation analysis will be performed to identify pairs of features that are highly collinear (e.g., Pearson correlation coefficient $|r| > 0.85$). Highly correlated features can over-emphasize a single underlying behavioral characteristic. In such cases, one of the features from the pair may be considered for removal to reduce redundancy. For instance, Forwarding Ratio and Forwarding

Consistency Score are mathematically related; if they are found to be perfectly correlated, one might be dropped.

Third, to complement the clustering algorithms, Principal Component Analysis (PCA), as introduced by Jolliffe [52], will be used as an exploratory tool. By projecting the high-dimensional feature vectors onto the principal components that capture the most variance, we can visualize the data's structure and assess its inherent clusterability. More importantly, analyzing the component loadings will reveal which original features contribute most heavily to the overall variance in the dataset. A feature that consistently shows high loadings across the first two or three principal components is empirically more relevant to the overall behavioral structure of the network.

This feature selection process is not intended to pre-define a rigid subset of features before clustering. Instead, it serves as a diagnostic step to ensure the dataset's integrity. Clustering will be performed on both the full feature set and the reduced, non-redundant feature set to compare the stability and interpretability of the results. This approach directly addresses the potential pitfall of relying on a single, dominant indicator and ensures that the discovered behavioral patterns are driven by a confluence of cross-layer evidence.

4.2.5. Data Clustering

4.2.5.1. Clustering Algorithms

Five clustering algorithms are applied to the same standardized feature vectors, so that the groupings they find can be compared: DBSCAN as the primary method, K-Means as a comparative baseline, agglomerative hierarchical clustering with Ward linkage, a Gaussian mixture model (GMM) and spectral clustering. Each is described below, with the figures that illustrate it.

4.2.5.1.1. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is selected as the primary clustering method due to its ability to identify density-connected regions in feature space without requiring a predefined number of clusters. This property is particularly appropriate for exploratory behavioral analysis, where the true number of underlying behavioral states is not assumed a priori.

DBSCAN offers the following advantages:

- **No predefined number of clusters:** The algorithm determines the number of clusters based on the density structure.
- **Noise detection capability:** Points that do not belong to any dense region are labeled as noise, which may correspond to transitional or unstable behavioral windows.

- **Arbitrary cluster shapes:** Unlike centroid-based methods, DBSCAN does not assume spherical clusters.
- **Suitability for anomaly discovery:** Normal operation is expected to form dense regions, while attack behaviors may either form separate dense clusters or appear as sparse anomalies.

Table 4.19 DBSCAN Parameter Selection

Parameter	Description	Selection Method
ε (eps)	Maximum distance between two samples to be considered neighbors	Determined using the k-distance graph (k = min_samples)
min_samples	Minimum number of points to form a dense region	Evaluated within the range D + 1 to 2D, where D is feature dimensionality

To determine ε, the distances to each point's k-th nearest neighbor (where k = min_samples) are computed and sorted in ascending order. The resulting k-distance graph typically exhibits a knee (elbow) point separating dense regions from sparse noise. The value of ε corresponding to this knee is selected as the optimal threshold.

Final parameter choices are validated using internal cluster evaluation metrics (silhouette score and Davies–Bouldin index) to ensure structural stability.

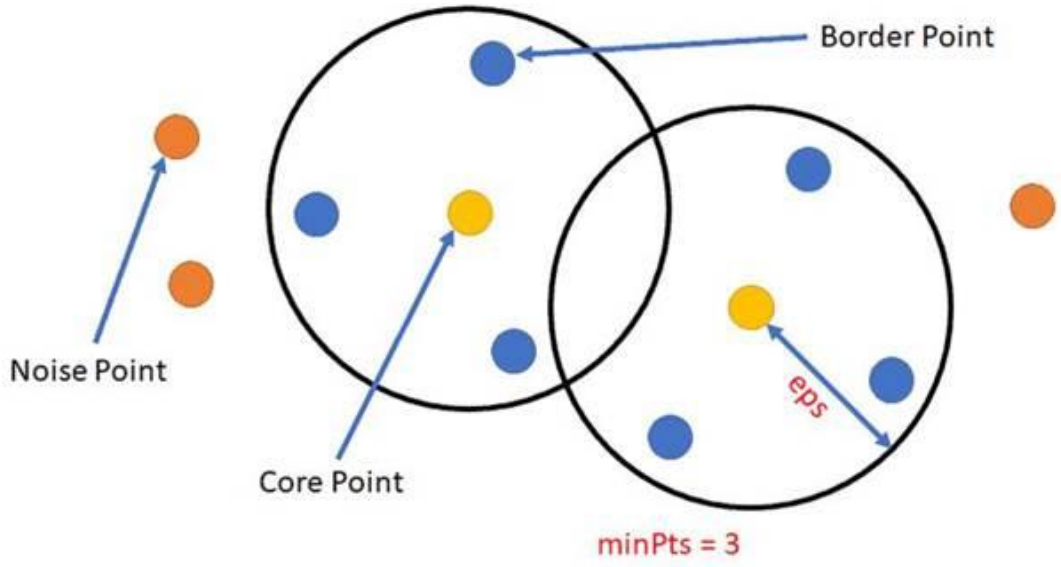


Figure 4.29 Core, Border, and Noise Points in DBSCAN

Illustrates how DBSCAN classifies points into **core points, border points, and noise points** using the ε-neighborhood radius and the minimum number of points (*minPts*). In the diagram, the **two black circles represent the ε-neighborhoods**, which define the radius within which neighboring points are counted. The parameter **minPts = 3**, shown in the figure, indicates the minimum number of neighboring points required for a point to be considered part of a dense region.

The elements shown in the figure correspond to the following DBSCAN point classifications:

- **Core Points** – The **yellow points located at the center of each ε circle** represent core points. These points have at least *minPts* neighboring points within their ε-neighborhoods. Because they lie in dense regions of the dataset, core points serve as the starting points for cluster formation.

- **Border Points** – The **blue points located inside the ε circles but not at the center** represent border points. These points fall within the ε-neighborhood of a core point but do not themselves have enough neighbors to satisfy the *minPts* condition. Border points are therefore assigned to a cluster, but cannot expand the cluster further.

- **Noise Points** – The **orange points located outside the ε circles** represent noise points. These points do not fall within the ε-neighborhood of any core point and, therefore, are not assigned to any cluster. In DBSCAN, such points are treated as outliers.

The **overlapping ε circles** in the figure illustrate how dense regions can connect clusters through shared neighborhoods. By distinguishing between core, border, and noise points, DBSCAN is able to identify clusters of arbitrary shapes while simultaneously detecting outliers in sparse regions of the dataset (Shameer, n.d.).

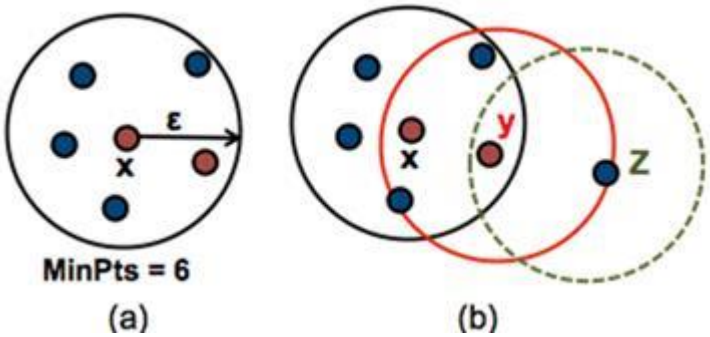


Figure 4.30 DBSCAN Cluster Formation with Noise

This figure illustrates how clusters are formed in the DBSCAN algorithm using the ε-neighborhood radius and the minimum number of points (*MinPts*). The diagram is divided

into two parts, labeled **(a)** and **(b)**, which demonstrate how DBSCAN identifies dense regions and expands clusters through density reachability.

In **Figure 4.30**, the black circle represents the **ε-neighborhood** around the central point. The surrounding points inside the circle represent neighboring data points. Since the number of points within the ε radius satisfies the required **MinPts = 6**, the central point is classified as a **core point**. Core points indicate regions of high density and serve as starting points for cluster formation in the DBSCAN algorithm (Soni, n.d.).

In **Figure 4.30**, the diagram shows three overlapping ε-neighborhoods corresponding to points **X, Y, and Z**, illustrating how clusters expand through density relationships:

- **Point X** lies inside the dense region and satisfies the minimum density requirement, making it a **core point**. Its ε-neighborhood is represented by the black circle containing several nearby points.

- **Point Y** lies within the ε-neighborhood of point X and also has sufficient neighboring points inside its red ε circle. As a result, Y is also considered a **core point**, allowing the cluster to expand further through density reachability.

- **Point Z** lies within the ε-neighborhood of point Y but does not contain enough neighboring points to satisfy the MinPts requirement. Therefore, Z is classified as a **border point** and becomes part of the cluster without initiating further cluster expansion.

The **overlapping circles** in the figure illustrate how clusters grow by connecting core points whose ε-neighborhoods intersect. Points that are reachable through chains of core points become part of the same cluster, while points located outside these dense regions would be classified as **noise**.

This density-based expansion allows DBSCAN to detect clusters of arbitrary shapes while distinguishing sparse points that do not belong to any cluster (Soni, n.d.).

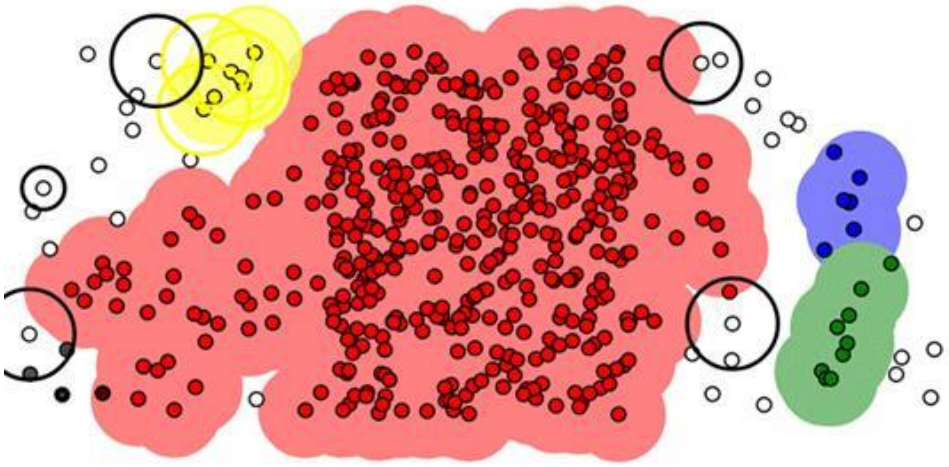


Figure 4.31 DBSCAN Density-Based Cluster Detection

The DBSCAN algorithm detects clusters by identifying **dense regions of data points** and separating them from sparse areas. In the visualization, clusters are represented by **colored shaded regions**, while points located outside dense areas remain unclustered and may be classified as noise. The black circles shown around certain points represent **ε-neighborhoods**, which define the radius used to measure local density.

The elements visible in the figure correspond to the following DBSCAN clustering concepts:

- **Dense Cluster Regions** – The **large red shaded region** in the center of the figure represents a dense cluster containing many closely spaced data points. Because the number of neighboring points within the ε-neighborhood satisfies the required *minPts* threshold, DBSCAN identifies this area as a cluster.

- **Smaller Cluster Groups** – On the right side of the figure, the **blue and green shaded regions** represent smaller clusters that also meet the density requirement. These clusters form when groups of points satisfy the ε and *minPts* conditions within localized regions.

- **Core Points and ε-Neighborhoods** – The **black circles surrounding some points** indicate the ε-neighborhood used by DBSCAN to evaluate point density. Points inside these circles represent neighboring data points that contribute to determining whether a point qualifies as a core point.

- **Border Points** – Points located along the **outer edges of the shaded cluster regions** represent border points. These points fall within the ε-neighborhood of core points but do not contain enough neighbors to satisfy the *minPts* requirement themselves.

- **Noise or Outlier Points** – The **isolated white or gray points scattered outside the colored regions** represent noise points. These points do not fall within the ε-neighborhood of any core point and, therefore, are not assigned to any cluster.

The figure demonstrates how DBSCAN forms clusters by expanding from dense regions through chains of density-reachable points while leaving sparse points unclustered. This density-based approach enables the algorithm to detect clusters of irregular shapes and varying sizes without requiring a predefined number of clusters (UpGrad, n.d.).

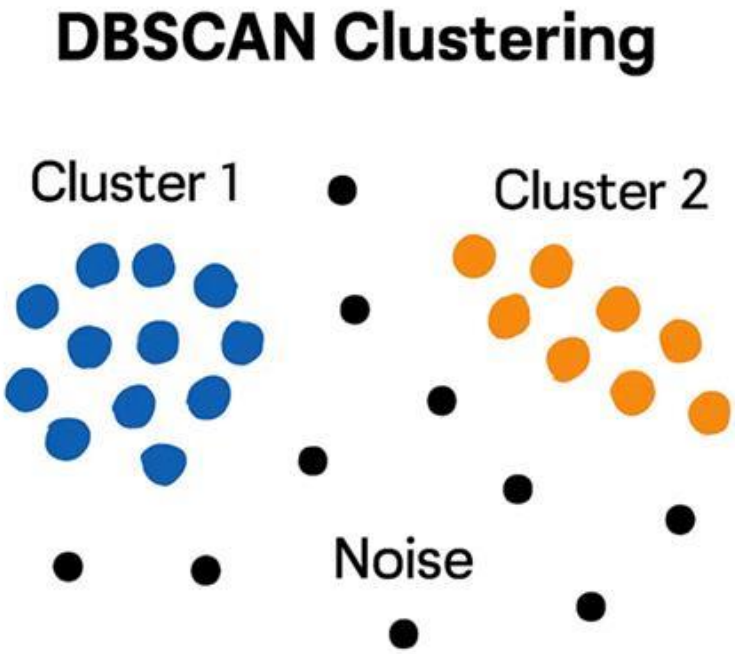


Figure 4.32 Density Reachability in DBSCAN

The concept of **density reachability in the DBSCAN clustering algorithm**, showing how clusters are formed through chains of density-connected points while sparse points remain classified as noise. In the diagram, the data points are divided into two dense groups labeled **Cluster 1** and **Cluster 2**, while scattered black points represent noise or outlier observations.

The elements shown in the figure correspond to the following DBSCAN clustering relationships:

- **Cluster 1 (Blue Points)** – The group of **blue points on the left side** represents a dense region of data points that are close enough to satisfy the ε-neighborhood and *minPts* requirements. Because these points are density-connected, DBSCAN groups them together to form a cluster.

- **Cluster 2 (Orange Points)** – The **orange points on the right side** form another dense region that satisfies the density conditions required for cluster formation. These points are connected through chains of density-reachable points, forming a second cluster separate from Cluster 1.

- **Density Reachability** – Points within each cluster are connected through neighboring points that satisfy the density requirement. A point is considered **density-reachable** from another point if it can be reached through a sequence of neighboring core points within the ε-neighborhood.

- **Noise Points (Black Points)** – The **scattered black points located between and around the clusters** represent noise or outliers. These points do not fall within the ε-neighborhood of any dense region and, therefore, are not assigned to any cluster.

This figure demonstrates how DBSCAN forms clusters based on **density connectivity rather than distance to a centroid**, allowing the algorithm to identify clusters of arbitrary shapes while effectively separating sparse data points as noise (STHDA, n.d.).

4.2.5.1.2. K-Means (Comparative Baseline)

K-Means clustering is applied as a centroid-based comparative baseline to evaluate whether the extracted cross-layer behavioral features form separable groupings under a partition-based clustering model. Unlike DBSCAN, which detects density-connected regions and identifies noise, K-Means partitions the dataset into k compact clusters by minimizing within-cluster variance (WCSS).

Clustering is performed on the standardized feature vectors aggregated per node per one-second window, with the same inputs as for the other algorithms (Section 4.2.5.1). Separate experiments on the wormhole runs may add the tunnel features to assess separability that rests on explicit manipulation indicators.

Number of clusters selection. The optimal number of clusters is determined using two complementary methods:

- Elbow Method: Plot of within-cluster sum of squares (WCSS) against increasing k; the elbow point indicates diminishing returns.
- Silhouette Analysis: Average silhouette width across all points for each k, with higher values indicating better cluster separation.

For this study, k values from 2 to 8 are evaluated. The lower bound ($k = 2$) reflects the minimum plausible behavioral separation (normal vs. anomalous), while higher values allow for potential differentiation between forwarding suppression, topology distortion, and

transitional states. The final value of k is selected based on agreement between elbow behavior, silhouette score peaks, and cluster interpretability.

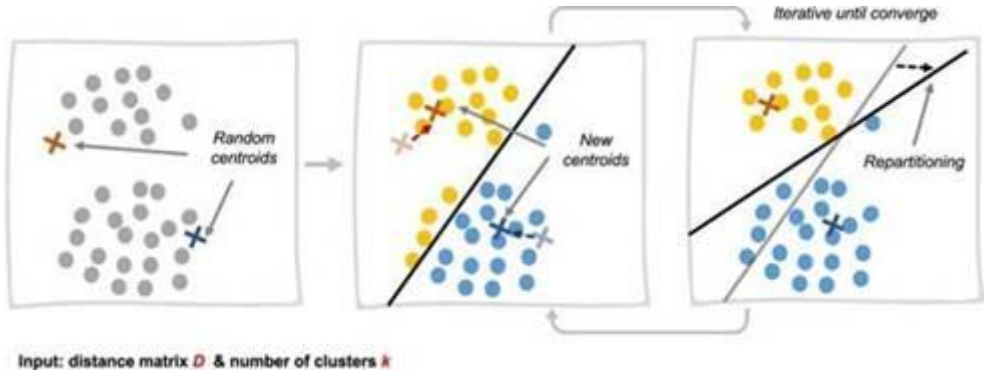


Figure 4.33 K-Means Iterative Cluster Formation in Three Stages

The iterative process of cluster formation in the **K-Means clustering algorithm**. The diagram presents three stages showing how clusters are formed and refined through repeated centroid updates until convergence is achieved. Each panel represents a step in the clustering process, demonstrating how data points are assigned to clusters based on their distance to cluster centroids.

The elements shown in the figure correspond to the following steps of the K-Means algorithm:

- **Initial Centroid Selection (Left Panel)** – The first panel shows a dataset of scattered points along with **randomly initialized centroids**, represented by cross symbols. These centroids serve as the initial centers of the clusters before the algorithm begins assigning data points.

- **Cluster Assignment and Repartitioning (Middle Panel)** – In the second panel, the algorithm assigns each data point to the **nearest centroid** based on distance. The black boundary line indicates the decision boundary that separates the clusters. Points closer to one centroid are assigned to that cluster, forming preliminary cluster groups.

- **Centroid Update and Iteration (Right Panel)** – In the final panel, the centroids are **recomputed based on the mean position of the points assigned to each cluster**. After updating the centroids, the algorithm reassigns data points and recalculates centroids repeatedly. This iterative process continues until the centroids stabilize and the cluster assignments no longer change significantly.

The arrows between panels indicate the **repetition of the assignment and centroid update steps**, highlighting the iterative nature of the K-Means algorithm. Through this process, clusters gradually form as centroids move toward the centers of their respective groups of data points.

This visualization demonstrates how K-Means partitions a dataset into clusters by minimizing the distance between data points and their assigned centroids during each iteration (Bishop, 2006).

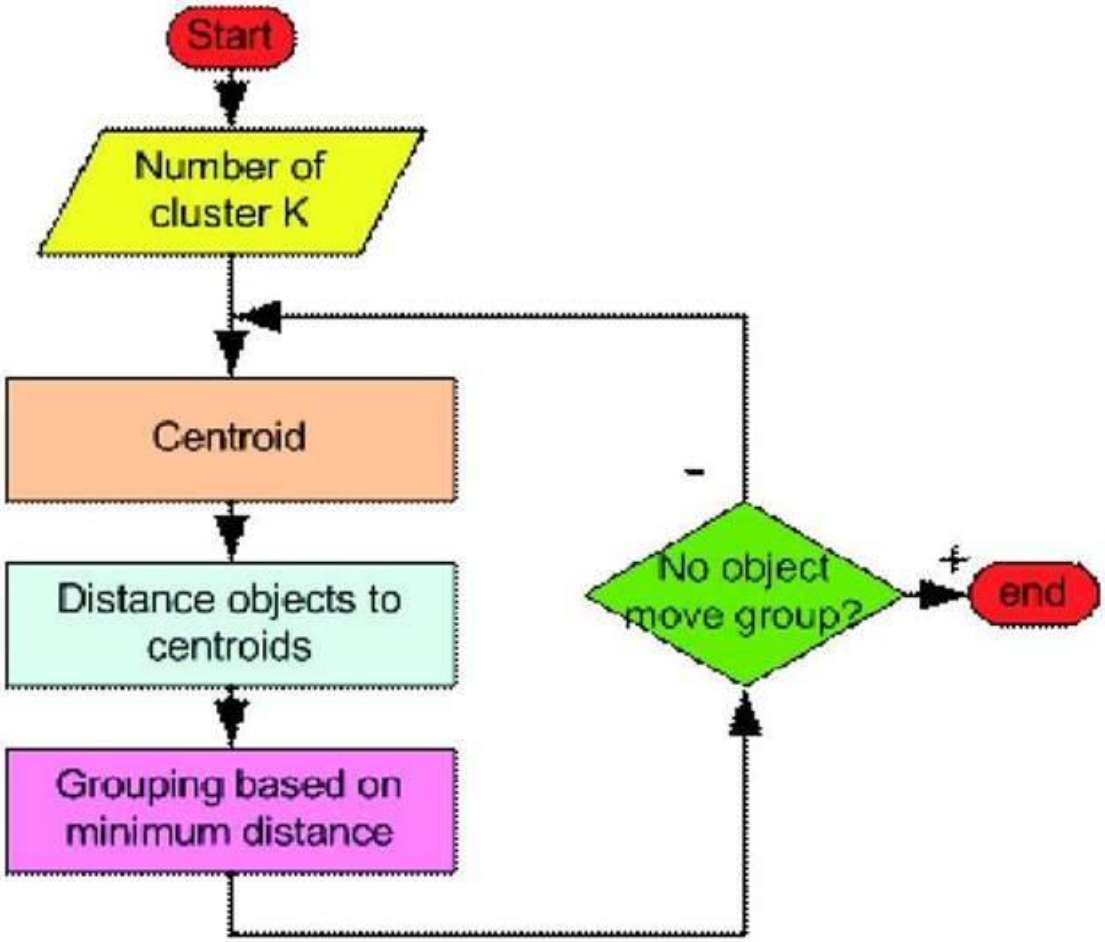


Figure 4.34 K-Means Algorithm Flowchart. Adapted from Lloyd (1982).

A flowchart illustrating the step-by-step procedure of the **K-Means clustering algorithm**. The diagram shows how data points are iteratively assigned to clusters based on their distance to cluster centroids until the cluster assignments stabilize. The flowchart begins with initialization and proceeds through repeated cluster assignment and centroid updates until convergence is reached.

The elements shown in the figure correspond to the following steps of the K-Means algorithm:

- **Start and Cluster Initialization** – The process begins at the **Start node**, followed by specifying the **number of clusters K.** This parameter determines how many clusters the dataset will be partitioned into.

- **Centroid Initialization** – The **"Centroid" process box** represents the step where initial centroids are selected. These centroids act as the starting centers of the clusters.

- **Distance Calculation** – The **"Distance objects to centroids" step** calculates the distance between each data point and the centroids. This distance measurement is typically computed using Euclidean distance.

- **Cluster Assignment** – In the **"Grouping based on minimum distance" step**, each data point is assigned to the cluster corresponding to the nearest centroid.

- **Convergence Check** – The **decision diamond labeled "No object move group?"** evaluates whether any data points change cluster assignments during the iteration. If cluster assignments continue to change, the algorithm returns to the centroid update step and repeats the process.

- **End Condition** – When no data points change clusters, indicating that the centroids have stabilized, the algorithm reaches the **End node**, and the final cluster configuration is produced.

The loop shown in the flowchart highlights the **iterative nature of the K-Means algorithm**, where centroid positions and cluster memberships are repeatedly updated until convergence is achieved. This iterative optimization process minimizes the distance between data points and their corresponding cluster centroids (Lloyd, 1982).

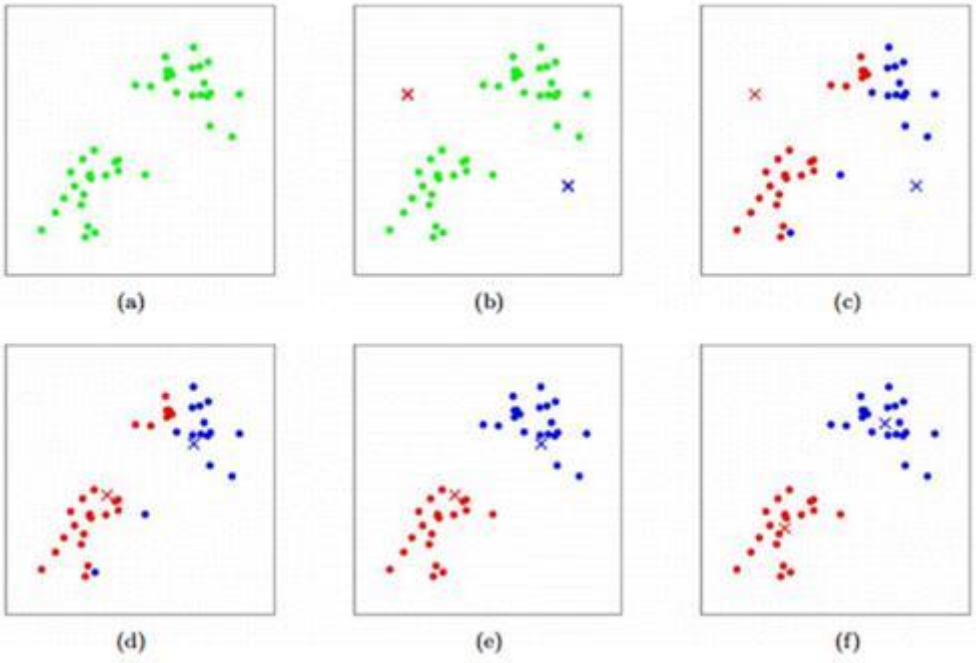


Figure 4.35 Step-by-Step K-Means Cluster Formation in Six Stages. Adapted from Bishop (2006).

The **iterative process of cluster formation in the K-Means clustering algorithm**, showing how clusters gradually form as centroids are updated and data points are reassigned during successive iterations. The six panels labeled **(a) to (f)** represent different stages in the clustering process.

The steps illustrated in the figure correspond to the following stages of the K-Means algorithm:

- **(a) Initial Dataset Distribution** – The first panel shows the original dataset consisting of scattered data points before clustering begins. At this stage, no cluster assignments have been made.

- **(b) Random Centroid Initialization** – In the second panel, two centroids (marked with "×") are randomly placed within the dataset. These centroids serve as the initial cluster centers from which the algorithm begins the clustering process.

- **(c) First Cluster Assignment** – The algorithm assigns each data point to the nearest centroid based on distance. As shown in the figure, points begin to separate into two groups represented by different colors (red and blue).

- **(d) Centroid Recalculation** – After the initial assignment, the centroids are updated by computing the mean position of the data points assigned to each cluster. The new centroid positions are then used for the next iteration.

- **(e) Reassignment of Data Points** – Using the updated centroids, the algorithm recalculates distances and reassigns data points to the closest centroid. This step may change the cluster membership of some points.

- **(f) Convergence of Clusters** – In the final panel, the clusters stabilize as centroid positions stop changing significantly. At this stage, the algorithm converges, and the final cluster structure is obtained.

The figure demonstrates how the K-Means algorithm repeatedly performs **cluster assignment and centroid update steps** until the cluster configuration stabilizes. This iterative process minimizes the within-cluster variance by ensuring that data points remain close to their corresponding centroids (Bishop, 2006).



Figure 4.36 Example of K-Means Clustering

Example of clustering using the **K-Means algorithm**, which partitions a dataset into a predefined number of clusters by minimizing the distance between data points and their corresponding cluster centroids. The visualization shows how data points are grouped into clusters based on similarity in the feature space.

In the figure, each **data point represents an observation plotted in a two-dimensional feature space**, where the horizontal and vertical axes correspond to two features used for clustering. The points form several dense groups that the K-Means algorithm identifies as clusters.

The elements shown in the figure correspond to the following characteristics of the K-Means clustering process:

- **Clustered Data Points** – The scatter plot shows four distinct groups of points represented by different colors. Each color corresponds to a cluster identified by the algorithm, indicating that the observations within the same group share similar feature characteristics.

- **Cluster Centroids** – The **large "×" markers located at the center of each cluster** represent the centroids. These centroids correspond to the mean position

of all data points assigned to each cluster and serve as the reference points for cluster assignment.

- **Cluster Partitioning** – Each data point is assigned to the cluster whose centroid is closest in terms of distance, typically measured using Euclidean distance. This results in the formation of clearly separated groups within the dataset.

- **Distance Minimization** – The K-Means algorithm iteratively updates centroid positions and reassigns data points until the clusters stabilize, minimizing the total within-cluster variance.

The figure demonstrates how K-Means divides a dataset into clusters by grouping nearby observations around central centroid points, producing partitions that represent similar patterns within the data (StrataScratch, n.d.).

4.2.5.1.3. Hierarchical Clustering

Hierarchical clustering is applied as an additional unsupervised clustering approach to analyze the structural relationships between behavioral observations without requiring an explicit assumption about the number of clusters during the clustering process. Unlike partition-based algorithms such as K-Means, hierarchical clustering builds a nested hierarchy of clusters by iteratively merging or splitting data points based on similarity.

For this study, agglomerative hierarchical clustering is employed. In this approach, each observation initially forms its own cluster. Clusters are then progressively merged according to a defined linkage criterion until all observations are grouped into a single hierarchy. The resulting structure is represented through a dendrogram, which visualizes the sequence of cluster merges and the relative distances between clusters.

Clustering is performed on the standardized feature vectors derived from telemetry observations aggregated per node per one-second window, with the same inputs as for the other algorithms (Section 4.2.5.1). An additional run on the wormhole runs may add the tunnel features to observe separability that rests on explicit manipulation indicators.
Linkage method selection. Cluster merging is determined using Ward's linkage criterion, which minimizes the increase in within-cluster variance when clusters are combined. Ward linkage is selected because it produces compact clusters that are comparable to those produced by K-Means, enabling meaningful comparison between partition-based and hierarchical clustering approaches.
The hierarchical clustering procedure is implemented as follows:

1. Each observation begins as an individual cluster.
2. Pairwise distances between clusters are computed using Euclidean distance.

3. The two clusters that produce the smallest increase in total within-cluster variance are merged.

4. Steps 2–3 are repeated until all observations are merged into a hierarchical structure.

Cluster selection. Although hierarchical clustering produces a full cluster hierarchy, the dendrogram is cut at selected distance thresholds to produce cluster partitions for evaluation. Similar to the K-Means baseline experiment, cluster counts between 2 and 8 clusters are evaluated. These values allow the analysis to determine whether hierarchical clustering reveals interpretable behavioral groupings corresponding to normal operation, forwarding suppression, topology distortion, or intermediate behavioral states.

Cluster partitions obtained from the dendrogram are evaluated using the same internal validation metrics used for other clustering methods, including silhouette score and Davies–Bouldin index, allowing direct comparison with results obtained from DBSCAN and K-Means.

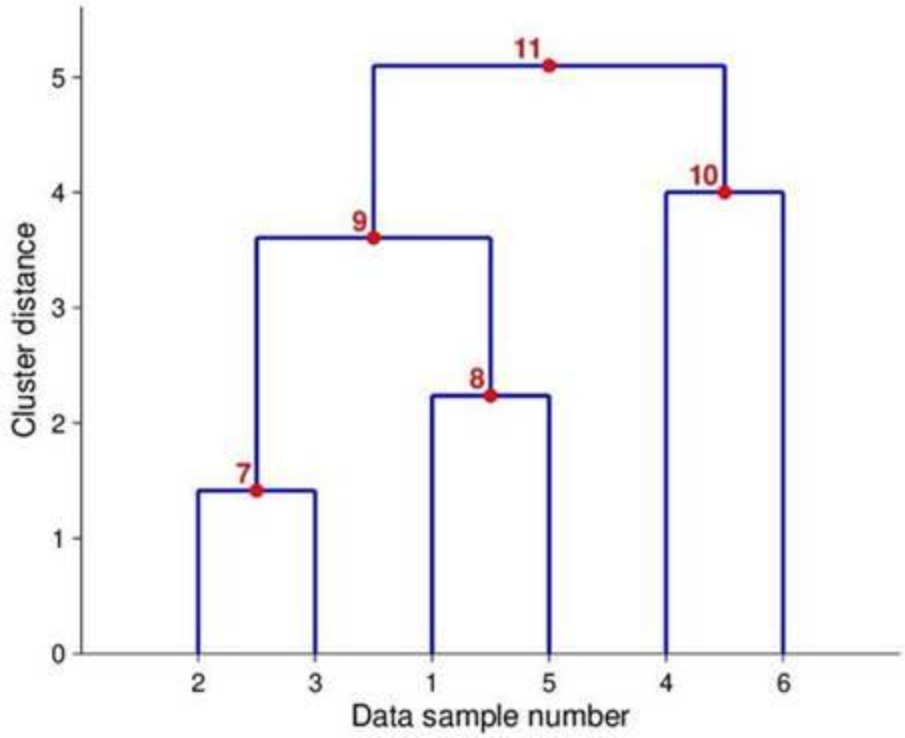


Figure 4.37 Example Dendrogram

Dendrogram used in hierarchical clustering to visualize how data points are grouped into clusters based on their similarity or distance. A dendrogram is a tree-like diagram that illustrates the sequence of cluster merging steps and the relative distances at which clusters are combined.

In the diagram, the **horizontal axis labeled "Data sample number"** represents the individual observations in the dataset. Each sample initially begins as its own cluster at the bottom of the dendrogram. The **vertical axis labeled "Cluster distance"** represents the distance or dissimilarity between clusters, indicating the level at which clusters are merged during the clustering process.

The clustering process illustrated in the figure proceeds as follows:

- **Initial Clusters** – The individual data samples (1, 2, 3, 4, 5, and 6) appear along the bottom of the dendrogram, each representing a separate cluster at the start of the hierarchical clustering process.

- **First Merging Step (Point 7)** – Data samples **2 and 3** are merged to form a cluster at a relatively small distance value, indicating that these observations are highly similar.

- **Second Merging Step (Point 8)** – Data samples **1 and 5** are combined to form another cluster at a slightly higher distance level.

- **Intermediate Cluster Formation (Point 9)** – The cluster formed by **samples 2 and 3** merges with the cluster formed by **samples 1 and 5**, producing a larger cluster that groups four observations.

- **Additional Cluster Formation (Point 10)** – Data samples **4 and 6** merge to form another cluster at a moderate distance level.

- **Final Cluster Merge (Point 11)** – The two larger clusters eventually merge at the highest distance level shown in the dendrogram, forming a single cluster that contains all observations.

The height at which clusters merge represents the **distance threshold between clusters**, allowing researchers to determine the number of clusters by selecting a horizontal cut across the dendrogram. Clusters formed below the chosen threshold are considered separate groups within the dataset.

This dendrogram demonstrates how hierarchical clustering builds clusters progressively by merging the most similar observations first and gradually combining larger clusters at higher distance levels (ResearchGate, n.d.).

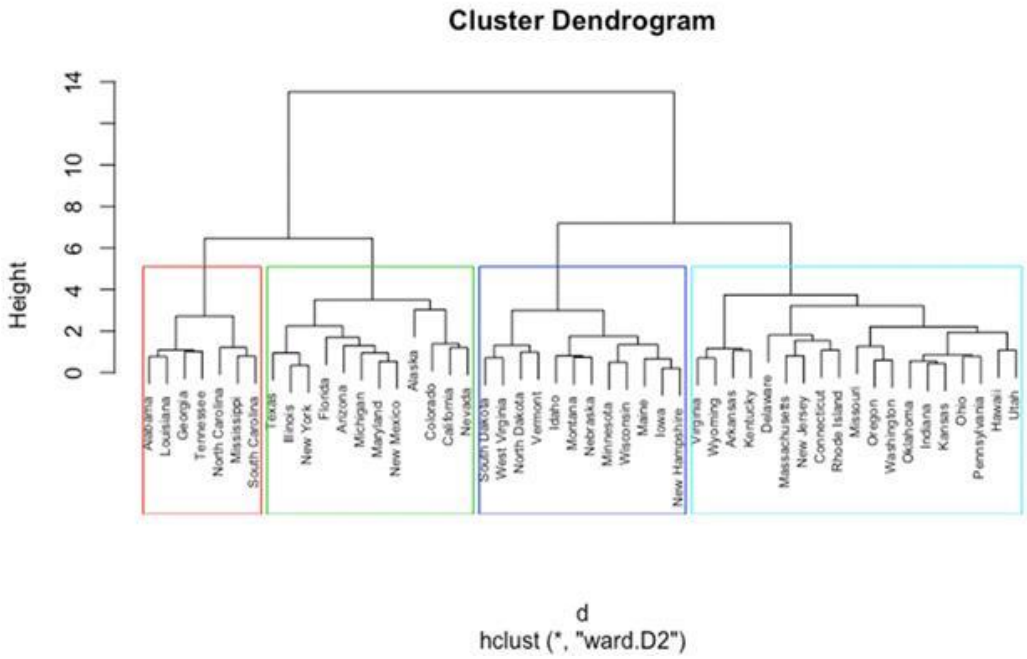


Figure 4.38 Hierarchical Clustering Dendrogram (Ward Method)

A **hierarchical clustering dendrogram generated using Ward's linkage method**, which groups observations by minimizing the variance within clusters during the merging process. The dendrogram visually represents how clusters are formed through successive merging steps based on the similarity between observations.

In the diagram, the **horizontal axis displays the individual observations**, represented by labeled data points positioned at the bottom of the dendrogram. Each observation initially begins as its own cluster. The **vertical axis labeled "Height"** represents the distance or dissimilarity level at which clusters are merged during the hierarchical clustering process.

The structure of the dendrogram illustrates the following clustering behavior:

- **Initial Individual Observations** – At the bottom of the dendrogram, each labeled item begins as a separate cluster. These individual observations are gradually merged with other similar observations as the algorithm proceeds.

- **Cluster Merging Process** – As the height increases along the vertical axis, branches of the dendrogram merge. Each horizontal linkage line represents the point at which two clusters are combined based on their similarity.

- **Ward's Linkage Method** – Ward's method merges clusters in a way that minimizes the total within-cluster variance. This approach results in clusters that are relatively compact and balanced compared to other linkage methods.

- **Cluster Groupings** – The colored rectangular boxes in the figure highlight **four major clusters** identified in the dataset. These colored regions show how the dendrogram can be "cut" at a specific height threshold to determine the final number of clusters.

- **Cluster Hierarchy Representation** – The hierarchical tree structure demonstrates how smaller clusters merge into larger clusters as the distance threshold increases, ultimately forming a single cluster that contains all observations at the top of the dendrogram.

This dendrogram provides a visual representation of the hierarchical relationships between data points and helps determine the appropriate number of clusters by selecting a cutoff level along the height axis. Ward's method is commonly used because it produces clusters with relatively small internal variance and well-balanced groupings (UC Business Analytics R Programming Guide, n.d.).

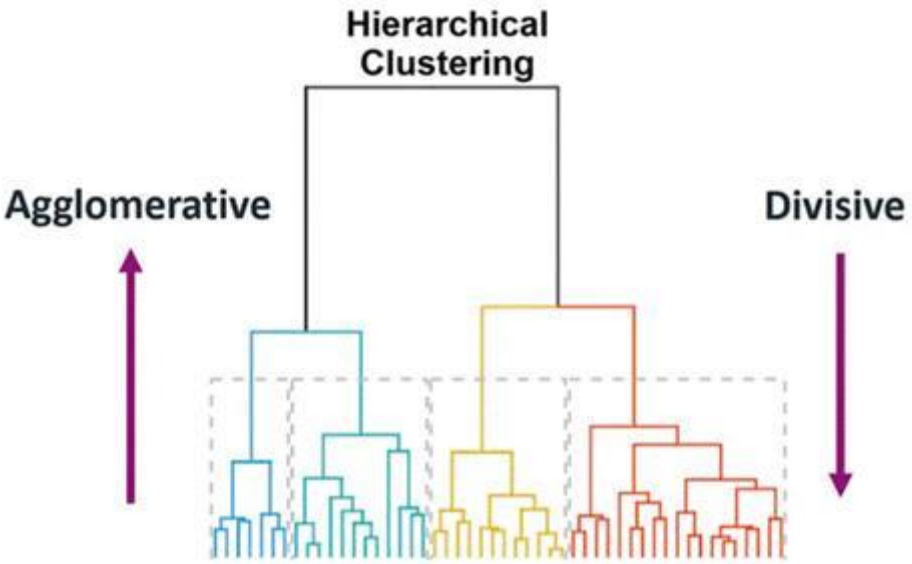


Figure 4.39 Agglomerative vs Divisive Hierarchical Clustering

A comparison between the two main approaches used in **hierarchical clustering: agglomerative clustering and divisive clustering**. The diagram highlights how clusters are

formed either by **merging smaller clusters into larger ones** or by **splitting larger clusters into smaller groups**.

In the center of the figure, the **hierarchical tree structure (dendrogram)** illustrates how data points can be grouped into clusters at different levels of similarity. The colored branches at the bottom represent groups of observations that form clusters as the hierarchical process progresses.

The figure highlights the two clustering strategies as follows:

- **Agglomerative Clustering (Bottom-Up Approach)** – The arrow on the **left side labeled "Agglomerative" pointing upward** indicates that clustering begins with individual data points. Each observation initially forms its own cluster, and the algorithm repeatedly merges the most similar clusters until larger clusters are formed. This bottom-up process continues until all data points are combined into a single cluster.

- **Divisive Clustering (Top-Down Approach)** – The arrow on the **right side labeled "Divisive" pointing downward** represents the opposite strategy. In divisive clustering, the process begins with all observations grouped into one large cluster. The algorithm then recursively splits this cluster into smaller subclusters based on dissimilarity until individual clusters are formed.

- **Hierarchical Tree Representation** – The dendrogram in the center visually represents the hierarchical relationships between clusters. Each branching point in the tree indicates a merging or splitting step, depending on the clustering approach used.

The figure demonstrates how both agglomerative and divisive methods produce hierarchical cluster structures, but they differ in the direction in which clusters are formed. Agglomerative clustering is the more commonly used approach due to its computational efficiency and simpler implementation (Minitab, n.d.).

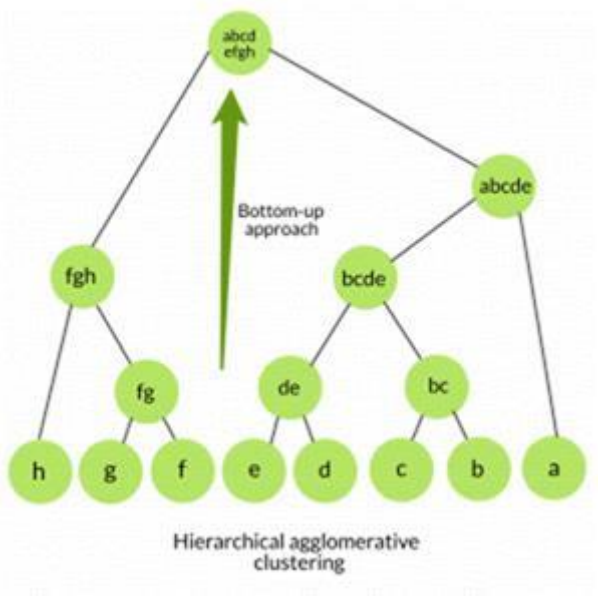


Figure 4.40 Agglomerative Hierarchical Clustering Tree

The structure of **Agglomerative Hierarchical Clustering** is a clustering technique that groups data points through a **bottom-up merging process**. In this approach, each data point initially forms its own cluster, and clusters are progressively merged based on similarity until a hierarchical structure is formed.

In the diagram, the **leaf nodes at the bottom of the tree (labeled a, b, c, d, e, f, g, h)** represent individual data points at the start of the clustering process. Each of these points begins as a separate cluster.

The clustering process then proceeds through the following stages:

- **Initial Cluster Formation** – Individual data points at the bottom level form the smallest clusters. For example, points **b and c merge to form cluster "bc"**, while **d and e merge to form cluster "de."**

- **Intermediate Cluster Merging** – As the algorithm continues, smaller clusters combine to form larger clusters. In the figure, clusters such as **bc and de merge into a larger cluster labeled "bcde."** Similarly, clusters **f and g merge into "fg,"** which then combines with **h** to form a larger cluster.

- **Higher-Level Cluster Formation** – These intermediate clusters are further merged into larger groupings, such as **"abcde"** and **"abcd efgh."** Each level of the tree represents a new stage in the merging process.

- **Bottom-Up Hierarchical Structure** – The **green arrow labeled "Bottom-up approach"** highlights the direction of the clustering process. Starting from individual data points at the bottom, clusters are gradually merged upward until a hierarchical tree structure is formed.

This tree structure, commonly referred to as a **dendrogram**, visually represents the relationships between clusters and shows how clusters combine at different levels of similarity. Agglomerative hierarchical clustering does not require a predefined number of clusters, allowing the final number of clusters to be determined by selecting an appropriate level in the hierarchy (Singh, n.d.).

4.2.5.1.4. Gaussian Mixture Model (GMM)

Gaussian Mixture Models (GMM) are used as a probabilistic clustering approach that models the dataset as a combination of multiple Gaussian distributions in feature space. Unlike hard clustering methods such as K-Means, which assign each observation to a single cluster, GMM performs soft clustering, assigning each observation a probability of belonging to each cluster.

This probabilistic representation allows the model to capture overlapping behavioral patterns that may occur in network telemetry data. In the context of ESP32 mesh network behavior, some observation windows may exhibit characteristics that lie between normal operation and anomalous routing behavior. GMM provides a mechanism for representing these transitional states through probabilistic cluster membership.

The model assumes that the dataset is generated from a mixture of k Gaussian components, where each component represents a latent behavioral state. Each component is characterized by:

- a mean vector
- a covariance matrix
- a mixture weight representing its relative contribution

Parameter estimation is performed using the Expectation–Maximization (EM) algorithm, which iteratively refines cluster parameters until convergence.

Expectation–Maximization procedure. The clustering process proceeds through two alternating steps:

Expectation step (E-step):

The probability that each observation belongs to each Gaussian component is estimated based on the current model parameters.

Maximization step (M-step):

The parameters of the Gaussian components (means, covariances, and mixture weights) are updated to maximize the likelihood of the observed data given the current cluster assignments.

This iterative process continues until the log-likelihood improvement between successive iterations falls below a convergence threshold.

Cluster number selection. To maintain comparability with the K-Means baseline experiment, candidate cluster counts ranging from 2 to 8 components are evaluated. The optimal number of mixture components is determined using both cluster validation metrics and the interpretability of resulting behavioral groupings.

Cluster quality is assessed using the silhouette score and Davies–Bouldin index, while additional inspection of cluster membership probabilities allows identification of observations that exhibit mixed behavioral characteristics. Such transitional patterns may indicate periods where network behavior shifts between normal and anomalous conditions.

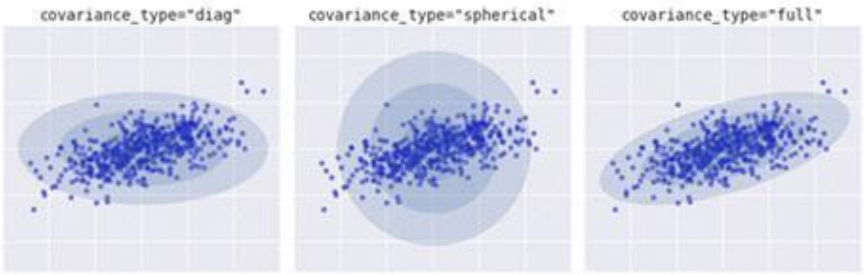


Figure 4.41 Covariance Types in Gaussian Mixture Models

The different **covariance structures used in Gaussian Mixture Models (GMM)** for modeling clusters in a dataset. Gaussian Mixture Models represent clusters as probability distributions rather than fixed boundaries, allowing clusters to take on different shapes depending on the covariance configuration. The shaded ellipses in the figure represent the estimated covariance of each Gaussian component, indicating the spread and orientation of the cluster.

The figure presents three common covariance types used in GMM clustering:

- **Diagonal Covariance (Left Panel – covariance_type = "diag")** – In this configuration, the covariance matrix is restricted to diagonal values, meaning each feature dimension varies independently while correlations between

dimensions are ignored. As shown in the left panel, clusters appear as **axis-aligned ellipses**, where the spread of data differs along each axis but cannot rotate.

- **Spherical Covariance (Middle Panel – covariance_type = "spherical")** – In the spherical covariance structure, each cluster has a single variance value applied equally across all dimensions. This results in clusters that appear as **circular shapes** in the visualization. The middle panel shows clusters that expand uniformly in all directions, assuming equal variance across the feature space.

- **Full Covariance (Right Panel – covariance_type = "full")** – In the full covariance structure, the covariance matrix allows correlations between features. This enables clusters to take **elliptical shapes that can rotate and stretch in any direction**, as illustrated in the right panel. This configuration provides the most flexibility for modeling complex cluster structures.

The comparison shown in the figure demonstrates how different covariance assumptions affect the shape and orientation of clusters identified by Gaussian Mixture Models. Choosing an appropriate covariance type is important for accurately capturing the distribution of data within each cluster (VanderPlas, n.d.).

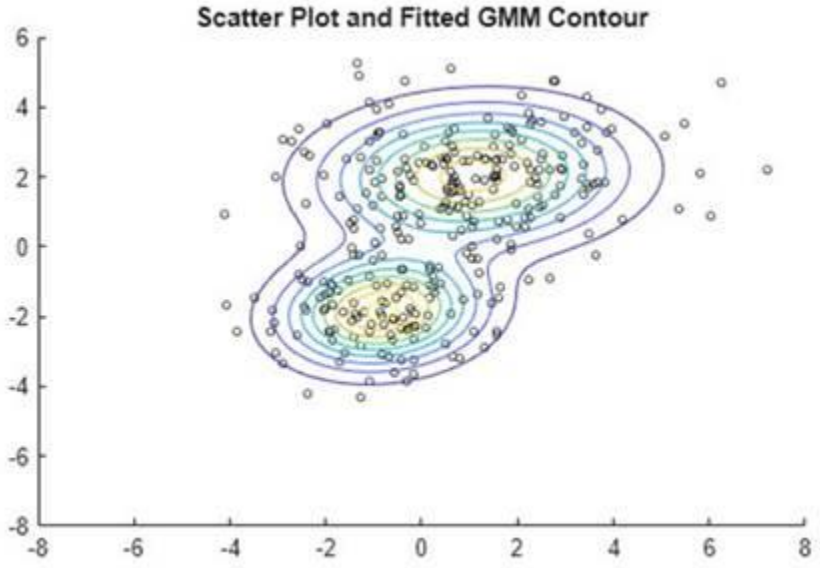


Figure 4.42 GMM Probability Density Contours

The clustering behavior of a **Gaussian Mixture Model (GMM)** using a scatter plot combined with probability density contours. The visualization shows how GMM models cluster as probability distributions rather than assigning rigid cluster boundaries.

In the figure, the **scatter points represent individual data observations** distributed across the feature space. These observations form two main groups of points, indicating the presence of two underlying clusters in the dataset.

The elements shown in the figure correspond to the following features of the Gaussian Mixture Model:

- **Scatter Data Points** – The individual points plotted on the graph represent data samples from the dataset. These points form visible groupings that the GMM algorithm attempts to model using Gaussian distributions.

- **Probability Density Contours** – The **elliptical contour lines surrounding the clusters** represent the probability density levels of the fitted Gaussian distributions. Each contour line indicates regions where the probability density of the cluster remains constant.

- **Cluster Centers** – The **central regions of the contour shapes** indicate the mean location of each Gaussian component. These means act as the centers of the clusters estimated by the GMM algorithm.

- **Elliptical Cluster Shape** – Unlike clustering methods such as K-Means that assume spherical clusters, GMM allows clusters to take **elliptical shapes with varying orientations and spreads**, which better represent complex data distributions.

The figure demonstrates how Gaussian Mixture Models estimate the probability distribution of clusters and visualize cluster boundaries using density contours. This probabilistic approach allows GMM to capture overlapping clusters and complex data structures more effectively than many distance-based clustering methods (MathWorks, n.d.).

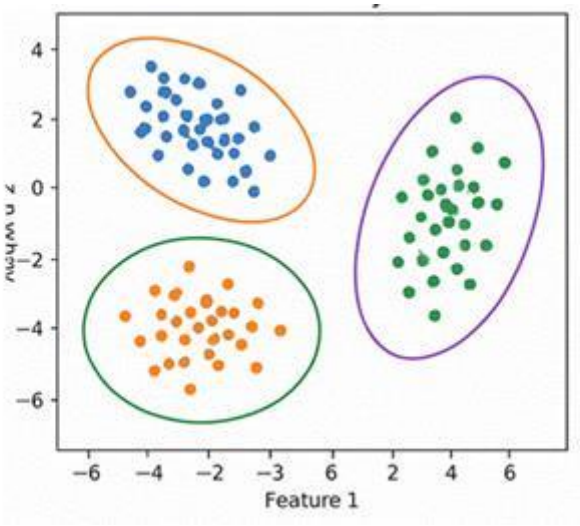


Figure 4.43 Gaussian Mixture Model Clustering Example

An example of clustering using a **Gaussian Mixture Model (GMM)**, where clusters are represented as Gaussian probability distributions rather than fixed geometric boundaries. The visualization shows how GMM identifies clusters in a dataset by estimating the probability distribution of data points within the feature space.

In the figure, the **scatter points represent individual data observations** plotted according to two features labeled *Feature 1* and *Feature 2*. The points form three visible groups that correspond to the clusters detected by the Gaussian Mixture Model.

The elements shown in the figure correspond to the following characteristics of GMM clustering:

- **Clustered Data Points** – The data points form three main groups in the scatter plot. These groups represent observations that share similar characteristics within the feature space.

- **Elliptical Cluster Boundaries** – The **elliptical shapes surrounding each cluster** represent the Gaussian distributions estimated by the model. These ellipses indicate the spread and orientation of the probability density for each cluster.

- **Cluster Centers and Orientation** – The center of each ellipse represents the **mean of the Gaussian component**, while the orientation and shape of the ellipse reflect the covariance structure of the cluster.

- **Flexible Cluster Shapes** – Unlike clustering algorithms such as K-Means that assume spherical clusters, Gaussian Mixture Models allow clusters to take **elliptical shapes with different orientations**, enabling the algorithm to better model complex data distributions.

The figure demonstrates how Gaussian Mixture Models represent clusters probabilistically and capture variations in cluster shape, size, and orientation within the dataset. This flexibility makes GMM particularly useful for datasets where clusters overlap or exhibit non-spherical structures (AI Learning Hub, n.d.).

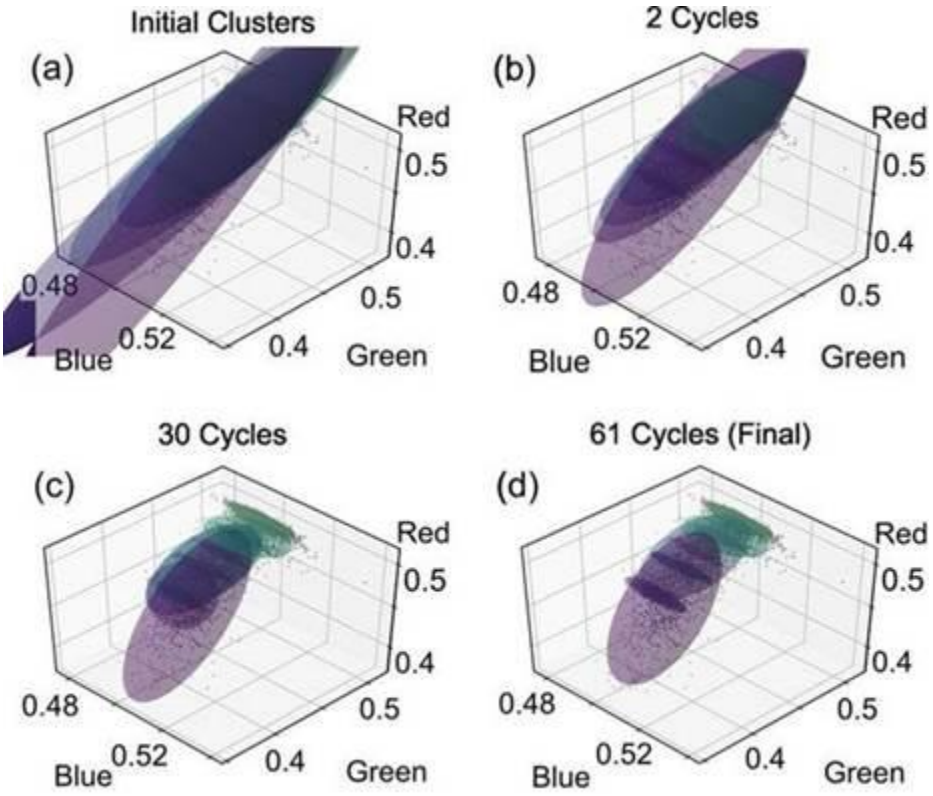


Figure 4.44 Cluster Fitting Using Gaussian Mixture Model

The **cluster fitting process of a Gaussian Mixture Model (GMM)** is the algorithm that iteratively updates its Gaussian components during the Expectation–Maximization (EM) procedure. The visualization shows how the Gaussian distributions gradually adjust their position, orientation, and spread to better represent the structure of the data.

In the figure, each panel represents a stage in the iterative clustering process, where the axes labeled **Red, Green, and Blue** correspond to three feature dimensions used in the dataset. The shaded ellipsoids represent the **Gaussian components**, while the scatter points correspond to the observed data samples.

The panels illustrate the following stages of the GMM fitting process:

- **(a) Initial Clusters** – The first panel shows the initial placement of Gaussian components before the algorithm begins adjusting them. At this stage, the ellipsoids represent preliminary estimates of the cluster distributions and may not yet align well with the underlying data.

- **(b) Early Iterations (2 Cycles)** – After two EM cycles, the Gaussian components begin to shift and rotate to better match the distribution of data points. The algorithm adjusts the parameters of each Gaussian component based on the likelihood of the observed data.

- **(c) Intermediate Iterations (30 Cycles)** – After additional iterations, the Gaussian components more closely align with the dense regions of the dataset. The ellipsoids begin to capture the structure and orientation of the clusters more accurately.

- **(d) Final Convergence (61 Cycles)** – In the final panel, the Gaussian components stabilize as the algorithm converges. At this stage, the ellipsoids closely represent the underlying data distribution, indicating that the model parameters have converged and the clustering process is complete.

The figure demonstrates how the Gaussian Mixture Model iteratively refines cluster parameters through the EM algorithm, adjusting the means, covariances, and mixture weights until the model converges to a stable solution that best fits the observed data (ResearchGate, n.d.).

4.2.5.1.5. Spectral Clustering

Spectral clustering is employed as a graph-based clustering technique capable of detecting complex cluster structures that may not be captured effectively by centroid-based or density-based methods. Unlike K-Means and GMM, which rely primarily on geometric distances in feature space, spectral clustering operates by analyzing the connectivity structure of the dataset represented as a similarity graph.

In this approach, each observation is treated as a node in a graph, and edges between nodes represent similarity relationships between feature vectors. The clustering process is performed by analyzing the eigenstructure of the graph Laplacian derived from this similarity matrix.

Similarity graph construction. To construct the similarity graph, pairwise similarities between standardized feature vectors are computed using a radial basis function (RBF) kernel:

Equation 4.20

$$S_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

where x_i and x_j are the standardized feature vectors of windows *i* and *j*, and σ sets how quickly the similarity falls as the distance grows.

Similarity between observations increases as the Euclidean distance between their feature vectors decreases. This representation allows spectral clustering to capture non-linear relationships in the data that may correspond to subtle behavioral patterns within the mesh network.

Spectral embedding. Once the similarity matrix is constructed, the normalized graph Laplacian is computed. Eigenvalue decomposition is then performed on the Laplacian matrix to obtain a lower-dimensional representation of the data known as the spectral embedding.

This transformation projects the observations into a new space in which cluster structure becomes more separable. After projection, a conventional clustering algorithm (typically K-Means) is applied to the embedded representation to produce the final cluster assignments.

Cluster number selection. As with the previous clustering methods, candidate cluster counts ranging from 2 to 8 clusters are evaluated. This consistent range allows comparison across all clustering algorithms used in the study.

Cluster quality is assessed using the same internal validation metrics applied to other clustering methods, including the silhouette score and Davies–Bouldin index. The results are further examined to determine whether spectral clustering reveals behavioral structures not detected by the other clustering techniques, particularly in cases where clusters exhibit non-spherical or irregular shapes.

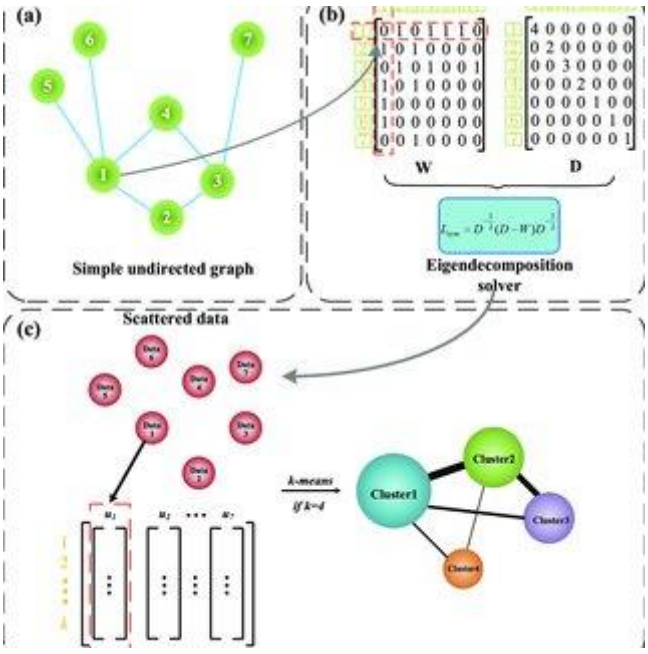


Figure 4.45 Spectral Clustering Algorithm Overview

Overview of the Spectral Clustering algorithm, which groups data points by analyzing the structure of a similarity graph. Spectral clustering transforms the dataset into a graph

representation, computes eigenvectors from the graph Laplacian matrix, and then performs clustering in the transformed feature space.

The diagram is divided into three main stages labeled **(a), (b), and (c)**, each representing a step in the spectral clustering workflow.

- **(a) Graph Construction** – The upper-left section shows a **simple undirected graph** where nodes represent individual data points, and edges represent similarity relationships between points. This graph representation captures how strongly each data point is connected to others in the dataset.

- **(b) Matrix Representation and Eigen Decomposition** – The upper-right section illustrates the conversion of the graph into matrix form. The **weight matrix (W)** represents the similarity between nodes, while the **degree matrix (D)** summarizes the total connection strength for each node. These matrices are used to compute the **graph Laplacian**, and eigen decomposition is applied to extract eigenvectors that capture the structural properties of the graph.

- **(c) Feature Embedding and Clustering** – In the lower section of the figure, the eigenvectors obtained from the spectral decomposition form a new feature space. The data points are projected into this space and then clustered using a standard clustering algorithm, such as **K-Means**, to produce the final cluster groups.

The figure demonstrates how spectral clustering combines **graph theory and linear algebra** to identify clusters based on connectivity patterns rather than simple distance measures. This approach is particularly effective for datasets where clusters have complex or non-convex shapes (ResearchGate, n.d.).

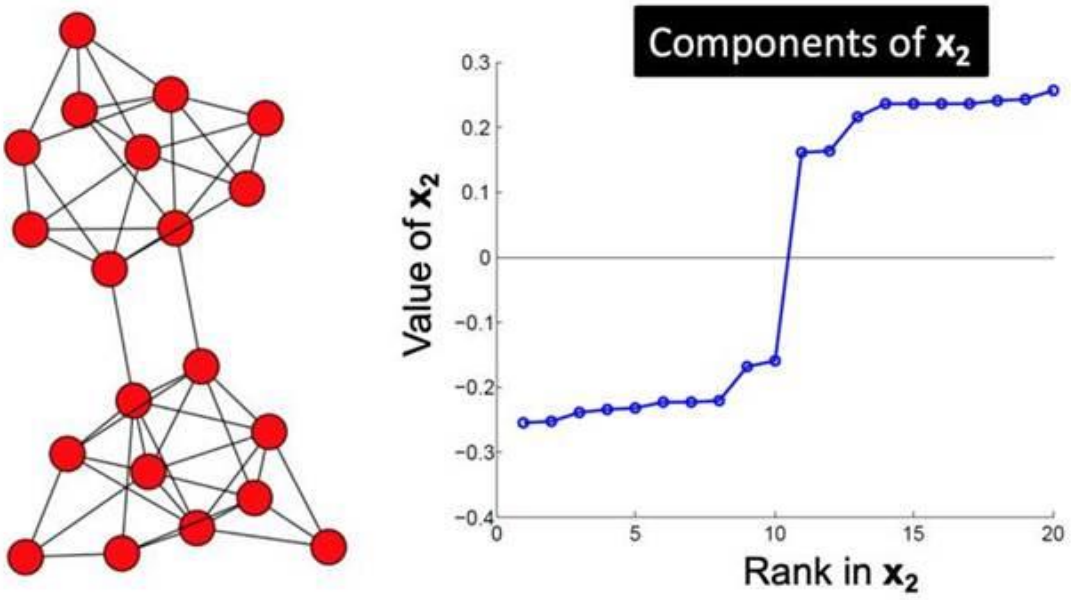


Figure 4.46 Eigenvector Components Used in Spectral Clustering

Demonstrates how **eigenvector components are used in spectral clustering to identify clusters within a graph structure**. Spectral clustering analyzes the connectivity between nodes in a graph and uses eigenvectors derived from the graph Laplacian matrix to reveal underlying cluster structures.

In the figure, the **left panel shows a network graph consisting of multiple nodes connected by edges**, where each node represents a data point, and edges represent similarity relationships between nodes. The structure of the graph suggests that the nodes form two main groups that are more densely connected internally than with nodes from other groups.

The **right panel displays a plot labeled "Components of x_2"**, which represents the values of the second eigenvector obtained from the spectral decomposition of the graph Laplacian matrix. The horizontal axis labeled **"Rank in x_2"** orders the nodes according to their eigenvector component values, while the vertical axis shows the corresponding **eigenvector value for each node**.

The figure highlights the following characteristics of spectral clustering:

- **Graph Representation of Data** – The network graph on the left illustrates how data points are represented as nodes connected by edges based on similarity relationships.

- **Eigenvector Component Values** – The plot on the right shows the values of the second eigenvector for each node in the graph. These values capture important structural information about the connectivity patterns within the graph.

- **Cluster Separation** – A noticeable **gap between eigenvector values** appears in the middle of the plot, indicating a natural division between two groups of nodes. Nodes with similar eigenvector values tend to belong to the same cluster.

- **Spectral Partitioning** – By using the eigenvector values as features, the spectral clustering algorithm can separate nodes into clusters based on their position relative to this separation point.

This visualization demonstrates how spectral clustering leverages eigenvectors of the graph Laplacian to reveal cluster structure within graph-based data, allowing clusters to be identified even when they are not easily separable using traditional distance-based methods (Syllogismos, n.d.).

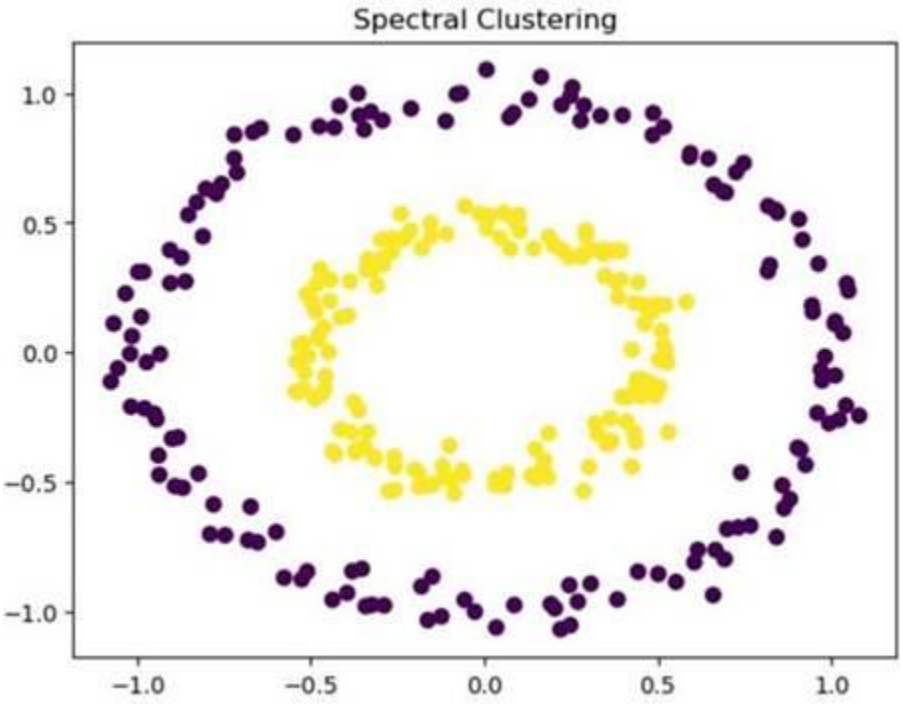


Figure 4.47 Spectral Clustering Result on Non-Linear Data

The application of **spectral clustering on a dataset with non-linearly separable structures**. Unlike traditional clustering algorithms such as K-Means that rely on distance-based partitioning, spectral clustering identifies clusters by analyzing the connectivity relationships between data points in a graph representation.

In the figure, the **scatter plot displays data points arranged in two concentric circular patterns**, forming an inner ring and an outer ring. Each point represents an observation in a two-dimensional feature space.

The visualization highlights the following aspects of spectral clustering:

- **Non-Linear Cluster Structures** – The dataset consists of two circular clusters that cannot be separated using a simple linear boundary. The **inner circular cluster (yellow points)** and the **outer circular cluster (purple points)** represent two distinct groups of observations.

- **Cluster Identification Through Graph Connectivity** – Spectral clustering constructs a similarity graph that captures the relationships between nearby data points. By analyzing the structure of this graph, the algorithm is able to detect clusters that share strong internal connections.

- **Separation of Complex Cluster Shapes** – The algorithm successfully separates the inner and outer circular groups despite their non-linear arrangement. This demonstrates the ability of spectral clustering to identify clusters with complex geometric shapes.

- **Cluster Visualization** – The different colors assigned to the two groups illustrate how spectral clustering partitions the dataset into clusters based on structural similarity rather than simple geometric distance.

The figure demonstrates the effectiveness of spectral clustering in detecting **non-linear and irregular cluster shapes**, making it particularly suitable for datasets where traditional centroid-based clustering methods fail to correctly separate clusters (StrataScratch, n.d.).

4.2.5.2. Cluster Evaluation Metrics

Even in unsupervised settings, internal validation metrics assess the quality and coherence of the clusters.

4.2.5.2.1. Silhouette Score

The silhouette score measures how similar a data point is to its own cluster compared to other clusters. The score ranges from −1 to 1, where higher values indicate better clustering structure.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$ **Equation 4.21**

where $a(i)$ is the mean distance from point i to the other points in its own cluster, and $b(i)$ is the mean distance from point i to the points of the nearest other cluster.

Interpretation

- $> 0.7 \rightarrow$ Strong cluster separation
- $0.5 – 0.7 \rightarrow$ Reasonable cluster structure

- 0.2 – 0.5 → Weak but potentially meaningful structure
- < 0.2 → Poor cluster structure
- < 0 → Possible misclassification (point may be assigned to the wrong cluster)

4.2.5.2.2. Davies–Bouldin Index

The Davies–Bouldin index measures average similarity between each cluster and its most similar counterpart, with lower values indicating better separation.

Equation 4.22

$$DB = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i}\left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)}\right)$$

where k is the number of clusters, σ_i is the average distance of the points in cluster i to its centroid, and $d(c_i, c_j)$ is the distance between the centroids of clusters i and j.

Interpretation:

- Lower values → Better cluster separation
- Values below 1.0 typically indicate well-separated clusters

4.2.5.2.3. Cluster Purity (Post-Analysis Only)

After clustering, cluster assignments are compared against ground-truth phase labels (baseline, blackhole, wormhole) to compute purity scores. This is strictly a post-analysis validation step, not used during clustering itself, and serves to assess whether discovered clusters align with experimental conditions.

Equation 4.23

$$\text{Purity} = \frac{1}{N}\sum_{i=1}^{k} \max_{j} \left|\text{cluster}_i \cap \text{class}_j\right|$$

where N is the number of windows, cluster_i is the set of windows assigned to cluster i, and class_j is the set of windows with ground-truth label j. DBSCAN noise points are counted as one additional group.
Higher purity indicates stronger alignment between unsupervised groupings and ground truth.

4.2.5.2.4. Adjusted Rand Index (Post-Analysis Only)

The Adjusted Rand Index (ARI) is a chance-corrected measure of agreement between two clusterings. It evaluates the similarity between the clustering produced by the algorithm and the ground-truth partition, while adjusting for the possibility that some agreement could occur purely by random chance.

Equation 4.24

$$ARI = \frac{\sum_{i,j}\binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\right]/\binom{N}{2}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\right]/\binom{N}{2}}$$

where n_{ij} is the number of windows that are in cluster i and in ground-truth class j, $a_i = \Sigma_j\, n_{ij}$ and $b_j = \Sigma_i\, n_{ij}$ are the cluster and class sizes, N is the number of windows, and $C(x, 2) = x(x - 1)/2$ is the binomial coefficient, the number of unordered pairs from x items.

Measures similarity between cluster assignments and ground-truth labels, corrected for chance. Range: [-1, 1]; 1 indicates perfect agreement.

Interpretation

- **ARI = 1** indicates perfect agreement between the clustering and the ground truth (up to permutation of labels).
- **ARI = 0** indicates that the agreement is no better than what would be expected from random clustering.
- **ARI < 0** indicates agreement worse than chance

4.2.5.3. Cluster Characterization Using Ground Truth

After clustering, each cluster is profiled to understand its behavioral meaning. This step uses the ground-truth phase labels (baseline, blackhole, wormhole) for interpretation only—they are not used during clustering.

Label distribution per cluster. For each cluster, we compute the percentage of windows belonging to each ground-truth phase. This reveals whether clusters correspond to distinct attack types or mix them. For example, a cluster may contain 95% baseline windows and 5% attack windows, indicating it predominantly represents normal behavior.

Feature centroid analysis. The mean value of each feature within a cluster is computed and compared to the global mean (across all windows). Features that are significantly elevated or depressed in a cluster help characterize its behavior. For instance, a cluster with a low forwarding ratio at the attacker and a low PDR at its child would be interpreted as "blackhole-affected."

Heatmap visualization. A heatmap of feature means per cluster (standardized values) is generated to visually summarize the behavioral signature of each cluster. Rows represent clusters, columns represent features, and color intensity indicates deviation from the global mean.

Interpretation of results. If the clustering yields only two clusters (e.g., benign vs. attack), we still characterize each cluster by its dominant label and feature profile. The ground-truth labels are used only after clustering to aid interpretation, not to guide the algorithm. This approach ensures that the discovered structure is data-driven, while still allowing us to map clusters to known experimental conditions.

4.2.5.4. Expected Outcomes

Based on the theoretical framework established in Chapter 3, the following outcomes are anticipated:

- Normal operation windows should form a dense, cohesive cluster characterized by forwarding ratios near 1.0, stable RSSI-hop relationships, and minimal parent switching.

- Forwarding suppression windows should form a distinct cluster (or be identified as noise) characterized by low forwarding ratios at attacker nodes and a reduced packet delivery ratio at the nodes whose traffic the attacker handles.
- Topology distortion windows should form a separate cluster characterized by tunnel activity and duplicate arrivals at the root (TunnelIntensity, TunnelBytes and TunnelLatency) and possibly by a latency-hop mismatch. Parent switching and layer changes are not expected, because the tunnel is an application-layer replay and does not alter the routing tree (Section 4.2.7.4).

If these behavioral states are indeed separable in feature space, the clustering analysis will reveal distinct groupings aligning with the three experimental conditions. This separation would validate that the collected cross-layer features capture meaningful distinctions between normal and manipulated network behaviors.
Because the wormhole tunnel does not change routing, wormhole windows may not separate from baseline windows unless the tunnel features are included. If they do not, that outcome is reported as a finding and not as a failure of the method.

4.2.6. Exploratory Data Analysis (EDA)

Before clustering, exploratory data analysis is conducted to characterize the structure and quality of the extracted feature set. EDA serves as a descriptive sanity check, ensuring that subsequent clustering interpretation is grounded in an understanding of feature distributions and relationships. All EDA is performed offline using Python-based analysis scripts and is strictly descriptive; no inferential claims or detection rules are derived from this stage.
EDA is run on each cell of the design (attack, topology, site and scenario) first, and then on the runs pooled. Only windows labelled baseline, attack or cooldown are analysed. Formation windows are left out. The following analyses are performed:

- **Descriptive statistics.** Mean, median, variance and range of every feature, by phase and node role.
- **Distribution visualization.** Histograms and box plots of ForwardingRatio and RSSI_Hop_Diff, by phase and node role. RetryRate is not plotted, because retry counts are application-layer counters.
- **Time-series plots.** PDR, ForwardingRatio, root arrivals and ParentSwitchRate over the run, so the effect can be checked against the logged attack window.
- **Cross-layer correlation analysis.** Pearson (straight-line) and Spearman (rank-based) matrices over three views: all labelled windows, baseline only, and attack only. A feature that has one value for the whole run has no correlation and is left off, and this is stated on the chart. Some pairs are linked by their formulas and not by network behaviour, for example ForwardingRatio and IngressEgressDelta, which use the same two counters,

and RSSI_mean and RSSI_Hop_Diff. These are reported as such. Correlation is not treated as cause.

- **Dimensionality reduction visualization.** PCA and t-SNE projections of z-scored features, clipped at ±5. Columns with missing values and role-specific columns (the tunnel features, which exist only on the wormhole ends) are left out of the projection. The axes of t-SNE have no units and only nearness is read from them.
- **Single-feature check.** For each feature alone, how well it could label a window, as described in Section 4.2.7.3.

Per-cell figures are reproduced for one representative cell of each attack in the paper, and the rest are placed in the repository [79]. The figures of the 80 runs are in Chapter 6 (Section 6.3). Any statement that attack windows lie apart from normal windows in these views is a description of the picture and not a detection result.

These analyses are conducted exclusively to understand the dataset structure and to validate that the engineered features capture meaningful behavioral variation. The results inform parameter choices for clustering (e.g., normalization confirmation and outlier identification), but they do not constitute intrusion detection or behavioral classification.

4.2.7. Attack Validation

Validation has two steps. The first is an internal check against the known schedule of each run. The second corroborates the result against published studies. The published studies describe other platforms, so the comparison is one of consistent behaviour, and the study does not claim proof by those works. *Table 4.20* lists the checks.

Table 4.20 Attack Validation Checks

Check	Method	Basis
Ground-truth schedule	Compare each feature across the baseline and attack phases of the same run	Section 4.2.3.4
Statistical test	3-sigma test of attack windows against the baseline mean and standard deviation	Zhukabayeva et al. [71]
Setup check	Confirm that the run matched its design: the topology during the attack, one attacker with victims behind it, and for the wormhole one entry end and one exit end	Section 4.2.1.4

	with the tunnel carrying the probes	
Independent observation	Sniffer frame record from outside the mesh	Section 4.2.3.3
Published definitions	Criterion-by-criterion conformance	*Table 4.4* and *Table 4.5*
Published behaviour	Compare with testbed and deployment results	Airehrour et al. [70], Khan et al. [22], Ramírez Gómez et al. [72], Zhukabayeva et al. [71]
Leakage audit	Single-feature threshold accuracy against the majority rate	*Section 6.2.3*

4.2.7.1. Ground-Truth and Statistical Check

Every run has a known phase schedule, so the expected effect of an attack can be stated before the data are read. In the G402 linear stationary run, the blackhole attacker's ForwardingRatio falls from 1.000 in the baseline to 0.001 in the attack phase, and the PDR of the six nodes behind it falls from 1.000 to 0.000. The 3-sigma verifier flags a feature when an attack window lies more than three baseline standard deviations from the baseline mean. When the baseline standard deviation is zero the z-score is undefined, and it is reported as undefined and not as an infinite value. For the wormhole, TunnelLatency cannot be tested with the 3-sigma method, because the baseline has no tunnel and so no spread, and its attack value, a median of 2.0 to 9.9 ms between the two copies over the 36 wormhole runs (typically 3.7 ms), is reported descriptively (Section 6.2.2). LatencyHopRatio cannot be tested either, because each node of the line sits at a different depth and the baseline therefore spreads widely, so neither enters the verdict.

The verifier also runs a setup check before it accepts a result. It reads the topology during the attack, the number and position of the attacker, whether victims were alive but cut off, and for the wormhole whether the entry end is deeper than the exit end and whether the exit end received what the entry end sent over the cable. A run that fails the setup check is reported as not confirmed, however clear the statistics look.

The verdict rules are fixed in the verifier before results are read. A run is confirmed when at least one primary signature passes: ForwardingRatio or PDR for the blackhole, and TunnelIntensity or the root-measured DuplicateRate for the wormhole (D-21). A pass needs

the move in the expected direction beyond three standard deviations, an absolute change of at least 0.10 and at least six attack points, and each point pools five one-second windows. A primary whose baseline is broken, with a mean below 0.90 or a standard deviation above 0.15 of the mean, is excluded and not counted as a failure. TunnelLatency and LatencyHopRatio are reported and not scored.

Each run then receives one outcome. PASS means that the signature, the setup check and the data all pass. FAIL means that the run could be tested and the attack did not happen as designed. INCONCLUSIVE means that the data needed to decide are missing. Missing data never turns into a pass or a fail. An impact check repeats the test without every board that has a gap; when the result holds, the gaps are graded minor and footnoted.

A blackhole is invisible to its victims, because the attacker accepts each probe at the link layer and discards it, so the sender sees no failure. The attack is therefore measured at the attacker, whose forwarding ratio is confirmed independently by its neighbours' counts, and at the root, from the victims' sequence numbers, with the boards outside the attacker's path as a control group (Karlof and Wagner [81]). A board that changes parent during the attack is tolerated when it keeps its class, victim or bystander (D-23). The attacker changing its own parent is tolerated only when no board changes class and the attacker was without an uplink for at most 10% of its attack, and the seconds in which a victim had no path to the root are left out of the test (D-31). A missing board is graded by its role: a missing attacker requires a recapture, a missing victim or bystander is footnoted while other victims and another control remain, and a board whose position cannot be established leaves the run undetermined (D-32). The verifier also reports, from the root's run log, whether a run was started with START_ANYWAY; earlier captures have no run log, and this is noted per run.

Thresholds for which no published value exists were set as engineering bounds before the results were examined. A one-at-a-time sensitivity analysis that moves the verdict thresholds by about ±50% changed no verdict in 53 analysed runs across 14 settings, while a deliberately extreme control setting (1,000 standard deviations) did change 18 verdicts, which shows that the test can fail, and a placebo test that relabels the second half of each baseline as an attack produced no false confirmation in 106 tests. When the verifier was run again on the final captures, all 72 attack runs had the outcome PASS, 67 of them with complete data and 5 with minor gaps that the impact check showed do not change the result, and all 8 benign runs were confirmed as clean normal traffic (Section 6.2.6). Section 4.2.7.7 sets out the four checks in order.

4.2.7.2. Corroboration with Published Studies

Airehrour et al. [70] ran a blackhole attack on a testbed and reported packet loss of 60 to 75% under the standard protocol, against under 28% for their trust-based protocol, together with 16 to 246 rank changes against 15 to 55. A collapse in delivery under a placed blackhole is therefore consistent with published hardware results. Parent switching is the exception: the data show almost none, because forwarding suppression does not alter routing, so this difference is explained by the attack design. Khan et al. [22] report delivery above 97% at

every node of a healthy ESP-MESH campus network except node H, an outdoor node, and this is the reference for the baseline phase. Table 6.34 sets each measured value of this study beside the published one.

For the wormhole, Ramírez Gómez et al. [72] propose duplicated routing packets as a signature for detecting a wormhole in a ZigBee network. This is related to the duplicate data arrivals that our tunnel produces, but it is a proposed signature and not a measured result. Zhukabayeva et al. [71] report that a wormhole shortened a route from four hops to three and raised the average delay by about 40% (their abstract; the route-change row of their Table 7 gives 120 to 200 ms against 80 to 100 ms). Our tunnel does not reproduce a shorter route (Section 4.2.7.4). The tunnel features have no counterpart in these studies, so they rest on the internal check alone.

4.2.7.3. Single-Feature Leakage Audit

If one feature alone could separate attack from normal windows, the clustering would be unnecessary. Each feature is therefore tested as a single-threshold classifier and compared with the majority-class rate, which is about 70% here. The audit is made on the windows of the 36 blackhole runs and of the 36 wormhole runs, each in three groups of boards, and it also tests the run clock (Section 6.2.3). On the victims of the blackhole runs, PDR alone reaches 99.9% against a majority rate of 70.1%. This is expected for a blackhole, which drops every packet for the whole attack window, so a complete delivery collapse is separable by construction. Every other feature that has a value in nearly all windows stays at the majority rate. LatencyHopRatio reaches 100.0% only on the windows that have a value, and its information is in its missing values, so it is kept out of the clustering. On the non-attacker nodes of the wormhole runs, no feature gets more than 0.01 percentage points above the majority rate. On the two tunnel ends, the tunnel features decide the label alone (99.7%), because the tunnel carries messages only in the attack, and they are excluded from the model inputs of the non-attacker nodes. The aligned run clock alone gives 80.0%, because every run follows the same phase schedule, and it is kept out of the model inputs. The tables and figures of the audit are in Section 6.2.3.

4.2.7.4. Predictions Tested and Not Observed

Table 3.4 predicted more victim retransmissions under the blackhole, and Table 3.5 predicted a distorted topology under the wormhole. Neither was observed. The independent sniffer shows no consistent rise in victim MAC retransmissions: it recorded the 802.11 retry rate in 7 of the 36 blackhole runs, and the change of the victims from the baseline to the attack lies between −0.10 and +0.33, up in 3 runs and down in 4, as expected when the attacker still acknowledges every frame at the link layer (Section 3.3.1.2). The application counters (D-15) agree, since a failed send of a victim occurs in only 9 of its 21,742 attack windows (Section 6.2.4). In the 36 wormhole runs, the nodes outside the tunnel recorded no parent change and no hop change in either the baseline or the attack (ParentSwitchRate and HopChangeCount were 0 in every window), and their PDR stayed at 0.9999 in both phases. Both tables are kept

as they were written at proposal time, before any capture. The result is that an application-layer replay tunnel reproduces the duplicate-delivery signature of a wormhole without its topological signature.

4.2.7.5. Limits

Early seven-board highload captures did not recover during cooldown, even in a run without an attack; the highload runs of the final dataset recover within 3 s of the end of the attack (Table 6.10). Wormhole conformance is established on all four topologies, since the tunnel skipped 1 to 5 hops in the 36 wormhole runs (Section 4.2.1.4). In the star runs it skipped one hop, because end A is the hub, one hop below the root, and end B is a board behind it (D-28). The partial-mesh runs are structurally plain trees, because no board was seen under a second parent in any of the 19 partial-mesh runs (Section 4.2.2.4). The clustering quality metrics (silhouette score, Davies–Bouldin index, purity and adjusted Rand index) were computed on the final dataset (Section 6.4). The attack starts at the same time in every run, so position in the run could reveal the phase and is kept out of the model inputs.

4.2.7.6. Reference Exhibits from the Validation Papers

The exhibits below are the published results that our measurements are compared against. Each is cropped from the cited paper and carries an adapted-from line. The figure and table numbers were checked against the source papers.

Airehrour et al. [70] ran a blackhole attack on a testbed that the paper lists as 14 motes, and *Table 4.21* lists their setup.

Table 4.21 Testbed Parameters. Adapted from Airehrour et al. [70, Table 2]

Testbed tool	Contiki/Cooja 3.0
Testbed coverage area	30m x 30m
Total number of XM1000 motes	14
Blackhole motes	2 (Motes 13 and 14)
Mote deployment environment	Smart building
RX Ratio	30-100%
TX Ratio	100%
TX Range	50m
Interference Range	55m
Routing Protocols	MRHOF-RPL and Trust-based RPL
Network protocol	IP based

They report packet loss under the blackhole attack for the standard protocol (MRHOF-RPL) and for their trust-based protocol. Under the standard protocol, which has no defence, loss was 60 to 75%.

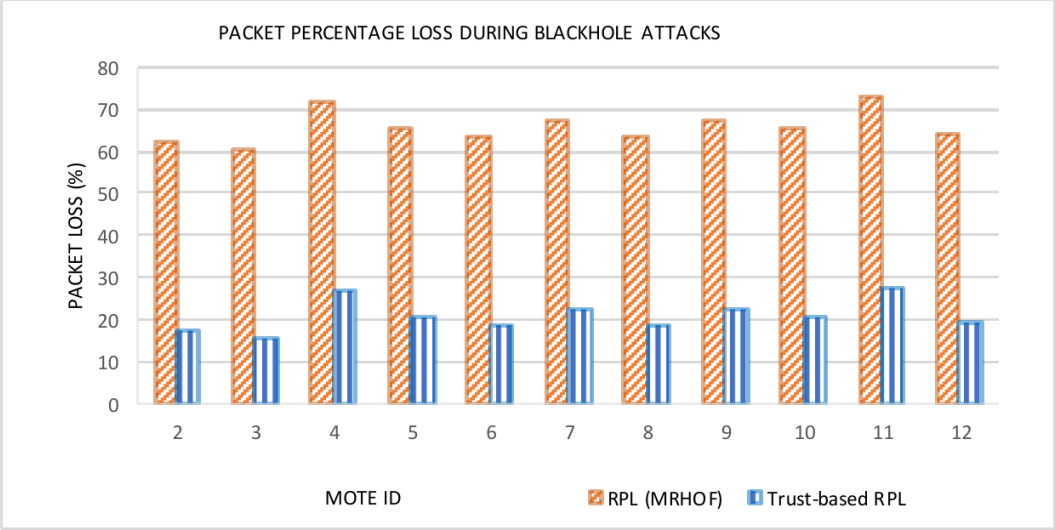


Figure 4.48 Packet Loss under Blackhole Attacks. Adapted from Airehrour et al. [70, Figure 10]

They also report how often each mote changed its routing rank, which ranged from 16 to 246 under the standard protocol.

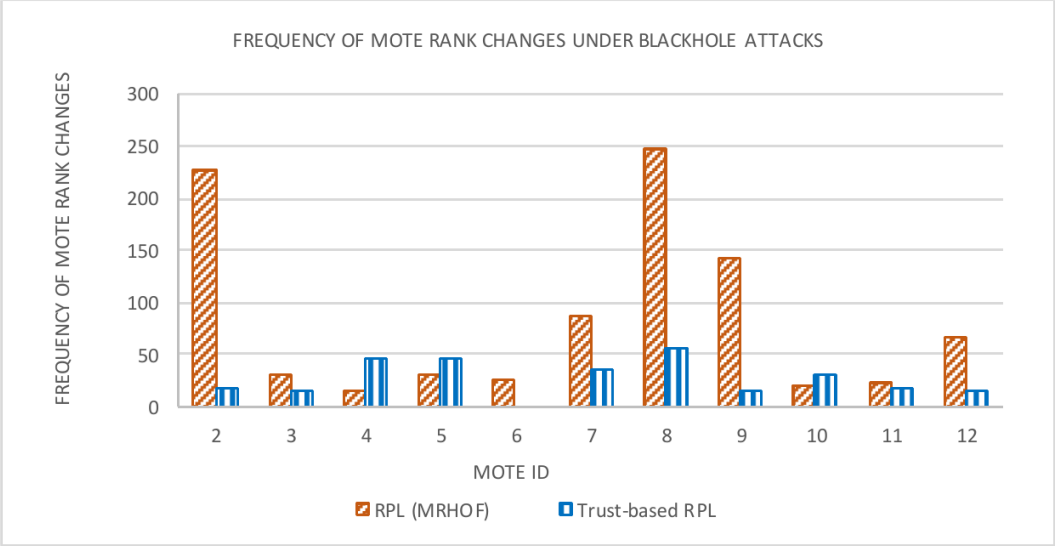


Figure 4.49 Frequency of Mote Rank Changes under Blackhole Attacks. Adapted from Airehrour et al. [70, Figure 8]

Throughput per mote is shown in *Figure 4.50*. We do not measure throughput in kilobits per second, so it is shown for completeness and is not compared.

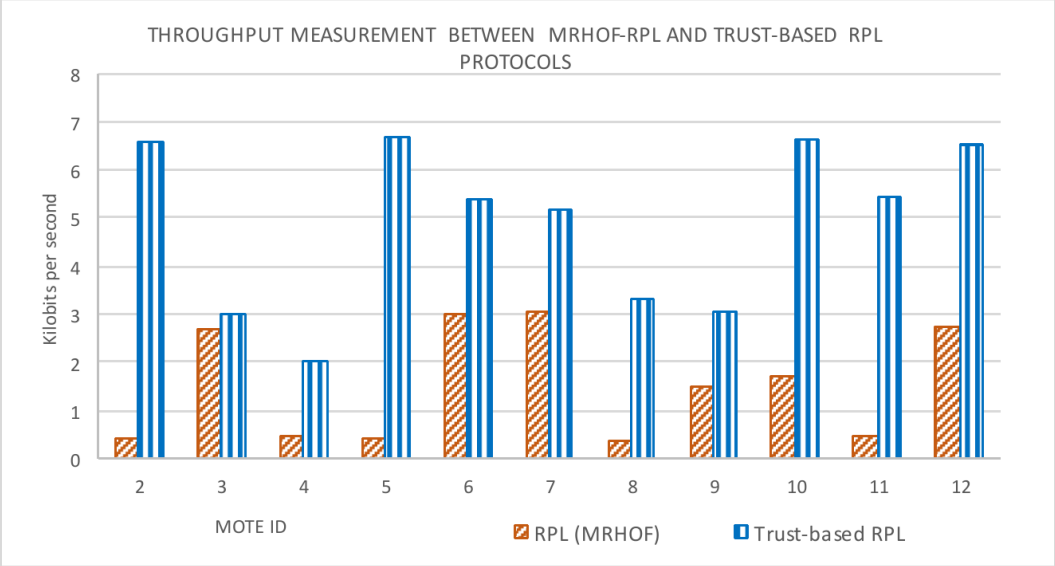


Figure 4.50 Throughput under Blackhole Attacks. Adapted from Airehrour et al. [70, Figure 9]

Table 6.34 in Chapter 6 sets each measured value of this study beside the published value, with the testbed of each source and the reading of the difference.

Khan et al. [22] measured a healthy ESP-WIFI-MESH network with the same ESP32 platform. Their sensor readings were taken every 2 minutes, and their delivery figures come from a stability test that sent every 60 s over five days. They report *Figure 4.51* and *Table 4.22*. Delivery stayed above 97%, and packet loss and package faults stayed under 1.8%, at every node except node H, which the authors place 100 m from the root and suspect was affected by its outdoor location. The values for each node are in their Figure 20.

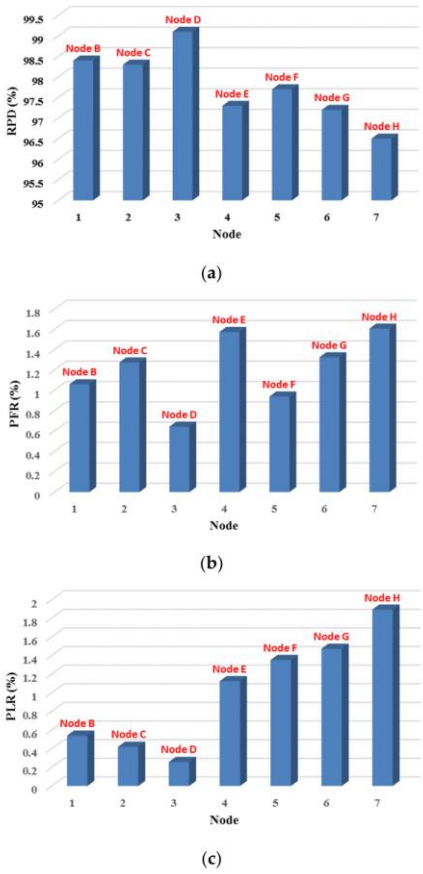


Figure 4.51 Received Packet Delivery, Packet Loss and Package Fault Rate per Node. Adapted from Khan et al. [22, Figure 20]

Table 4.22 Performance of the ESP-MESH Protocol. Adapted from Khan et al. [22, Table 6]

Performance Parameter	Time Spent
Network-building time	17.32 s
Healing time after the sink node failure	4.18 s
Healing time after the intermediate node failure	3.46 s

Zhukabayeva et al. [71] attacked a wireless sensor network with a wormhole and report how the route changed. *Figure 4.52* shows the hop count between the two ends and *Table 4.23* lists the measured changes. Hop count fell from four to three. Their abstract reports that the average delay rose by about 40% and that false readings appeared in over 30% of cases, while the route-change row of their Table 7 lists a delay of 120 to 200 ms against 80 to 100 ms before the attack, with 20 to 30% of the data modified. The source is not fully consistent, since the text with its Figure 19 says that the hop count increased slightly.

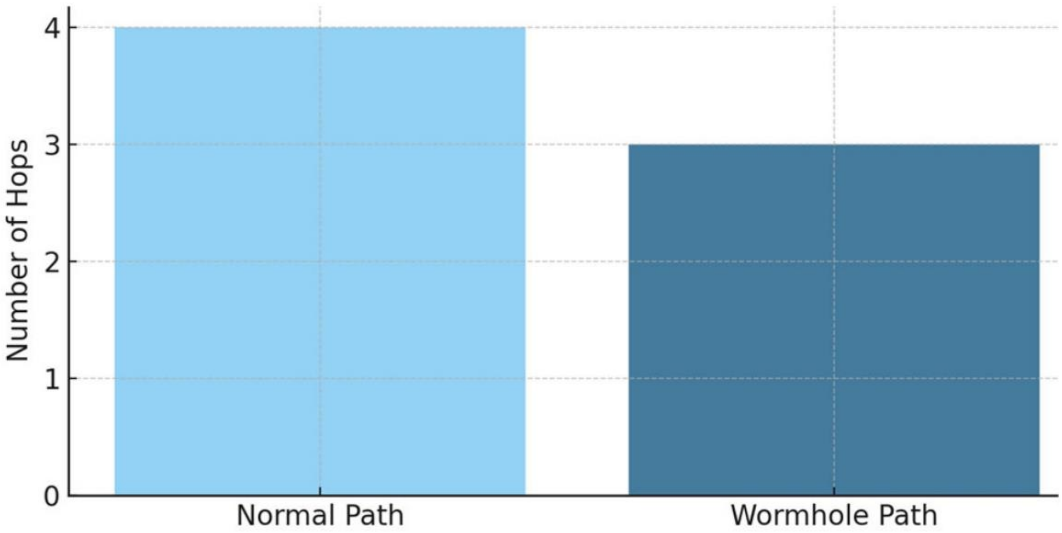


Figure 4.52 Hop Count under the Wormhole Attack. Adapted from Zhukabayeva et al. [71, Figure 4]

Table 4.23 Route Changes and Comparative Traffic Analysis under the Wormhole Attack. Adapted from Zhukabayeva et al. [71, Table 7]

Component	Description
Route Changes During Attack	After the Wormhole tunnel is activated, data are routed in 3 hops instead of 4: Old route: R3 → R2 → R1 → R0 → C; New route: R3 → R4 → R5 → C. This alters routing, bypasses some nodes, and allows the attacker to intercept and manipulate data.
Comparative Traffic Analysis	- Before the attack: 4 hops, 80–100 ms delay, accurate data. - After the attack: 3 hops, 120–200 ms delay, 20–30% of data modified by the attacker. - XCTU visualization displays the new R4–R5 link and route changes.

Figure 4.53 compares the same network before and after the attack.

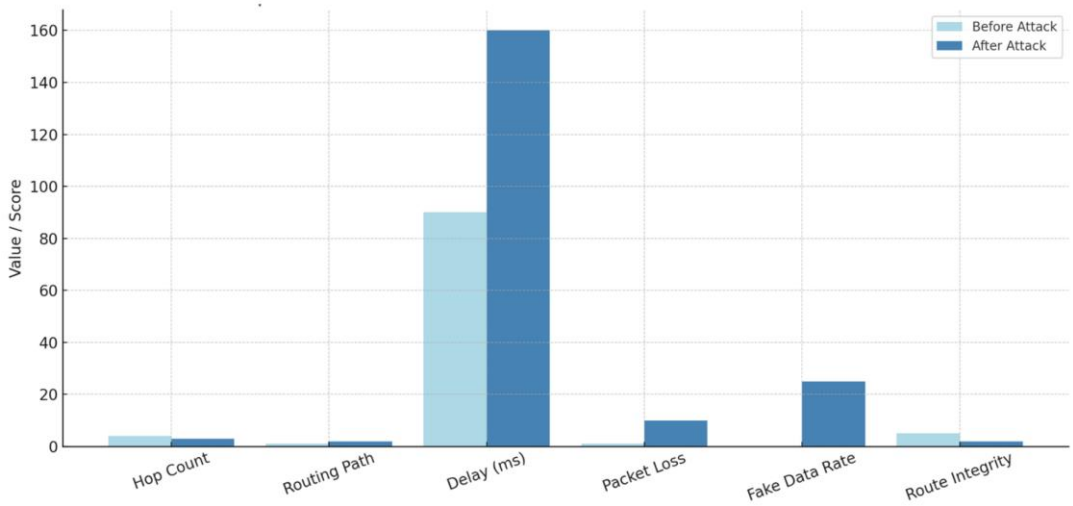


Figure 4.53 Network Behaviour before and after the Attack. Adapted from Zhukabayeva et al. [71, Figure 19]

Ramírez Gómez et al. [72] implemented a wormhole on a ZigBee network of XBee S2C devices, using a single malicious node. *Figure 4.54* shows their diagram of the attack, and their test setups are in their Figures 7 and 8. They propose duplicated routing packets, two source-routing packets for each data packet, as a signature for detecting the wormhole. This is a proposed signature and not a measured result, and it is related to, but not the same as, the duplicate data arrivals our tunnel produces.

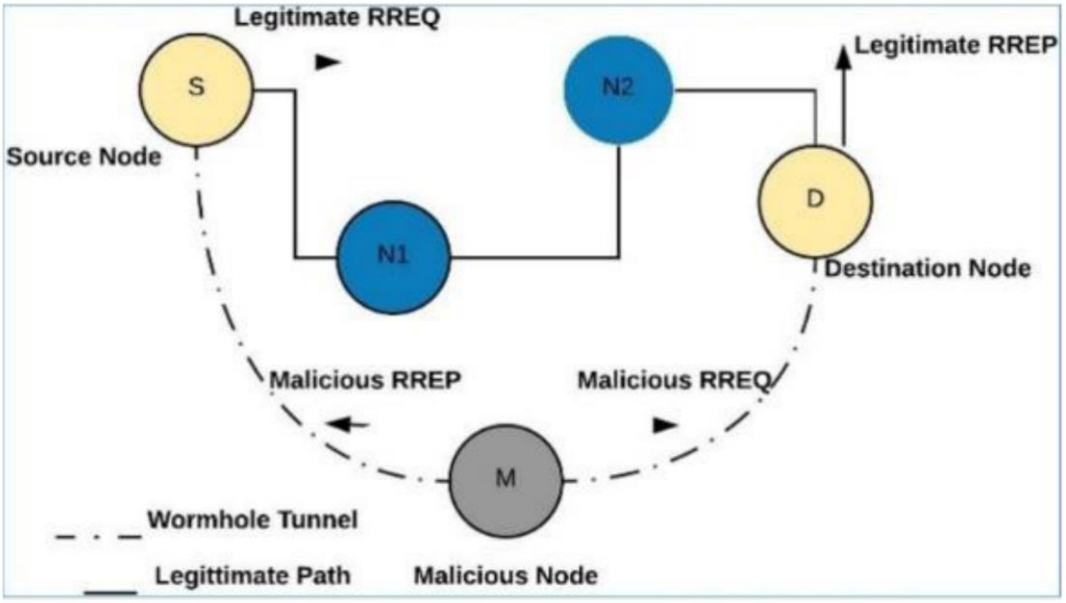


Figure 4.54 Wormhole Attack with a Malicious Node. Adapted from Ramírez Gómez et al. [72, Figure 2]

4.2.7.7. Verification of Each Run

Every captured run was checked by the verifier before it was accepted into the dataset. The verifier reads the feature table of the run and the run log of its root, and it answers four questions in a fixed order. The first is whether the attack signature moved (attack signature). The second is whether the boards formed the topology under which the run is filed (topology). The third is whether every board

recorded enough of each phase for the result to be trusted (dataset). The fourth is whether the traffic matched the scenario of the run (scenario). Two supporting checks are reported together with these four. The setup check confirms that the attack ran as designed: for the blackhole, that the attacker relayed and then dropped the probes and that its victims were alive but cut off; for the wormhole, that exactly one Node A and one Node B were present, that Node A sat shallower than Node B, and that the tunnel frames sent by Node B reached Node A. The execution check of the capture tools confirms that the file of every board was imported. Figure 4.55 shows the order of the checks and how their results are combined into one outcome.

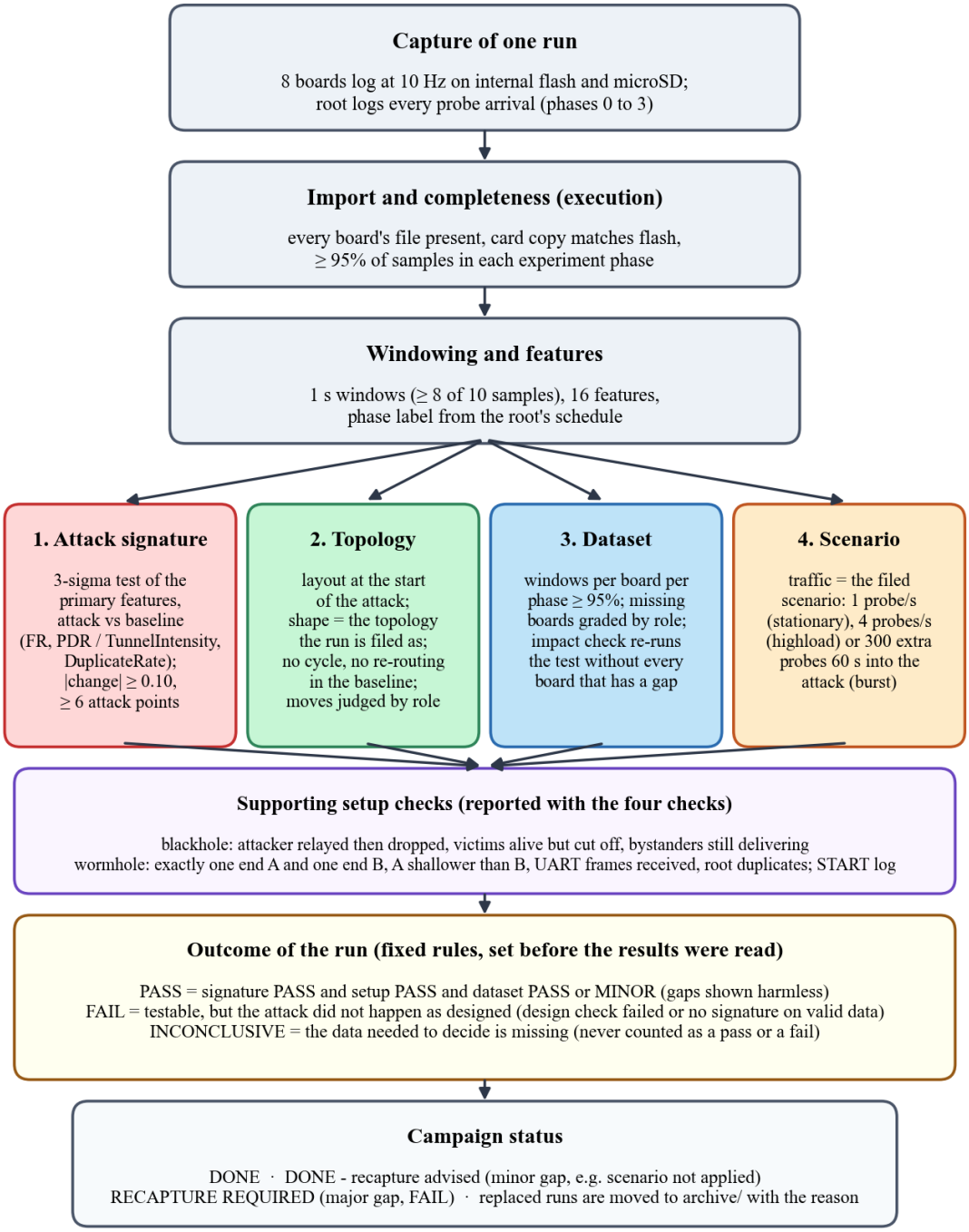


Figure 4.55 Order of the Checks That the Verifier Applies to Every Run

How to read it. Each box is one step, read from the top. A run is first imported and cut into one-second windows. The four checks in the middle row are then applied to the same windows, and the setup check below them confirms the roles of the boards. The results are combined by the rules in the yellow box, which were fixed before any result was read, and the campaign status at the bottom decides whether the run is kept or captured again.

4.2.7.7.1. Attack Signature

The attack signature check applies the 3-sigma test of Section 4.2.7.1 to the primary features of each attack. For the blackhole, the primary features are the ForwardingRatio of the attacker and the PDR of the victims. For the wormhole, they are TunnelIntensity, which is counted by the tunnel ends, and

DuplicateRate, which is measured at the root (D-21). The windows of the attack phase are compared with those of the normal phase, and every five one-second windows are pooled into one point (Table 4.10). A primary feature passes when its attack mean lies more than three baseline standard deviations from the baseline mean in the expected direction, when it changes by at least 0.10 in absolute terms, and when it has at least six attack points. When the baseline did not vary at all, the z-score is undefined, and any move in the expected direction is accepted. A run passes the check when at least one primary feature passes. The support features (ConsistencyScore, IngressEgressDelta and NeighbourForwardingRatio for the blackhole, and TunnelBytes for the wormhole) are reported but do not decide the verdict. TunnelLatency and LatencyHopRatio are measured and not scored, because their normal phase has no usable spread. Figure 4.56 shows the check on one blackhole run and on one wormhole run.

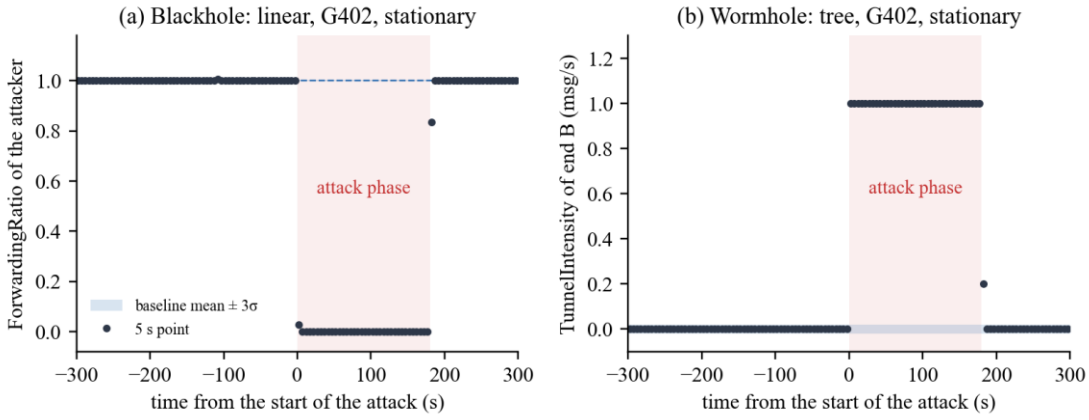


Figure 4.56 Attack Signature Check on Two Runs: (a) ForwardingRatio of the Blackhole Attacker, Linear Topology, G402, Stationary; (b) TunnelIntensity of Wormhole End B, Tree Topology, G402, Stationary

How to read it. The horizontal axis is the time in seconds from the start of the attack (negative values are the baseline), and the shaded area is the 180 s attack phase. Each dot is the mean of five one-second windows. The light blue band is the baseline mean plus or minus three baseline standard deviations (3σ); it is very thin here because the baseline barely varied. (a) The attacker forwarded every probe it received in the baseline (ratio 1.0, no unit) and none during the attack (0.0), so every attack point lies far outside the band. (b) The tunnel end sent nothing through the tunnel in the baseline (0 messages per second) and one message per second during the attack.

4.2.7.7.2. Topology

The topology check rebuilds the tree of the mesh at the start of the attack from the parent address that every board logs, and it compares the shape of that tree with the topology under which the run is filed. A linear run must form a chain with one board on each layer. A star run must have every child under one hub that sits behind the root (D-16, D-28). A tree or a partial-mesh run must branch over several hops without a cycle. The check also reads the parent changes during the experiment. A board that re-routes during the baseline fails the topology gate (D-27). A board that changes parent during the attack is judged by its role: the move is tolerated when the board stays a victim or stays a bystander, and the run fails when the board switches between the two (D-23, D-31). For the partial mesh, the

verifier also records whether any board was seen under a second parent, and it adds a note when the run formed a plain tree (D-29). Figure 4.57 shows the layouts that the verifier rebuilt for six runs.

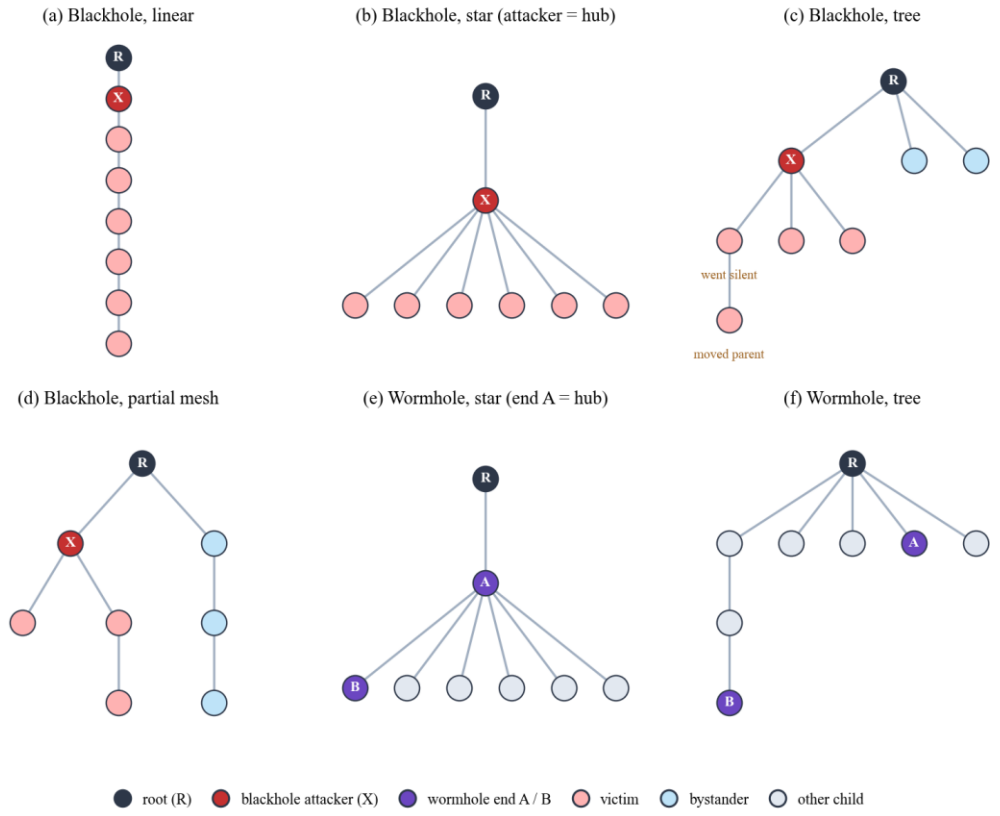


Figure 4.57 Layouts Rebuilt by the Topology Check at the Start of the Attack (Six Runs at G402)

How to read it. Each circle is a board, and each line joins a board to its parent, with the root (R) at the top. X is the blackhole attacker and A and B are the two ends of the wormhole. Red boards are victims, whose path to the root passes through the attacker, and blue boards are bystanders, whose path does not. Grey boards are the other children of a wormhole run, which are not tunnelled. Panels (a) to (d) and (f) are stationary runs and panel (e) is a highload run. In panel (c), one victim went silent 28 s into the attack and another moved to a new parent while it stayed a victim; the move was tolerated (D-23).

4.2.7.7.3. Dataset

The dataset check asks whether every board recorded enough of each phase for the result to be trusted. For each board, the number of one-second windows in the baseline (300 s), the attack (180 s) and the cooldown (120 s) is compared with the length of the phase, and a board below 95% in any phase is reported as a gap. A board that the root heard but whose file is missing is found from the arrival log of the root, and it is placed in the tree from the logged parent links and never from its delivery (D-32). Each gap is graded by the role of the board. A gap on the attacker or on a tunnel end is major. A gap on a victim or a bystander is minor while other boards of the same role still carry the evidence or the control. An impact check then repeats the attack signature test without every board that has a gap. When the result holds, the gaps are graded minor and are footnoted. When it does not hold, or when a major gap exists, the run cannot be validated and is captured again. Figure 4.58 shows a run with one minor gap.

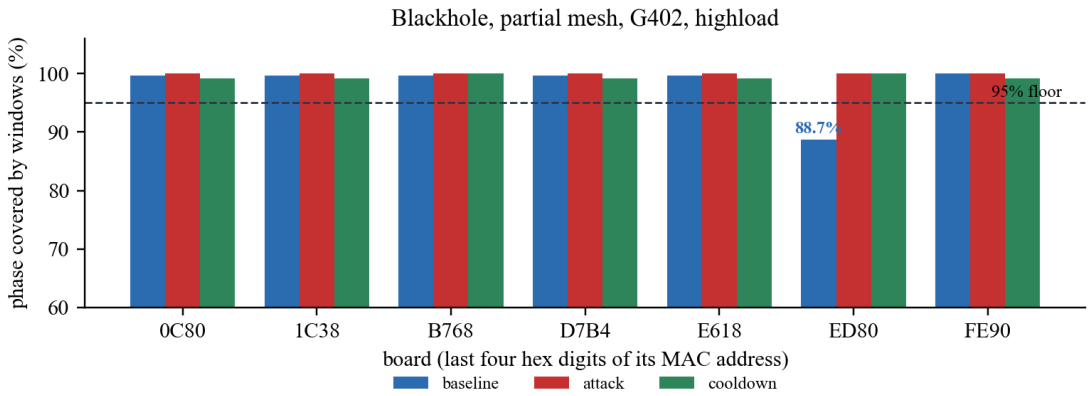


Figure 4.58 Dataset Check: Share of Each Phase Covered by Windows, per Board (Blackhole, Partial Mesh, G402, Highload)

How to read it. Each group of bars is one board, named by the last four hexadecimal digits of its MAC address, and each bar is the share (%) of a phase that the board covered with one-second windows. The dashed line is the 95% floor. Board ED80, a victim, covered only 266 of the 300 s of its baseline (88.7%), so the verifier reported a gap. The impact check repeated the attack test without ED80, the attack was still confirmed, and the run was kept with a minor gap.

4.2.7.7.4. Scenario

The scenario check confirms that the traffic of a run matched the scenario under which it is filed (Section 4.2.2.5). It reads the probe counters of the child boards. In a stationary run, each child sends about one probe per second for the whole run. In a highload run, each child sends about four probes per second, one every 250 ms. In a burst run, one child sends 300 extra probes within 10 s, about 60 s into the attack. A run whose scenario was not applied, for example a burst that fired outside the attack window, keeps its attack result, but it is graded as having a minor gap and a recapture is advised (D-34). The same check is applied to the benign runs, where the burst is expected in the part of the baseline that matches the attack window of an attack run. Figure 4.59 shows the three scenarios on three blackhole runs.

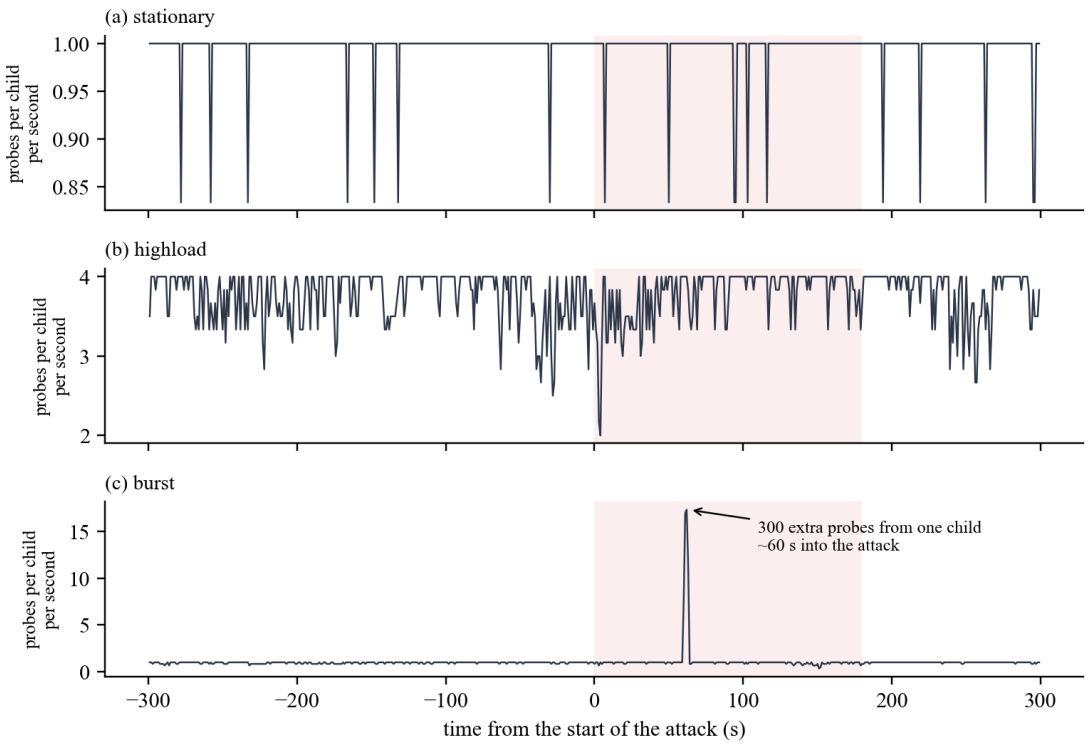


Figure 4.59 Scenario Check: Probes Sent per Child Board per Second in Three Runs (Blackhole, Star, G402)

How to read it. The horizontal axis is the time in seconds from the start of the attack, and the shaded area is the attack phase. The vertical axis is the mean number of probes that one child board sent in each second. (a) In the stationary run each child sent about one probe per second. (b) In the highload run each child sent about four probes per second; the short dips are seconds in which a board sent fewer probes. (c) In the burst run the line stays near one probe per second except for one spike about 60 s into the attack, when one child sent 297 extra probes within 10 s.

4.2.7.7.5. Outcome and Grade

The results of the checks are combined by fixed rules that were written into the verifier before the results were read (Table 4.24). A run is PASS when the attack signature passes, the setup check passes, and the data are complete or have only minor gaps. It is FAIL when the run could be tested but the attack did not happen as designed, and INCONCLUSIVE when the data needed to decide are missing. A missing piece of data is never counted as a pass or as a fail. Each run also receives a grade that states how complete its data are. Benign runs are checked the other way round: the same two signatures are tested on the cooldown of the benign run against its baseline, and a benign run is confirmed only when neither signature moves and the delivery stays healthy, with a PDR of at least 0.90 in both phases and every relay forwarding at least 90% of what it received. The outcomes of all 80 runs are reported in Section 6.2.6.

Table 4.24 Rules for the Outcome and the Grade of a Run

Result	Rule	What happens to the run
PASS (outcome)	Attack signature passes, setup check passes, and the data are	Kept in the labelled dataset

	complete or have only minor gaps	
FAIL (outcome)	The run could be tested, but a design check failed (the attacker forwarded, no victim sat behind the attacker, a board switched between victim and bystander, the topology was wrong or the tunnel was reversed), or the signature did not move on valid data	Archived with the reason and captured again
INCONCLUSIVE (outcome)	The data needed for the decision are missing (attacker telemetry missing, too few victim-seconds, a major gap, or the role of a missing board cannot be placed)	Archived with the reason and captured again
STRONG or GOOD (grade)	Every board has usable data for the whole attack; STRONG when nothing needed a note, GOOD when a note was added (for example, a plain tree or a run with no run log)	Kept
MINOR GAPS (grade)	A gap on a victim, a bystander or the scenario that the impact check shows does not change the result	Kept; the gap is footnoted and a recapture is advised
MAJOR GAPS (grade)	A gap on data that the result depends on (the attacker, a tunnel end or the last control board)	Not validated; captured again

Legend:
- Outcome = the verdict of the verifier on the attack evidence of a run; grade = how complete the data of the run are.
- Design check = a check of the setup that the experiment depends on, such as the position of the attacker; impact check = the attack test repeated without the boards that have gaps.
- Victim = a board whose path to the root passes through the attacker; bystander = a board whose path does not; control = a bystander used to show that the loss is specific to the path of the attacker.
- Victim-seconds = seconds of the attack in which a victim had data; run log = the serial log of the root, saved by the capture tool.

4.3. Theoretical Analysis

This section provides the theoretical justification for the selected cross-layer features and clustering methodology. While Section 4.2 detailed the implementation procedures, this section explains the conceptual rationale linking routing manipulations to observable behavioral deviations in wireless mesh networks. The analysis is grounded in the theoretical framework established in Chapter 3 and demonstrates how the empirical methodology aligns with fundamental principles of wireless mesh networking and routing behavior.

4.3.1. Theoretical Justification of Selected Features

The features engineered in Section 4.2.4 were not chosen arbitrarily; each is grounded in the cross-layer inconsistency framework developed in Chapter 3. Routing attacks in wireless mesh networks do not produce isolated anomalies within a single protocol layer; instead, they generate correlated inconsistencies across PHY, MAC, and Network layers due to the

interdependent nature of wireless communication (Al-Anzi [5]; Amouri et al. [8]). The selected features are designed to capture these cross-layer disruptions by targeting the specific relationships that routing manipulations violate.

4.3.1.1. Forwarding Behavior Features (Blackhole Theory)

Forwarding suppression attacks directly violate the fundamental routing assumption of multi-hop networks: that intermediate nodes forward transit traffic. In ESP-WIFI-MESH, forwarding compliance is implicit in the routing design; nodes are expected to relay packets received from children toward the root and vice versa. The theoretical concept of packet conservation—that packets received should approximately equal packets forwarded—serves as an invariant of normal operation (Kurosawa et al. [43]; Baadache & Belmehdi [44]).

The following features operationalize this invariant:

- **Forwarding Ratio** directly measures the proportion of received transit packets that are forwarded. A value significantly below 1.0 indicates a violation of the conservation principle.
- **Ingress-Egress Delta** quantifies the absolute packet imbalance, capturing the magnitude of packet absorption at the attacker node.
- **Forwarding Consistency Score** provides a unified deviation metric, useful for visualization and cluster interpretation.

These features were selected because they directly measure deviation from expected forwarding behavior. In normal mesh routing, packet conservation holds; a blackhole disrupts this conservation property. Thus, forwarding-based features operationalize the theoretical routing invariants of multi-hop networks.

4.3.1.2. Link Reliability Features (Indirect Propagation Theory)

Routing misbehavior does not affect only the attacker; it propagates stress both upstream and downstream. Nodes that send packets through a suppressing attacker lose those packets after the first hop, so the delivery they achieve falls even though the link-layer exchange with the attacker still succeeds. Cross-layer telemetry theory also predicts that network-layer disruptions can show up as MAC-layer stress (Section 3.4.1). This prediction was tested and was not observed (Section 4.2.7.4).

The following features capture these second-order effects:

- Packet Delivery Ratio (PDR) measures end-to-end success from the victim to the root. Prior studies report high baseline delivery under stable conditions (e.g., above 97% at every node but one in Khan et al. [22]); in this work, baseline PDR is established empirically per run, and deviations from that baseline indicate degradation.
- Retry Rate is the share of application-layer send attempts that failed. It is kept as a dataset column only and is not used in the analysis, because ESP-IDF exposes

no MAC retransmission counter and no cited study uses it as an attack signature. MAC retransmissions are read from the sniffer as MacRetryRate in sniffer runs.

PDR captures the indirect, yet observable, consequences of forwarding suppression on nodes beyond the attacker.

4.3.1.3. Topology Stability Features (Routing Adaptation Theory)

ESP-WIFI-MESH dynamically adapts parent selection, favoring the shallowest layer among candidate parents with an acceptable signal and then the fewest children. This adaptive behavior is a fundamental characteristic of wireless mesh routing protocols (Akyildiz et al. [1]). A node that perceives an artificially attractive path, such as a wormhole shortcut, might be expected to re-evaluate its parent.

In this study the wormhole is emulated at the application level by an out-of-band tunnel that the mesh does not use for routing. The routing tree is therefore not expected to change, and the parent and layer features act as a check on this: they should stay at their baseline values (Section 4.2.7.4).

The following features capture deviations from expected routing stability dynamics:

- Parent Switch Rate measures the frequency of upstream parent changes. Normal operation exhibits stable parent relationships; increased switching indicates instability in the topology.
- Hop Change Count captures fluctuations in hop count, reflecting nodes moving up or down in the routing hierarchy.
- Hop Stability Duration quantifies the longest continuous period of stable topology and provides a complementary measure of instability intensity.

These features are grounded in the theoretical behavior of adaptive mesh routing protocols, which are designed to maintain stable hierarchies under normal conditions.

4.3.1.4. Physical Layer Features (Signal Consistency Theory)

In ESP-WIFI-MESH the RSSI that a node reports is the signal strength of the link to its parent. It reflects the distance and obstacles between those two boards, and it does not fall with each extra hop, because every hop is a separate radio link (Espressif [21]). RSSI therefore does not rank nodes by depth. What ties signal strength to the topology is parent selection: a node joins a parent only when the link passes the RSSI threshold, so the parent link of a node is expected to stay in a stable range for its layer in a given deployment.

In a true wormhole, logical topology no longer matches physical distance, and a node may hear packets from a distant source with a signal far stronger than expected (Khalil et al. [45]; Hu et al. [46]). The wormhole in this study is an application-layer replay and does not change which parent a node uses, so the RSSI of the parent link is expected to stay at its baseline. The RSSI features therefore act as a check on the physical layer: if they shifted during the attack, the cause would be something other than the tunnel, for example a parent change or interference.

The following features capture this inconsistency:

- **RSSI Mean** provides the average signal strength, serving as a baseline for link quality.
- **RSSI Variance** measures signal stability; high variance may indicate interference or topology reconfiguration effects.
- **RSSI-Hop Inconsistency** directly quantifies the deviation between observed RSSI and the value expected based on the node's reported layer. Expected values are derived empirically from stable baseline windows, making the metric environment-specific and robust to testbed variations.

These features are theoretically grounded in the spatial signal-propagation properties of wireless networks and the assumption that physical distance should constrain logical topology.

4.3.1.5. Cross-Layer Inconsistency Theory

The core theoretical assumption of this study is that routing manipulation creates measurable cross-layer inconsistencies. Instead of relying on attack-specific signatures, the methodology measures structural deviations from expected relationships:

- Forwarding invariant violation (Network layer)
- Latency-to-hop mismatch (Cross-layer)
- RSSI-to-hop mismatch (Cross-layer)

These features collectively operationalize the concept of behavioral inconsistency across PHY, MAC, and Network layers. They were selected because they target the fundamental relationships that routing attacks disrupt, as identified in the theoretical framework of Chapter 3.

4.3.1.6. Cross-Layer Feature Relationship Preservation

A central premise of this study is that routing misbehavior manifests as broken relationships between protocol layers rather than isolated anomalies within a single layer. Accordingly, the feature set is designed to preserve these cross-layer relationships, enabling clustering algorithms to detect structural inconsistencies without imposing attack-specific signatures.

Two examples illustrate this design principle:

- **Receive–forward relationship:** The Forwarding Ratio and Forwarding Consistency Score operationalize the expected invariant that the number of transit packets received equals the number forwarded (packet conservation). Deviation from this relationship directly indicates forward suppression, independent of absolute traffic volume or timing characteristics.
- Physical–logical relationship: The RSSI-Hop Inconsistency feature compares the signal strength of a node’s parent link with the level that the same layer shows in the baseline of the run. A significant mismatch means that the parent link is

unusual for the node's depth, which can reflect a parent change, interference or, in a true wormhole, a shortcut.

These features encode structural expectations derived from wireless mesh theory (Sections 3.1 and 3.2) and attack theory (Section 3.3). They do not embed detection rules (e.g., fixed thresholds for "attack"), but instead preserve the relationships whose violations characterize misbehavior. This design allows exploratory clustering to discover whether such violations naturally separate normal from manipulated windows in the collected dataset.

4.3.2. Theoretical Basis for Data Clustering

The use of unsupervised clustering rather than supervised classification is a deliberate methodological choice grounded in the exploratory nature of this research. This subsection justifies why clustering is an appropriate concept, independent of algorithm mechanics.

4.3.2.1. Behavioral State Hypothesis

This study assumes that wireless mesh behavior exists in discrete operational states:

1. Normal operation – Characterized by stable forwarding ratios, consistent RSSI-hop relationships, and minimal parent switching.
2. Forwarding suppression – Characterized by attacker-side forwarding ratio degradation and an end-to-end delivery reduction at the nodes whose traffic the attacker handles.
3. Topology distortion – Characterized by tunnel activity and duplicate arrivals at the root, and possibly a latency-hop mismatch. Increased parent switching is not expected, because the tunnel does not alter routing.

If cross-layer telemetry captures meaningful deviations, these states should form separable regions in the multidimensional feature space. Clustering tests this hypothesis by attempting to discover groupings without using ground-truth labels during the clustering process.

4.3.2.2. Absence of Pre-Defined Signatures

This study does not assume that the behavioral signatures of routing attacks in ESP-WIFI-MESH are fully known a priori. While theoretical expectations exist (Section 4.3.1), how these expectations manifest in real hardware under controlled conditions is an empirical question. Unsupervised discovery allows patterns to emerge from the data without imposing pre-conceived classifications, aligning with the exploratory objective stated in Section 1.3.

4.3.2.3. Density-Based Justification (DBSCAN)

Theoretically, normal operation is stable and frequent, forming a dense region in feature space. Attack windows differ structurally and may form separate, dense regions. Transitional behavior—such as network recovery or partial manipulation effects—may appear as sparse noise points.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) aligns with this behavioral density assumption. It does not assume spherical clusters, does not require a predefined number of clusters, and explicitly identifies points that do not belong to any dense region. This aligns with the exploratory objective of determining whether natural groupings emerge from the data.

4.3.2.4. Partition-Based Justification (K-Means)

K-Means is included as a comparative structural test. If behavioral states are compact and separable in Euclidean space, partition-based clustering should also reveal coherent groupings. The agreement between DBSCAN and K-Means strengthens the validity of discovered clusters, indicating that separation is not an artifact of a specific algorithm.

DBSCAN is used as the primary method due to its noise handling and ability to discover clusters of arbitrary shape. At the same time, K-Means serves as a comparative baseline to test whether the same behavioral separation emerges under a centroid-partition model.

4.3.2.5. Justification for the Other Algorithms

Three further algorithms test whether a grouping depends on the cluster model. Ward hierarchical clustering gives compact clusters that can be compared with K-Means, and its dendrogram shows how groups nest. A Gaussian mixture model allows overlapping, elliptical clusters and soft membership, which suits windows that lie between normal and attacked behavior. Spectral clustering uses the connectivity between windows, so it can find groups that are not compact in feature space. Agreement between these methods strengthens a finding, and disagreement shows that the grouping depends on the cluster model.

4.3.2.6. Validation Theory

Internal validation metrics assess cluster quality independently of ground truth:

- **Silhouette Score** measures how similar points are to their own cluster compared to other clusters.
- **The Davies-Bouldin Index** measures the average similarity between clusters, with lower values indicating better separation.

These metrics provide evidence that the intra-cluster distance is coherent.

Post-analysis, cluster purity and the adjusted Rand index are computed against experimental phase labels. This two-level validation ensures that discovered clusters are not artifacts of parameter tuning and that they align with the controlled experimental conditions.

4.3.2.7. Alignment with Research Objectives

The study's primary objective is to generate a dataset and explore its structure, not to build a deployable classifier. Clustering directly supports this objective by revealing whether the collected features capture meaningful distinctions between behavioral states. The results inform future work on intrusion detection by highlighting which features are most

discriminative and whether attack-induced behaviors naturally separate from normal operation.

4.3.2.8. Theoretical Expectation

If attacks create measurable cross-layer deviations, then these deviations should manifest as separable behavioral states in multidimensional feature space. Clustering provides an empirical test of this proposition: distinct clusters aligned with experimental phase labels would validate that the feature set captures attack-induced behavioral patterns. In contrast, overlapping clusters would suggest either insufficient feature discrimination or that the theoretical expectations are not reflected in the hardware behavior. Either outcome contributes valuable knowledge to the research community.

4.4. Cluster Visualization and Interpretation

To facilitate interpretation of the clustering results, several visualization techniques will be used to examine the structure of the clusters and the behavioral characteristics associated with each cluster. These visualizations allow the study to analyze how telemetry observations are distributed across clusters and to determine whether the clusters correspond to meaningful network behaviors.

First, dimensionality reduction techniques such as Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE) will be applied to project the high-dimensional feature vectors into a two-dimensional space. These projections will be visualized using scatter plots where each point represents a node–window observation, and colors indicate the cluster assignments produced by the clustering algorithms. This visualization provides an intuitive representation of cluster separation and density structure within the dataset.

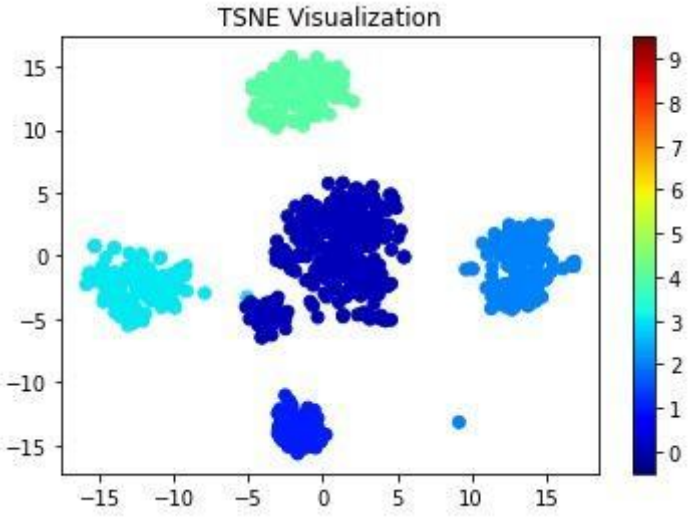


Figure 4.60 t-SNE Cluster Visualization (Illustrative Example)

A t-Distributed Stochastic Neighbor Embedding (t-SNE) visualization is used to display clustered data in a two-dimensional space. t-SNE is a dimensionality-reduction technique that maps high-dimensional feature vectors into a lower-dimensional representation while preserving local similarities between data points. This allows clusters present in the original high-dimensional dataset to become visually distinguishable (van der Maaten & Hinton, 2008).

In the figure, each **dot represents a data point** projected into two dimensions using the t-SNE embedding. The **horizontal and vertical axes correspond to the two t-SNE dimensions**, which do not represent specific original features but instead reflect the relative similarity relationships between observations.

The figure shows several **distinct clusters of points**, indicating groups of data observations that share similar characteristics:

- The **large dark-blue cluster near the center** represents a dense grouping of closely related observations.
- The **cyan cluster on the left side** and the **light-blue cluster on the right side** represent additional groups that are separated from the central cluster based on feature similarity.
- The **green cluster located near the top of the plot** forms another clearly separated group.
- A **small cluster near the bottom of the plot** represents another localized grouping of similar data points.

The **color bar on the right side of the figure** indicates cluster labels or category identifiers assigned to each group. Different colors help visually distinguish clusters and highlight how the algorithm separates groups within the dataset.

The separation between clusters in the t-SNE space demonstrates how high-dimensional feature relationships can be visualized in two dimensions, making it easier to examine cluster structure and assess the separability of patterns within the dataset.

To further characterize the clusters, feature distribution analysis will be performed for each cluster. Summary statistics and box plots will be used to examine how key telemetry features differ across clusters. This helps identify the behavioral patterns associated with each cluster, such as variations in forwarding ratios, delivery ratios, hop counts, or signal strength indicators.

Additionally, heat maps of feature values per cluster will be generated to visualize the relative magnitude of each feature across clusters. In these heat maps, rows correspond to clusters and

columns correspond to features, allowing direct comparison of feature intensities between clusters. This visualization assists in identifying which features contribute most strongly to cluster differentiation.

Cluster characterization will also involve examining the instances assigned to each cluster. By analyzing the distribution of experimental labels (e.g., baseline, blackhole attack, or wormhole attack) within each cluster, the study can determine whether clusters correspond to specific behavioral states of the network. This analysis provides insight into how the clustering algorithms group normal and anomalous network behaviors.

Together, these visualization and analysis techniques enable the study to interpret the clustering results beyond numerical evaluation metrics. They provide qualitative insight into the behavioral patterns present in the dataset and help determine whether the clusters reflect meaningful distinctions between normal network operation and attack scenarios.

The figures of the final clustering run are in Chapter 6 (Section 6.4): the PCA and t-SNE projections in Figures 6.22, 6.23, 6.25, 6.30 and 6.31, the label share and the feature means of each cluster in Figures 6.21, 6.27 and 6.28, and the feature values inside each cluster in Figure 6.29.

5. Methodological Deviations

This chapter states what was changed from the approved proposal and what was added to it. The results that follow from these changes are reported in Chapter 6. Each change carries its identifier from Appendix C (D-1 to D-37), where the full entry is kept. A change is labelled Modified when it replaces something that the proposal specified, and New when it adds a rule, a procedure or a data item that the proposal did not contain.

5.1. Overview of the Changes

The 37 entries fall into five groups (Table 5.1): data collection and sampling, feature definitions, the implementation of the attacks, the experimental design, and run control and validation. Three reasons recur in Appendix C. The first is that a planned measure could not be taken on the ESP32 platform as planned, as with the MAC retry count (D-15) or the round-trip time (D-2 and D-3). The second is that the planned design would not have produced the intended attack or would have exposed the label, as with the relay feature that existed on the attacker alone (D-12) and the wormhole ends at the same depth (D-17). The third is that a check or a gate was needed to make a run a valid experiment, as with the pre-run gates (D-22) and the grading of missing boards (D-32).

Table 5.1 Groups of Changes and Where Their Effect Is Reported

Group	Entries	Main change	Reported in
Data collection and sampling	D-1, D-9, D-13, D-14, D-25	Boards log at 10 Hz and the analysis uses one-second windows of ten samples (the proposal had five-second windows of five samples). The radio is fixed at 20 MHz.	Section 6.1, Table 6.2
Feature definitions	D-2, D-3, D-7, D-8, D-10, D-11, D-15	Latency is one-way, not a round trip. The retry rate comes from application counters, not from the MAC layer. Run identifiers are columns on every row.	Section 6.1 (Table 6.7), Section 6.2.4
Implementation of the attacks	D-12, D-16, D-17, D-20, D-28, D-30, D-36	Both attacks are emulated at the application layer, and every node relays probes hop by hop so that forwarding can be counted. The wormhole ends join the mesh as leaf nodes.	Sections 6.2.1, 6.2.2 and 6.5.3
Experimental design	D-5, D-6, D-18, D-19, D-26, D-29, D-33, D-35, D-37	The baseline is phase 0 of each attack run, and 8 matched benign runs are added, which gives 80 runs. Runs are made at three sites with three scenarios, and the tree builds have their own rules for parents.	Sections 6.1 and 6.5.4

Group	Entries	Main change	Reported in
Run control and validation	D-4, D-21, D-22, D-23, D-24, D-27, D-31, D-32, D-34	A run starts only after pre-run gates are met, and the verifier grades each run, including runs with a missing board and runs whose scenario was not applied.	Tables 6.4, 6.6 and 6.9

Legend:

- Entries = the identifiers of Table C.1 in Appendix C; D-n stands for deviation number n from the approved proposal. The tables of Sections 5.2 to 5.6 list the entries of each group.
- Hz = hertz, samples per second; 10 Hz = one sample every 0.1 s.
- Window = one second of recording from one board, with up to ten samples.
- MHz = million hertz; here the width of the Wi-Fi channel that the radio uses.
- Latency = the time a probe takes to travel from its sender to the root; one-way = measured in the direction of the root only; round trip = there and back.
- MAC layer = the Wi-Fi layer that controls access to the radio channel and retransmits frames that were lost; retry rate = the share of send attempts that failed in a window.
- Application layer = the software of the study that runs on each board, above the mesh stack; application counters = the counts that this software keeps.
- Run identifiers = the columns that name the attack, topology, site, scenario and repeat of a run.
- Relay = to receive a probe and send it on one hop towards the root; the forwarding of a node is counted from this.
- Baseline (phase 0) = the first 300 s of an attack run, before the attack starts.
- Benign run = a run with no attack; matched = set up like an attack run, so that it can be compared with the attack runs.
- Pre-run gates = the checks that the root makes before the timed phases of a run can start, for example that every child has joined.
- Verifier = the script that checks every run against its schedule (Section 4.2.7.1).

Chapters 5 and 6 cite each change by its identifier, in the form D-20. D stands for deviation from the approved proposal. Table 5.2 lists all 37 identifiers in numerical order with the topic name used in Appendix C, what the change means in one line, and the table of this chapter that gives the entry in full.

Table 5.2 Index of the Deviation Identifiers D-1 to D-37

ID	Topic	In plain words	Given in full in
D-1	Sampling rate	Each board records 10 samples per second, so a one-second window holds ten samples.	Table 5.3
D-2	LatencyHopRatio	Latency is the one-way delay of a probe per hop, not a round-trip time.	Table 5.4
D-3	TunnelLatency	Tunnel latency is the gap in arrival time between a probe and its tunnelled copy at the root.	Table 5.4
D-4	Topology check	Parent changes in the first 60 s count as the mesh forming, not as re-routing.	Table 5.7
D-5	Baseline control	The first 300 s of every attack run serve as the baseline.	Table 5.6
D-6	Repeats and parents	The topology type is fixed for a run, but which board sits where is left to the mesh.	Table 5.6
D-7	run_repeat column	Every row records which repeat of a run it belongs to.	Table 5.4
D-8	PDR attribution	A window gets a delivery ratio, even 0, when the node sent in it and the root’s log covers it.	Table 5.4

ID	Topic	In plain words	Given in full in
D-9	Grid and window	The data are cut into one-second windows, and a window needs at least 8 of its 10 samples.	Table 5.3
D-10	Run columns	Every row records the attack, topology, site and scenario of its run.	Table 5.4
D-11	Hop, not layer	`The feature counts hop changes, where hop = mesh layer − 1.`	Table 5.4
D-12	Application-layer relay	Every node forwards probes itself at the application layer, so that forwarding can be counted.	Table 5.5
D-13	microSD mirror	The memory-card copy of the log is forced to be written to the card.	Table 5.3
D-14	Radio width	The Wi-Fi channel is fixed at 20 MHz and recorded in every capture.	Table 5.3
D-15	Application counters	Retry counts come from the software on the boards, not from the Wi-Fi hardware.	Table 5.4
D-16	Star and blackhole	In the star topology the attacker is the hub and the root sits behind it.	Table 5.5
D-17	Wormhole ends	The two ends of the wormhole tunnel are chosen at run time from mesh depth and then locked.	Table 5.5
D-18	Matched benign runs	Benign runs without any attack are added as a matched control; the final plan has 8.	Table 5.6
D-19	Sites	Runs are made at three sites: G402, the DLSU Library and the Yuchengco Building.	Table 5.6
D-20	Attack emulation	Both attacks act on the study's own probes at the application layer, not on the routing.	Table 5.5
D-21	Wormhole verification	The wormhole is verified by two independent measures, one from the attackers and one from the root.	Table 5.7
D-22	Pre-run gates and START	A run starts only when the setup checks pass and the operator types START.	Table 5.7
D-23	Parent changes in the attack	A board may change parent during the attack if it stays a victim or stays a bystander.	Table 5.7
D-24	START lock	A failed check blocks START unless a reason is typed and logged.	Table 5.7
D-25	Tick alignment	Each sample takes the earliest free time slot, at most 300 ms late.	Table 5.3
D-26	Sticky tree parents	In tree runs the boards switch parent less readily.	Table 5.6
D-27	Topology gate	A board passes the topology check if it is attached at the start of the baseline and does not re-route.	Table 5.7
D-28	Star wormhole	In the star the exit end of the wormhole is the hub.	Table 5.5
D-29	Partial mesh	The partial mesh is defined by placement and at most two children per node.	Table 5.6
D-30	Wormhole end depth	In tree and partial-mesh wormhole runs the entry end sits deeper than the exit end.	Table 5.5
D-31	Attacker re-parenting	The attacker may change its own parent only if no board changes class and its uplink is down at most 10% of the attack.	Table 5.7
D-32	Missing boards	A missing board is graded by its role, and not by a plain board count.	Table 5.7

ID	Topic	In plain words	Given in full in
D-33	Jitter scenario	The jitter scenario is left out of the plan, which leaves three scenarios.	Table 5.6
D-34	Scenario not applied	A run whose scenario was not applied is graded as minor gaps.	Table 5.7
D-35	Benign run plan	The benign runs are spread over the sites and the scenarios; 8 remain in the final plan.	Table 5.6
D-36	Wormhole ends as leaves	In the tree and the partial mesh, the wormhole ends join as leaf nodes.	Table 5.5
D-37	Tree fan-out	The number of children that a tree board accepts is set for each build.	Table 5.6

Legend:

- Topic = the name used in Table C.1 of Appendix C. Given in full in = the table of this chapter that gives the entry in full and shows whether it is Modified or New.
- D-n = deviation number n from the approved proposal; the full entry is in Table C.1 of Appendix C.
- Kind: Modified = replaces what the proposal specified; New = added to the proposal.
- Window = one second of recording from one board, with up to ten samples.
- Hz = hertz, samples per second; 10 Hz = one sample every 0.1 s.
- Tick = one slot of the 10 Hz sampling grid (0.1 s).
- Probe = the small packet that a node sends towards the root about once a second to measure delivery.
- Root = the board that receives every probe and logs the arrivals (one in each run).
- Hop = one wireless link on the path to the root.
- Mesh layer = the depth of a board in the mesh tree, with the root as layer 1; hop = layer − 1.
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1).
- LatencyHopRatio = the relative one-way delay of a probe divided by its hop count; TunnelLatency = the difference in arrival delay between the two copies of one probe at the root.
- run_repeat = the column that gives the number of the repeat of a run; HopChangeCount = the number of changes of the hop of a board in a window.
- Topology = the shape in which the boards are linked to the root: linear (a chain), star, tree or partial mesh.
- Wormhole = the emulated tunnel in which a probe is also copied over a wired link from the entry end (Node B) to the exit end (Node A).
- Hub = the board in the centre of the star, through which every other board reaches the root.
- Application layer = the software of the study that runs on each board, above the mesh stack; application counters = the counts that this software keeps.
- microSD mirror = the copy of the log that each board writes to a memory card.
- START = the operator's command that begins the timed phases of a run.
- Pre-run gates = the checks that the root makes before the timed phases of a run can start, for example that every child has joined.
- Verifier = the script that checks every run against its schedule (Section 4.2.7.1).

The 37 entries of Appendix C were not planned at the start of the study. They were made during the study, each time that a capture or a review showed that the approved plan could not be followed as written, or that following it would yield data that could not be trusted. The earliest entries arose from the first captures (D-1, D-2, D-3 and D-4), several from the later development work (D-8, D-11, D-14 and D-15), and the last from the capture campaign, in which failed runs and reviews were turned into rules so that the same failure could not recur, and in which the plan itself was adjusted (D-21 to D-37). For each entry, Appendix C records what the proposal said, what is done instead, why, and what the change costs.

Three reasons recur. The first is that a planned quantity could not be measured on the platform: a round-trip time requires a reply that the probes never receive (D-2 and D-3), and ESP-IDF provides no MAC retry count (D-15). The second is that the planned design would have allowed the label to be read from the data without any learning, or would not have produced the intended attack: a relay feature that exists on the attacker alone identifies the attacker (D-12), a wormhole whose two ends lie at the same depth has no shortcut (D-17, D-28 and D-30), and the ends of a wormhole in a tree or a partial mesh did not join the mesh in a way that gave a valid run (D-36 and D-37). The third is that a run must constitute one valid experiment, whereas a mesh heals itself, boards reset and cards fail, so that rules had to be written for when a run may start and for what a run may contain (D-4, D-21 to D-32 and D-34). Some entries record a decision on the scope of the study and not a problem of the platform, for example the choice of sites (D-19), of benign runs (D-18 and D-35) and of scenarios (D-33). Of the 37 entries, 17 are labelled Modified and 20 New in the tables of this chapter.

Sections 5.2 to 5.6 take the five groups of Table 5.1 in turn. Each begins with a table that states, for every entry, what the study does and why, and continues with a narration that treats the entries of the group one by one. The narration supplies what the table cannot: what the proposal specified, what happened in the captures, how the change works, the evidence that it worked, and what it costs. Section 5.7 states which of the changes limit how the results of Chapter 6 can be read.

5.2. Data Collection and Sampling

The proposal planned five-second windows of five samples. The first captures showed that this left no room for a dropped sample, so the sampling rate, the grid and the window were changed (D-1, D-9, D-25), and two protections of the data were added (D-13, D-14).

Table 5.3 Changes to Data Collection and Sampling

ID and topic	Kind	What the study does	Why
D-1 Sampling rate	Modified	The boards log at 10 Hz, and the analysis keeps the 10 Hz samples (see D-9).	Five samples per window left no room for a dropped sample.
D-9 Grid and window	Modified	A 10 Hz grid and one-second windows are used; a window with fewer than 8 of 10 samples is discarded.	The 80% completeness ratio is kept, with more support per row.
D-25 Tick alignment	New	Each sample takes the earliest free tick, at most 300 ms late.	Placement on the nearest tick discarded healthy windows.
D-13 microSD mirror	New	The microSD mirror is flushed with fsync().	It fixes a failure in which a card file held 0 rows. It has not been validated by a mid-run reset.

ID and topic	Kind	What the study does	Why
D-14 Radio width	New	The radio is fixed at 20 MHz (HT20), and every capture records its width.	The sniffer captures 20 MHz only, and the two widths differ in PHY behaviour.

Legend:

- Kind: Modified = replaces what the proposal specified; New = added to the proposal.
- Window = one second of recording from one board, with up to ten samples.
- Sample = one reading of a board (one telemetry row); the boards take 10 samples per second.
- Hz = hertz, samples per second; 10 Hz = one sample every 0.1 s.
- Tick = one slot of the 10 Hz sampling grid (0.1 s).
- Interpolation = filling a short gap with values computed from the samples on either side; forward fill = repeating the last value.
- Baseline run = a capture with no attack.
- microSD mirror = the copy of the log that each board writes to a memory card.
- fsync() = a call that forces the data in a file to be written to the card.
- Mesh formation = the time in which the boards join the mesh and choose their parents.
- HT20 = the 20 MHz Wi-Fi channel width; HT40 = the 40 MHz width, which is the default of the ESP32; MHz = million hertz.
- PHY = the physical layer of Wi-Fi, which sets the radio rates and the use of the channel.
- Sniffer = a separate radio that records the Wi-Fi frames.

Entries D-1 and D-9 concern the sampling rate and the analysis window. The proposal planned raw telemetry at 1 Hz and windows of five seconds, which gives five samples per window, with a window being discarded when fewer than 4 of its 5 samples are present. With so few samples, a single lost sample costs 20% of a window and can trigger the discard rule. The boards therefore sample at 10 Hz, one sample every 100 ms.

In its first form, the change (D-1) preserved the analysis rate of the proposal by reducing the data to 1 Hz before windowing, which discarded 9 of every 10 samples that had been captured. D-9 replaced that step: the data are analysed on the 10 Hz grid in windows of one second, as the panel and the adviser requested, and a window with fewer than 8 of its 10 samples is discarded, which is the same 80% completeness ratio as 4 of 5.

The number of rows provided a further reason. The panel asked for about 10,000 rows per run and the team had proposed 6,000, and the number of rows is determined by the number of nodes and by the analysed seconds divided by the window length. With five-second windows and the six nodes then in use, 6,000 rows would have required a run of about 83 minutes.

Each window now holds ten samples instead of five and therefore carries more support. Nothing is invented to reach this: gaps of one or two samples are filled, by interpolation or forward fill, which at 10 Hz covers 0.2 s where at 1 Hz it covered 2 s, so that less of the data is synthetic than before. On a clean baseline run, the change raised the number of rows from 577 to 2,894 (five times as many) and lowered the share of discarded windows from 1.0% (6 of 583 windows) to 0.1%.

One consequence must be kept in mind. A probe is sent about once a second, so a window of one second holds about one probe, and the delivery ratio of a single window is close to 0 or 1. The delivery ratio is therefore read through the mean over many windows and not as a fine-grained rate.

Entry D-25 concerns the alignment of the samples to the 0.1 s grid. The sampling task of the firmware runs on a fixed period. When the task is held up, it takes the missed samples back to back, so that the gaps between consecutive rows can be, for example, 258 ms, 1 ms and 41 ms. The first version of the analysis placed every sample on the nearest tick of the grid. Two samples that arrived in a burst therefore shared one tick and left an empty tick behind, and three empty ticks were sufficient to discard the window. Windows were consequently lost although no sample was missing on the card.

The rule is now that each sample takes the earliest free tick for which it could have been scheduled, at most 3 ticks (300 ms) late and never early. The lateness limit derives from the data, since the 99th percentile of the measured delay was 278 ms. There is one sample per tick, and nothing is invented, interpolated further or counted twice. The 80% rule and the filling of gaps of one or two samples remain as before. On two boards with clustered timestamps, FE90 and ED80, the share of discarded windows fell from 10% to 0% and from 15% to 0%, and boards that were already clean did not change. No verdict changed, but the grades of some runs became more accurate. The reason for the change is that the old rule measured the resampling and not the board, whereas a missing window should now correspond to a missing sample.

Entry D-13 concerns the microSD mirror of the log. Each board writes every telemetry row to its flash memory and mirrors it to a microSD card. Operators found that a file pulled from a card sometimes held no rows, with no pattern that could be identified, and the run had to be repeated. The cause lay in the logger, which flushed with fflush() alone. On the file system that ESP-IDF uses for the card, this pushes the bytes to the card but does not update the directory entry of the file, which holds its recorded size and is updated only by fsync() or by closing the file. A run that did not end cleanly, because of a brownout, a reset, a card pulled while the board was on, or a killed run, therefore left a file whose size still read 0 bytes although its rows were on the card, and every host tool counted it as empty. Clean runs were never affected, which is why the failure appeared random.

The logger now calls fsync() on the mirrored files at most every 5 s and at every phase boundary, so that the worst case falls from the whole run to the last few seconds. The change repairs nothing that was captured before it, and a file from before the change that holds 0 rows has to be treated as lost and not as a board that logged nothing. None of the 435 campus capture files has 0 rows. At import, the number of rows streamed is now compared with the number in the manifest of the run, and a

difference is printed. The firmware compiles. Among the 637 telemetry files of the 80 runs of the final set, none has 0 rows; the one board without a file is B768 in the blackhole run in the tree topology at G402 with highload (Table 6.6). The change has not been validated by a reset in the middle of a run followed by a read-back of the card, which is the test that would show that a file survives an unclean end of a run.

Entry D-14 concerns the width of the radio channel. The proposal does not state a channel width, so every early capture used the ESP32 default of 40 MHz (HT40). The only independent observer available to the study, a sniffer, captures 20 MHz only, and a 20 MHz receiver cannot decode 40 MHz data frames. In one capture it heard the beacons of every board but almost none of their data frames (0 for the root and 3 of 1,848 for a child). Without a change, the sniffer could not have served as the third-party check on the dataset.

The radio is now fixed at 20 MHz (HT20) on both of its interfaces before it starts. Every boot prints the width, and the status report of the card records it, so that every capture carries its own width. The order matters: a first version fixed the width after the radio had started, and no node joined the mesh in a full 11-minute run, whereas fixing it before the start resolved this.

Captures at 40 MHz differ in the PHY (the rates, the airtime and possibly the RSSI by a few dB). They are archived, labelled by width and never pooled unlabelled, and the dataset uses the captures at 20 MHz only. The verifier is not affected, because it compares phases within one run. Mesh formation at 20 MHz was verified on hardware in a test of 3 minutes. The 80 runs of the final set were all captured after the change, and each holds windows in the baseline, the attack (or the point at which it would start) and the cooldown, so that full runs at 20 MHz are in the dataset. The sniffer decoded data frames in 7 of the 36 blackhole runs (Section 6.2.4).

5.3. Features and Measurements

Several features were defined differently from the original planned implementation because the platform does not provide the quantity that was planned. Probes travel one way to the root, so latency is one-way (D-2, D-3), and ESP-IDF returns no MAC retry count, so the retry rate comes from application counters (D-15). Three entries add rules for when a feature has a value (D-8) and for naming and labelling (D-7, D-10, D-11).

Table 5.4 Changes to Feature Definitions and Measurements

ID and topic	Kind	What the study does	Why
D-2 LatencyHopRatio	Modified	LatencyHopRatio is the relative one-way delay divided by the hop count (Equation 4.14).	Probes travel one way, so no round trip exists in the data.
D-3 TunnelLatency	Modified	TunnelLatency is the difference in arrival delay between the two copies of one probe.	The tunnel is a one-way UART write with no echo.
D-8 PDR attribution	New	A window gets a PDR, including 0, when the node sent, was associated and the root's log covers it.	The old run-wide rule hid the zeros of the nodes most affected.
D-11 Hop, not layer	Modified	`The feature is named HopChangeCount, and hop = layer − 1 is added next to the raw layer.`	The old name suggested network-layer data.
D-15 Application counters	Modified	retry_count and tx_count are application counters, and RetryRate is not a model input.	ESP-IDF returns no MAC retry count.
D-7 run_repeat column	New	The repeat number is a column in the windowed and feature tables.	Pooled runs cannot be separated again, so per-repeat variance would be lost.
D-10 Run columns	New	Attack, topology, location and scenario are columns on every row.	Without them, runs of different scenarios collapsed into one group.

Legend:

- Kind: Modified = replaces what the proposal specified; New = added to the proposal.
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1).
- UART = the wired serial link between the two tunnel boards.
- MAC = medium access control, the Wi-Fi layer that controls access to the radio channel and retransmits frames that were lost.
- Sniffer = a separate radio that records the Wi-Fi frames.
- LatencyHopRatio = the relative one-way delay of a probe divided by its hop count (Equation 4.14); relative = counted from the fastest delivery of the same sender in the run.
- TunnelLatency = the difference in arrival delay between the two copies of one probe at the root.
- One-way = the time to go to the root only; round trip = the time to go to the root and come back.
- Probe = the small packet that a node sends towards the root about once a second to measure delivery.
- Root = the board that receives every probe and logs the arrivals (one in each run).
- Hop = one wireless link on the path to the root.
- Mesh layer = the depth of a board in the mesh tree, with the root as layer 1; hop = layer − 1.
- HopChangeCount = the number of changes of the hop of a board in a window; RetryRate = the share of send attempts that failed in a window.
- Application counters = counts kept by the software of the study, here retry_count (failed sends; on Node B, frames sent into the tunnel) and tx_count (frames sent).
- Mesh association = the state in which a board holds a parent in the mesh.
- Network layer (IP) = the layer of the OSI model that carries IP addresses and routing; the mesh works below it.
- ESP-IDF = the software framework of Espressif, the maker of the ESP32.
- Windowed table = the table of one-second windows made by the preprocessing; feature table = the same windows with the features added.

Entry D-2 concerns LatencyHopRatio. Equation 4.14 defines it as the mean round-trip time divided by the hop count, computed from probe-response exchanges between the victim nodes and the root. In the system that was built, a probe goes from a node to the root, and the root logs the arrival and

sends nothing back, so no round trip exists in the data. The root does log a delay for every arrival, but it is the difference between the clock of the root and the clock of the sender, and each clock starts at the boot of its own board and is never synchronised with the other. The raw value is therefore the true one-way delay plus a large constant, which is strongly negative in practice because the children are powered before the root (−194 s on one wormhole run).

The constant is the same for all arrivals of one sender in one run, so subtracting the smallest delay of that sender cancels it and leaves the delay relative to the fastest delivery of that sender. LatencyHopRatio is this relative delay divided by the hop count. Each arrival is assigned to the window of the sender whose probe sequence numbers contain it, which is the same clock-free join that PDR uses. An earlier version joined on time, which mixed the two clocks and placed the latencies of the cooldown in attack windows; it was corrected.

On an early baseline run, the median was between 2.4 and 2.8 ms per hop at every depth from 1 to 5 hops. A flat ratio is what Equation 4.14 describes as normal forwarding. The range of 10 to 30 ms per hop that the proposal expected was an estimate for a round trip, and a one-way leg is about half of that, on boards that are faster than the estimate assumed. What the feature is for, namely detecting a delay that does not match the reported path length, is preserved, because it depends on the ratio varying and not on its size. Its limit is that the zero point belongs to each node and each run, so the values cannot be compared in absolute terms between nodes or runs. Only their shape can be compared, and the clustering of Chapter 6 uses standardised features.

Entry D-3 concerns TunnelLatency. The proposal defines it as the round-trip latency of the tunnel itself, measured with periodic echo messages. The tunnel between the two wormhole boards is a one-way write on a UART line, from Node B to Node A, and no echo frame is defined or sent, so there is no round trip to measure. What does exist is the signature of a wormhole: the same probe reaches the root twice, once by the multi-hop path and once through the tunnel, with a measurable difference in delay. The root does not remove the duplicate, so both arrivals reach the log.

TunnelLatency is therefore the largest minus the smallest delay among the arrivals of one probe (one sender and one sequence number). The two copies share one sender clock and one root clock, so the offset of D-2 cancels exactly, and the value is a measurement and not an estimate. On an early linear wormhole run there were 541 pairs of duplicates with a median difference of 12.2 ms and a range of 1.2 to 242 ms, and 102 of the 109 rows that have a value fall in the attack window.

The proposal gives the tunnel fields to the attacker nodes only. TunnelLatency is instead assigned to the sender whose probes were duplicated, as PDR is, because the difference belongs to the manipulated traffic and because the arrival log does not mark which copy came through the tunnel. TunnelIntensity and TunnelBytes remain with the attacker as specified. For the same reason, the difference has no sign: it does not indicate whether the tunnel was the faster path.

Entry D-8 concerns the attribution of PDR (Equation 4.5), which joins what each node sent with what the root logged. In the first code, a window could have a PDR only if the root had logged something from that node somewhere in the run. A node that the root never heard from therefore received no PDR in any window, and this is the node that a blackhole affects most severely. On a linear blackhole capture, three of the four victims sent in nearly every window and were treated as having no data, so that PDR = 0 appeared in none of the 446 rows. The feature that exists to detect the suppression of forwarding could not record the value it exists to detect.

A window now receives a PDR, including 0, when three conditions hold. First, the node sent in the window, because a ratio of 0 over 0 is undefined. Second, the node held a mesh association (a layer above 0 and a parent address that is not empty), because the undelivered probes of a detached node show lost connectivity and not suppressed forwarding. Third, the window lies inside the span in which the root was logging arrivals, which still prevents a missing root file from being read as a failure of delivery.

The first code also divided by the sum of 0 and a small constant, so that a window in which a node sent nothing came out as 0.0, the same as total failure, and a false blackhole signature could arise in this way. Such windows are now missing values. On that capture, the windows with a PDR rose from 41 to 238 of 446 and the windows with PDR = 0 from none to 201, and the PDR of the victims in the three phases read 0.217, 0.000 and 0.750, which is the expected shape of suppression and recovery. The definition of PDR is unchanged. Only the rule for when a window can be judged changed, and it is conservative in both directions, because it stops hiding real zeros and stops inventing false ones.

Entry D-11 concerns the naming of the hop count. ESP-WIFI-MESH numbers the depth of a node in its tree as a layer, with the root as layer 1, so that a direct child of the root is layer 2. Reviewers read the column name as an OSI layer. It is not, because the mesh runs entirely below IP and the dataset holds no network-layer data, and each time the name is misread, the reader believes that the cross-layer claim is larger than it is. The adviser asked for the change and a second reviewer asked for it independently, so the problem lay in the name and not in the reader.

A new column hop = layer − 1 is added next to the raw layer, which is kept for traceability to what the firmware reported, and LayerChangeCount is renamed HopChangeCount. The offset of one is essential: a direct child is one hop from the root and the root is 0 hops from itself, and a rename that kept the values would have made every hop count wrong by one. The no-parent value of the firmware (layer = −1) becomes a missing hop value. HopChangeCount counts changes in depth, which a constant offset does not alter, so no feature value changed, and the pipeline gave identical values under the new name.

Entry D-15 concerns the application counters. The proposal takes retry_count, the cumulative MAC-layer retransmissions, and tx_count, the cumulative frames transmitted, from the Wi-Fi statistics of ESP-IDF. It also gives a fallback for a version that does not expose them: record zero, flag it in the metadata, leave RetryRate out of the core set, and use an application-layer proxy if wanted. The firmware reads no driver statistics. This was checked: the only statistics call in ESP-IDF 5.3.5 and in the documentation of version 5.5, esp_wifi_statis_dump(), prints the statistics to the log and returns only a status, so no number reaches the application, and no change to the firmware could put a MAC retry count into the data.

Both columns are therefore filled with application counters. On every role except the wormhole entry node B, retry_count is the number of failed esp_mesh_send() calls, that is, sends that the mesh stack refused, and not radio retransmissions. On Node B it holds the number of frames sent into the tunnel, because the tunnel features read it from there. The columns are not zero and are not flagged in the metadata, so this departs from the fallback of the proposal, and the MAC layer of the cross-layer claim rests on the RSSI and on the state of the link and the parent, and not on retransmissions.

RetryRate remains a column of the dataset. It is not a model input, as the fallback prescribes, and the verifier no longer uses it as a check, because none of the attack studies cited (Airehrour et al. [70], Zhukabayeva et al. [71], Ramírez Gómez et al. [72]) names a retry rate as a signature. The predicted rise in the retransmissions of the victims (Table 3.4) cannot be seen with these counters. A blackhole that drops at the application layer still acknowledges every frame, so the radio of the victim never sees a failure; the prediction is reported in Section 6.2.4 as predicted and not observed, and not as missing data.

Real retries come from the sniffer: a retransmitted 802.11 frame carries the retry bit, and counting it per link and per phase gives the retry rate of an independent observer. Where a sniffer capture exists, the pipeline can add its retry rate and the number of frames it heard (MacRetryRate and

MacFramesHeard) to the feature table, and a run without a capture remains a missing value and never 0.

Entries D-7 and D-10 concern the columns that identify a run. One analysis folder holds all repeats of a cell, and the preprocessing reads the whole folder, so the windows of the three repeats, and of the different scenarios, landed in one table with nothing to tell them apart. The script that gathered the folders re-derived the cell from the folder path and ignored the scenario, so a run with mobility and a run with no variation, of the same attack, topology and site, collapsed into one group. Runs can always be pooled but can never be separated again.

The preprocessing now writes run_repeat, taken from the file name, and attack, topology, location and scenario, taken from the path of the export, as columns on every windowed row, and they pass unchanged into the feature table. A run with no scenario folder is read as stationary, and a path that is not an export tree receives no value, so that an unknown origin is never labelled as if it were known. These columns are identifiers: no feature value, count of windows or label changed. They make every row self-describing, which the dataset needs in order to be reused, and they allow Chapter 6 to report results by topology, site and scenario.

5.4. Implementation of the Attacks

Both attacks are application-layer emulations (D-20). The mesh stack relays packets by itself and documents no way to write a node's routing table, so the attacks act on the probes that the study sends and not on the routes. To make forwarding visible, every node relays probes at the application layer (D-12). The remaining entries fix where the attacker and the wormhole ends sit in each topology (D-16, D-17, D-28, D-30) and how the ends join the mesh (D-36).

Table 5.5 Changes to the Implementation of the Attacks

ID and topic	Kind	What the study does	Why
D-12 Application-layer relay	Modified	Every node relays at the application layer and counts packets received, forwarded and dropped.	The relay features existed only on the attacker.
D-20 Attack emulation	Modified	Both attacks act on the data plane only, and no route or parent selection is changed.	ESP-WIFI-MESH documents no way to write a routing table.
D-16 Star and blackhole	Modified	In the star, the attacker is the hub and the root sits one hop behind it.	The root cannot drop instead of forward, and it is the referee.

ID and topic	Kind	What the study does	Why
D-17 Wormhole ends	New	The wormhole ends are assigned at run time from mesh depth, locked at the first phase and recorded in every row.	The mesh decides which board is near the root, and a fixed assignment can give no shortcut.
D-30 Wormhole end depth	New	In tree and partial-mesh builds, Node B never joins the root and stays deeper than Node A.	Both ends kept landing at the same depth.
D-28 Star wormhole	Modified	In the star, wormhole end A is the hub and the root sits one hop behind it.	In a flat star, the tunnel skips no hop.
D-36 Wormhole ends as leaves	New	In tree and partial-mesh builds, the wormhole ends join as leaf nodes, and the telemetry stack is 6144 bytes.	Boards under an end moved whenever the end restarted, and the telemetry stack ran low.

Legend:

- Kind: Modified = replaces what the proposal specified; New = added to the proposal.
- Hub = the board in the centre of the star, through which every other board reaches the root.
- Hop = one wireless link on the path to the root.
- Wormhole = the emulated tunnel in which a probe is also copied over a wired link from the entry end (Node B) to the exit end (Node A).
- Node A = the exit end of the wormhole, on the root side; Node B = the entry end, on the leaf side.
- Star = every other board is linked to one central board, the hub; tree = boards branch from the root over several hops; partial mesh = a branched tree in which a board can hear several candidate parents and has at most two children (Section 5.5).
- Blackhole = the emulated attack in which a board that relays probes silently drops them; the attacker is that board.
- Relay = to receive a probe and send it on one hop towards the root; the forwarding of a node is counted from this.
- Application layer = the software of the study that runs on each board, above the mesh stack; application counters = the counts that this software keeps.
- Referee = the root, which drives the phases and logs the arrivals from which PDR is measured.
- Routing = the choice of the path that a packet takes; parent selection = the way in which a board chooses the board it sends to.
- Shortcut = a path to the root with fewer hops than the normal path.
- Mesh depth = the layer of a board in the mesh tree; locked = a role that is fixed at the first phase and does not change for the rest of the run.
- UART = the wired serial link between the two tunnel boards.
- Baseline (phase 0) = the first 300 s of an attack run, before the attack starts.
- Verifier = the script that checks every run against its schedule (Section 4.2.7.1).
- Leaf node = a board that forwards no traffic for other boards, so that no board can attach below it; child = a board that attaches below another board.
- Telemetry task = the task on each board that reports its state to the root; stack = the memory reserved for a task; backtrace = the record of the calls that led to a restart.
- Layout = which board sits at which place in the mesh; recapture = to run the capture again.

Entry D-20 concerns the way in which the attacks are emulated. In the studies that the thesis cites, a blackhole advertises a false short route to attract traffic and then drops it, and the tunnel of a wormhole causes the routing protocol to adopt a false shortcut, so that distant nodes appear to be neighbours. Both attacks therefore act on the control plane. This study implements only the behaviour on the data plane, on real hardware. The blackhole is a legitimate mesh node that silently drops the probes it relays during the attack phase, and traffic reaches it because of where it is placed, between the victims and the root, and not because of a false route. The wormhole is a real out-of-band wired tunnel on a

UART line between two boards: Node B sends a copy of each probe through it, Node A re-injects the copy near the root, and the root keeps both copies. The routing of the mesh never adopts the tunnel.

The reason is the platform. ESP-WIFI-MESH is a self-organising tree, so there are no route request and reply messages to falsify. Each node chooses its parent from information in the mesh beacons, and the routing tables are kept inside the closed-source mesh library of Espressif. To our knowledge, the documentation of the public interface lists a call that reads the routing table, and none that writes a route or changes what a node advertises, so that false routes could be created only by forging the management frames of the mesh or by reverse-engineering a proprietary stack, which lies outside an exploratory data analysis.

The dataset therefore records the data-plane signature of each attack (for the blackhole, probes that vanish; for the wormhole, duplicated probes with a difference in latency) and not the stage in which the attacker attracts traffic, so no parent changes or shifts in the topology that the attacker caused are in the data. The position of the attacker becomes an experimental variable instead, which is why D-12, D-16 and D-17 matter. Section 6.5.3 discusses what an emulation can and cannot show.

Entry D-12 concerns the relaying of traffic at the application layer. Three of the 16 features (ForwardingRatio, IngressEgressDelta and ConsistencyScore) are relay features. In the first traffic model, the honest nodes sent to the root with esp_mesh_send(), and the mesh stack relayed that traffic below the application layer, so the code of an intermediate node never saw what it forwarded. Only the blackhole attacker saw transit packets, and only because the victims were built to address it by its MAC address. The three features therefore existed on one node role, and a column that is empty identified the attacker and with it the label. This is the objection that the panel raised: if one feature decides whether an instance is an attack, machine learning is unnecessary. It could not be repaired in Python, because the numbers did not exist.

A shared relay component now runs on every node. Each node sends to its own parent with the MESH_DATA_P2P flag and relays what it receives one hop further up, and it counts the packets received, forwarded and dropped. The blackhole attacker runs the same relay and differs from an honest node by one decision, whether to forward. Both wormhole ends relay as well, with their tunnel logic unchanged. This follows the forwarding discipline that the proposal itself requires of intermediate nodes. Every hop goes to the node’s own parent, so traffic only ascends the tree and no loop is possible, and a node without a parent counts the packet as dropped. Two effects follow. The relay features are now present on every node, so that the attacker is an outlier in a populated

distribution and not the only value that exists. And the attacker intercepts traffic because of where it sits in the tree and not because the victims were built to address it.

A fault was found and corrected during the work: the relay at first forwarded only ordinary probes, so the duplicate that Node A re-injects, which carries a different marker, would have been dropped silently by every intermediate relay. The duplicates that constitute the wormhole signature would then never have reached the root, and the wormhole runs would have shown no wormhole signature. The relay now forwards both kinds. Captures made before this change are not comparable with later captures, because the traffic model differs. The change also departs from the text of the milestone form, in which victims address the attacker directly, although the milestone criteria are all still met.

Entry D-16 concerns the blackhole in the star topology. The star figure of the proposal makes the root the attacker, with the victims connected directly to it as their parent. This cannot be done as written, for three reasons. First, the mesh is standalone, with no router, so the root is the destination of the probes and there is nothing onward to forward to, and ‘drop instead of forward’ has no meaning at the root. Second, the root is also the referee, because it drives the phases and PDR is measured from its log of arrivals, so that a root that drops is indistinguishable from a root that stops logging, and the attacker would be grading itself. Third, the other reading, one ring node as the attacker, cannot work either, because the star was limited to a depth of 2 and a node in layer 2 has no children, so nothing ever passes through it.

In the star blackhole builds, the star is therefore built one hop down: the root, as the sink and the referee, in layer 1; the attacker as the hub in layer 2; and every victim in layer 3. The victims do not organise themselves. Each scans for the mesh access point of the attacker and fixes it as its parent with esp_mesh_set_parent(), the sequence of the manual networking example of ESP-IDF. A victim that cannot find the attacker keeps scanning and never joins the root directly, because it would then be an unexposed bystander, and the root holds the baseline until every board is in.

The first hardware run formed the hub shape, but only the victims built with the address of the real attacker joined, and a victim built with an old address scanned indefinitely. The setup tool now passes the address of the chosen attacker into every blackhole build. What the proposal intended is preserved: one central node that every victim uses as its direct parent and that drops everything during the manipulation window. Every victim is behind the attacker, so in the star the PDR is not averaged with unaffected nodes, unlike the tree and the partial mesh.

Entry D-17 concerns the assignment of the two ends of the wormhole. Which board ends up near the root is chosen by the mesh and not by the experimenter. Originally, the end of each board was fixed when the board was flashed. A linear wormhole setup formed with Node B next to the root (hop 1) and Node A at hop 6, so the tunnelled copy had to climb back up 6 hops and arrived later than the normal copy of B, which is no shortcut and gives no latency signature, while every check passed. A tree capture had both ends at hop 1 for the whole attack. A fixed assignment cannot guarantee the shortcut that defines a wormhole.

One firmware image now serves both ends. The two boards exchange a message with a check code (HELLO) over the tunnel cable every second, carrying their mesh layer, their role and whether it is locked. Before phase 0, the deeper board becomes B, the entry, and the shallower board becomes A, the exit, and at equal depth the board with the lower MAC address is A. The role locks at the first real phase and never changes, because a swap during a run would corrupt the capture, and a board that restarts in a running run takes the locked role that its peer reports. If no HELLO arrives, because of a cable fault or old firmware, the role flashed on the board is used. The role that was decided is in the heartbeat and in the role column of every telemetry row, so that the dataset records which board was which.

The assignment worked on hardware on a linear run at the DLSU Library: A in layer 5, B in layer 7, roles stable for the whole run, and all 181 of B's attack probes duplicated. The labelling changed with it. The firmware tunnels only the probes of B, so a wormhole run has no victims: every other non-attacker child is labelled as not tunnelled, and the attack is the duplication of B's probes. The automatic assignment fixes the direction, with A always the shallower, but not the separation, which still depends on where the mesh places the boards, so that the number of hops skipped is reported for each run.

Entry D-30 concerns the depth of the wormhole ends in the tree and partial-mesh topologies. The tunnel is a shortcut only if A is closer to the root than B. Placement was expected to provide this, but the two ends hang off one UART cable and therefore lie close together, and ESP-WIFI-MESH joins the shallowest parent that it can hear. A partial-mesh setup repeatedly placed both ends at hop 1, with B remaining on the root even at −82 to −89 dBm, so that the root held phase 0 because of the 'same depth' condition and boards had to be moved and unplugged repeatedly. The automatic assignment of D-17 cannot help with equal depth, because it only renames the ends.

In tree and partial-mesh builds, the board flashed as B therefore no longer organises itself. It scans and fixes as its parent the strongest mesh node that is not the root, so that B is always at hop 2 or

deeper, and if no such node is within reach it waits and the root holds phase 0. The board flashed as A fixes as its parent the shallowest open parent it hears at −85 dBm or better, which is the root whenever the root has room, and the parent of B must be at least as deep as A, so that B is at least one hop deeper. The two ends do not use the automatic switching of parents of ESP-WIFI-MESH. They keep a parent while it works, even if the link weakens, and change only when the parent is lost (three drops in a row) or full, and the new parent must still keep A shallower than B. Every other board organises itself as before.

Linear needs no rule, because with one child per node every board has a different depth, and star is covered by D-28. The price is that the position of B is constrained by design, so the tunnel skips at least one hop by construction, and B can still hang directly under A, in which case A relays the normal copy of B as in the star.

Entry D-28 concerns the wormhole in the star topology. In a flat star every child is one hop from the root, so both tunnel ends lie at hop 1. The normal copy of B reaches the root in one hop, and the tunnelled copy, B through the cable to A and then to the root, is also one hop. No hop is skipped and no latency advantage can exist. The gate of D-22 would hold phase 0 indefinitely, and the verifier would mark every such run as the same depth and duplication only. Moving boards cannot help, because a flat star has no deeper layer for B.

The star wormhole is therefore built as the star blackhole of D-16: the root in layer 1, wormhole end A as the hub in layer 2, and every other board, B included, in layer 3. Every non-root board other than the hub scans for the access point of the hub and fixes it as its parent, and the hub organises itself and can reach only the root. The hub is the A board of the preset, whose address the setup tool passes to the build in the same way as the address of the blackhole attacker, and the automatic assignment of D-17 needs no change, because the hub is the shallower end. The normal copy of B now takes two hops (B, A, root) and the tunnelled copy one (B through the cable to A, then the root), a shortcut of one hop. It remains a star in structure, with one centre, every other node one link from it and no links among the leaves; what changes is where the sink sits, which is behind the centre and not at it, as in D-16.

Two alternatives were rejected. A star kept flat with the gate relaxed would give duplicated probes with no shortcut, which is a replay and not a wormhole, and the latency side of the signature could never appear. Dropping the wormhole in the star would lose one cell of the 2 by 4 matrix of attacks and topologies.

Three side effects have to be disclosed. First, A also relays the normal copy of B, because A is the only parent of B, so that both copies pass through A; neither primary check depends on this, since DuplicateRate is measured at the root and TunnelIntensity on B. Second, the shortcut of one hop is the smallest possible, so the difference in latency is small and may lie within the jitter, and TunnelLatency and LatencyHopRatio are measured and not scored (D-21). Third, the control boards are two hops from the root, through A, so their PDR and latency baseline differs from that of a flat star; they are labelled as not tunnelled and are not victims (D-17).

Entry D-36 concerns the way in which the wormhole ends join the mesh. Originally, the two tunnel ends joined as ordinary mesh nodes, so that other boards could attach below them. In a tree run at G402 with stationary traffic, end A, at hop 1, carried two boards (1C38 and FE90) as children, and the normal path of B ran through 1C38. A restarted three times during the run, and each time its children re-parented during the attack phase, which the verifier reports as a layout that changed during the capture and which requires a recapture. The tunnel was also down while A rebooted, and the root printed a warning that the telemetry task had only 504 bytes of its 4096-byte stack left. The cause of the restarts of A was not proven when the entry was written, because no backtrace had been retrieved yet, and exhaustion of the telemetry stack was one candidate. The boards had already been placed by hand, and a team member asked for a fix in the firmware because the problem remained.

In the tree and partial-mesh builds, the wormhole ends, which are parented by hand (D-30), now declare themselves leaf nodes of the mesh. In ESP-IDF a leaf has no forwarding ability, so no board can attach below A or B. The stack of the telemetry task is raised from 4096 to 6144 bytes on every board, which adds about 2 KB of memory to each board. The star builds and all other parent rules are unchanged. The reason is that an end that also relays the traffic of honest boards mixes two roles. As a leaf, it carries only its own probes and the tunnelled probes, and a restart of an end no longer moves honest boards, while the normal copy of B still goes through honest relays. Captures made before this change may have had children under A or B. When the entry was written, the firmware had been compile-checked and not yet tested on hardware, and every board had to be flashed again because the stack change is in every build. The entry was checked with the adviser.

Because of D-20, the blackhole results of Chapter 6 describe the suppression of forwarding and not the attraction of traffic (Section 6.5.3).

5.5. Experimental Design

The design of Section 4.2.2.5 crosses 4 topologies, 2 attacks, 3 sites and 3 scenarios, which gives 72 attack runs, and adds matched benign runs. The plan for the benign runs changed several times (D-18, D-33 and D-35) and ended with 8 benign runs, which gives 80 runs in all. The changes concern the baseline and the benign control (D-5, D-18, D-35), the scenarios (D-33), the sites (D-19) and the definition of the topologies (D-6, D-26, D-29, D-37).

Table 5.6 Changes to the Experimental Design

ID and topic	Kind	What the study does	Why
D-5 Baseline control	Modified	The baseline is phase 0 (300 s) of each attack run, with the benign runs of D-18 and D-35 added.	Phase 0 shares firmware, roles, session and placement with the attack.
D-18 Matched benign runs	New	Each site and topology gets one benign run, under different scenarios (the final plan is in D-35).	Every topology needs a stand-alone benign capture, under more than one condition.
D-35 Benign run plan	New	The benign runs are spread over the sites and the scenarios; 4 uncaptured ones were dropped, which leaves 8 in a plan of 80 runs.	The team wanted one benign run at each site under a different scenario; the campaign ended before the other 4 were captured.
D-33 Jitter scenario	Modified	The jitter scenario is left out of the plan, which leaves three scenarios.	A scope decision of the team, checked with the adviser.
D-19 Sites	Modified	Runs are made at G402, the DLSU Library and Yuchengco.	The three sites differ in Wi-Fi interference.
D-6 Repeats and parents	Modified	The topology class is fixed and checked; the position of each board is not fixed.	Pinning parents would override the parent selection under study.
D-26 Sticky tree parents	New	In tree runs, the parent-switch and weak-link thresholds are 6 dB lower and the watch is three times longer.	Children re-parented on weak but working links.
D-29 Partial mesh	Modified	The partial mesh is defined by placement and at most two children per node.	No processed run showed a node under a second parent.
D-37 Tree fan-out	New	The number of children that a tree board accepts is a setting of each build, and the fan-out that forms is read from the logs of each run (Table 6.35).	A tree run with the stock limit of 10 children came out flat like a star, whereas partial-mesh runs with a limit of two formed several hops.

Legend:

- Kind: Modified = replaces what the proposal specified; New = added to the proposal.
- Scenario = the traffic pattern of a run: stationary, burst or highload (Section 4.2.2.5).
- Sites: G402 (Room G402, Gokongwei College of Computer Studies (GCCS)), the DLSU Library and the Yuchengco Building.
- Weak-link watch = the time a link must stay weak before a board changes parent.
- Baseline (phase 0) = the first 300 s of an attack run, before the attack starts.
- Benign run = a run with no attack; matched = set up like an attack run, so that it can be compared with the attack runs.
- Originate = to send probes of one's own, as opposed to relaying those of other boards.

- Topology class = linear, star, tree or partial mesh, fixed by a limit on the depth and on the number of children of a node.
- Star = every other board is linked to one central board, the hub; tree = boards branch from the root over several hops; partial mesh = a branched tree in which a board can hear several candidate parents and has at most two children (Section 5.5).
- Sticky parents = tree parents that a child keeps unless the link is really bad.
- dB = decibels; the thresholds for switching parent are 6 dB lower.
- Parent switching = a board changing the board it sends to.
- Firmware = the program stored on the boards.
- Hop = one wireless link on the path to the root.
- Root = the board that receives every probe and logs the arrivals (one in each run).
- Mesh layer = the depth of a board in the mesh tree, with the root as layer 1; hop = layer − 1.
- Depth = the highest layer of the mesh; a mesh of depth 2 is flat like a star.
- Jitter = a fourth traffic scenario of the working plan, which is not one of the three scenarios of Section 4.2.2.5.
- Plan = the list of runs that the campaign tool draws for every site, topology, attack and scenario; skip = a setting that leaves a scenario out of the plan.
- Confounded = mixed with another factor, so that the effect of one cannot be told from the effect of the other.
- Fan-out = the number of children that a board accepts; stock limit = the default of the mesh software, which is 10 children.
- Soft access point = the part of a board to which other boards connect.
- dBm = decibel-milliwatts, the unit of the signal strength of a link.

Entry D-5 concerns the baseline control. The milestone form implies a separate capture without an attack for each topology, as the reference of normal behaviour. The study instead takes phase 0 of each attack run, the first 300 s in which nothing is attacked, and labels it normal. This control is better matched. Phase 0 shares the firmware, the node roles, the mesh session and the placement of the attack that follows, so that the attack that is switched on is the only variable.

A separate baseline run differs in two ways at once: it has a different firmware, and it has a different number of probing nodes, because in an attack run one board is the attacker or a tunnel end and relays probes instead of originating them. In the first captures of the linear topology, a baseline run had 5 probing nodes that together sent 4.90 probes per second, and phase 0 of an attack run had 4 that together sent about 3.9. Comparing the windows of a baseline run with the windows of an attack run would therefore confound the attack with a change in the composition of the traffic, whereas comparing within a run does not. The cost is that, for the star, the tree and the partial mesh, there is no reference of how the topology behaves with no attack firmware at all. The matched benign runs of D-18 are meant to supply a stand-alone control.

Entry D-18 concerns the matched benign runs. At first, a benign run was added only to the cells that drew the burst scenario, and every other cell used phase 0 of its attack run as its control. The team decided that every topology should have a whole benign run of its own and that the benign runs should cover different conditions, and not all be burst. Every combination of a site and a topology was given exactly one benign run. The four topologies of a site were given four different scenarios, each of them a scenario that the attack runs of that cell had drawn, so that the benign run is still matched. With the three sites of D-19, this came to 12 benign runs and 108 runs in all (96 attack runs and 12 benign runs). A benign run has the attack set to none and is otherwise run like an attack run, with the same phases and the same traffic scenarios, so that it is matched to the attack runs at its site and scenario.

It gives the dataset a stand-alone class of normal behaviour, apart from the phase 0 and the cooldown of the attack runs.

Entry D-35 concerns the distribution of the benign runs over the sites and the scenarios. With the jitter scenario taken out of the plan (D-33), three of the 12 benign slots of D-18 had been jitter and were dropped, which left 9 benign runs. Later, the team asked for the benign runs to be arranged differently: for each topology, the three benign runs, one at each site, use three different scenarios, and the assignment of the scenarios to the sites is shuffled for each topology. Each scenario then has four benign runs, and each site has one or two benign runs of each scenario. The attack runs are not changed and still cover every scenario. This restored the number of benign runs to 12 and brought the plan to 84 runs (72 attack runs and 12 benign runs). The pairing with the attack runs holds, because every attack cell runs all three scenarios, but within one topology the benign scenario and the site are confounded, so that the attack runs at the same site and scenario provide the matched comparison. This point was checked with the adviser.

The campaign ended with fewer benign runs than planned. At the end of the campaign, the four benign runs that had never been captured were removed from the plan: the DLSU Library with the tree (burst) and with the partial mesh (highload), and Yuchengco with the tree (highload) and with the partial mesh (stationary). The final plan has 8 benign runs and 80 runs in all, and all 8 benign runs have been captured. Four of the benign runs are at G402 (linear with highload, star with burst, tree with stationary and partial mesh with highload), two at the DLSU Library (linear and star, both with stationary) and two at Yuchengco (linear with stationary and star with highload). There are therefore 4 stationary, 3 highload and 1 burst benign runs, and the tree and the partial mesh each have one benign run, both at G402.

The final list differs from the draw above in five of the 8 cells. The star was captured with burst at G402, stationary at the DLSU Library and highload at Yuchengco, where the draw had stationary, highload and burst; the partial mesh at G402 was captured with highload, where the draw had burst; and the linear topology at Yuchengco was captured as stationary, where the draw had burst. The plan was set to match the captures. The rule of three different scenarios for each topology therefore holds for the star, which has burst, stationary and highload, but no longer for the linear topology, whose three benign runs are highload at G402 and stationary at both the DLSU Library and Yuchengco. The final plan of 8 benign runs and 80 runs in all is the one analysed in Chapter 6.

Entry D-33 concerns the jitter scenario. The plan of the campaign had four scenarios for every combination of site, topology and attack: stationary, highload, burst and jitter, which came to 108

planned runs (96 attack runs and 12 matched benign runs). Two further scenarios, mobility and power cycle, had been dropped from the plan earlier, so that every combination of site, topology and attack ran the same four. The team decided to take jitter out of the plan for the time being. This is a decision about the plan and not a response to a problem in the data. A setting of the planning tool now lists the scenarios that are skipped, and with jitter skipped the plan comes to 81 runs (72 attack runs and 9 benign runs). The drawn order of the slots is kept, so that restoring jitter restores the plan of 108 runs exactly. The five jitter captures that had already been made remain on disk and are counted as outside the plan. The three scenarios that remain are those of Section 4.2.2.5, stationary, burst and highload, and the results of Chapter 6 use them. Each of the 24 combinations of topology, attack and site then runs the three scenarios once, so that the 72 attack runs of the plan file are the crossed design of Section 4.2.2.5, one run for each combination, and all 72 were captured (Table 6.1). The entry is a scope decision of the team, checked with the adviser.

Entry D-19 concerns the sites. The panel-change plan listed four sites: home, G402, the DLSU Library and a site called Goks. The team later dropped home as a campaign site and named the site called Goks as Yuchengco. The campaign is now designed around three sites, which differ in Wi-Fi interference. The tools that plan a run and the pickers of the setup tool now offer G402, the DLSU Library and Yuchengco. The draws of the attack scenarios for these three sites were not changed, the draws for home were removed, and the benign runs were drawn again for three sites. The captures already made at home remain on disk, and the tools still accept home for them, but the results of Chapter 6 come from the three campaign sites. In the firmware, the name Goks was replaced by Yuchengco, and no data were ever captured under the old name. Boards that carry older firmware do not know the new name and have to be flashed again before it is written to the card as the location.

Entry D-6 concerns the repeats and the parents. The milestone form implies three repeats of the same configuration for each cell. The topology class, whether linear, star, tree or partial mesh, is fixed at build time by a limit on the depth and on the number of children of a node, and it is verified in every run, because the verifier rebuilds the structure from the parent links that were logged. Which board sits at which position is not fixed, because every board chooses its parent by signal strength each time it boots. Pinning parents would mean overriding the parent selection of the mesh, which is the behaviour under study, and the self-organising structure is what makes the data mesh data and not data of a fixed route.

The evidence is in three repeats of a linear wormhole setup, in which the distance between Node A and Node B was adjacent, then 2 hops and then 3 hops, and all three still passed the check of one

node per layer and depth 6. The attack signature did not change (181, 181 and 180 duplicates), because the tunnel is a physical wire whose behaviour does not depend on the distance in the mesh. The features of the radio path, RSSI_Hop_Diff, LatencyHopRatio and HopStabilityDuration, do vary between repeats. This is real behaviour of the mesh and not noise to be removed, and the run_repeat column of D-7 allows the variance to be measured.

Entry D-26 concerns the stability of parents in the tree topology. In the standard ESP-WIFI-MESH, a child starts to look for a better parent when the RSSI of its parent falls into the bands of −78, −82 and −85 dBm. The links of the testbed lie at a median of −71 to −78 dBm and many boards are below −78 dBm, so the children of a tree re-parented in the middle of a run on a link that still worked, and this changes who is a victim without any fault.

In the tree builds, on the boards other than the root, the defaults of the mesh are read at boot and the thresholds for keeping or switching a parent are made 6 dB weaker, and the weak-link watch lasts three times as long. The old and the new values are printed on the serial log of every board. A parent that crashes or vanishes still makes its children move, so the self-healing of the mesh is kept (D-23). A first version also lowered the thresholds that a board uses when it first picks a parent. The root then counted as good enough from 6 dB farther away, more boards joined it directly, and a tree run stayed two hops deep and flat like a star, so the change was narrowed to the keeping and switching of a parent, and the selection of a parent is the standard one again.

The trade-off is that a child may stay on a weak link and lose some packets, which is why the setup tool warns below −78 dBm so that the placement can be corrected first. Tree runs captured before this firmware used the standard thresholds, and this has to be stated for each such run.

Entry D-29 concerns the partial mesh. The proposal describes the partial mesh as the primary experimental environment, with branched, semi-structured connectivity in which nodes can attach to a subset of parents and switch between them. In all 9 partial-mesh captures examined, the structure was a plain tree: no child was seen under a second parent during the experiment, and the verifier prints a warning that it is still a plain tree. Switching cannot be checked before START, because every mesh looks like a tree until a node actually switches, so no gate before the start can enforce it.

The team chose to keep the captures and the method, and to describe the partial mesh by how it is built: boards placed so that each is in radio range of several candidate parents, at most two children per node, which forces branching and depth instead of a flat star, and ESP-WIFI-MESH left free to re-parent. Parent switching was allowed but not observed. Forcing a switch, or checking before the

start that a board can hear two parents, would need new firmware and a new capture of all 9 runs, and a forced switch would be an artefact and not behaviour of the mesh, so this was rejected. The results for the partial mesh are reported as placement-defined branched trees with a limit of two children, and adaptive parent switching is not claimed but listed as a limit (Section 6.5.4).

Entry D-37 concerns the number of children that a tree board accepts. ESP-WIFI-MESH organises itself by signal strength and joins the shallowest parent that it hears, so that with a free root every board in one room lands at hop 1. In a tree wormhole run at G402 with stationary traffic and the stock limit of 10 children, the root took five boards at hop 1, and the only deep branch (one board at hop 1, a second at hop 2 and B at hop 3) dropped during the baseline. The root then reported that the layout was flat like a star (depth 2) and marked the run for recapture. Partial-mesh runs in the same room, with a limit of two children, had formed the root, two boards at hop 1, and then hops 2 and 3 every time.

Limiting the connections of the soft access point of each node is the documented lever in ESP-IDF, and it is what made the partial-mesh runs multi-hop. The number of children is therefore a setting of each build. Because the fan-out that forms also depends on the placement of the boards, the structure of every run is read from the parent addresses in its logs and reported in Table 6.35. The entry was checked with the adviser.

The tree runs do not all have the same fan-out. The root has 2 to 5 children in the tree wormhole runs and in the tree blackhole runs, and 5 in the benign tree run (Table 6.35). The fan-out of the tree therefore differs between runs, and the comparison of the tree with the partial mesh in the wormhole runs is limited accordingly (Section 6.5.4).

The logs of the runs also show how the wormhole ends and the parents behaved. The tunnel ends are leaves (D-36) in all 9 tree wormhole runs. In the 19 tree runs of the final set, 1 of the 132 boards changed parent in the labelled phases, in the blackhole run at G402 with stationary traffic.

5.6. Run Control and Validation

Most of the new work lies here. A mesh heals itself, boards reset and cards fail, so a run is valid only if the topology, the attack and the boards are checked against the run’s own schedule. The proposal did not contain the pre-run gates, the START lock, the grading of missing boards, the rules for parent changes during an attack or the grading of a scenario that was not applied. They were added one by one as captures failed (D-21 to D-32 and D-34).

Table 5.7 Changes to Run Control and Validation

ID and topic	Kind	What the study does	Why
D-4 Topology check	Modified	Parent changes in the first 60 s of a node's log count as mesh formation.	Initial parent acquisition was read as baseline re-routing.
D-22 Pre-run gates and START	New	The root holds the baseline until the gates pass, and the operator then types START.	Runs that would have been invalid can no longer start.
D-24 START lock	New	A failed pre-run check locks START, and an override needs a typed, logged reason.	START could be pressed before the requirements were met.
D-27 Topology gate	Modified	A board passes if it is attached at the start of the baseline and does not re-route during it.	The time taken to form is no longer part of the data.
D-21 Wormhole verification	New	The wormhole has two independent primary checks, TunnelIntensity and DuplicateRate [72].	A self-reported counter is weak evidence, and the root's view is independent.
D-23 Parent changes in the attack	New	A parent change during the attack is tolerated if the board keeps its class; a class flip fails the run.	The mesh heals itself, and what matters is whom the attack reaches.
D-31 Attacker re-parenting	New	The attacker's own parent change is tolerated only if no board changes class and its uplink was down for at most 10% of its attack windows.	An attacker outage drops probes in the same way as the attack.
D-32 Missing boards	New	A missing board is placed by structure only and graded by its role.	A board count hid two missing boards and could not tell evidence from control.
D-34 Scenario not applied	New	A run whose scenario was not applied is graded as minor gaps.	The attack result stands, but the run is not the scenario it is filed as.

Legend:

- Kind: Modified = replaces what the proposal specified; New = added to the proposal.
- Verifier = the script that checks every run against its schedule (Section 4.2.7.1).
- Victim = a board other than the root and the attacker whose path to the root passes through the attacker; bystander = such a board whose path does not (the terms are defined at the start of Chapter 6).
- Transition window = a window whose rows belong to two phases.
- START = the operator's command that begins the timed phases of a run.
- START_ANYWAY = the command that overrides the pre-run gates; it is logged.
- Pre-run gates = the checks that the root makes before the timed phases of a run can start, for example that every child has joined.
- Node A = the exit end of the wormhole, on the root side; Node B = the entry end, on the leaf side.
- UART OK = the wired link between the two wormhole boards works in both directions.
- Topology table = the list, printed by the root, of every board with its layer and parent.
- Operator = the person who runs the capture.
- TunnelIntensity = the number of messages per second sent through the tunnel, counted by the attacking boards; DuplicateRate = the share of the windows of Node B in which the root receives a duplicate arrival of a probe.
- TunnelLatency and LatencyHopRatio are defined in Table 5.4; reported and not scored = shown for description and not used to decide the verdict.
- 3-sigma test = the check that an attack value lies at least three baseline standard deviations from the baseline mean (Section 4.2.7.1).
- Class = victim, bystander, detached (rejoining) or unknown (the chain of parents cannot be resolved), decided for each board and each second from its chain of parents; re-parenting = a board changing its parent.
- Uplink = the link from a board to its parent.

- Undetermined = the grade of a run whose outcome cannot be judged, because the role of a missing board, or the evidence it held, cannot be determined.
- Scenario = the traffic pattern of a run: stationary, burst or highload (Section 4.2.2.5).
- Grade = the verdict of the verifier on a run: strong, good, minor gaps or major gaps; a run with minor gaps is done, with a recapture advised.
- Scenario check = the part of the verifier that tests whether the traffic of a run was the scenario under which the run is filed; NOT APPLIED = the check found that it was not.
- Impact check = the part of the verifier that repeats the test of the attack without the boards that have gaps, to see whether the result depends on them.

Entries D-4 and D-27 concern the topology check. The milestone form asks that each topology converge to its intended structure within 60 seconds, and that the mesh stay stable through a full baseline of 5 minutes with no spontaneous re-routing. The verification tool was treating these as one criterion. The firmware stamps a row as baseline before the first broadcast of the root arrives, so the first acquisition of a parent by a node was counted as re-routing in the baseline. On an early baseline run, all 6 changes happened between 0 and 12.8 s, with no re-parenting in the remaining 470 s or so. The mesh was stable and the tool said it was not. By D-4, parent changes in the first 60 s of the log of a node count as formation, and the number is printed on its own line and never hidden.

D-27 went further. The tool timed convergence from the first log row of each board and failed any board that took longer than 60 s, even one that had settled minutes before the experiment, and it failed 31 of 54 captures. Since D-22, the operator presses START only after the mesh has settled, so the time to form depends on the setup and is not part of the data. A board now passes when it has a parent at the moment phase 0 begins and does not re-route during the baseline. The time to form is printed (‘settled N s before phase 0’) and not judged. Moves after the start of phase 0 are judged by the verifier (D-23), so that the two tools cannot contradict each other. Of these 31 captures, 29 changed from fail to pass, because nothing moved during their experiments, 2 remain as fail for genuine reasons, and no verdict or grade changed, since the verifier never used the rule of 60 s.

Entries D-22 and D-24 concern the conditions under which a run may start. The proposal starts the baseline when the mesh has stabilised. Wormhole ends lock their roles at phase 0, and a tree run is corrupted when boards are moved after it. Some linear runs started with both tunnel ends set as B because of a UART jumper that carried data one way only. The root now holds phase 0 until the following conditions are met: every child is in (the roster gate); in a tree blackhole run, the attacker has a child; in a wormhole run, there is exactly one Node A and one Node B, the UART works in both directions, and A is closer to the root than B; and the operator has typed START, in both attacks, after checking the topology table. START_ANYWAY overrides the gates and is announced prominently in the log.

D-24 moved the checks of the setup tool from a question to a lock. A check that would put a known fault into the data (children missing, the wrong location on a card, a tree blackhole attacker without a child, victims aiming at another attacker address, a wormhole that is not set up) locks START, shown in red, and unlocks it again when the check turns green. An override needs a typed reason of at least 15 characters and the word OVERRIDE, the reason is written to the run log, and the verifier prints it as a warning. Checks shown in amber, such as the estimate of the clock and a weak link, only ask. Neither change touches the data. They stop a run that would have been invalid from starting, and the procedure can state that the operator confirmed the layout before the baseline.

Entry D-21 concerns the verification of the wormhole. Windows are labelled by phase, and an attack is confirmed by a 3-sigma test of the signature features, following Zhukabayeva et al. [71]. For the wormhole there are two independent primary checks. The first, TunnelIntensity, is the count of the attacking boards themselves. The second, DuplicateRate, is the share of the windows of Node B in which the root records a duplicate arrival, measured at the root as in Ramírez Gómez et al. [72]. A single self-reported counter is weak evidence, and the view of the root is independent of the attacking boards.

TunnelLatency and LatencyHopRatio are reported and not scored. TunnelLatency exists only while the tunnel runs, so there is no distribution for the normal phase, and the unsynchronised relative delay of LatencyHopRatio is so dispersed that the largest attainable |z| was between 0.23 and 0.64; a test that can never pass is not evidence in either direction. A window whose raw rows span two phases is marked as a transition window and receives no label, because the first tunnelled probe was being counted as baseline. A board whose exit from a phase fell in discarded windows is anchored on that exit, so that its valid baseline is kept, because a stall in the telemetry had been deleting valid baseline windows. On a dry run over 44 attack cells, every live verdict was unchanged, and the wormhole cells now have 2 of 2 primary checks.

Entry D-23 concerns parent changes during the attack. A mesh heals itself, so boards change parent for reasons that have nothing to do with the attack. Originally, any change of parent by a board that the attack depends on, an attacker or a victim, during the attack failed the run as an invalid setup, even when the board had only rejoined under the same attacker after a neighbour crashed. The verifier now classes every board in every window by following its chain of parents at that moment: a victim if its path to the root runs through the attacker, a bystander if it does not, detached if it is rejoining, and unknown if the chain cannot be resolved.

A parent change of a victim or of a wormhole end is tolerated, with a warning, only if all of the following hold. The board stays in one class, with fewer than 3 windows in the other, and a flip of 3 or more in each direction fails the run. The windows with no traceable path to the root are left out of the 3-sigma test, which makes the test harder and not easier, and up to 10% of them is reported as a brief rejoin and more as a long outage. And the 3-sigma test, repeated on the attack windows before the first re-route alone (at least 60 s are needed), still confirms the attack; otherwise the run fails as not robust.

The rule is outcome-blind, which is why it cannot make an invalid run acceptable. The decision to tolerate or to reject reads only the parent address and the hop. PDR and ForwardingRatio, which the test scores, are shown and never used to accept a run, which is tested. The thresholds were fixed in advance and are the same for every run. The rule is symmetric, because a flip from victim to bystander and from bystander to victim fail alike. The outcome is used once, in the robustness test, and it can only lower a verdict. In a wormhole run, a change is tolerated while A stays shallower than B, and 3 seconds or more of the opposite fail the run. A board whose class in the last 60 s of the baseline, or the first 60 s of the cooldown, differs from its class in the attack receives a warning, because that confounds the features of the board. A victim that drops out is a warning while other victims remain, and losing the attacker or every victim is a failure.

A lock that pins the parents at START was rejected, because it would contradict the self-healing that the proposal describes, it could not help when the parent itself crashes, and it is a larger change of method than a rule of the verifier. What decides whether a run is one experiment is who the attack reaches, and not which address a board has as its parent. A check over 53 analysed captures changed one verdict. A tree blackhole run at G402 was an invalid setup because two victims rejoined after another board crashed. They stayed victims, they were detached for 5 s and 7 s against a limit of 18 s, and the 156 s before the re-route confirm the attack on their own, so the run is confirmed with 2 tolerated warnings. The tree run at G402 with highload remains an invalid setup, because board B768 really did flip between victim and bystander.

Entry D-31 concerns the re-parenting of the attacker. D-23 never tolerated the attacker moving, because its position is the treatment. A review held that a change in the relations of the nodes should not invalidate a run unless it breaks an essential requirement of the experiment or compromises the evidence. The treatment of a blackhole is who sits behind the attacker, its subtree of victims, and who does not, the bystanders. An attacker that takes a different parent above it changes neither by itself, and what can change them is already measured every second by the classes.

A second flaw was found in the same code. A board counted as a victim as soon as its chain of parents met the attacker, also while the attacker had no uplink or had crashed, when every probe is lost for a reason that is not the blackhole. A board is now a victim in a given second only if its whole path to the root runs through the attacker, and the seconds in which the attacker has no uplink or is silent count as no path and are left out of the 3-sigma test, which can only make the test stricter.

The attacker changing its own parent is tolerated, with a warning, only if all three conditions hold: no board switched between victim and bystander during the attack, at least one board stayed a victim, and the attacker was without an uplink for at most 10% of its attack windows. Otherwise the run fails with the reason named. The decision uses the layout only and never delivery or forwarding. Re-running the verifier over all 56 cells then live changed no verdict, grade or campaign status, because no capture had a moved attacker.

Entry D-32 concerns missing boards. Completeness used to mean a root, the file of arrivals, at least 3 child files, and at least 95% coverage of each. A board with no file at all was invisible to the verifier, and a board that went silent made one blanket major gap in every blackhole run. A missing board should not by itself invalidate an attack that is otherwise valid, but missing data that could compromise the signature must never be overlooked. A separate tool now finds the boards that the root heard (at least 100 arrivals in phases 0 to 3) but that have no file, and the boards with no attack windows. It places each by structure only, from its own file, then from the run log of the root, then from the boards that name it as their parent, and never from its delivery.

The tool grades by role. A missing attacker is critical. A missing victim is yellow if the other victims keep at least 60 traceable seconds, and critical if not. A missing bystander, or a relay above the attacker, is yellow if another control board keeps at least 60 s, and undetermined if it was the last control. A board in an unknown position is undetermined, and a plain child of a wormhole is yellow. The new grade, undetermined, means that the outcome is inconclusive and the run is flagged for investigation, and a capture that is incomplete only in the coverage of a board is marked for checking and not for repeating.

Two real runs had a board that the root heard but that had no file (a tree blackhole run at G402 with highload, board B768, and a partial-mesh wormhole run at G402 with highload, board 2805), and both had been graded without anyone knowing. Both now carry a named yellow footnote. The thresholds of 100 arrivals and 60 s are engineering decisions.

Entry D-34 concerns a scenario that was not applied. The scenario check of the verifier only labelled the verdict line of a run, for example ‘scenario burst: NOT APPLIED’, and the grade ignored it, so that a run could still be graded strong or good although its traffic was not the scenario under which it was filed. Later, a scenario that was not applied became a minor gap, which grades the run as done with a recapture advised. The attack result stands, but the run is not the scenario it is filed as. The scenario check was added after the impact check, which repeats the test of the attack without the boards that have gaps, had been merged into the grade. The scenario check concerns no board, so the impact check never tests it again, and it never raises a run to a major gap by itself. A scenario that is unclear or not visible remains a note. Two blackhole runs changed from good to minor gaps, linear at G402 with burst and linear at home with burst (home was removed as a campaign site by D-19), and both had already been marked for checking in the campaign table by design. No other run changed.

These rules explain why Chapter 6 reports a run as confirmed, as carrying a minor or major gap, or as inconclusive (Tables 6.4 and 6.6), and not only as passed or failed.

5.7. What the Changes Mean for the Results

Four of the changes limit how the results can be read. The first is D-20: the dataset records the effect of the emulated attacks on traffic and not the attraction of traffic by a routing protocol. The second is D-15: no MAC retry count is available on the boards, so retransmissions are read from the application counters and, where a sniffer was present, from the sniffer. The third is D-18, as revised by D-35: the 8 benign runs of the final plan are a control and not a counterpart of every cell (Section 6.5.4), and within one topology the benign scenario and the site are confounded. The fourth is D-37: the tree runs do not share one fan-out, since the root has 2 to 5 children (Table 6.35), and the comparison of the tree with the partial mesh in the wormhole runs is limited accordingly (Section 6.5.4).

6.Results and Discussion

This chapter reports what the collected data show and what they do not show. The changes made to the approved proposal are described in Chapter 5, so only results are reported here. All 80 runs of the final plan were captured and analysed: 36 blackhole runs, 36 wormhole runs and 8 benign runs (repository snapshot THESIS3, commit 970fb15). Section 6.1 describes the dataset as collected. Section 6.2 reports whether each attack behaved as designed. Section 6.3 describes the data through exploratory analysis and states no detection result. Section 6.4 reports the clustering. Section 6.5 sets the results against the research objectives and the published studies, and Chapter 7 concludes the study and gives the recommendations for future work. Both attacks are emulated at the application layer, that is, they act on the study's own probes and not on the routing (deviation D-20, Section 5.4), so every result describes what the emulated attack does to the traffic and to the cross-layer features, and not what an attack on the routing protocol would do. A reference such as D-20 points to the deviation log of Appendix C, and Table 5.2 explains all 37 in plain words.

Several terms are used throughout the chapter in a fixed sense. The root is the board that receives every probe and logs the arrivals, and there is one in each run. The child nodes are the boards other than the root, 7 in a run of 8 boards. In a blackhole run, one child node is the attacker, which drops the probes it should relay, and the other 6 are non-attacker nodes. In a wormhole run, two child nodes are the tunnel ends, the exit end (Node A) and the entry end (Node B), and the other 5 are non-attacker nodes. The tunnel ends are reported on their own, because their tunnel counters and the duplicate copies of the entry end's probes at the root are the direct record of the attack. Wherever a table or a figure refers to non-attacker nodes, these boards are meant. In a blackhole run, a non-attacker node is a victim when its path to the root passes through the attacker, and a bystander (also called an unaffected node) when it does not. In a wormhole run only the probes of the entry end are sent through the tunnel, so no non-attacker node is a victim and none of them is touched by the attack. A benign run has no attack and no attacker, and its child nodes are all treated as normal.

A window is one second of recording from one board, with up to ten samples, and every count of windows in this chapter counts such windows. Baseline and cooldown windows are labelled normal (0), blackhole attack windows are labelled 1 and wormhole attack windows are labelled 2. The analysis of one attack alone uses attack against normal, and the pooled analysis of Section 6.4.7 uses the three labels. The label is set for every board of the run, including the bystanders and the other non-attacker nodes of a wormhole run.

6.1. Dataset Collected

The design of Section 4.2.2.5, as changed by D-18 and D-35 (Section 5.5), crosses 4 topologies, 2 attacks, 3 sites and 3 scenarios, which gives 72 attack runs, and adds 8 benign runs, which gives 80 runs. All 80 were captured (Table 6.1). Each attack has one run in each of its 36 cells: the 4 topologies (linear, star, tree and partial mesh) at the 3 sites (Room G402 (GCCS), the DLSU Library and the Yuchengco Building), each in the 3 scenarios (stationary, burst and highload). The 8 benign runs are 4 stationary, 3 highload and 1 burst, spread over the 4 topologies and the 3 sites (Table 6.3). Every run has 8 boards, the root and 7 child nodes, except the benign run in the star topology at Yuchengco with highload, which has 6, and the blackhole run in the tree topology at G402 with highload, which has 7 because board B768 recorded no data. Every attack run follows the same schedule: 60 s of mesh formation, 300 s of baseline, 180 s of attack and 120 s of cooldown, and a benign run has the same schedule without the attack phase. Ten samples per second are taken by each board and grouped into one-second windows (10 Hz sampling, D-1, and one-second windows, D-9).

Table 6.1 Runs Captured against the Plan, by Topology, Site and Scenario

Group	Value	Blackhole (runs)	Wormhole (runs)	Benign (runs)	All runs
Topology	Linear	9 of 9	9 of 9	3 of 3	21 of 21
Topology	Partial mesh	9 of 9	9 of 9	1 of 1	19 of 19
Topology	Star	9 of 9	9 of 9	3 of 3	21 of 21
Topology	Tree	9 of 9	9 of 9	1 of 1	19 of 19
Site	G402	12 of 12	12 of 12	4 of 4	28 of 28
Site	DLSU Library	12 of 12	12 of 12	2 of 2	26 of 26
Site	Yuchengco	12 of 12	12 of 12	2 of 2	26 of 26
Scenario	stationary	12 of 12	12 of 12	4 of 4	28 of 28
Scenario	burst	12 of 12	12 of 12	1 of 1	25 of 25
Scenario	highload	12 of 12	12 of 12	3 of 3	27 of 27
All runs		**36 of 36**	**36 of 36**	**8 of 8**	**80 of 80**

Legend:

- Each entry gives the runs captured out of the runs planned.
- Blackhole and wormhole = the two emulated attacks (Section 4.2.2); benign = a run with no attack, used as a control.
- Topology = the shape in which the boards are linked to the root (Section 5.5); site = where the run was made (G402 = Room G402 (GCCS), DLSU Library, Yuchengco = the Yuchengco Building).
- Scenario = the traffic pattern of the run (Section 4.2.2.5): stationary, burst or highload.
- The plan is the final plan of D-18 and D-35: 36 cells for each attack and 8 benign runs.

The 80 runs hold 456,195 one-second windows (Table 6.2). The windows labelled baseline and cooldown are treated as normal, and the windows labelled attack as attack. The windows recorded before the baseline, which include the formation of the mesh, and the windows on a phase boundary

or without a phase label are not analysed. The windows analysed number 369,678: 266,491 normal windows and 103,187 attack windows.

Table 6.2 Windows in the Dataset by Phase and Kind of Run

Phase	Blackhole (windows)	Wormhole (windows)	Benign (windows)	All runs (windows)	Share of all windows (%)
Before the baseline (not analysed)	40,666	34,065	10,704	85,435	18.7
Baseline (normal)	85,613	86,255	18,555	190,423	41.7
Attack (blackhole or wormhole)	51,438	51,749	0	103,187	22.6
Cooldown (normal)	34,200	34,435	7,433	76,068	16.7
Phase boundary or no phase label (not analysed)	502	521	59	1,082	0.2
All windows	**212,419**	**207,025**	**36,751**	**456,195**	**100.0**

Legend:

- Window = one second of recording from one board, with up to ten samples. The number of windows is also the number of board-seconds recorded.
- Share = the window count of all runs divided by the 456,195 windows recorded.
- Baseline = the 300 s before the attack (or before the point at which an attack would start, in a benign run); attack = the 180 s in which the attack is on; cooldown = the 120 s after it.
- Normal = baseline plus cooldown. A benign run has no attack phase.
- Phase boundary or no phase label = windows whose rows belong to two phases (transition windows), or windows that carry no phase label. The first and the fifth row are not analysed.
- Blackhole = the emulated attack in which a board that relays probes silently drops them; wormhole = the emulated attack in which one board sends the probes of another through a side channel (Section 4.2.2).

Every window kept has at least 8 of its 10 samples, and the mean number of samples in a window is 9.97 or more in every run. The lowest value is that of the blackhole run in the tree topology at G402 in the stationary scenario.

The 8 benign runs serve as a stand-alone control (Section 6.3.3). They hold 18,555 baseline windows and 7,433 cooldown windows, and the verifier rates all 8 as benign and confirmed (Table 6.3). The benign runs cover the three scenarios, but each topology and each site is covered by fewer runs than in the attack runs, so a benign run is compared with the attack runs of its own cell wherever the comparison needs the same placement (Section 6.3.3).

Table 6.3 The 8 Benign Runs

Topology	Site	Scenario	Boards (count)	Baseline windows (count)	Cooldown windows (count)	Verifier grade
Linear	DLSU Library	stationary	8	2,392	960	Strong

Topology	Site	Scenario	Boards (count)	Baseline windows (count)	Cooldown windows (count)	Verifier grade
Linear	G402	highload	8	2,392	960	Strong
Linear	Yuchengco	stationary	8	2,397	960	Strong
Partial mesh	G402	highload	8	2,393	959	Strong
Star	DLSU Library	stationary	8	2,396	960	Strong
Star	G402	burst	8	2,399	956	Strong
Star	Yuchengco	highload	6	1,794	720	Strong
Tree	G402	stationary	8	2,392	958	Strong
All				**18,555**	**7,433**	

Legend:

- Benign run = a run in which no attack is switched on; baseline = the 300 s of normal operation, cooldown = the 120 s that follow (the point at which an attack would have started is the end of the baseline).
- Boards = the boards that recorded in the run (the root and the child nodes); the star run at Yuchengco has 6 boards.
- Windows = one-second windows of all boards (Table 6.2).
- Verifier grade = the confidence grade that the verifier gives to its verdict (Section 4.2.7.1).
- Scenario = stationary, burst or highload (Section 4.2.2.5).

Whether a node is a victim depends on where the attacker sits in that run, so the number of victims differs by topology (Table 6.4). In the star, every non-attacker node sits behind the attacker, which is the hub, so the star has no bystanders. In the other topologies some non-attacker nodes are not behind the attacker. During the attack window these nodes carry the attack label, because the label is set for the whole run, but nothing is done to their traffic. This matters for the clustering results of Section 6.4. The blackhole runs hold 123 victim boards and 92 bystander boards. In a wormhole run, every non-attacker node is untouched, and the 36 runs hold 180 such boards. The verifier confirmed the attack in all 36 blackhole runs and all 36 wormhole runs, and found no attack signature in any of the 8 benign runs (Section 6.2).

Table 6.4 Runs and Boards by Topology

Topology	Blackhole runs, confirmed in brackets (count)	Blackhole victim boards (count)	Blackhole bystander boards (count)	Wormhole runs, confirmed in brackets (count)	Wormhole non-attacker boards (count)	Benign runs, confirmed in brackets (count)
Linear	9 (9)	33	21	9 (9)	45	3 (3)
Partial mesh	9 (9)	18	36	9 (9)	45	1 (1)
Star	9 (9)	54	0	9 (9)	45	3 (3)
Tree	9 (9)	18	35	9 (9)	45	1 (1)
All	**36 (36)**	**123**	**92**	**36 (36)**	**180**	**8 (8)**

Legend:

- Topology = the shape in which the boards are linked to the root (Section 5.5).
- Confirmed = the verifier found the attack in the run (Section 6.2); for a benign run, confirmed = the verifier found no attack signature.
- Verifier = the script that checks every run against its schedule (Section 4.2.7.1).
- Victim and bystander are defined at the start of the chapter; boards are non-attacker nodes. The blackhole runs hold 6 non-attacker nodes each, and the tree run at G402 with highload has 5 labelled boards, because board B768, a bystander, has no data (Table 6.6).
- Wormhole non-attacker boards = the 5 boards of each wormhole run that are neither the root nor a tunnel end; none of them is a victim.
- Board names such as B768 = the last four hexadecimal digits of the MAC address of the board.
- G402 = Room G402; highload = one of the traffic scenarios (Section 4.2.2.5).

Some cells were captured more than once. Fifteen earlier captures of 11 cells have been replaced by a newer capture and moved to an archive folder of the repository: 7 of the blackhole runs, 7 of the wormhole runs and 1 of the benign runs (Table 6.5). Only the newest capture of a cell is analysed, and every run counted in this chapter is the capture that stands in the dataset. The table gives the verdict that the verifier recorded for the earlier capture where one exists. The reasons for each replacement are kept in the verification log of the repository and are not repeated here.

Table 6.5 Earlier Captures Replaced by a Newer Capture

Attack	Run (topology, site, scenario)	Earlier captures replaced (count)	Verdict recorded for the earlier capture
Blackhole	Linear, Yuchengco, stationary	1	confirmed
Blackhole	Tree, G402, stationary	2	confirmed; not confirmed, setup invalid
Blackhole	Tree, G402, burst	1	confirmed
Blackhole	Tree, G402, highload	3	2 × not confirmed, setup invalid; confirmed
Wormhole	Partial mesh, G402, stationary	1	confirmed
Wormhole	Partial mesh, G402, highload	1	confirmed
Wormhole	Star, G402, highload	2	no verdict recorded; confirmed
Wormhole	Tree, G402, stationary	1	not confirmed, setup invalid
Wormhole	Tree, G402, highload	1	confirmed
Wormhole	Tree, Yuchengco, burst	1	confirmed
Benign	Partial mesh, G402, highload	1	no verdict recorded

Legend:

- Replaced = the earlier capture was archived (folder named replaced in the archive of the repository) and a new capture of the same cell was made; only the new capture is analysed.
- Verdict = the verdict that the archive note of the earlier capture records; no verdict recorded = the archive note of that capture holds no verdict line.
- Not confirmed, setup invalid = the verifier found that the roles or the topology were not as planned.
- Run = topology, site and scenario as in Table 6.1.

The export folders of the repository also hold 12 captures that are not part of the final plan: 5 captures of the jitter scenario of the earlier design (blackhole runs in the tree, partial-mesh and linear topologies), 6 blackhole runs and 1 benign run recorded at home, which served as tests. None of them is analysed or counted in this chapter.

Seven groups of runs carry a note from the verifier or from the capture, and all of them are kept in the dataset and in the pooled analyses (Table 6.6). In the blackhole runs, five runs are rated minor gaps, 27 good and 4 strong, and in the wormhole runs 15 are rated good and 21 strong. In the tree run at G402 with highload, board B768 has no data; the run log of the root places it as a bystander, so the gap is minor, and the attack is confirmed on the other boards (D-32). In the partial-mesh run at Yuchengco with highload, the entry end of the wormhole and one non-attacker node sent about one probe per second, where the scenario asks for four. The scenario check of the verifier is satisfied by the rate of the other boards, so the run is rated good, but the highload effect on these two boards is not present, and the run is read with that in mind.

Table 6.6 Runs Carrying a Note

Attack	Run (topology, site, scenario)	Note
Blackhole	Linear, G402, burst	The scenario was not applied in the attack phase: board 0C80 sent about 297 probes outside the attack window. The run stays confirmed with minor gaps (D-34).
Blackhole	Partial mesh, G402, highload	Victim ED80 has baseline windows for 266 of 300 s (88%), below the 95% that the verifier requires.
Blackhole	Tree, G402, stationary	Victim B768 has attack windows for only 28 of 180 s and no cooldown windows; board ED80 moved to a new parent in the attack.
Blackhole	Tree, G402, highload	Board B768 has no data; the run log of the root places it as a bystander, so the verifier rates the gap minor, and the attack is confirmed on the other boards (D-32).
Blackhole	Tree, Yuchengco, burst	Bystander 0C80 has baseline windows for 40 of 300 s (13%).
Blackhole and wormhole	All 9 partial-mesh runs of each attack	Every node was seen under a single parent, so the topology is a plain tree (Section 6.5.4).
Wormhole	Partial mesh, Yuchengco, highload	The entry end (0C80) and the child 1C38 sent 1 probe per second instead of 4 (TunnelIntensity of B is 1.0 in the attack); the scenario check of the verifier passes.

Legend:

- Note = a remark of the verifier or of the capture on a run; every run in the table is kept in the dataset.
- Minor gaps = a gap that does not change the verdict; grades are those of the verifier (deviation D-32).
- Victim, bystander and non-attacker node are defined at the start of the chapter.
- Plain tree = the topology that the mesh reported contains one parent for every node and no other link (Section 1.4.1).
- Probes per second = the rate at which a board sends its probes; in the other runs a board sends about 1 probe per second in the stationary and burst scenarios and about 4 in the highload scenario (Table 6.21).
- Board names such as 0C80 = the last four hexadecimal digits of the MAC address of the board.

The dataset defines 16 features (Table 4.17). Table 6.7 gives, for each feature, its unit, a short description and its use in this chapter. Table 6.8 gives the share of windows in which each feature has a value, for the kinds of board that the runs contain, over all windows recorded, including those not analysed. The text after the tables explains what each feature measures and why it is used or left out. The tunnel features exist only on the two ends of a wormhole, and eight features can enter the clustering of non-attacker nodes (Section 6.4).

Table 6.7 Features, Their Units and Their Use in This Chapter

Feature (unit)	What it measures	Use in this chapter
ForwardingRatio (ratio, 0 to 1)	Packets forwarded divided by packets received.	Direct record of the blackhole at the attacker (Section 6.2.1). Not clustered (leakage guard, Section 6.2.3).
IngressEgressDelta (packets per window)	Packets received minus packets forwarded.	Support check of the blackhole. Not clustered (leakage guard, Section 6.2.3).
RetryRate (ratio, 0 to 1)	Share of send attempts that failed (application counters, D-15).	Means reported only (Section 6.2.4). Not clustered. On a tunnel end it counts something else.
PDR (ratio, 0 to 1)	Share of a node's probes that reached the root.	Primary check of the blackhole on the non-attacker nodes. FULL and REDUCED sets.
ParentSwitchRate (parent changes per second)	Parent changes of the node per second.	FULL set only; dropped from REDUCED (identical values).
HopChangeCount (count per window)	Changes of the node's mesh layer (hop) in the window.	FULL set only; dropped from REDUCED (identical values).
HopStabilityDuration (s)	Longest interval with the same layer and parent.	FULL set only; dropped from REDUCED (identical values).
RSSI_mean (dBm)	Mean RSSI of the link to the parent.	Every feature set.
RSSI_var (dBm²)	Variance of the RSSI samples in the window.	Every feature set.
RSSI_stability (s)	Longest interval with the RSSI within ±3 dBm of the window mean.	FULL set only; dropped from REDUCED (identical values).
RSSI_Hop_Diff (dB)	Observed RSSI minus the baseline median RSSI at the node's reported layer.	Every feature set.

Feature (unit)	What it measures	Use in this chapter
LatencyHopRatio (ms per hop)	One-way delay of a probe per hop (D-2).	Not clustered; where it is missing, it reveals the label of a blackhole victim.
ConsistencyScore (ratio, 0 to 1)	|ForwardingRatio − 1|.	Support check of the blackhole. Not clustered (leakage guard, Section 6.2.3).
TunnelIntensity (messages per second)	Messages sent through the tunnel by a tunnel end.	Primary check of the wormhole (Section 6.2.2). Tunnel ends only; not clustered with the non-attacker nodes.
TunnelBytes (bytes per window)	Bytes sent through the tunnel in the window.	Support check of the wormhole. Tunnel ends only.
TunnelLatency (ms)	Spread in arrival time between the two copies of one probe at the root (D-3).	Entry end of the wormhole only (Section 6.2.2).

Legend:

- The features are those of Table 4.17; a feature has no value in a window where it is undefined, for example a ratio with nothing received.
- FULL and REDUCED are the feature sets of Table 6.24.
- Leakage guard = the list in the analysis code of the columns that may not be model inputs, with the reason for each; the single-feature audit tests whether one feature alone gives the label away (Section 6.2.3).
- RSSI = received signal strength indicator; dBm = decibel-milliwatts; dB = decibels.
- Relay counters = the counts of packets received, forwarded and dropped that every node keeps (D-12).
- Parent change = a board moving to another board as its parent; hop = layer − 1 (Section 5.3).
- Tunnel end = a board at one end of the wormhole tunnel: the entry end (Node B) and the exit end (Node A).
- Cross-layer feature = a feature that combines the physical link (signal strength) with the state of the mesh (layer).
- Probe = the small packet that a node sends towards the root about once a second (or about four times a second in the highload scenario) to measure delivery.
- Identical values = more than 90% of the windows of the pool hold the same value (the low-variance filter of Section 6.4.1).

Table 6.8 Windows with a Value for Each Feature, by Kind of Board (%)

Feature	Blackhole attacker	Blackhole non-attacker	Wormhole exit end (A)	Wormhole entry end (B)	Wormhole non-attacker	Benign child
ForwardingRatio	94.4	36.3	50.9	24.8	48.7	39.1
IngressEgressDelta	100.0	100.0	100.0	100.0	100.0	100.0
RetryRate	70.4	96.3	26.1	87.7	98.8	98.5
PDR	0.0	94.8	0.0	0.0	97.3	97.5
ParentSwitchRate	100.0	100.0	100.0	100.0	100.0	100.0
HopChangeCount	100.0	100.0	100.0	100.0	100.0	100.0
HopStabilityDuration	100.0	100.0	100.0	100.0	100.0	100.0
RSSI_mean	98.6	99.1	99.7	99.8	98.8	99.4
RSSI_var	100.0	100.0	100.0	100.0	100.0	100.0
RSSI_stability	100.0	100.0	100.0	100.0	100.0	100.0
RSSI_Hop_Diff	97.8	98.7	99.6	99.7	98.5	99.1
LatencyHopRatio	0.0	66.3	0.0	88.4	83.8	73.6
ConsistencyScore	94.4	36.3	50.9	24.8	48.7	39.1

Feature	Blackhole attacker	Blackhole non-attacker	Wormhole exit end (A)	Wormhole entry end (B)	Wormhole non-attacker	Benign child
TunnelIntensity	0.0	0.0	100.0	100.0	0.0	0.0
TunnelBytes	0.0	0.0	100.0	100.0	0.0	0.0
TunnelLatency	0.0	0.0	0.0	26.6	0.0	0.0

Legend:

- Each entry is the share of windows (%) of that kind of board in which the feature has a value; all windows recorded are counted, including those before the baseline and on a phase boundary.
- A feature has no value when it is undefined, for example a ratio with nothing received, or when its role gate excludes the board (the tunnel features exist only on the tunnel ends).
- Blackhole attacker = the board that drops probes; non-attacker = the child nodes that are neither the attacker nor a tunnel end; tunnel ends are defined at the start of the chapter.
- Benign child = a child node of a benign run.
- Features are defined in Table 6.7.

Three of the features describe the relaying of probes. ForwardingRatio is the number of packets a node forwarded divided by the number it received in the window. It is defined only in windows in which the node received packets, so that the share of windows with a value is 94.4% for the blackhole attacker and 36.3% for the blackhole non-attacker nodes (Table 6.8). For the attacker, it is the direct record of what it did with the probes (Section 6.2.1). IngressEgressDelta is the number of packets received minus the number forwarded in the window. It is computed from the same two relay counters as ForwardingRatio and therefore repeats that information. ConsistencyScore is the distance of the forwarding ratio from 1, |ForwardingRatio − 1|, and is computed from ForwardingRatio. The leakage guard of the analysis code keeps all three out of the model inputs of the non-attacker nodes (Section 6.2.3): ConsistencyScore and IngressEgressDelta add no information of their own, and the guard excludes the relay features unless more than one kind of node reports them. So none of them is used in the clustering of non-attacker nodes.

PDR is the share of the probes a node sent in the window that reached the root. It is the direct measure of delivery, which a blackhole is expected to reduce for the nodes behind it (Table 3.4), and it is used in the FULL and REDUCED feature sets. It has no value on the blackhole attacker or on the wormhole ends, which are not scored on delivery. RetryRate is the share of send attempts that failed in the window, taken from application counters and not from the MAC layer (deviation D-15). These counters stay near zero in every phase on the non-attacker nodes, so RetryRate is not used in the clustering, and its means are reported only to test the prediction of Table 3.4 (Section 6.2.4). On the entry end of the wormhole the same counter holds something else: it counts the frames sent through the tunnel, so on that board it records the attack and is not a retry rate (Section 6.2.3). LatencyHopRatio is the one-way delay of a probe divided by the number of hops (deviation D-2). It

has no value when no probe is delivered, so that a missing value alone reveals the attack label of a blackhole victim (Section 6.2.3), and it is therefore not used in the clustering.

ParentSwitchRate, HopChangeCount and HopStabilityDuration describe the stability of the topology. ParentSwitchRate is the number of parent changes of the node per second, HopChangeCount is the number of times the node's mesh layer (hop) changed in the window, and HopStabilityDuration is the longest interval in the window in which the node kept the same layer and parent. They are included in the FULL set because parent changes indicate an unstable topology and a short interval indicates reconfiguration (Section 4.2.4.2.3). The low-variance filter drops them from REDUCED, because almost every window of the pool has the same value (Table 6.24).

The four RSSI features describe the radio link. RSSI_mean is the mean received signal strength (RSSI) of the link to the parent in the window, and it reflects link quality and distance to the parent (Section 4.2.4.2.4), so it shows whether the attack changes the radio link. RSSI_var is the variance of the RSSI samples in the window and serves as the signal-stability measure of the physical-layer group (Section 4.2.4.2.4): a high variance points to interference, mobility or an unstable topology. RSSI_stability is the longest interval in the window in which the RSSI stays within ±3 dBm of the window mean, and a short interval points to a fluctuating signal, for example from interference or a parent switch. RSSI_Hop_Diff is the difference between the RSSI observed and the median RSSI that the same run showed at the node's reported layer during the baseline. It is the cross-layer feature: a large difference (above 10 dB) would mean that a node hears a signal that does not fit its layer, the physical–logical mismatch expected under a wormhole (Section 4.2.4.2.5). RSSI_mean, RSSI_var and RSSI_Hop_Diff are used in every feature set, whereas RSSI_stability is used in FULL only and dropped from REDUCED, because almost all of its windows have the same value (0.9 s) (Table 6.24).

The three tunnel features exist only on the two ends of a wormhole, and they are empty on every other board (Table 6.8). TunnelIntensity is the number of messages per second that a tunnel end sends through the tunnel, and TunnelBytes is the number of bytes sent through it in the window, which is 30 bytes for every message. TunnelLatency is the difference in arrival time between the two copies of one probe at the root (deviation D-3). It is defined only on the entry end, and only in a window in which a copy of one of its probes arrived twice, so it has a value in 6,508 windows of the wormhole runs, 6,468 of them in the attack phase, and in no window of the blackhole or the benign runs. They are the direct record of the wormhole and are reported in Section 6.2.2.

6.2. Attack Validation Results

Validation has two parts, as set out in Chapter 4: a check of every run against its own schedule and setup, and a comparison with published studies. This section reports the first part for the blackhole, the wormhole and the benign runs, and the single-feature audit. The comparison with published studies is in Section 6.5. Every run has a known schedule, so the expected effect of the attack is stated before the data are read, and the verifier tests it with the 3-sigma test of Chapter 4: a feature passes when its value in the attack phase lies at least three baseline standard deviations from its baseline mean, in the expected direction. The 3-sigma test pools five one-second windows into each point, and the score z is the distance in baseline standard deviations. When the baseline never varies, the standard deviation is 0 and z is infinite. Each attack has primary checks, which decide the verdict of a run, and support checks, which do not.

6.2.1. Blackhole: Forwarding Suppression

The attacker's own forwarding ratio is the direct record of the suppression. It is 1.000 to 1.004 in the baseline of every run and 0.000 to 0.007 during the attack, so between 0.0% and 0.7% of the probes received were forwarded by the attacker. Where the 3-sigma test gives a finite z-score (21 runs), the attack value lies at least 22.9 baseline standard deviations from the baseline mean (median 111). In 15 runs the baseline value never varied, so the z-score is infinite there. Table 6.9 lists the five checks of the verifier. Two are primary and decide the verdict, and three are support only.

Table 6.9 Attack Checks of the Verifier over the 36 Blackhole Runs

Check	Role	Passed (runs)	Detail
ForwardingRatio of the attacker	Primary	36 of 36	Baseline 1.000 to 1.004; attack 0.000 to 0.007.
PDR of all non-attacker nodes	Primary	35 of 36	One run excluded because of an unstable baseline (0.930 ± 0.253).
ConsistencyScore	Support	31 of 36	The 5 misses have z between 1.4 and 2.7.
IngressEgressDelta	Support	31 of 36	The 5 misses have z between 0.4 and 2.9.
NeighbourForwardingRatio of the attacker	Support	32 of 36	4 runs have no attack-phase values; no run fails.

Legend:

- Primary checks decide the verdict of a run, and support checks do not.
- Passed = runs that passed the check, out of the 36 blackhole runs; excluded = the test was not applied; no data = the board did not report the value in the attack phase.
- z = the distance between the attack value and the baseline mean, in baseline standard deviations (the 3-sigma test); a check passes at z of 3 or more in the expected direction.
- ForwardingRatio = packets forwarded divided by packets received in the window; ConsistencyScore = the distance of the forwarding ratio from 1; IngressEgressDelta = packets received minus packets forwarded.

- NeighbourForwardingRatio = the forwarding ratio of the attacker as reported by its neighbours.
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1).
- ± = the mean plus or minus the standard deviation of the baseline.
- Baseline = the 300 s before the attack; attack = the phase in which the blackhole is on.

Of the five checks, the forwarding ratio of the attacker passed in all 36 runs, and the PDR of all non-attacker nodes in 35 of 36. The remaining run, partial mesh at Yuchengco with stationary traffic, is excluded from the PDR test because its baseline is unstable (0.930 ± 0.253), and it is confirmed on the attacker's forwarding ratio. Each of the two support checks on the relay counters passed in 31 of the 36 runs. For ConsistencyScore, the 5 misses (linear at G402 with stationary traffic, linear at G402 with highload, linear at the DLSU Library with highload, partial mesh at G402 with stationary traffic and partial mesh at Yuchengco with highload) have z between 1.4 and 2.7, and for IngressEgressDelta the 5 misses (linear at G402 with highload, linear at the DLSU Library with stationary traffic, linear at the DLSU Library with highload, partial mesh at G402 with stationary traffic and partial mesh at Yuchengco with highload) have z between 0.4 and 2.9. For NeighbourForwardingRatio of the attacker, 4 runs have no attack-phase values (partial mesh at G402 with highload, partial mesh at the DLSU Library with highload, partial mesh at Yuchengco with stationary traffic and tree at the DLSU Library with burst), and the other 32 pass.

The verifier confirmed the blackhole in all 36 runs, in every topology, at every site and in all three scenarios (Table 6.4). It grades 4 runs as strong, 27 as good and 5 as having minor gaps. The 5 runs with minor gaps (linear at G402 with burst, partial mesh at G402 with highload, tree at G402 with stationary traffic, tree at G402 with highload and tree at Yuchengco with burst) are listed in Table 6.6: in each of them a board has too few windows in one phase or no data at all, or the scenario was not applied in the attack phase, and the impact check shows that the verdict is not affected (deviations D-32 and D-34). In the tree run at G402 with highload, board B768 has no data, and the run log of the root places it as a bystander, so the run is confirmed on the other boards (Table 6.6).

Delivery behind the attacker collapses at once and returns at once. In the 123 victim boards of the 36 runs, the PDR fell below 0.5 in the first second of the attack (121 boards in the first window, the other 2 within 1 s). After the attack ended, it was back above 0.5 within 3 s (median 2 s) in the 122 boards that kept reporting. One board went silent and has no recovery data (tree at G402 with stationary traffic, Table 6.10). The mean PDR of the victims is 0.998 in the baseline and 0.000 in the attack phase of every run, and every one of the 123 victim boards has a mean attack-phase PDR of exactly 0. The bystanders stay at 1.000 during the attack (0.996 in the baseline).

Table 6.10 Collapse and Recovery of Delivery in the Victim Boards of the 36 Blackhole Runs

Topology	Victim boards (count)	PDR below 0.5: median (max), s after the attack starts	Boards with recovery data (count)	PDR above 0.5 again: median (max), s after the attack ends
Linear	33	0 (0)	33	2 (2)
Partial mesh	18	0 (0)	18	2 (2)
Star	54	0 (1)	54	2 (3)
Tree	18	0 (1)	17	2 (2)
All	**123**	**0 (1)**	**122**	**2 (3)**

Legend:

- Each time is counted in seconds from the start (or the end) of the attack phase.
- Median (max) = the middle value and the largest value over the victim boards of that topology.
- A victim board that went silent has no recovery time.
- Collapse = the time from the start of the attack until the PDR of a victim board falls below 0.5; recovery = the time from the end of the attack until it rises above 0.5 again.
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1).
- Boards with recovery data = the victim boards for which a recovery time exists.
- In the tree run at G402 with highload, board B768 has no data and is a bystander, so it is not counted here (Table 6.6).

Figure 6.1 shows the mean PDR of the victims in the attack phase of each of the 36 runs. It is 0.00 in all 36 runs. Figure 6.2 follows the PDR over the run for each topology, and Figure 6.3 shows the forwarding ratio of the attacker, which falls from about 1 to about 0 for the 180 s of the attack and recovers when the attack ends.

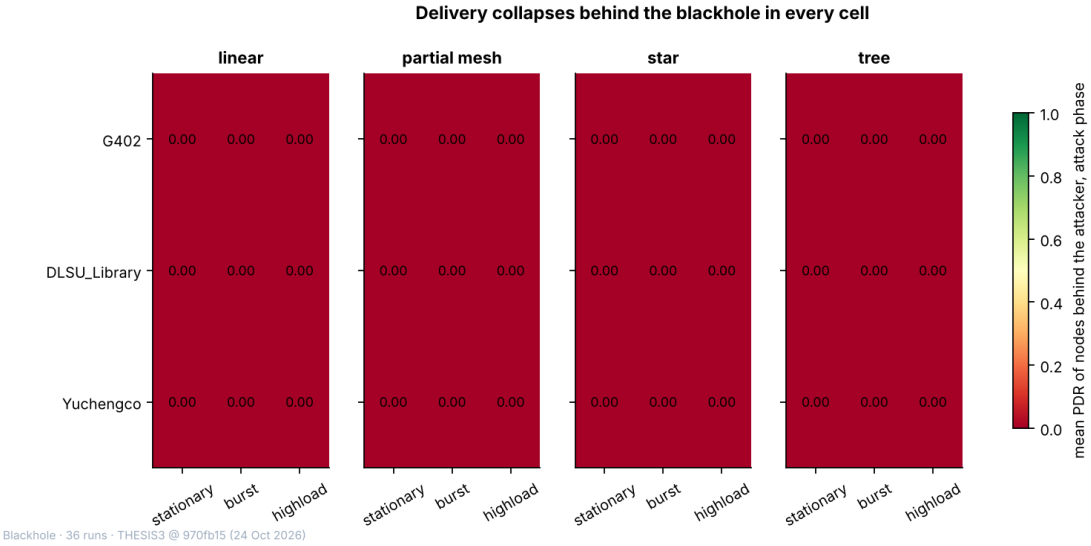


Figure 6.1 Mean PDR of the Victim Boards in the Attack Phase, for Each of the 36 Blackhole Runs

In Figure 6.1, each rectangle is one run. The panels are the topologies, the rows are the sites and the columns are the scenarios. The colour and the number give the mean PDR (ratio, 0 to 1) of the victim boards during the attack phase, where 0 means that no probe from them reached the root.

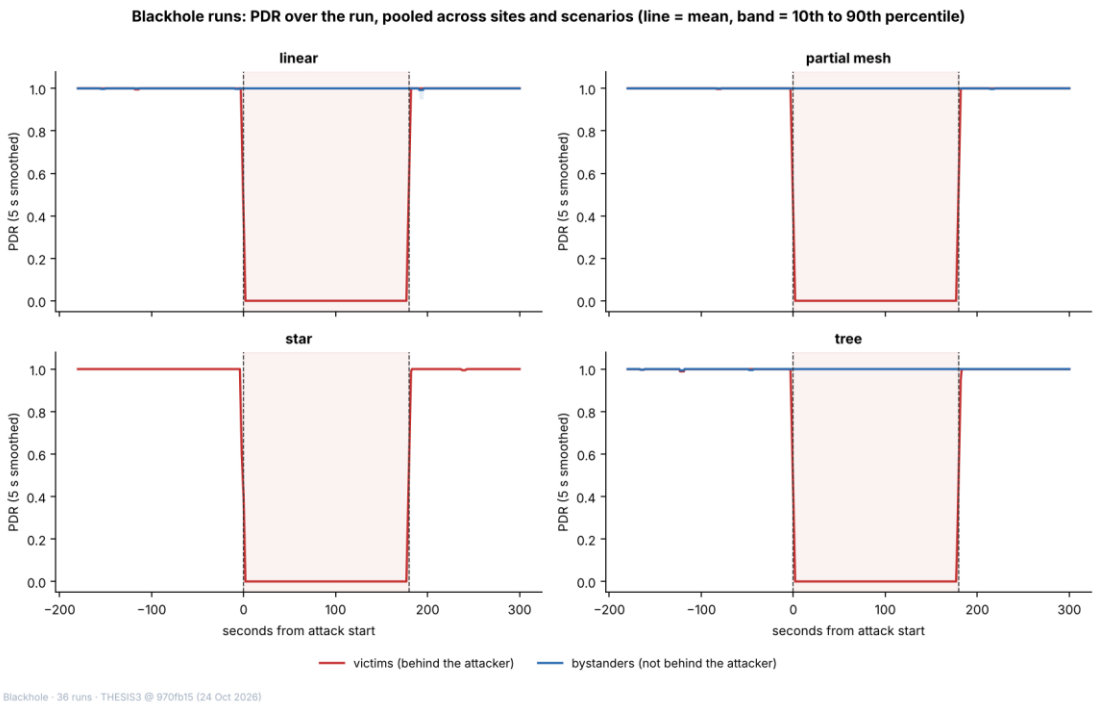


Figure 6.2 PDR of the Victim and Bystander Boards over the Run, by Topology

In Figure 6.2, the horizontal axis is time in seconds from the start of the attack (0 s = start, 180 s = end, negative values = baseline), and the vertical axis is the PDR, the share of probes that reached the root (ratio, 0 to 1), as a 5 s moving average. The red line shows the victims (the boards behind the attacker) and the blue line the bystanders (the boards that are not behind it), with a band from the 10th to the 90th percentile of the windows. The star has no bystanders. The shaded area is the attack phase.

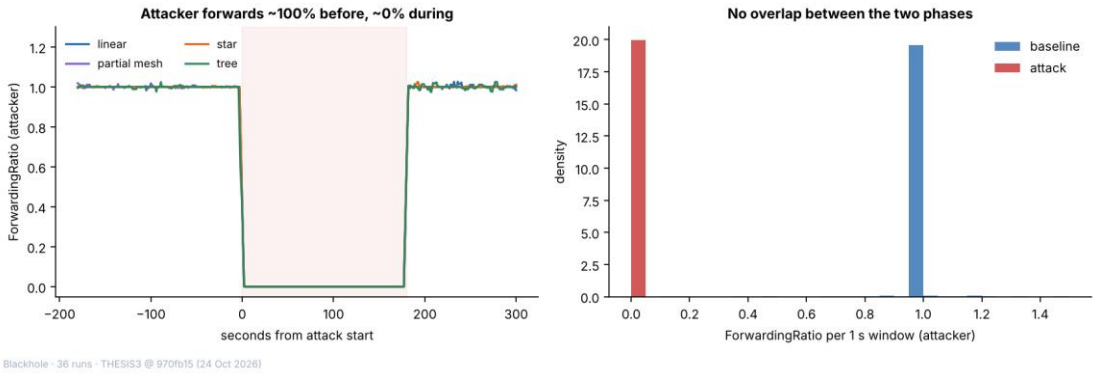


Figure 6.3 Forwarding Ratio of the Attacker over the Run (Left) and in Each One-Second Window (Right)

In the left panel of Figure 6.3, the horizontal axis is time in seconds from the start of the attack, and the vertical axis is the forwarding ratio of the attacker (packets forwarded divided by packets received, 5 s moving average; 1 = everything forwarded, 0 = nothing forwarded), with one line for each topology. In the right panel, each bar counts the one-second windows of a phase whose forwarding

ratio lies in that range, as a density, and values above 1.5 are drawn at 1.5. The two phases do not overlap.

6.2.2. Wormhole: Tunnel Replay

In a wormhole run, the entry end (Node B) sends each of its own probes to the root in the normal way and also over a wire to the exit end (Node A), which sends a copy on from its own place in the mesh (Section 4.2.2). The root therefore receives each probe of B twice, once by each route, and both ends count the messages that go through the wire. Only the probes of B are tunnelled, so the other 5 boards of the run are not touched. The expected evidence is therefore of three kinds: tunnel messages and bytes on the two ends and none in the baseline, duplicates of the probes of B at the root, and an entry end that is deeper in the mesh than the exit end, so that the tunnel skips hops (D-17). The 3-sigma test is applied to the duplicate rate and the tunnel intensity as primary checks and to the tunnel bytes as a support check (Table 6.11). The duplicate rate is not a feature of the dataset. It is computed by the verifier as the share of the probes of B that reach the root twice in a window.

Table 6.11 Attack Checks of the Verifier over the 36 Wormhole Runs

Check	Role	Passed (runs)	Detail
DuplicateRate of the entry end	Primary	36 of 36	Baseline 0 in every run; attack 0.986 to 1.000 (median 1.000).
TunnelIntensity of the tunnel ends	Primary	36 of 36	Baseline at most 0.001 messages per second; attack 0.994 to 4.0 messages per second.
TunnelBytes of the tunnel ends	Support	36 of 36	Baseline at most 0.035 bytes per window; attack median 30, largest 120 bytes per window.

Legend:

- Primary checks decide the verdict of a run, and support checks do not.
- Passed = runs that passed the check, out of the 36 wormhole runs.
- DuplicateRate = the share of the probes of the entry end that reach the root twice in a window (computed by the verifier; ratio, 0 to 1).
- TunnelIntensity = tunnel messages per second; TunnelBytes = bytes sent through the tunnel in the window (30 bytes per message).
- The 3-sigma test (the attack value at least three baseline standard deviations from the baseline mean) is applied in the expected direction, upwards. The baseline never varied in 36 runs for DuplicateRate and in 35 runs for TunnelIntensity and TunnelBytes, so z is infinite there.
- Baseline = the 300 s before the attack; attack = the 180 s in which the wormhole is on.
- Entry end = Node B, which sends the probes into the tunnel; exit end = Node A, which sends them on.

All three checks passed in all 36 runs. The verifier also confirmed the setup of every run: the scenario and the tunnel check are satisfied in all 36, and the topology check is satisfied in 27 runs. In the 9 partial-mesh runs it gives a warning, because every node was seen under a single parent, so that the topology is a plain tree (Section 6.5.4). The runs are confirmed in every topology, at every site and in every scenario, and 21 are graded strong and 15 good (Table 6.4). The 15 good runs are the 9

partial-mesh runs, with their plain-tree note, and 6 runs for which no run log of the root was saved, so a forced start cannot be ruled out. In the baseline there are practically no tunnel messages (the highest baseline mean is 0.001 messages per second) and no duplicates. In the attack, the tunnel intensity of each run lies between 0.994 and 4.0 messages per second, which is the probe rate of the entry end: about 1 per second in the stationary and burst scenarios and 4 per second in highload. The tunnel bytes follow, at 30 bytes for each message. In one run, partial mesh at Yuchengco with highload, the entry end sent about 1 probe per second instead of 4 (Table 6.6), so its tunnel intensity is 1.0.

The tunnel acted as designed in every cell (Table 6.12). In every run the entry end was deeper in the mesh than the exit end, and the hop of both ends stayed constant for the whole run (36 of 36 runs, no hop change in any phase). The tunnel skips from 1 to 5 hops: 2 to 5 in the linear runs (median 4), 1 to 2 in the partial-mesh runs, 1 in the star runs, where the exit end is the hub (D-28), and 1 to 2 in the tree runs. The wire between the two ends delivered the tunnel messages: the exit end received 99% to 101% of the messages that the entry end sent in all 36 runs. In the attack, 6,468 of the 6,469 windows of the entry end (99.98%) hold a duplicate, against none in the baseline or the cooldown (Figure 6.5).

Table 6.12 Evidence of the Tunnel by Topology and Scenario, over the 36 Wormhole Runs

Group	Value	Runs (count)	Hops skipped: min to max (median)	Messages received by the exit end (% of those sent), min to max	Windows of the entry end with a duplicate, lowest run (%)	Probes of the entry end arriving twice, lowest run (%)	Median spread between the two copies: median of runs (range), ms
Topology	Linear	9	2 to 5 (4)	99 to 101	100.0	99.4	5.4 (2.8 to 9.9)
Topology	Partial mesh	9	1 to 2 (1)	99 to 101	100.0	98.8	4.0 (2.4 to 6.4)
Topology	Star	9	1 to 1 (1)	99 to 101	99.4	100.0	3.7 (3.0 to 4.6)
Topology	Tree	9	1 to 2 (1)	100 to 101	100.0	99.7	3.3 (2.0 to 4.3)
Scenario	Stationary	12	1 to 5 (1)	99 to 101	100.0	99.4	3.4 (2.0 to 9.9)
Scenario	Burst	12	1 to 4 (1)	100 to 101	99.4	100.0	3.4 (2.1 to 6.4)
Scenario	Highload	12	1 to 4 (1)	99 to 101	100.0	98.8	4.0 (3.1 to 7.4)
All runs		**36**	**1 to 5 (1)**	**99 to 101**	**99.4**	**98.8**	**3.7 (2.0 to 9.9)**

Legend:

- Group = the runs of one topology, of one scenario, or all runs; each run is one cell of the design.
- Hops skipped = the hop (layer − 1) of the entry end minus the hop of the exit end; the hop of each end is constant during the run.
- Messages received by the exit end = the tunnel messages counted by the exit end as a share of those counted by the entry end.
- Windows with a duplicate = the share of the attack windows of the entry end in which a probe of it reached the root twice (the duplicate rate of Table 6.11).
- Probes arriving twice = the share of the probes of the entry end that reached the root twice, from the arrival log of the root; the two copies of a probe have the same sequence number.
- Spread between the two copies = the difference between the delays of the two copies of one probe (TunnelLatency, ms); each run contributes its own median.
- Entry end and exit end are defined in Table 6.11.

The root's arrival log gives the same picture at the level of single probes. Of the 12,465 probes that the entry end sent in the attack, 12,448 (99.86%) reached the root twice, and none of the probes of the other 5 boards did. The two copies do not arrive together: the difference between their delays is TunnelLatency, with a median of 3.9 ms, a 90th percentile of 12.1 ms and a 99th percentile of 417 ms over the 6,468 windows that have a value. The tail is long, because 84 windows have a spread above 200 ms, but the typical spread is a few milliseconds in every topology (Figure 6.6). The spread does not grow with the hops skipped: the Spearman correlation between the hops skipped and the median spread of the 36 runs is 0.25 ($p = 0.14$), and 0.29 with the 90th percentile ($p = 0.09$), so no relation can be claimed from 36 runs. Because the two copies travel by different routes, some probes arrive out of order. 232 of the 24,913 arrivals of the entry end in the attack (0.93%) reached the root after a probe with a higher sequence number. The out-of-order arrivals are rare, and are concentrated in a few runs (tree at G402 with highload (94), partial mesh at G402 with highload (55) and partial mesh at the DLSU Library with highload (47)).

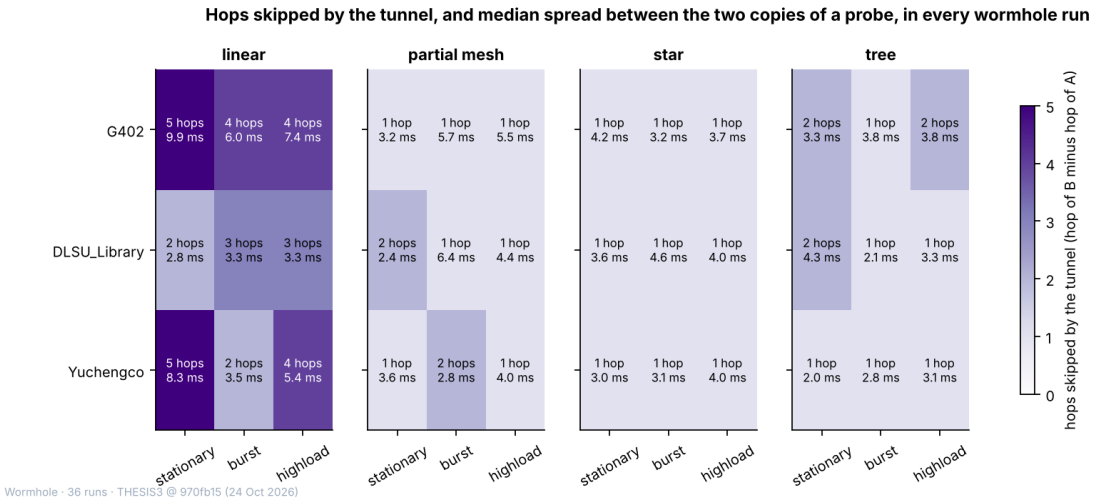


Figure 6.4 Hops Skipped by the Tunnel and Median Spread between the Two Copies of a Probe, in Each of the 36 Wormhole Runs

In Figure 6.4, each rectangle is one run. The panels are the topologies, the rows are the sites and the columns are the scenarios. The shade gives the hops skipped by the tunnel (the hop of the entry end minus the hop of the exit end, from 1 to 5), and the second number in each cell is the median spread between the two copies of the probes of the entry end, in milliseconds.

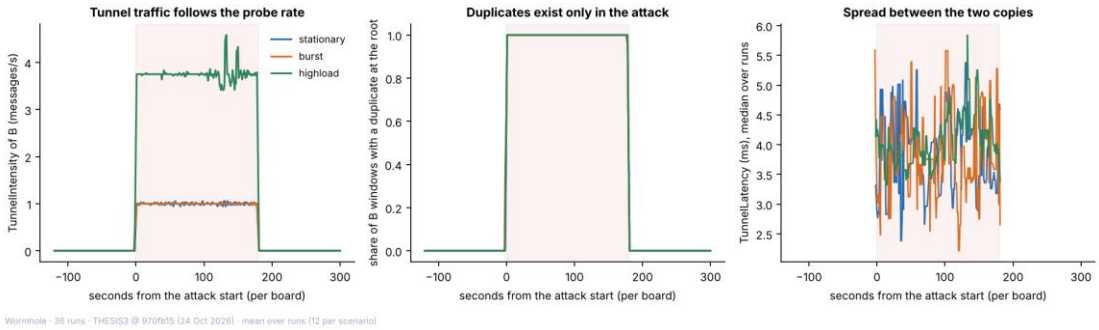


Figure 6.5 Tunnel Traffic, Duplicates and Spread over the Run, Averaged over the Runs of Each Scenario

In Figure 6.5, the horizontal axis is time in seconds from the start of the attack on each board (0 s = start, 180 s = end). The left panel shows the TunnelIntensity of the entry end (messages per second), the middle panel the share of its windows in which a probe reached the root twice, and the right panel the median spread between the two copies (ms). One line is drawn for each scenario, as the mean of its 12 runs. The shaded area is the attack phase.

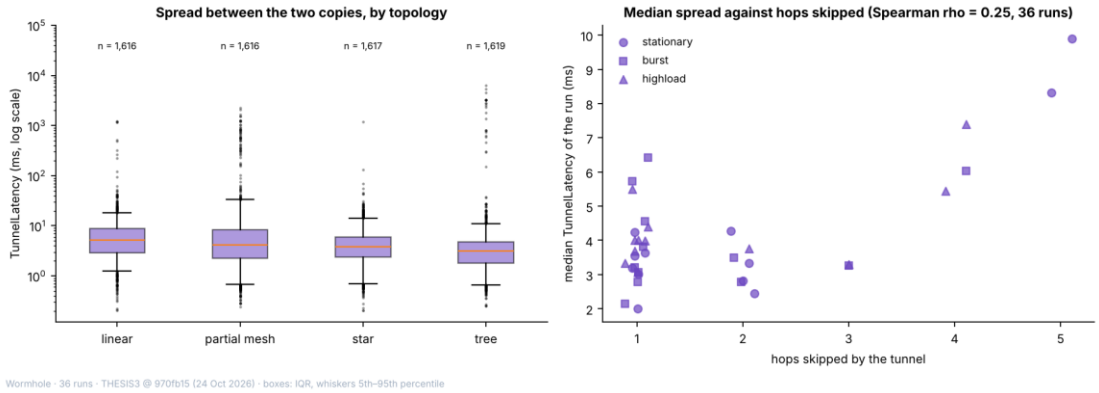


Figure 6.6 Spread between the Two Copies of a Probe, by Topology (Left) and against the Hops Skipped (Right)

In the left panel of Figure 6.6, each box spans the middle half of the windows of the entry end in the attack (25th to 75th percentile) with the median as a line, and the whiskers reach the 5th and 95th percentiles; the vertical axis is the spread in milliseconds on a logarithmic scale. In the right panel, each point is one run, placed at the hops skipped by its tunnel and at the median spread of the run, with the scenario as the marker.

6.2.3. Single-Feature Audit

If one feature alone could separate attack windows from normal windows, the clustering would be unnecessary. Each feature is therefore tested alone. A single threshold on the feature is chosen, with sight of the labels, so that as many windows as possible are classified correctly (normal = 0, attack = 1), and the share classified correctly is the accuracy of the feature. The accuracy is compared with the majority rate, the accuracy of a rule that always answers with the larger class (about 70% here, because normal windows are the larger class). An accuracy at the majority rate means that the feature carries no information about the label by itself, and an accuracy well above it means that the feature alone separates the windows. A high accuracy does not by itself show that the feature reveals the attack, because it can also come from an artefact of how the data were made. Arp et al. [80] list spurious correlation, where a feature that is an artefact of how the data were made decides the result, among the common pitfalls of machine learning in security, and this audit tests for it. A feature that is undefined in some windows is judged on the windows in which it has a value, so for these features the majority rate of the same windows is also given, together with the accuracy of the rule that predicts the label from whether the value is missing (Table 6.15).

The analysis code contains a leakage guard, a list of the columns that may not be model inputs, with the reason for each. It excludes RetryRate, because it counts failed sends at the application layer and, on the entry end of the wormhole, holds the frames sent through the tunnel (D-15); the three tunnel features, because they exist only on the tunnel ends, so that their presence identifies the kind of run; and the relay features ForwardingRatio, IngressEgressDelta and ConsistencyScore, which the guard excludes unless more than one kind of node reports them. The last two are computed from ForwardingRatio and add no information of their own. The audit applies the guard to each group of boards in turn: where a group holds the attacker as well as other nodes, two kinds of node report the relay features and the guard does not flag them in that group. A bar is drawn in grey where the guard flags the feature in the group.

For the blackhole runs, the audit is made on three groups of boards (Table 6.13): all boards other than the root (the attacker and the non-attacker nodes), the non-attacker nodes, and the victims only.

Table 6.13 Best Single-Threshold Accuracy of Each Feature in the Blackhole Runs (%)

Feature	All boards but the root (%)	Non-attacker nodes (%)	Victims only (%)
PDR	86.9†	86.9	99.9
LatencyHopRatio	84.3†	84.3†	100.0†
ForwardingRatio, ConsistencyScore	80.4†	71.7†	70.0†

Feature	All boards but the root (%)	Non-attacker nodes (%)	Victims only (%)
IngressEgressDelta	73.7	70.0	70.1
RetryRate	73.1	70.0	70.1
RSSI_mean, RSSI_var, RSSI_stability, RSSI_Hop_Diff, ParentSwitchRate, HopChangeCount, HopStabilityDuration (highest of the seven)	70.0	70.0	70.1
Run clock: raw board clock (window_start) / aligned (t_anchor_s)	70.2 / 80.0	70.3 / 80.0	70.4 / 80.0
Majority rate	70.0	70.0	70.1

Legend:

- Accuracy (%) = the share of windows labelled correctly by the best single threshold on the feature (normal = 0, attack = 1).
- Best single threshold = the cut-off value on one feature that labels the largest number of windows correctly.
- Majority rate = the accuracy of always answering with the larger class; a value at this rate means that the feature alone says nothing about the label.
- All boards but the root = the 7 child nodes of each run (the attacker and the 6 non-attacker nodes); non-attacker nodes = the 6 boards that are neither the root nor the attacker; victims only = the non-attacker nodes behind the attacker.
- † = the feature has no value in more than 5% of the windows, so the accuracy is that of the windows that have a value; the majority rate of those windows is in Table 6.15.
- The run clock is the position of a window in the run; it is not a feature and is tested only to show how much the fixed phase schedule gives away.
- Raw board clock (window_start) = the time on the clock of each board, which starts when that board starts; aligned clock (t_anchor_s) = the position of a window in the run on a time axis that the boards of one run share.
- Highest of the seven = the best accuracy among the seven features named in the row.
- Features are defined in Table 6.7.

On the victims, one feature alone decides the label: PDR reaches 99.9%, against a majority rate of 70.1%, and every other feature that has a value in nearly all windows stays at the majority rate. This is expected for a blackhole, which drops every packet for the whole attack window, so a complete collapse of delivery is separable by construction. LatencyHopRatio decides the label for a different reason. Its threshold accuracy on the windows that have a value is 100.0%, but those are the 69% of the windows in which a probe was delivered, and almost all of them are normal windows (the majority rate of those windows is 100%), so the threshold carries no information. The information is in the missing values, because a probe that is not delivered has no delay: whether the value is missing predicts the label with 99.2% accuracy on the victims, and it is missing in 100.0% of their attack windows. For that reason LatencyHopRatio is kept out of the clustering. The aligned run clock alone gives 80.0%, about 10 percentage points above the majority rate, because every run follows the same phase schedule and one cut can remove the end of the run. It cannot isolate a window in the middle of the run, and it is kept out of the model inputs. The raw board clock gives 70.4%, because the boards start at different times.

For the non-attacker nodes and for all boards other than the root, PDR reaches only 86.9%, because the bystanders carry the attack label but behave as normal nodes (Section 6.4.3). The forwarding ratio

and the consistency score reach 80.4% when the attacker is included, because the attacker's own forwarding ratio falls to 0 in the attack; on the non-attacker nodes they reach 71.7% on the 37% of the windows that have a value, which is the majority rate of those windows, so they carry no information there. IngressEgressDelta, which is computed from the same counters, reaches 73.7% when the attacker is included and the majority rate on the other boards, and RetryRate stays at the majority rate of its windows. Figure 6.7 shows the audit for the three groups of boards.

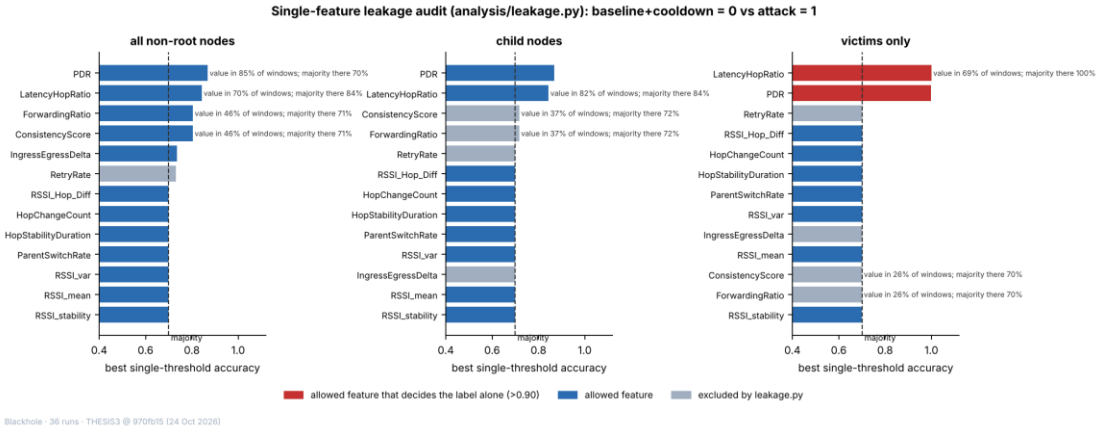


Figure 6.7 Best Single-Threshold Accuracy of Each Feature in the Blackhole Runs, for Three Groups of Boards

In Figure 6.7, each bar is the accuracy (%) of the best single threshold on one feature in telling attack windows from normal windows. The dashed line is the majority rate, about 70%, which is the accuracy of always answering "normal". A bar that ends at the dashed line means that the feature alone says nothing about the label, and a bar that goes beyond it means that the feature alone separates the windows to that degree. Red bars are allowed features that decide the label alone (above 90%), blue bars are the other allowed features, and grey bars are features that the leakage guard excludes from the model inputs in that group of boards (Table 6.7). Where a feature has a value in fewer than 95% of the windows, the text beside the bar gives the share of windows with a value and the majority rate of those windows. The left panel covers all boards other than the root, the middle panel the non-attacker nodes and the right panel the victims.

For the wormhole runs, the audit is made on the non-attacker nodes, on the two tunnel ends together, and on the entry end alone (Table 6.14). The tunnel features exist only on the ends, so they have no value on the non-attacker nodes.

Table 6.14 Best Single-Threshold Accuracy of Each Feature in the Wormhole Runs (%)

Feature	Non-attacker nodes (%)	Tunnel ends A and B (%)	Entry end B only (%)
TunnelIntensity, TunnelBytes	no value	99.7	99.7

Feature	Non-attacker nodes (%)	Tunnel ends A and B (%)	Entry end B only (%)
TunnelLatency	no value	100.0†	100.0†
RetryRate	70.0	76.9†	100.0
ForwardingRatio, ConsistencyScore	68.7†	70.0†	70.0†
IngressEgressDelta	70.0	70.0	70.0
PDR	70.0	no value	no value
LatencyHopRatio	70.0	69.9†	69.9
RSSI_mean, RSSI_var, RSSI_stability, RSSI_Hop_Diff, ParentSwitchRate, HopChangeCount, HopStabilityDuration (highest of the seven)	70.0	70.0	70.0
Run clock: raw board clock (window_start) / aligned (t_anchor_s)	70.0 / 80.0	72.1 / 80.0	73.1 / 80.0
Majority rate	70.0	70.0	70.0

Legend:

- Accuracy (%) = the share of windows labelled correctly by the best single threshold on the feature (normal = 0, attack = 1).
- Majority rate = the accuracy of always answering with the larger class; a value at this rate means that the feature alone says nothing about the label.
- Non-attacker nodes = the 5 boards of each run that are neither the root nor a tunnel end; tunnel ends A and B = the exit end and the entry end; entry end B only = the board that sends the probes into the tunnel.
- No value = the feature is not defined on these boards.
- † = the feature has no value in more than 5% of the windows, so the accuracy is that of the windows that have a value; the majority rate of those windows is in Table 6.15.
- The run clock is the position of a window in the run; it is not a feature and is tested only to show how much the fixed phase schedule gives away; raw and aligned board clocks are defined in Table 6.13.
- Highest of the seven = the best accuracy among the seven features named in the row.
- Features are defined in Table 6.7.

On the two tunnel ends, the tunnel features decide the label alone: TunnelIntensity and TunnelBytes reach 99.7%, against a majority rate of 70.0%. This is expected, because the tunnel carries messages only in the attack. TunnelLatency exists only on the entry end and only in the attack, so its presence alone gives the label (the rule that predicts the label from whether it is missing is correct for 100% of the windows of the entry end); this is why it is flagged by the audit, and why all three tunnel features are excluded from the model inputs of the non-attacker nodes. RetryRate reaches 76.9% on the two ends together and 100.0% on the entry end alone, although the retry counter is not a signature of the wormhole: on the entry end the counter that RetryRate reads holds the frames sent through the tunnel (Section 6.2.4), so the feature records the attack by construction and it is excluded from the model inputs. Every other feature stays at the majority rate on the ends, except for the raw board clock of the entry end (73.1%). On the non-attacker nodes no feature gets more than 0.01 percentage points above the majority rate (70.0% at most), which means that the wormhole leaves no trace on them that a threshold on one feature can use. The aligned run clock gives 80.0%, as in the blackhole runs. Figure 6.8 shows the audit.

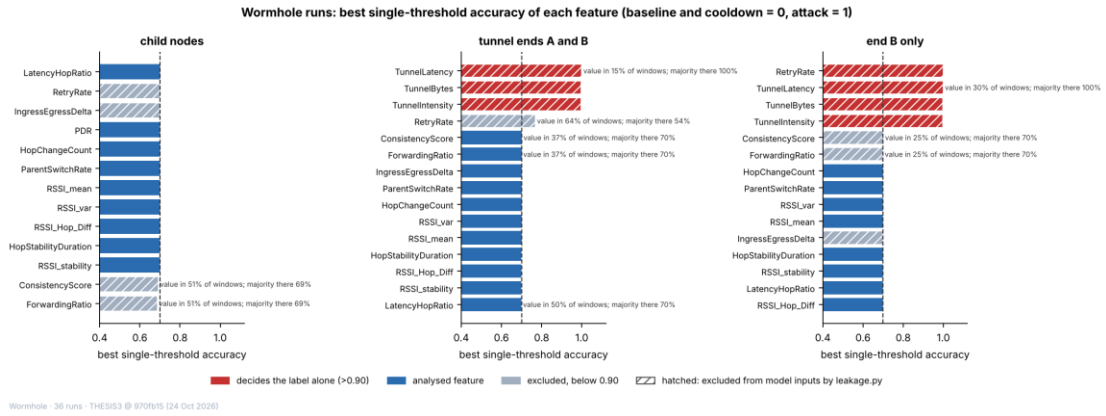


Figure 6.8 Best Single-Threshold Accuracy of Each Feature in the Wormhole Runs, for Three Groups of Boards

In Figure 6.8, each bar is the accuracy (%) of the best single threshold on one feature in telling attack windows from normal windows. The dashed line is the majority rate, about 70%. Red bars are features that decide the label alone (above 90%), blue bars are the other analysed features, and grey bars are features that are excluded from the model inputs and stay below 90%; a hatched bar is a feature that the leakage list of the analysis excludes from the model inputs. Where a feature has a value in fewer than 95% of the windows, the text beside the bar gives the share of windows with a value and the majority rate of those windows. The left panel covers the non-attacker nodes, the middle panel the two tunnel ends and the right panel the entry end alone.

Table 6.15 gives, for every feature that has no value in more than 5% of the windows of a group, the accuracy of its best threshold on the windows that have a value, the share of windows with a value, the majority rate of those windows, and the accuracy of the rule that predicts the label from whether the value is missing. The threshold carries information only where its accuracy is above the majority rate of the same windows. This is the case for PDR, which is missing only on the attacker and the tunnel ends, for the forwarding ratio of the attacker, and for RetryRate on the tunnel ends. LatencyHopRatio and TunnelLatency carry their information in the missing values, which is the finding of the rule on the right of the table.

Table 6.15 Features with Missing Values in the Single-Feature Audit

Runs	Group of boards	Feature	Accuracy of the best threshold, windows with a value (%)	Windows with a value (%)	Majority rate of those windows (%)	Accuracy of the rule "is the value missing?" (%)
Blackhole	All boards but the root	PDR	86.9	85	70.0	64.0
Blackhole	All boards but the root	LatencyHopRatio	84.3	70	84.3	78.4

Runs	Group of boards	Feature	Accuracy of the best threshold, windows with a value (%)	Windows with a value (%)	Majority rate of those windows (%)	Accuracy of the rule "is the value missing?" (%)
Blackhole	All boards but the root	ForwardingRatio	80.4	46	71.1	50.5
Blackhole	All boards but the root	ConsistencyScore	80.4	46	71.1	50.5
Blackhole	Non-attacker nodes	LatencyHopRatio	84.3	82	84.3	86.5
Blackhole	Non-attacker nodes	ConsistencyScore	71.7	37	71.7	53.9
Blackhole	Non-attacker nodes	ForwardingRatio	71.7	37	71.7	53.9
Blackhole	Victims only	LatencyHopRatio	100.0	69	100.0	99.2
Blackhole	Victims only	ConsistencyScore	70.0	26	70.0	59.6
Blackhole	Victims only	ForwardingRatio	70.0	26	70.0	59.6
Wormhole	Non-attacker nodes	ConsistencyScore	69.2	51	68.7	50.8
Wormhole	Non-attacker nodes	ForwardingRatio	68.7	51	68.7	50.8
Wormhole	Tunnel ends A and B	TunnelLatency	100.0	15	100.0	85.0
Wormhole	Tunnel ends A and B	RetryRate	76.9	64	53.9	65.0
Wormhole	Tunnel ends A and B	ConsistencyScore	70.1	37	70.0	55.0
Wormhole	Tunnel ends A and B	ForwardingRatio	70.0	37	70.0	55.0
Wormhole	Tunnel ends A and B	LatencyHopRatio	69.9	50	70.0	50.0
Wormhole	Entry end B only	TunnelLatency	100.0	30	100.0	100.0
Wormhole	Entry end B only	ConsistencyScore	70.2	25	70.1	60.0
Wormhole	Entry end B only	ForwardingRatio	70.0	25	70.1	60.0

Legend:

- Only features with no value in more than 5% of the windows of the group are listed; a feature that is not defined on the boards of a group is not listed.
- Accuracy of the best threshold = the share of the windows that have a value that the best single threshold labels correctly; it is compared with the majority rate of the same windows, which is given in the sixth column.
- Accuracy of the rule "is the value missing?" = the share of all windows of the group that are labelled correctly by predicting the attack label from the presence (or the absence) of a value; n/a = the feature has a value in every window or in none.
- Groups of boards are defined in Tables 6.13 and 6.14.
- Features are defined in Table 6.7.

6.2.4. Predictions Tested and Not Observed

Table 3.4 predicted that victim nodes would retransmit more under the blackhole. The table rests on the cross-layer premise that a routing attack leaves correlated traces in the physical, MAC and network layers. This premise is taken from Al-Anzi [5] and Amouri et al. [8], on which the feature design of Section 4.3.1 is based, and from Srinivasan [19], who combines physical, MAC and network-layer knowledge against blackhole and wormhole attacks. These studies were used as the basis because they treat the three layers together, as this study does. The retransmission row itself is a hypothesis of this study and not a published result: a victim whose frames are not acknowledged would repeat them, and any rise was expected to be small because the attacker still acknowledges each frame (Section 3.3.1).

The prediction was not observed. The retry rate of the victims (the share of send attempts that failed, from application counters, deviation D-15) was 0.0000 in the baseline, 0.0004 in the attack and 0.0000 in the cooldown, so it did not rise during the attack: a failed send occurs in only 9 of the 21,742 attack windows of the victims that have a value, and in none of their baseline or cooldown windows. For the bystanders it was 0 in every phase. A rise would be expected if frames were left unacknowledged, but the attacker acknowledges every frame at the link layer (Section 3.3.1.2), so the sender never sees a failed delivery. Parent changes were also almost absent: the mean ParentSwitchRate (parent changes per second) of the victims was 0.00000 in the baseline and 0.00009 in the attack (Cohen's d = 0.01). Table 3.4 is kept as it was written at proposal time, before any capture. The result is that an application-layer blackhole removes the delivery of the probes and leaves the link layer and the routing tree as they were.

One further measurement bears on the prediction. A sniffer captured the real 802.11 retry rate (MacRetryRate, the share of the frames of a board to its parent that carry the retry bit) in 7 of the 36 blackhole runs (17,204 windows with a value), and in none of the wormhole or benign runs. It is not one of the 16 features of Table 4.17 and not a model input, because the boards cannot measure it (D-15). In these runs, the frame-weighted retry rate of the victims was 0.295 in the baseline, 0.364 in the attack and 0.323 in the cooldown, and that of the bystanders was 0.355, 0.342 and 0.354. The change from the baseline to the attack differs in sign between runs: for the victims it lies between −0.10 and +0.33 (3 runs up, 4 down) and for the bystanders between −0.06 and +0.10 (2 runs up, 3 down). The data therefore show no consistent rise in the retransmissions of the victims, but 7 runs, with few frames heard in the attack in some of them, are too few to rule out a small effect, and this column is reported as supplementary.

Table 3.5 lists nine indicators that a wormhole was expected to leave in the physical, MAC and network layers. Table 6.16 gives each of them with what was measured in the 36 wormhole runs.

Table 6.16 Predicted Indicators of the Wormhole (Table 3.5) and What Was Measured in the 36 Wormhole Runs

Indicator	Expected (Table 3.5)	Measured	Observed
Hop count consistency	Nodes report unexpectedly low hop counts given physical placement	The hop of A and of B is constant during the attack in 36 of 36 runs; windows with a hop change on any board: baseline 0, attack 0, cooldown 0	No
Parent-child validity	Relationships violate physical constraints	Windows with a parent change on any board: baseline 0, attack 0, cooldown 0	No
Routing table entries	Self-referential entries or impossible neighbour relationships	The routing table is not logged by the firmware, so it is not measured	Not measured
RSSI-hop correlation	High RSSI observed from physically distant nodes	RSSI_Hop_Diff mean baseline → attack: A 2.79 → 2.76 dB (share of windows above 10 dB: 10.6% → 9.2%); B 5.41 → 5.68 dB (share of windows above 10 dB: 25.5% → 25.2%)	No
RSSI variance	Abnormally high RSSI values inconsistent with the expected path loss	RSSI_var mean baseline → attack: A 0.390 → 0.343 dBm² (d = −0.03); B 0.366 → 0.278 dBm² (d = −0.07)	No
Frame arrival pattern	Duplicates or out-of-order frames due to tunnelling	Probes of B reaching the root twice: baseline 0 of 20,717, attack 12,448 of 12,465 (99.86%); out-of-order arrivals of B in the attack 232 of 24,913 (0.93%); other boards: 0 probes twice	Yes for duplicates; out-of-order arrivals are rare
Retransmission count	May increase near tunnel endpoints due to timing issues	The RetryRate of B is 0.50 in the attack and 0.00 in the baseline, but on B the retry counter holds the frames sent through the tunnel (D-15), so it is not a retransmission count; MacRetryRate has no values in the wormhole runs; the RetryRate of the other boards is 0.000 in every phase	Not measurable on the ends; none on the children
Duplicate arrivals	The same probe reaches the root twice	B windows with a duplicate: 6,468 of 6,469 attack windows (100.0%) and 0 baseline windows	Yes
Latency mismatch	The two arrivals of one probe differ in latency	Spread between the two copies: median 3.9 ms, 90th percentile 12.1 ms, 99th percentile 417 ms over 6,468 windows; Spearman rho with hops skipped (run medians) = 0.25	Yes

Legend:

- Indicator and expected value are those of Table 3.5, which is kept as it was written at proposal time, before any capture.
- A and B = the exit end and the entry end of the tunnel; the other boards are the 5 non-attacker nodes of each run.
- RSSI_Hop_Diff = the observed RSSI minus the baseline median RSSI at the node's layer (dB); RSSI_var = the variance of the RSSI samples in the window (dBm²); d = Cohen's d, the shift of the mean in units of the spread (Section 6.3.2).
- Baseline → attack = the mean in the baseline phase and in the attack phase.

- Not measured = the quantity is not recorded by the boards.
- The routing table is not logged by the firmware.
- Out-of-order arrival = a probe that reaches the root after a probe with a higher sequence number from the same board.
- MacRetryRate = the 802.11 retry rate seen by a sniffer (Section 6.2.4); the sniffer captured none of the wormhole runs.

Three of the nine indicators are observed: the duplicate arrivals, the frame arrival pattern (duplicates, while out-of-order arrivals are rare) and the latency mismatch between the two copies. All three are properties of the traffic that the tunnel creates directly. Four indicators of the physical and network layers are not observed: the hop count, the validity of parent and child links, the relation between signal strength and hop, and the signal variance. The hop and the parent of every board stay the same in every phase of all 36 runs, and the RSSI features of the two ends and of the other five boards do not shift (|d| of at most 0.07 for the ends). The routing table is not logged, so that row is not measured, and the retransmission row cannot be tested on the two ends, because the retry counter of the entry end holds the frames sent through the tunnel, while on the other boards it is 0 in every phase. This is the pattern that an application-layer tunnel implies. The mesh does not see the tunnel, so its routing and its radio links are left as they were, and only the data plane carries the signature (D-20, Section 6.5.3). Table 3.5 is kept as it was written at proposal time.

6.2.5. Cells That Could Not Be Fully Tested

Some blackhole cells could be tested in part only. Partial mesh at Yuchengco with stationary traffic could not be tested on PDR, because its baseline is unstable, and it is confirmed on the attacker's forwarding ratio. In the run tree at G402 with highload, board B768 has no data; the run log of the root places it as a bystander, and the run is rated minor gaps. Four runs have no attack-phase values of the NeighbourForwardingRatio (Table 6.9), so that support check was not run for them. In the run linear at G402 with burst the scenario was not applied in the attack phase (board 0C80 sent about 297 probes outside it), and it is rated minor gaps (D-34), so the burst effect is not present in that attack phase. Three further runs carry a minor gap because one board has too few windows in a phase (Table 6.6). None of these gaps changes a verdict.

All 36 wormhole cells passed the three checks, and none carries a gap. Two limits remain. The routing table and the retransmissions of the two ends cannot be measured (Section 6.2.4). In the run partial mesh at Yuchengco with highload the entry end and one non-attacker node sent about one probe per second where the scenario asks for four, so the highload effect is missing on these two boards, although the scenario check of the verifier passes on the rate of the other boards. The nine partial-mesh runs of each attack are plain trees, so the partial-mesh cells test a branched tree with a limit of two children and not a mesh with alternative parents (Section 6.5.4).

6.2.6. Benign Runs: No Attack Signature

A benign run is the normal reference, so it has to be shown normal with the same tests that the attack runs are held to, pointed the other way. The verifier applies the 3-sigma test of both attack signatures, with the later phase of the run (the cooldown, where an attack would have run) against the baseline, and no signature may pass. It also checks that delivery is healthy in both phases, that every relay forwards at least 90% of what it receives, that every board has windows for at least 95% of each phase, and that the scenario was applied. All 8 benign runs are rated BENIGN CONFIRMED with the grade strong, which means that no signature passed and nothing is to be noted (Table 6.17). Of the 21 signature checks that could be evaluated, none passed, and the 43 checks that need a tunnel or an attacker had no data, as expected. Delivery is healthy: the mean PDR of each run is between 0.9994 and 1.0000 in both phases, and 4 of the 54 child boards ever had a window below 1, all of them in linear at G402 with highload (lowest window 0.75).

Table 6.17 Verifier Result for the 8 Benign Runs

Topology	Site	Scenario	PDR in the baseline	PDR in the cooldown	z of the PDR check	Signature checks passed (of those evaluated)	Verdict and grade
Linear	DLSU Library	stationary	1.0000	1.0000	+0.00	0 of 3	Benign Confirmed, strong
Linear	G402	highload	0.9994	0.9997	+0.06	0 of 3	Benign Confirmed, strong
Linear	Yuchengco	stationary	1.0000	1.0000	+0.00	0 of 3	Benign Confirmed, strong
Partial mesh	G402	highload	1.0000	1.0000	+0.00	0 of 3	Benign Confirmed, strong
Star	DLSU Library	stationary	1.0000	1.0000	+0.00	0 of 2	Benign Confirmed, strong
Star	G402	burst	1.0000	1.0000	+0.00	0 of 2	Benign Confirmed, strong
Star	Yuchengco	highload	1.0000	1.0000	+0.00	0 of 2	Benign Confirmed, strong
Tree	G402	stationary	1.0000	1.0000	+0.00	0 of 3	Benign Confirmed, strong

Legend:

- PDR in the baseline / cooldown = the mean PDR of all child boards in that phase (ratio, 0 to 1).
- z of the PDR check = the distance of the cooldown mean from the baseline mean in baseline standard deviations, as in the blackhole check (the attack signature needs z of −3 or lower); infinite = the baseline never varied.
- Signature checks = the checks of Tables 6.9 and 6.11 that can be evaluated on the data; the checks that need a tunnel or an attacker board have no data in a benign run.
- Verdict and grade = the result of the verifier for a benign run: BENIGN CONFIRMED means no signature passed and delivery and forwarding are healthy; strong = all board data present, nothing to note.
- The star run at Yuchengco has 6 boards, and the others have 8 (Table 6.3).
- Topology, site and scenario are defined in Table 6.1.

6.3. Exploratory Data Analysis

The exploratory analysis describes the three kinds of run with summary statistics and plots, and it states no detection result. For each feature it compares the baseline with the attack phase by the two means and by Cohen's d, which is the difference of the two means divided by the pooled standard deviation. By the common reading, an absolute d below 0.2 is a negligible shift, 0.5 a medium one and 0.8 or more a large one. The windows of one board follow one another and are not independent, so d describes the size of a shift and is not used for a significance test. A negative d means that the mean is lower in the attack phase. The blackhole runs are described for the victims and the bystanders separately (Section 6.3.1), the wormhole runs for the two tunnel ends and for the other five boards (Section 6.3.2), and the benign runs are described on their own and against the normal windows of the attack runs (Section 6.3.3).

6.3.1. Blackhole Runs

Table 6.18 gives, for each feature, the mean in the baseline and in the attack phase and the value of d, separately for the victims and the bystanders. Only the PDR of the victims moves: it falls from 0.998 in the baseline to 0.000 in the attack phase (d = −29.71). Every other feature of both groups shifts by less than 0.1 standard deviations, and the largest shift is RSSI_var of the bystanders (d = −0.09). The PDR of the bystanders is 0.996 in the baseline and 1.000 in the attack phase (d = 0.09), so the bystanders, which carry the attack label, are not affected by the attack. The collapse of the victims' PDR is the whole of the effect that the blackhole has on the features of the non-attacker nodes.

Table 6.18 Mean of Each Feature in the Baseline and in the Attack Phase, and Cohen's d, for the Victims and the Bystanders of the Blackhole Runs

Feature (unit)	Victims: baseline mean	Victims: attack mean	Victims: d	Bystanders: baseline mean	Bystanders: attack mean	Bystanders: d
PDR (ratio)	0.9977	0.0000	−29.71	0.9963	1.0000	0.09
RetryRate (ratio)	0.0000	0.0004	0.03	0.0000	0.0000	n/a

Feature (unit)	Victims: baseline mean	Victims: attack mean	Victims : d	Bystanders: baseline mean	Bystanders: attack mean	Bystanders: d
IngressEgressDelta (packets per window)	0.019	0.013	−0.01	0.031	0.018	−0.03
RSSI_mean (dBm)	−74.72	−74.95	−0.03	−72.24	−72.53	−0.04
RSSI_var (dBm²)	0.412	0.311	−0.07	0.332	0.199	−0.09
RSSI_stability (s)	0.894	0.896	0.04	0.895	0.898	0.07
RSSI_Hop_Diff (dB)	3.18	3.28	0.02	4.07	3.99	−0.01
ParentSwitchRate (changes per second)	0.00000	0.00009	0.01	0.00000	0.00000	n/a
HopChangeCount (count per window)	0.00000	0.00009	0.01	0.00000	0.00000	n/a
HopStabilityDuration (s)	0.9998	0.9998	0.01	0.9999	0.9999	0.00

Legend:

- Mean = the mean over the one-second windows of the group in that phase; d = Cohen's d, the difference of the means divided by the pooled standard deviation (negative = lower in the attack).
- Victims = the non-attacker nodes behind the attacker; bystanders = the non-attacker nodes that are not behind it (defined at the start of the chapter). The attacker is not included.
- Windows (baseline, attack): victims 36,774 and 21,962; bystanders 27,281 and 16,531. A feature that has no value in a window is left out of its own mean.
- n/a = d cannot be computed because the feature has one value in both phases (RetryRate of the bystanders, ParentSwitchRate and HopChangeCount of the bystanders).
- LatencyHopRatio is not listed, because the victims have no value of it in the attack phase (Section 6.2.3).
- RetryRate = the share of send attempts that failed, from application counters (D-15); PDR = the share of a node's probes that reached the root.
- Features are defined in Table 6.7.

The signal-strength features do not respond to the attack. The RSSI_mean of the victims is −74.7 dBm in the baseline and −74.9 dBm in the attack (d = −0.03), and that of the bystanders is −72.2 dBm and −72.5 dBm (d = −0.04). The victims are the deeper boards, so their mean signal is about 2.5 dB weaker than that of the bystanders already in the baseline, which reflects where the boards sit and not the attack. RSSI_var is lower in the attack phase in both groups (d = −0.07 for the victims and −0.09 for the bystanders), and because the bystanders are not touched by the attack, a change of this size cannot be attributed to it. The retry rate of the victims is 0.0000 in the baseline and 0.0004 in the attack phase, because a failed send occurs in 9 of their 21,742 attack windows (d = 0.03); the bystanders have none. LatencyHopRatio cannot be followed through the attack for the victims, since a probe that is dropped has no delay, but for the bystanders its median is 4.26 ms per hop in the baseline, 3.15 in the attack phase and 2.46 in the cooldown. The decline continues after the attack has ended and is also seen in the benign runs (Section 6.3.3), so it follows the course of the run and not the attack.

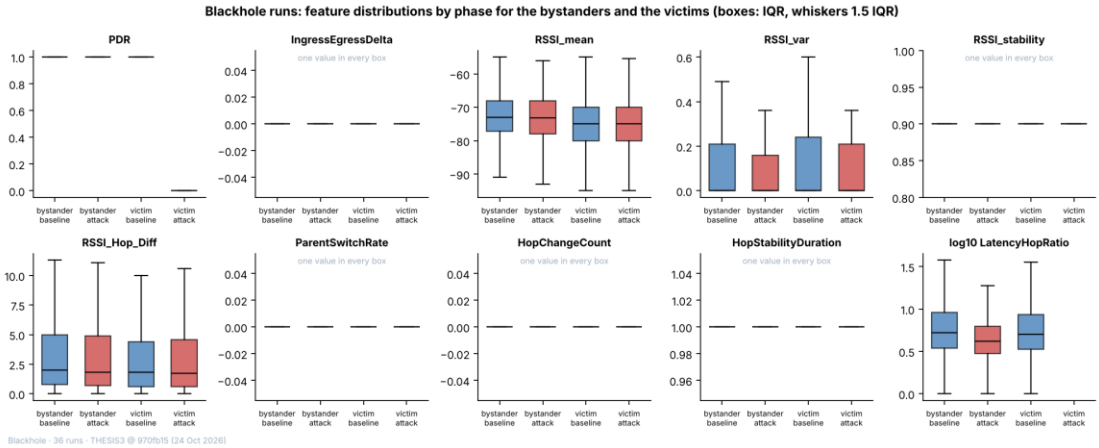


Figure 6.9 Distributions of Ten Features by Phase for the Bystanders and the Victims of the Blackhole Runs

In Figure 6.9, each panel is one feature and each box is one group in one phase: bystander or victim, baseline (blue) or attack (red). The box spans the middle half of the windows (25th to 75th percentile) with the median as a line, and the whiskers reach 1.5 times that range. Only the PDR panel shows a change: the victims' box in the attack lies at 0, and all other boxes lie at 1. The panels of IngressEgressDelta, RSSI_stability, ParentSwitchRate, HopChangeCount and HopStabilityDuration hold one value in every box and are marked as such. The boxes of RSSI_mean, RSSI_var and RSSI_Hop_Diff overlap between the phases. In the panel of LatencyHopRatio (plotted as the logarithm of 1 plus the value), the victims' box in the attack is missing because the victims have no value in that phase.

Four pairs of features are strongly correlated (Figure 6.10). ConsistencyScore is computed from ForwardingRatio as |ForwardingRatio − 1|, so the two are strongly rank-correlated (Spearman −0.94), although the Pearson coefficient is only 0.12 because the relation is not linear. IngressEgressDelta and ConsistencyScore have a Pearson coefficient of 0.65 over all windows (0.78 in the baseline), since both are built from the same relay counters. RSSI_var and RSSI_stability have a Pearson coefficient of −0.79 but a Spearman coefficient of only 0.05: the link comes from a few windows with a high variance and a short stable interval, and not from a general relation. ParentSwitchRate and HopChangeCount have a coefficient of 1.00 in the attack windows, where both are non-zero in the same windows; in the baseline both are constant and no coefficient exists. No other pair among the eight features of the FULL set of Section 6.4 has an absolute Pearson coefficient above 0.4 (the largest is 0.36 between ParentSwitchRate and HopStabilityDuration, in the attack windows). The correlation structure is the same in the baseline and in the attack windows, apart from the pairs that involve ParentSwitchRate or HopChangeCount, which have a coefficient only in the attack windows because both are constant in the baseline.

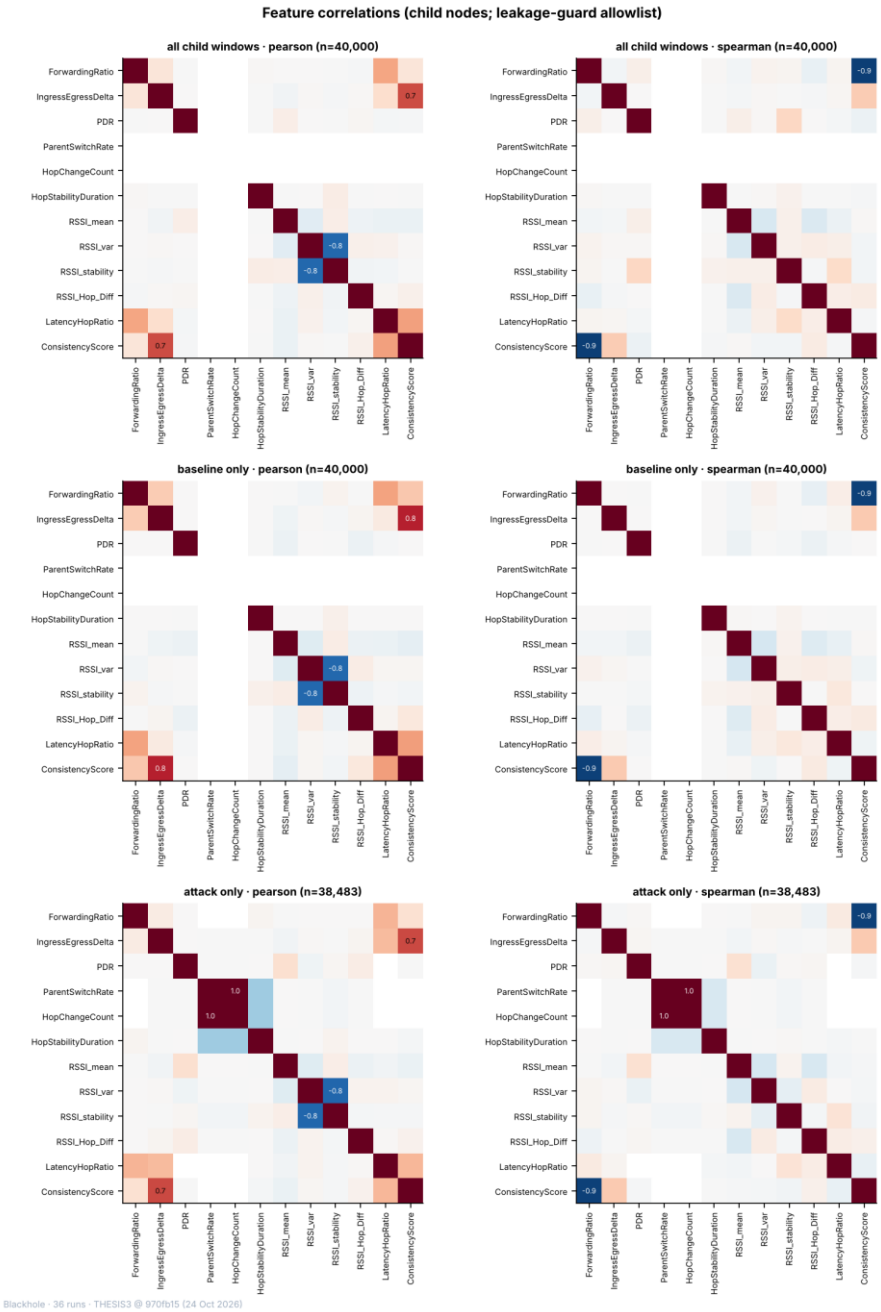


Figure 6.10 Correlations between the Features of the Child Nodes of the Blackhole Runs

In Figure 6.10, each square is the correlation between two features, from −1 (blue) to +1 (red), and values of 0.5 or more in absolute size are printed. The left column is the Pearson coefficient and the right column the Spearman coefficient. The rows are all windows, the baseline windows and the attack windows of the child nodes, each on a sample of up to 40,000 windows. The twelve features are those that the leakage guard allows in a group that holds the attacker (Section 6.2.3); RetryRate and the tunnel features are not shown. An empty row means that the feature is constant in those windows.

6.3.2. Wormhole Runs

Table 6.19 gives the same comparison for the wormhole runs, for the exit end (A), the entry end (B) and the non-attacker nodes. On the two ends, the tunnel features are the only ones that change. TunnelIntensity rises from 0 to 1.92 messages per second on both ends (d = 1.83), and TunnelBytes from 0 to 57.5 bytes per window on both ends, which is 30 bytes for each message. The shift is large, but d is limited by the spread of the attack values, which lie between 1 and 4 messages per second according to the scenario, while the baseline never varies. TunnelLatency exists only on B and only in the attack, with a mean of 21.1 ms and a median of 3.9 ms; the mean is drawn up by a few windows with a long delay (Section 6.2.2).

Table 6.19 Mean of Each Feature in the Baseline and in the Attack Phase, and Cohen's d, for the Two Tunnel Ends and the Non-Attacker Nodes of the Wormhole Runs

Feature (unit)	Exit end A: baseline → attack	Exit end A: d	Entry end B: baseline → attack	Entry end B: d	Non-attacker nodes: baseline → attack	Non-attacker nodes: d
TunnelIntensity (messages per second)	0.000 → 1.916	1.83	0.000 → 1.916	1.83	no value	n/a
TunnelBytes (bytes per window)	0.0 → 57.5	1.83	0.0 → 57.5	1.83	no value	n/a
TunnelLatency (ms)	no value	n/a	no value → 21.1	n/a	no value	n/a
RetryRate (ratio)	no value → 0.000	n/a	0.000 → 0.500	42.99	0.000 → 0.000	n/a
PDR (ratio)	no value	n/a	no value	n/a	0.9999 → 0.9999	0.00
IngressEgressDelta (packets per window)	0.068 → 0.086	0.03	0.009 → 0.007	−0.01	0.038 → 0.051	0.02
RSSI_mean (dBm)	−65.63 → −65.82	−0.02	−62.47 → −62.35	0.01	−69.18 → −69.13	0.01
RSSI_var (dBm²)	0.390 → 0.343	−0.03	0.366 → 0.278	−0.07	0.451 → 0.331	−0.06
RSSI_stability (s)	0.895 → 0.895	0.01	0.895 → 0.897	0.04	0.893 → 0.896	0.06
RSSI_Hop_Diff (dB)	2.79 → 2.76	−0.01	5.41 → 5.68	0.04	2.65 → 2.72	0.02
LatencyHopRatio (ms per hop)	no value	n/a	12.0 → 11.2	−0.01	11.3 → 9.9	−0.03
HopStabilityDuration (s)	1.0000 → 1.0000	0.00	0.9999 → 0.9999	0.01	0.9999 → 0.9999	−0.01
ParentSwitchRate (changes per second)	0.0000 → 0.0000	n/a	0.0000 → 0.0000	n/a	0.0000 → 0.0000	n/a
HopChangeCount (count per window)	0.0000 → 0.0000	n/a	0.0000 → 0.0000	n/a	0.0000 → 0.0000	n/a

Legend:

- Each pair gives the mean in the baseline and in the attack phase, over the one-second windows of the group; d = Cohen's d, the difference of the means divided by the pooled standard deviation (negative = lower in the attack).
- Exit end A = the board that sends the tunnelled probes on; entry end B = the board whose probes are tunnelled; non-attacker nodes = the 5 other child nodes of each run (defined at the start of the chapter).
- Windows (baseline, attack): end A 10,778 and 6,474; end B 10,779 and 6,469; non-attacker nodes 53,917 and 32,334.
- No value = the feature is not defined for that group in that phase (the tunnel features exist only on the ends, TunnelLatency only on B in the attack, and PDR is not scored on the ends).
- n/a = d is not defined, because the feature has no value or has one value in both phases. Where only the baseline never varies (the tunnel features), d is limited by the spread of the attack values.
- RetryRate of B counts the frames sent through the tunnel (D-15), so its shift records the tunnel and is not a retry rate (Section 6.2.4).
- Features are defined in Table 6.7.

Every other feature of the two ends and of the non-attacker nodes stays where it was. The largest shift on the ends is RSSI_var of B (d = −0.07) and the largest on the non-attacker nodes is RSSI_var (d = −0.06). The PDR of the non-attacker nodes is 0.9999 in the baseline and 0.9999 in the attack phase, whereas the PDR of the blackhole victims falls to 0 (Figure 6.11). RSSI_Hop_Diff is larger on B than elsewhere, 5.4 dB in the baseline and 5.7 dB in the attack phase, against 2.8 and 2.8 dB on A and 2.7 and 2.7 dB on the non-attacker nodes, which reflects where B sits, because it is as large before the attack as during it. The RetryRate of B changes from 0.00 to 0.50 (d = 42.99), but it is not a retry rate: on B, the counter holds the frames sent through the tunnel (Section 6.2.4), so it records the attack by construction. The wormhole leaves the delivery, the signal strength and the position of the nodes in the mesh unchanged, which is what an attack at the application layer is expected to do (D-20).

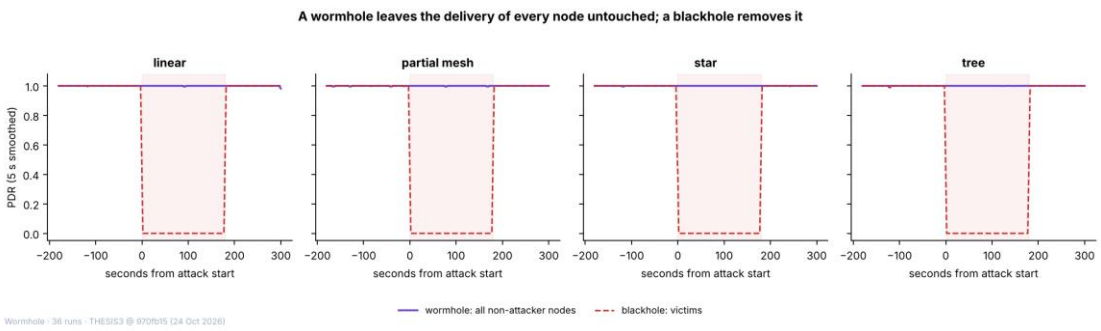


Figure 6.11 PDR of the Non-Attacker Nodes of the Wormhole Runs and of the Victims of the Blackhole Runs, by Topology

In Figure 6.11, the horizontal axis is time in seconds from the start of the attack (0 s = start, 180 s = end) and the vertical axis is the PDR, the share of probes that reached the root (ratio, 0 to 1), as a 5 s moving average. The solid purple line is the mean PDR of all non-attacker nodes of the wormhole runs of the topology, and the dashed red line is the mean PDR of the victims of the blackhole runs of the same topology. The shaded area is the attack phase. The wormhole line does not leave 1, whereas the blackhole line falls to 0 for the whole of the attack phase.

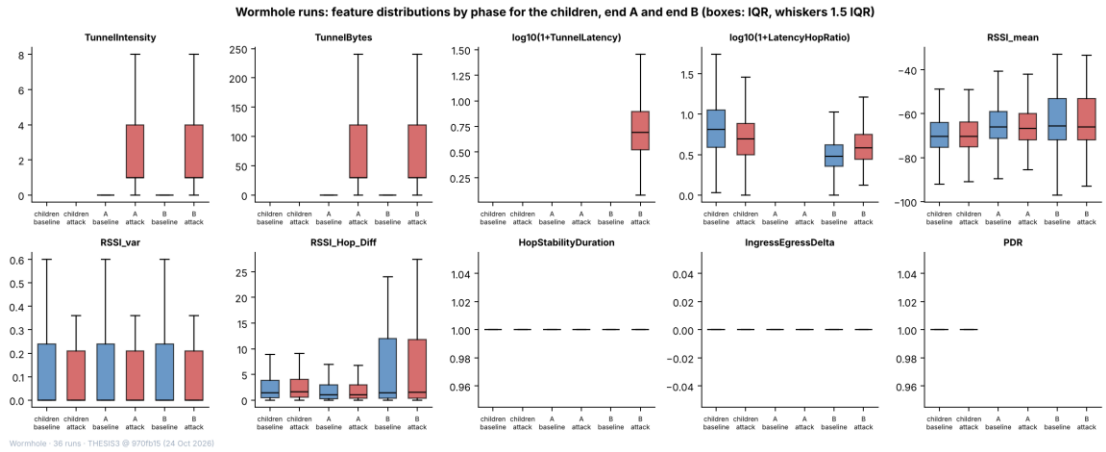


Figure 6.12 Distributions of Ten Features by Phase for the Non-Attacker Nodes and the Two Ends of the Wormhole Runs

In Figure 6.12, each panel is one feature and each box is one group in one phase: non-attacker nodes, end A or end B, in the baseline (blue) or in the attack (red). The box spans the middle half of the windows (25th to 75th percentile) with the median as a line, and the whiskers reach 1.5 times that range. TunnelIntensity, TunnelBytes and TunnelLatency exist only on the ends and take values only in the attack; TunnelLatency exists only on B (logarithmic scale of 1 plus the value). The boxes of RSSI_mean, RSSI_var and RSSI_Hop_Diff coincide between the phases for every group, and the boxes of LatencyHopRatio differ between the groups and shift only slightly between the phases. HopStabilityDuration and IngressEgressDelta hold one value in every box, and PDR has no value on the two ends.

6.3.3. Benign Runs

Table 6.20 describes each benign run by its delivery, its signal and the stability of its mesh. Delivery is complete: a window with a PDR below 1 occurs only in linear at G402 with highload (0.2% of its windows, lowest 0.75), and the other seven runs have none. The mean RSSI_mean of the baseline ranges from −74.4 dBm (linear at Yuchengco with stationary traffic) to −61.8 dBm (linear at the DLSU Library with stationary traffic), a spread of 12.5 dB between runs, which the matched comparison below shows to be a property of the capture. The mesh is stable: no run has a hop change, and one parent change is recorded in the 8 runs (tree at G402 with stationary traffic). The median LatencyHopRatio is lower in the cooldown than in the baseline in all 8 runs, which shows that the decline seen in the blackhole runs in Section 6.3.1 belongs to the course of a run.

Table 6.20 Delivery, Signal and Mesh Stability in Each of the 8 Benign Runs

Topology	Site	Scenario	Windows with PDR below 1 (%)	Lowest window PDR	RSSI_mean in the baseline (dBm)	RSSI_var in the baseline (dBm²)	Median LatencyHop Ratio, baseline → cooldown (ms per hop)	Parent changes (count)	Deepest hop (count)
Linear	DLSU Library	stationary	0.00	1.00	−61.8	0.158	1.9 → 1.3	0	7
Linear	G402	highload	0.20	0.75	−70.6	0.174	3.6 → 2.4	0	7
Linear	Yuchengco	stationary	0.00	1.00	−74.4	0.224	3.3 → 2.1	0	7
Partial mesh	G402	highload	0.00	1.00	−69.1	0.629	4.7 → 2.5	0	3
Star	DLSU Library	stationary	0.00	1.00	−64.9	0.173	4.9 → 3.4	0	1
Star	G402	burst	0.00	1.00	−71.6	0.414	6.3 → 4.7	0	1
Star	Yuchengco	highload	0.00	1.00	−65.7	0.087	6.7 → 4.8	0	1
Tree	G402	stationary	0.00	1.00	−68.2	1.114	5.9 → 3.3	1	2

Legend:

- Each value is computed over the one-second windows of the child boards of the run; the baseline is the first 300 s and the cooldown the last 120 s.
- Windows with PDR below 1 = the share of the windows of the run in which at least one probe of the board was not delivered; lowest window PDR = the lowest value of any window.
- Parent changes = the number of windows in which a board changed its parent; deepest hop = the largest hop (layer − 1) of any board in the run. No run has a hop change.
- PDR = the share of a board's probes that reached the root; RSSI_mean and RSSI_var are the mean and the variance of the signal strength of the link to the parent.
- LatencyHopRatio = the one-way delay of a probe divided by its hops (ms per hop).
- Topology, site and scenario are defined in Table 6.1.

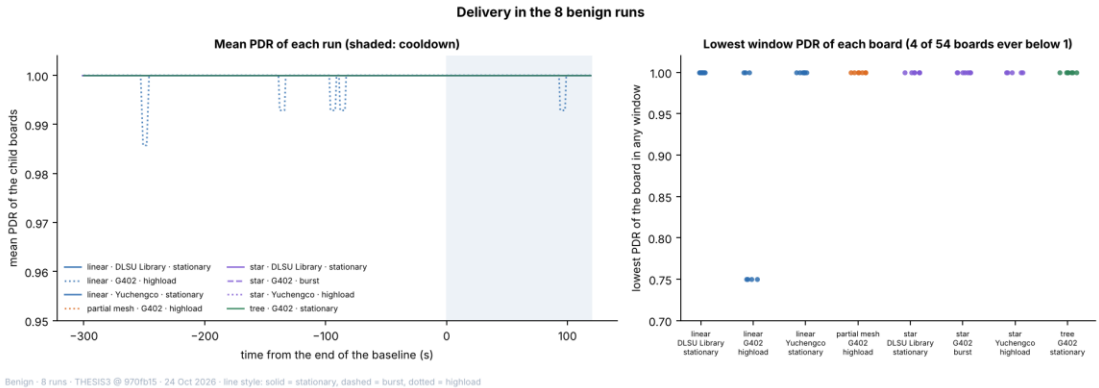


Figure 6.13 Delivery in the 8 Benign Runs over Time and the Lowest PDR of Each Board

In Figure 6.13, the left panel plots the mean PDR of the child boards of each run as a 5 s moving average against the time from the end of the baseline (0 s = end of the baseline, shaded = cooldown),

and the vertical axis is cut at 0.95 to make small changes visible. The line style gives the scenario (solid = stationary, dashed = burst, dotted = highload). The only departures from 1 are short dips in the linear run at G402 with highload, to 0.986 at the lowest. In the right panel, each point is one board and its lowest PDR in any window; four of the 54 boards fall below 1, all in the linear run at G402 with highload (to 0.75), and the other 50 stay at 1.

Table 6.21 sets the benign runs against the normal windows of the attack runs, scenario by scenario. The probe rate follows the scenario and is the same in both kinds of run: about 1.0 probes per board per second in the stationary scenario, 1.1 in the burst scenario and 3.9 in the highload scenario (benign runs; 1.0, 1.0 and 3.9 in the attack runs). Delivery is close to complete everywhere. The lower mean PDR of the stationary attack runs, 0.9967 against 1.0000 in the benign runs, comes from one run: the partial-mesh blackhole run at Yuchengco, whose baseline is unstable (mean PDR 0.931 over 2,513 windows, Table 6.9); without it the mean is 0.9999 and the share of windows below 1 falls from 0.33% to 0.01%. The medians of LatencyHopRatio and the mean RSSI_mean differ by scenario and by kind of run by a few ms per hop and a few dB, which is within the spread between runs described below.

Table 6.21 Normal Windows of the Benign Runs and of the Attack Runs, by Scenario

Scenario	Runs of	Runs (count)	Windows (count)	Mean PDR	Windows with PDR below 1 (%)	Median LatencyHopRatio (ms per hop)	Mean RSSI_mean (dBm)	Probes per board per second
stationary	Benign runs	4	11,739	1.0000	0.00	3.22	−67.5	1.00
stationary	Attack runs	24	55,229	0.9967	0.32	4.06	−72.0	1.00
burst	Benign runs	1	2,935	1.0000	0.00	5.78	−71.7	1.10
burst	Attack runs	24	55,096	0.9998	0.02	3.66	−72.1	0.99
highload	Benign runs	3	7,961	0.9998	0.08	3.94	−69.1	3.87
highload	Attack runs	24	54,860	0.9999	0.05	4.09	−70.9	3.92

Legend:

- Normal windows = the baseline and the cooldown windows of the child boards; attack runs = the blackhole and the wormhole runs together (24 runs in each scenario), their windows before and after the attack phase.
- Probes per board per second = the mean number of probes that a board sent in a one-second window, successful and failed sends together, from the application counters; it gives the probe rate of the scenario (Section 4.2.2.5).
- Windows with PDR below 1 = the share of the windows in which at least one probe of the board was not delivered.
- The stationary attack runs include the partial-mesh blackhole run at Yuchengco, whose baseline is unstable (mean PDR 0.931 over 2,513 windows).
- LatencyHopRatio = the one-way delay of a probe divided by its hops (ms per hop); RSSI_mean = the mean signal strength of the link to the parent (dBm).
- Scenario is defined in Section 4.2.2.5.

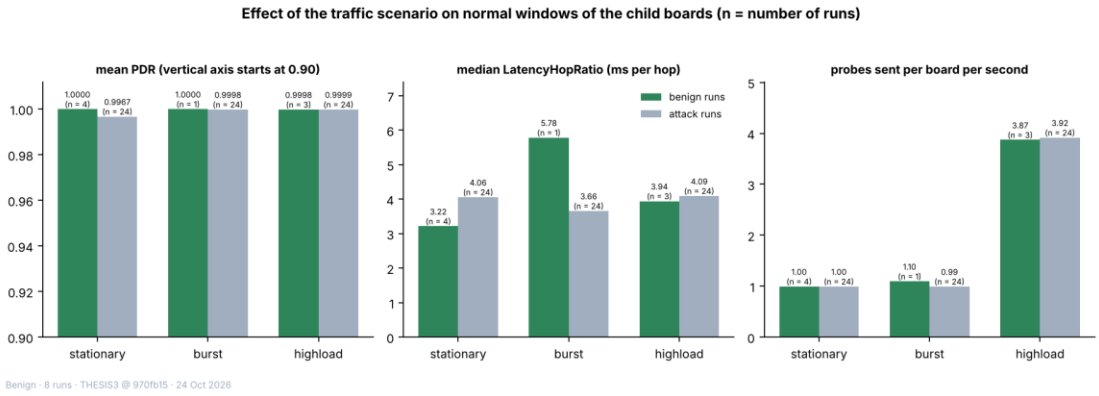


Figure 6.14 Mean PDR, Median LatencyHopRatio and Probe Rate of the Benign Runs and of the Normal Windows of the Attack Runs, by Scenario

In Figure 6.14, each group of bars is one scenario, with the benign runs in green and the normal windows of the attack runs in grey; the number of runs is printed over each bar. The left panel shows the mean PDR (the vertical axis starts at 0.90), the middle panel the median LatencyHopRatio (ms per hop) and the right panel the probes sent per board and second. The probe rate is the same in both kinds of run and about four times higher in highload; the mean PDR differs only for the stationary attack runs, because of the one run with an unstable baseline.

Table 6.22 compares the normal windows of the benign runs with those of the attack runs. The only large difference is that of the signal strength: the mean RSSI_mean is −68.6 dBm in the benign runs, against −73.8 dBm in the normal windows of the blackhole runs (d = 0.69) and −69.2 dBm in those of the wormhole runs (d = 0.07). Every other difference is below 0.1 standard deviations, except RSSI_Hop_Diff against the wormhole runs (d = 0.23). The benign runs are 8 runs in a few cells, so this difference is read together with the matched comparison that follows.

Table 6.22 Normal Windows of the Benign Runs against the Normal Windows of the Blackhole and the Wormhole Runs

Feature (unit)	Benign runs: mean	Blackhole runs, normal windows: mean	Benign against blackhole: d	Wormhole runs, normal windows: mean	Benign against wormhole: d
PDR (ratio)	0.9999	0.9979	0.06	0.9999	0.00
RSSI_mean (dBm)	−68.57	−73.78	0.69	−69.15	0.07
RSSI_var (dBm²)	0.393	0.343	0.03	0.413	−0.01
RSSI_Hop_Diff (dB)	3.50	3.61	−0.02	2.67	0.23
LatencyHopRatio (ms per hop)	11.28	9.45	0.03	10.33	0.02
RSSI_stability (s)	0.894	0.895	−0.03	0.894	0.01
HopStabilityDuration (s)	0.9999	0.9998	0.03	0.9998	0.02

Feature (unit)	Benign runs: mean	Blackhole runs, normal windows: mean	Benign against blackhole: d	Wormhole runs, normal windows: mean	Benign against wormhole: d
ParentSwitchRate (changes per second)	0.00009	0.00000	0.01	0.00000	0.01

Legend:

- Normal windows = baseline and cooldown windows of the child boards. Benign runs: 22,481 windows; blackhole runs: 89,140; wormhole runs: 75,211.
- d = Cohen's d, the benign mean minus the attack-run mean divided by the pooled standard deviation (positive = higher in the benign runs).
- n/a = d cannot be computed because the feature has one value in both groups. RetryRate and HopChangeCount are 0 in every one of these windows and are not listed.
- Features are defined in Table 6.7.

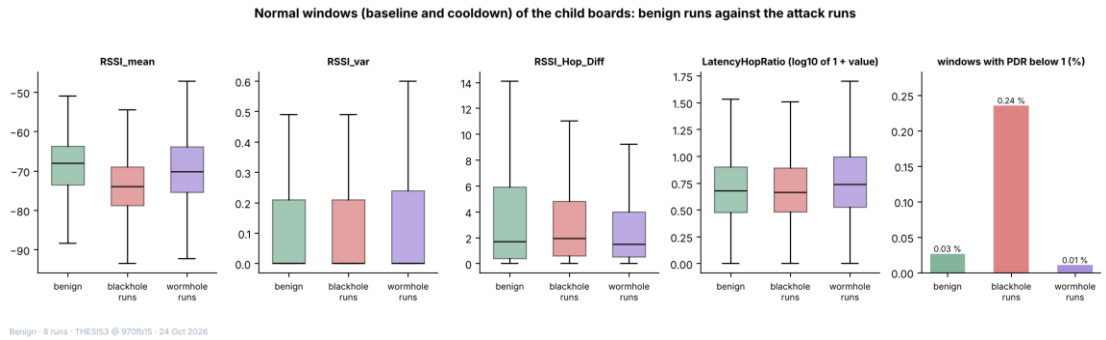


Figure 6.15 Normal Windows of the Child Boards: Benign Runs against the Blackhole and the Wormhole Runs

In Figure 6.15, the first four panels are box plots of RSSI_mean, RSSI_var, RSSI_Hop_Diff and LatencyHopRatio (logarithm of 1 plus the value) for the normal windows of the benign runs (green), the blackhole runs (red) and the wormhole runs (purple); the box spans the middle half of the windows with the median as a line, and the whiskers reach the 5th and 95th percentiles. The last panel gives the share of windows with a PDR below 1. Only the RSSI_mean boxes differ clearly, with the blackhole runs lower than the benign runs.

The benign runs share their topology, site and scenario with an attack run of the same cell, so each of them can be set against the baseline of that attack run, in which the attack has not started (Table 6.23). The baseline RSSI_mean of two captures of the same cell differs by 8.1 dB on average from the blackhole run and by 7.1 dB from the wormhole run, in both directions: the mean signed difference is +6.3 dB for the blackhole runs and −1.0 dB for the wormhole runs, and the largest absolute d is 1.97 and 3.29. The PDR differs by at most 0.0001. Each capture is a separate set-up of the boards, and the received signal strength depends on where they stand and on their surroundings, so a difference of this size between two runs is a property of the capture and not of an attack. This bears

on the clustering of the non-attacker nodes, because RSSI_mean can then separate the runs by their set-up and not the attack from the normal windows (Section 6.4).

Table 6.23 Baseline of Each Benign Run against the Baseline of the Attack Run of the Same Topology, Site and Scenario

Attack run compared	Feature (unit)	Runs (count)	Mean difference (benign − attack run)	Mean absolute difference	Median d	Largest absolute d
Blackhole	PDR (ratio)	8	−0.0000	0.0001	0.00	0.03
Blackhole	RSSI_mean (dBm)	8	6.34	8.08	1.29	1.97
Blackhole	RSSI_var (dBm²)	8	−0.092	0.359	−0.14	0.51
Blackhole	RSSI_Hop_Diff (dB)	8	0.59	0.81	0.12	0.62
Blackhole	LatencyHopRatio (ms per hop)	8	0.46	6.82	0.02	0.23
Wormhole	PDR (ratio)	8	−0.0000	0.0001	−0.01	0.07
Wormhole	RSSI_mean (dBm)	8	−1.00	7.12	−0.61	3.29
Wormhole	RSSI_var (dBm²)	8	−0.027	0.285	0.02	0.35
Wormhole	RSSI_Hop_Diff (dB)	8	1.79	1.94	0.65	1.27
Wormhole	LatencyHopRatio (ms per hop)	8	0.24	8.03	−0.04	0.30

Legend:

- Each benign run is compared with the baseline windows of the blackhole run and of the wormhole run of the same topology, site and scenario; the differences are taken between the benign mean and the attack-run baseline mean, one pair for each of the 8 runs.
- Mean difference = the average of the 8 signed differences; mean absolute difference = the average of their absolute values; d = Cohen's d of each pair, summarised by its median and its largest absolute value.
- The attack runs are separate captures of the same cell, each with its own set-up of the boards.
- Features are defined in Table 6.7.

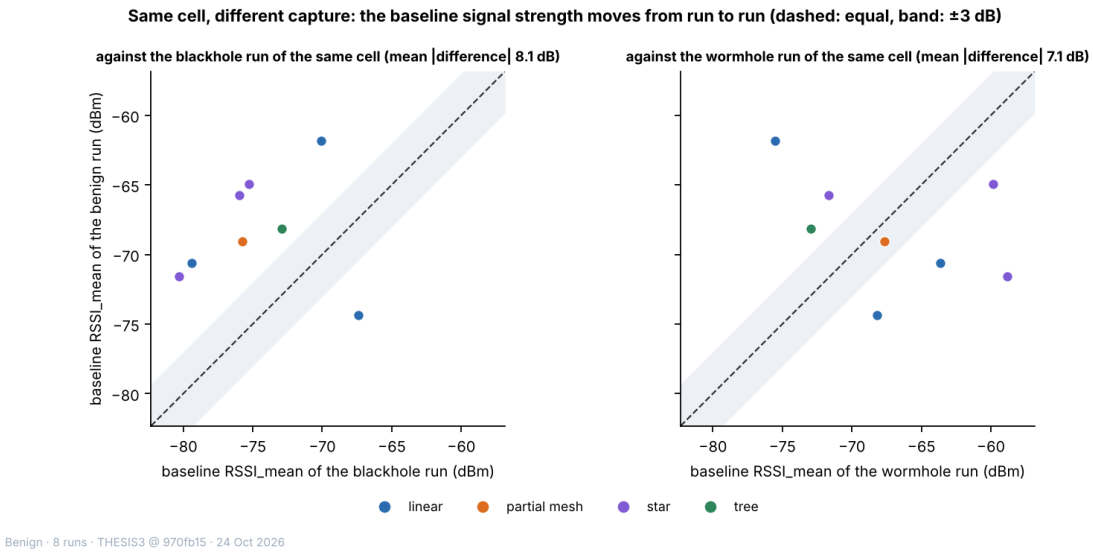


Figure 6.16 Baseline RSSI_mean of Each Benign Run against That of the Attack Run of the Same Cell

In Figure 6.16, each point is one benign run and the panels are its comparison with the blackhole run (left) and the wormhole run (right) of the same topology, site and scenario. The horizontal axis is the baseline RSSI_mean of the attack run (dBm) and the vertical axis that of the benign run; the dashed line marks equal values and the band is ±3 dB around it. The colour gives the topology. The points lie on both sides of the line and mostly outside the band, which shows a difference between captures of the same cell that goes in both directions.

The cooldown of a benign run is the stretch in which an attack would have run in the attack runs, so it is compared with the baseline of the same run. For RSSI_mean, RSSI_var and LatencyHopRatio the cooldown mean lies within 0.5 baseline standard deviations of the baseline mean in all 24 comparisons (8 runs, 3 features). The PDR can be compared in 1 run only, because in the other seven the baseline PDR never varies; there the distance is +0.06. The benign runs thus show no drift that a detector could mistake for an attack. The 8 benign runs are used again in the pooled clustering of Section 6.4.7, as a reference group of normal windows.

6.4. Clustering Results

Clustering asks whether the windows fall into groups by themselves, from the values of their features alone, and whether those groups coincide with the attack. The label is never given to the algorithms. It is used only afterwards, to score the clusters. The blackhole runs are clustered first (Sections 6.4.1 to 6.4.5). The wormhole runs follow (Section 6.4.6), and the last analysis pools the 80 runs, benign runs included, and scores the clusters against three labels: normal, blackhole and wormhole (Section 6.4.7).

6.4.1. Setup

The clustering follows the procedure of Chapter 4. A cluster is a set of windows that an algorithm puts together because their feature values are close. Five algorithms (K-Means, Ward hierarchical clustering, a Gaussian mixture model (GMM), DBSCAN and spectral clustering) are run on standardised features, with k, the number of clusters, from 2 to 8 and three seeds (42, 7 and 123). Each seed draws a new sample of 6,000 windows from the 127,372 windows of the non-attacker nodes that have every feature of the FULL set (of the 128,175 labelled windows of these nodes, 803 lack at least one feature), and restarts the algorithm. DBSCAN takes min_samples between D + 1 and 2D, where D is the number of features, and eps, the neighbourhood radius, at the knee of the k-distance graph (Figure 6.17); of the settings tried, the one with the highest silhouette score is kept. The cluster purity and the adjusted Rand index (ARI) are computed afterwards against the label

(baseline and cooldown = 0, attack = 1). All results below are means over the three seeds, with k = 2 for the four methods that need k.

Table 6.24 Feature Sets Used for Clustering

Feature set	Features	Count	Purpose
FULL	PDR, ParentSwitchRate, HopChangeCount, HopStabilityDuration, RSSI_mean, RSSI_var, RSSI_stability, RSSI_Hop_Diff	8	All features eligible for clustering (Table 6.7); the start of both filters.
REDUCED	PDR, RSSI_mean, RSSI_var, RSSI_Hop_Diff	4	FULL after the low-variance and correlation filters (Section 4.2.4.6).
REDUCED without PDR	RSSI_mean, RSSI_var, RSSI_Hop_Diff	3	Control: REDUCED without the delivery ratio.

Legend:

- A feature set is the group of features given to the clustering algorithms.
- The names FULL, REDUCED and Without PDR are used in Table 6.25 and in the figures of this section.
- Percentages in the text are the share of the 127,372 non-attacker-node windows of the blackhole runs that hold the most common value of the feature.
- Low-variance filter = drops a feature when more than 90% of the windows have the same value; correlation filter = drops a feature when $|r| > 0.85$ with another kept feature; r = the correlation coefficient between two features (−1 to 1).
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1).
- Control = a set used to test whether anything other than the delivery ratio separates the windows.

Three feature sets are compared (Table 6.24). FULL contains the 8 features that may enter the clustering of non-attacker nodes. The dataset defines 16 features, but ForwardingRatio, IngressEgressDelta, ConsistencyScore, RetryRate and the three tunnel features are kept out by the leakage guard of Section 6.2.3, and LatencyHopRatio is kept out because the fact that it is missing alone predicts the label in 86% of the windows (Section 6.2.3). REDUCED is FULL after the two filters of Section 4.2.4.6. The low-variance filter removed ParentSwitchRate (100% of the windows hold the same value), HopChangeCount (100%), HopStabilityDuration (99.8%) and RSSI_stability (98.5%). The correlation filter removed none, because the largest |r| among the four features that remain is 0.11. The third set, REDUCED without PDR, is a control that removes the delivery ratio from REDUCED, to show whether anything other than PDR separates the windows. The sets were derived on the blackhole windows by these rules and were kept for the wormhole and pooled analyses. On the pooled windows the low-variance rule would also remove PDR, because 91.5% of them have a PDR of 1, which is the case of the wormhole and benign runs (100.0% and 100.0%); PDR was kept so that the three analyses use the same features. Two populations are clustered: all non-attacker nodes, and victims only (72,687 windows). The first includes the windows of bystanders that carry the attack label (Section 6.1), and the second does not.

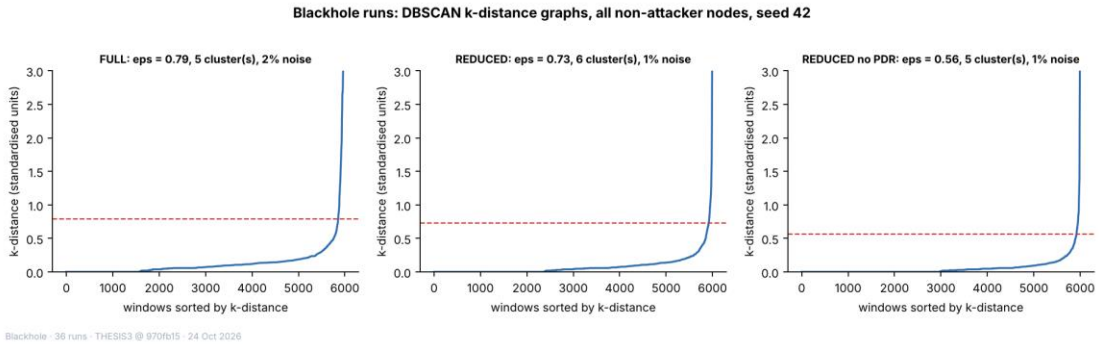


Figure 6.17 DBSCAN k-Distance Graphs for the Three Feature Sets (All Non-attacker Nodes, Seed 42)

In Figure 6.17, each curve orders the 6,000 sampled windows (horizontal axis) by their distance to the k-th nearest other window, where k is the min_samples value of the setting that was kept (vertical axis, standardised units). The dashed line is eps, the neighbourhood radius used by DBSCAN, set at the knee where the curve turns upward: 0.79 for FULL (5 clusters, 2% noise), 0.73 for REDUCED (6 clusters, 1% noise) and 0.56 for REDUCED without PDR (5 clusters, 1% noise). A window with fewer than min_samples neighbours within eps is treated as noise. The vertical axis is cut at 3 and the curves rise to 47, 35 and 33.

6.4.2. Quality and Agreement with the Label

Table 6.25 gives the quality metrics and the agreement with the label for every algorithm and feature set. A purity of 0.702 is the majority rate of the sample (0.705 for victims only), so it means no separation. The table is read in two steps: first the silhouette score and the Davies–Bouldin index, which describe the shape of the clusters, then the purity and the ARI, which describe their agreement with the label.

Table 6.25 Clustering Quality and Agreement with the Label, Means over Seeds 42, 7 and 123

Feature set	Algorithm	Silhouette (−1 to 1)	Davies–Bouldin (0 and up)	Purity, all non-attacker nodes (0 to 1)	ARI, all non-attacker nodes (−1 to 1)	Purity, victims only (0 to 1)	ARI, victims only (−1 to 1)
REDUCED	K-Means	0.489	0.917	0.870	0.514	0.999	0.996
REDUCED	Ward	0.478	0.887	0.867	0.503	0.973	0.891
REDUCED	GMM	0.489	0.917	0.870	0.514	0.904	0.659
REDUCED	DBSCAN (5 to 9 clusters, 1.3% noise)	0.510	0.447	0.866	0.485	0.992	0.940
REDUCED	Spectral	0.157	0.729	0.702	−0.001	0.707	0.008

Feature set	Algorithm	Silhouette (−1 to 1)	Davies–Bouldin (0 and up)	Purity, all non-attacker nodes (0 to 1)	ARI, all non-attacker nodes (−1 to 1)	Purity, victims only (0 to 1)	ARI, victims only (−1 to 1)
FULL	K-Means	0.773	0.558	0.757	0.172	0.705	−0.001
FULL	Ward	0.905	0.332	0.702	0.000	0.705	−0.002
FULL	GMM	0.790	0.409	0.757	0.173	0.705	0.002
FULL	DBSCAN (5 to 6 clusters, 2.0% noise)	0.524	0.499	0.865	0.476	0.991	0.929
FULL	Spectral	0.072	0.882	0.702	0.000	0.706	0.001
Without PDR	K-Means	0.315	1.269	0.702	0.000	0.705	0.000
Without PDR	Ward	0.728	0.687	0.702	0.000	0.705	−0.001
Without PDR	GMM	0.414	1.938	0.702	−0.016	0.705	−0.009
Without PDR	DBSCAN (5 to 7 clusters, 1.4% noise)	0.615	0.333	0.702	0.000	0.705	0.001
Without PDR	Spectral	0.165	0.927	0.702	0.000	0.705	0.000

Legend:

- Feature set: REDUCED = 4 features (PDR, RSSI_mean, RSSI_var, RSSI_Hop_Diff); FULL = 8 features; Without PDR = the 3 REDUCED features other than PDR (Table 6.24).
- K-Means = groups windows around k centres, each window going to the nearest centre; Ward = hierarchical clustering that merges the two clusters whose union adds the least spread; GMM = Gaussian mixture model, which assumes that the windows come from k bell-shaped groups; DBSCAN = groups windows that lie in dense regions and leaves isolated windows as noise; Spectral = groups windows using the eigenvectors of a graph of their similarities.
- k = 2 for K-Means, Ward, GMM and Spectral. DBSCAN chooses its own number of clusters: for it, the range of the number of clusters over the three seeds and the share of windows left as noise are given in brackets for all non-attacker nodes. On victims alone DBSCAN found 2 to 4 (FULL), 5 to 6 (REDUCED) and 3 to 5 clusters (Without PDR).
- Seeds 42, 7 and 123 = the three random starting values; each draws a new sample of 6,000 windows and restarts the algorithm.
- Silhouette = how well each window fits its own cluster compared with the nearest other cluster (higher is better); Davies–Bouldin = the average similarity of each cluster to its most similar cluster (lower is better).
- Purity = the share of windows that carry the majority label of their cluster; ARI (adjusted Rand index) = agreement between the clusters and the labels: 1 = exact match, 0 = no better than chance, negative = worse than chance.
- The two populations are clustered separately: all non-attacker nodes and victims only (Section 6.4.1). Silhouette and Davies–Bouldin are given for all non-attacker nodes.

With the delivery ratio among the features, clustering separates the windows in which the attack is felt. K-Means with k = 2 on the REDUCED set reaches an ARI of 0.514 on all non-attacker nodes and 0.996 on victims alone. GMM, Ward and DBSCAN agree on all non-attacker nodes (0.514, 0.503 and 0.485) and are less exact on victims alone (0.659, 0.891 and 0.940). Without the delivery ratio, no algorithm finds the attack: for every algorithm and every k from 2 to 8, on both populations, the ARI lies between −0.016 and 0.002, and the projection shows no structure (Section 6.4.5). The blackhole does not show in the signal-strength features. The ARI against k is given in Figure 6.18, the silhouette score in Figure 6.19 and the Davies–Bouldin index in Figure 6.20.

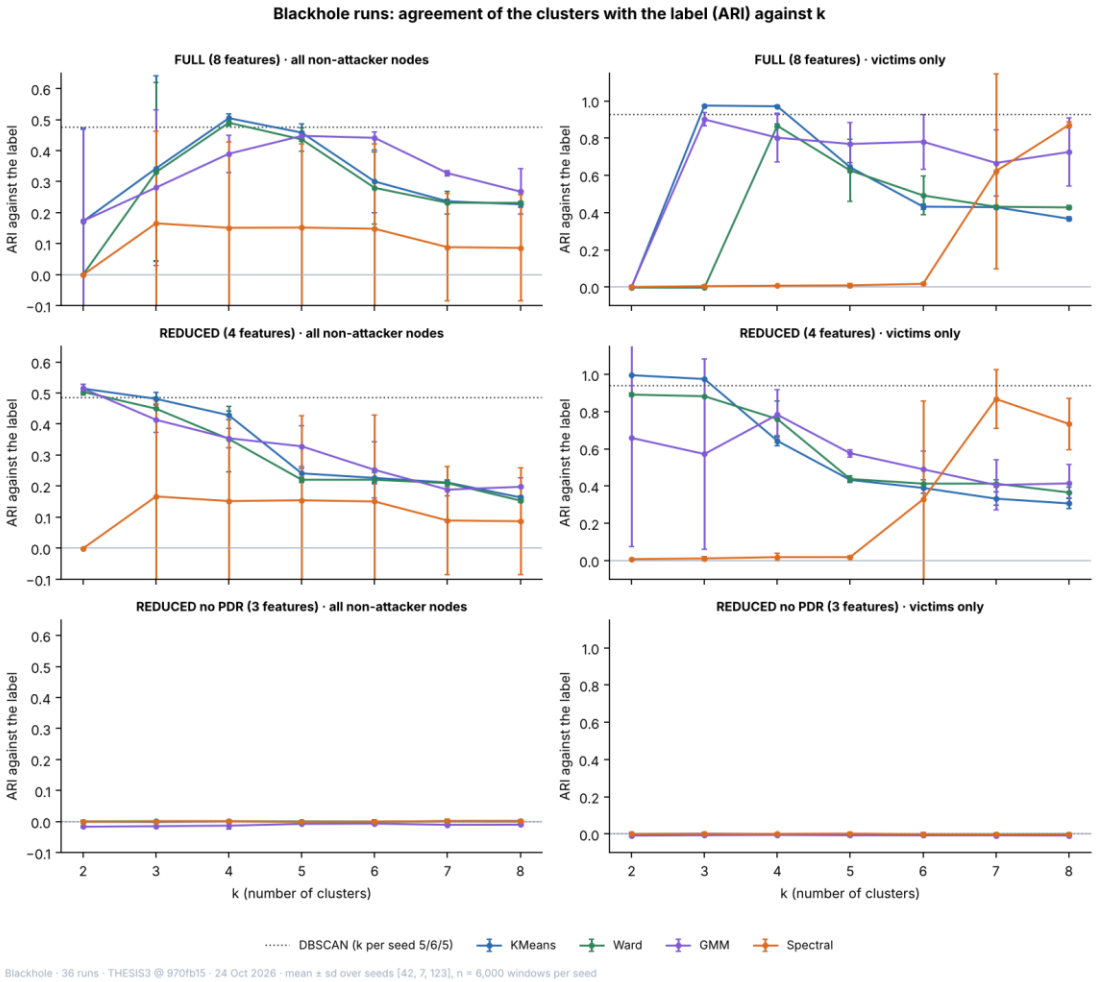


Figure 6.18 Adjusted Rand Index against the Number of Clusters, by Feature Set and Population (Mean and Spread over Three Seeds)

In Figure 6.18, the horizontal axis is k, the number of clusters, and the vertical axis is the ARI against the label (1 = the clusters match the labels exactly, 0 = no better than chance). The rows are the feature sets, the left column is all non-attacker nodes and the right column is victims only. Each coloured line is one algorithm, and its bars show ±1 standard deviation over the three seeds. The dotted line is DBSCAN, which chooses its own number of clusters. In the bottom row, all values lie between −0.016 and 0.002 and are drawn on the same scale as the rows above.

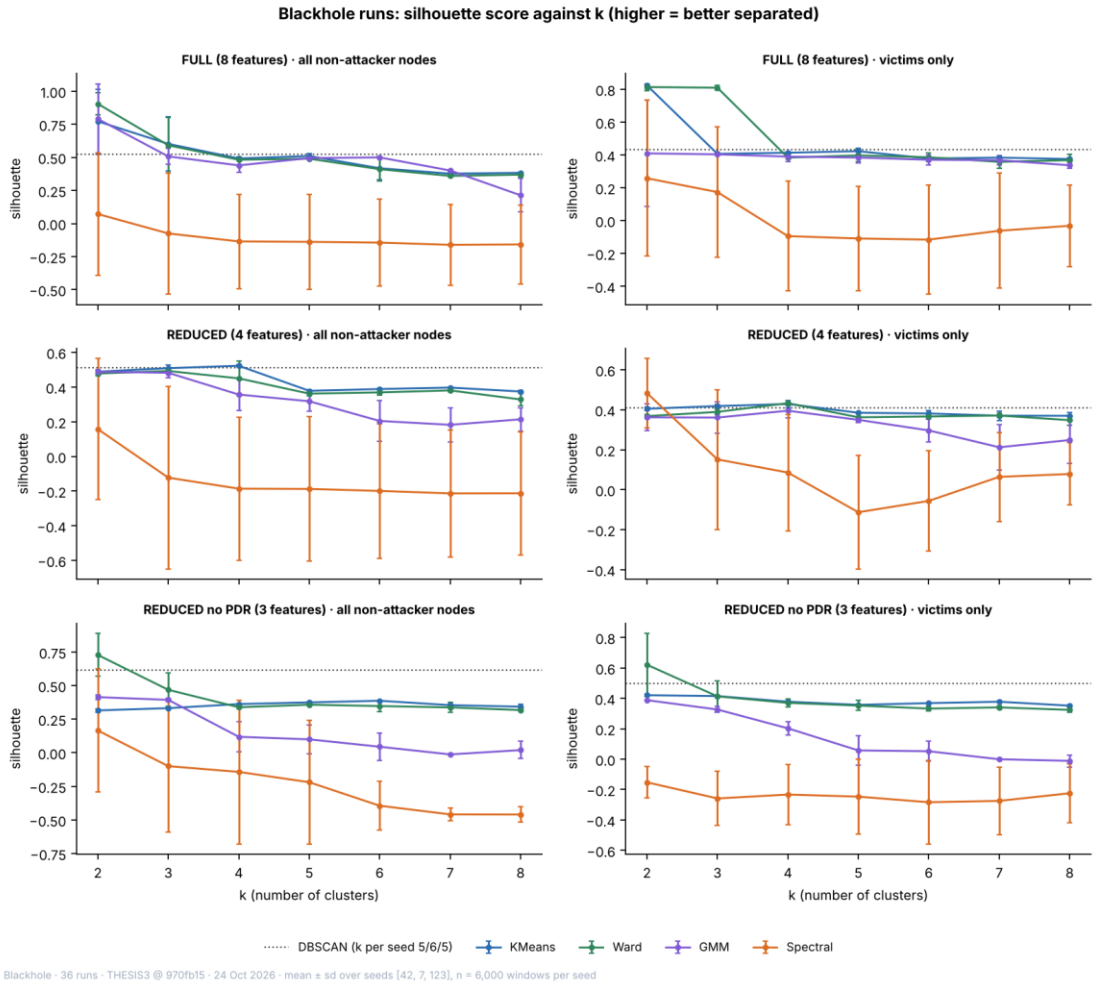


Figure 6.19 Silhouette Score against the Number of Clusters, by Feature Set and Population

Figure 6.19 has the same layout as Figure 6.18. The vertical axis is the silhouette score (−1 to 1, no unit), and a higher value means that the windows lie closer to their own cluster than to the nearest other cluster. The silhouette score describes the shape of the clusters and not their agreement with the label.

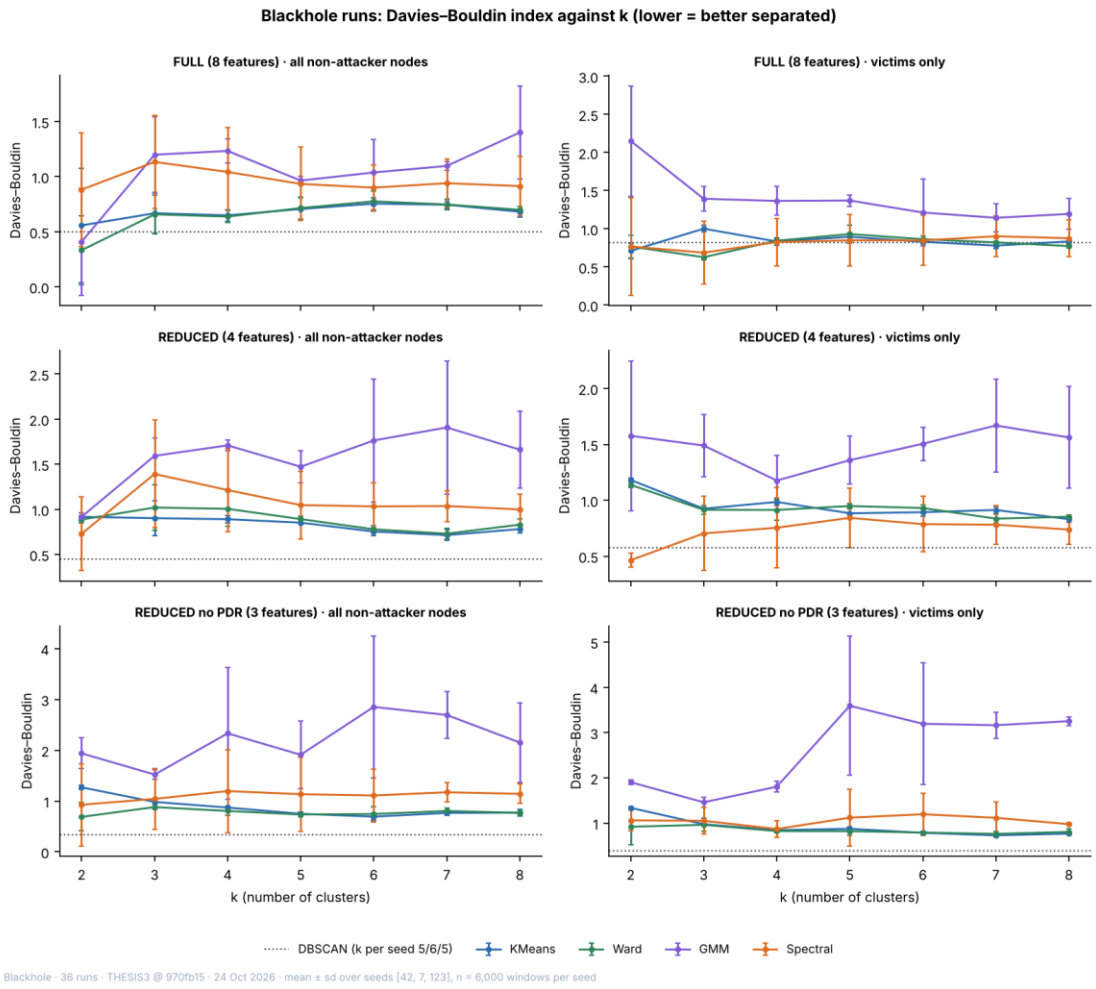


Figure 6.20 Davies–Bouldin Index against the Number of Clusters, by Feature Set and Population

Figure 6.20 has the same layout as Figure 6.18. The vertical axis is the Davies–Bouldin index (0 and above, no unit), and a lower value means that the clusters are more compact and further apart.

6.4.3. What Limits the Agreement

The ceiling on all non-attacker nodes is an ARI of 0.515, not 1. A bystander carries the attack label in the attack window but behaves like a normal node, and 43.2% of the attack-labelled non-attacker-node windows (16,499 of 38,232) come from bystanders. No method that reads only the features of a window can group them with the attack. A perfect separation of the windows in which the attack is felt scores 0.515 on all non-attacker nodes, and K-Means on the REDUCED set scores 0.514 (sd 0.014 over the seeds). On victims alone the same method scores 0.996. The victims-only figure is therefore reported next to the all-nodes figure.

The silhouette score alone picks the wrong k on the FULL set. K-Means and Ward score highest at k = 2 (0.77 and 0.90), but the split does not follow the attack. For K-Means the ARI at k = 2 is 0.515, 0.000 and 0.000 on the three seeds (mean 0.172, sd 0.298). On seed 42 the two clusters are the attack

windows of the victims and the rest. On seed 7 the smaller cluster holds 44 of the 6,000 windows, with a mean RSSI_var of 8.8 against 0.22 in the rest and a low RSSI_stability, and on seed 123 it holds a single window, in which the board changed its parent (ParentSwitchRate and HopChangeCount are non-zero in that window and zero in almost all others). Ward at k = 2 gives an ARI of −0.001, 0.001 and 0.000. The attack is recovered when k reaches 4: K-Means gives 0.505 (sd 0.014) and Ward 0.490. The choice of k therefore has to use the elbow, the silhouette peaks and interpretability together, as Chapter 4 sets out, and not the silhouette alone.

GMM is stable on the REDUCED set at k = 2 on all non-attacker nodes (ARI 0.520, 0.524 and 0.498 on the three seeds), but on victims alone its mean ARI is 0.659 with a standard deviation of 0.58 over the seeds, and on the FULL set it behaves like K-Means (0.520, −0.001 and 0.000). Spectral clustering finds nothing at k = 2 on any feature set (the largest absolute ARI is 0.008). For k = 3 to 6 on the FULL and REDUCED sets its mean ARI on all non-attacker nodes lies between 0.148 and 0.167, with a standard deviation of 0.27 to 0.30 over the seeds, which means that it finds the attack on one seed and not on the others; on victims alone it reaches 0.871 at k = 8 on the FULL set. It is therefore reported and not relied on. DBSCAN, which is given no k, agrees with K-Means: on the REDUCED set it finds 5 to 9 clusters (purity 0.866, ARI 0.485) and leaves 1.3% of the windows as noise.

Figure 6.21 shows what each K-Means cluster contains at k = 3. With the delivery ratio among the features, one cluster (1,008 windows in the FULL set and 1,000 in the REDUCED set) is almost entirely attack windows of the victims (99.4% and 99.4% of its windows are in the attack phase). A second cluster holds the rest. On the FULL set the third cluster has 12 windows and an extreme RSSI variance (a mean of 27.2), and on the REDUCED set it has 303 windows with a mean RSSI_Hop_Diff of 20.0 dB, so a small cluster of extreme windows appears in both. Without the delivery ratio, every cluster of 100 windows or more is a mixture of the four kinds of window, and the attack-phase share lies between 28% and 31% in them, against 29% in the sample. Table 6.26 gives the profile of the two K-Means clusters at k = 2 on the REDUCED set.

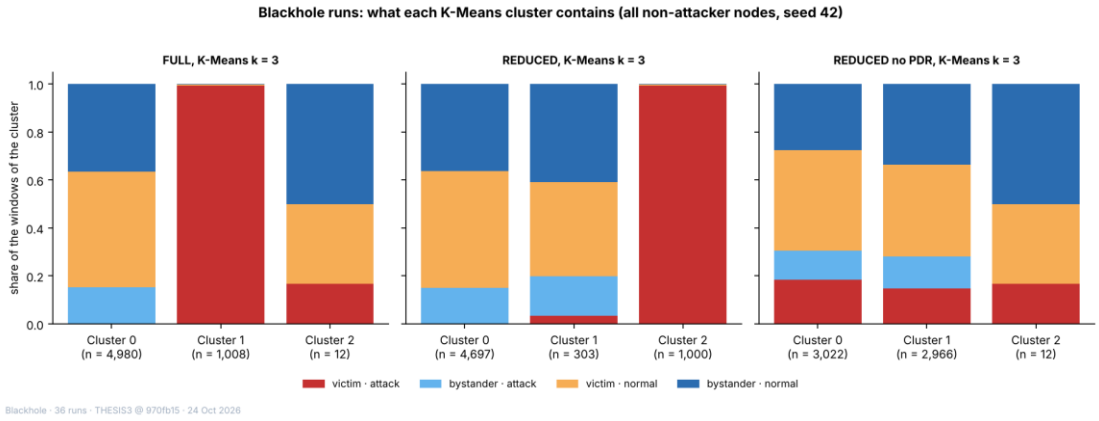


Figure 6.21 Composition of Each K-Means Cluster at k = 3, by Feature Set (All Non-attacker Nodes, Seed 42)

In Figure 6.21, each bar is one cluster, with the number of windows in it (n) under its name, and its height is split into the share of four kinds of window. Red = victim windows in the attack phase, light blue = bystander windows in the attack phase, orange = victim normal windows and dark blue = bystander normal windows. Normal means baseline or cooldown. A bar that is entirely red holds only attack windows of victims. With PDR (left and centre), one cluster is entirely red; without PDR (right), no cluster is.

Table 6.26 K-Means Clusters at k = 2 on the REDUCED Set (All Non-attacker Nodes, Seed 42)

Measure (unit)	Cluster A (no delivery loss)	Cluster B (delivery lost)
Windows (count)	4,990	1,010
Attack-phase windows (%)	15.2	99.4
Baseline windows (%)	59.5	0.6
Cooldown windows (%)	25.4	0.0
Windows from victims (%)	48.2	99.7
Mean PDR (ratio)	1.000	0.000
Mean RSSI_mean (dBm)	−73.6	−74.9
Mean RSSI_var (dBm²)	0.315	0.322
Mean RSSI_Hop_Diff (dB)	3.65	3.33

Legend:

- The percentages are shares of the windows in the cluster. The three phase rows (attack, baseline and cooldown) add up to 100% in each cluster.
- Cluster A is the cluster with a mean PDR of 1 and Cluster B the cluster with a mean PDR of 0; the cluster numbers given by the algorithm carry no meaning.
- Mean values are the mean of each feature over the windows of the cluster. Windows from victims = the share of the windows of the cluster that come from victim boards.
- K-Means = groups windows around k centres; k = 2 is the number of clusters; REDUCED = the feature set of Table 6.24; seed 42 = the random starting value that fixes the sample of 6,000 windows and the start of the algorithm.
- PDR = packet delivery ratio (ratio, 0 to 1); RSSI = received signal strength indicator; dBm = decibel-milliwatts; dB = decibels.

Cluster B holds 17% of the windows of the sample and is 99.4% attack-phase windows with a mean PDR of 0.000, and Cluster A, with a mean PDR of 1.000, holds the baseline and cooldown windows and the attack windows of the bystanders (15.2% of its windows are attack-phase windows). The other three features differ little between the two clusters.

6.4.4. Robustness across Sites, Topologies and Labelling

The result holds in every subgroup (Table 6.27). K-Means with k = 2 on the REDUCED set stays within 0.03 of the best possible ARI inside each site and each topology, and without the delivery ratio it stays at 0 inside each of them (between −0.006 and 0.002). The best possible figure is the ARI of one particular partition, so a method can edge past it, as it does in the linear topology. In the star the ARI is 0.999, because the star has no bystanders. Dropping the cooldown windows from the normal label keeps the same pattern: with the delivery ratio, K-Means purity is 0.836 against a majority rate of 0.628, and without it purity equals the majority rate (0.628).

Table 6.27 K-Means (k = 2) ARI inside Each Site and Topology, All Non-attacker Nodes

Subgroup	Best possible ARI (−1 to 1)	REDUCED: K-Means ARI (−1 to 1)	REDUCED without PDR: K-Means ARI (−1 to 1)
Site G402	0.531	0.516	0.002
Site DLSU Library	0.572	0.574	0.002
Site Yuchengco	0.445	0.420	−0.006
Linear	0.560	0.565	−0.001
Partial mesh	0.286	0.277	−0.001
Star	1.000	0.999	−0.003
Tree	0.279	0.278	0.000

Legend:

- ARI = adjusted Rand index, the agreement between the clusters and the labels (1 = exact match, 0 = no better than chance).
- Best possible ARI = the ARI of the partition that puts exactly the windows in which the attack is felt (victim windows in the attack phase) in one group and every other window in the other; it is the agreement that a method reading only the features of a window would reach if it found exactly those windows, and it is below 1 where there are bystanders, which carry the attack label but behave as normal nodes. Because it is the ARI of one particular partition, a method can edge past it.
- REDUCED and REDUCED without PDR are the feature sets of Table 6.24. K-Means (k = 2) = groups windows around two centres, each window going to the nearest.
- Each row clusters only the windows of that site or topology (mean over seeds 42, 7 and 123). Site = where the runs were made; topology = the shape in which the boards are linked to the root.

The seed spread of Section 6.4.2 measures only how much the result moves when the sample of 6,000 windows and the starting points of the algorithm change. It does not measure how much it moves from run to run, and the 127,372 windows come from only 36 runs. A run-level check was therefore added. One K-Means model (k = 2, REDUCED set, seed 42) was fitted on a sample of 6,000 windows

and applied to every window of the non-attacker nodes. Its ARI over all windows is 0.511. Resampling the 36 runs with replacement (300 times) gives a 95% interval of 0.41 to 0.63, which is much wider than the seed spread (sd 0.014) and still far from 0. Inside each run, the model comes within 0.05 of the best possible ARI of that run in 35 of the 36 runs. The exception is partial mesh at Yuchengco with stationary traffic (ARI 0.066 against a best possible 0.139), the run whose baseline delivery ratio is unstable (Section 6.2.1). Without the delivery ratio, the same check gives an ARI of 0.000 (95% interval −0.001 to 0.002) over all windows, and between −0.06 and 0.04 in the single runs; no run comes within 0.05 of its best possible value.

6.4.5. Projections

Figure 6.22 projects the standardised FULL features of a sample of 6,000 non-attacker-node windows to two dimensions with PCA and t-SNE, so that each window becomes a point on a plane. In the PCA, the first component is dominated by a few windows of extreme RSSI variance, so most points lie in a narrow column, and the attack windows of the victims lie along its lower part together with some normal windows. In the t-SNE, the attack windows of the victims form their own group at the bottom of the plot, and the attack windows of the bystanders lie among the normal windows. Figure 6.23 repeats the projection after the delivery ratio is removed: the four kinds of window overlap and no structure is visible. The axes of t-SNE have no units, and only nearness is read from them.

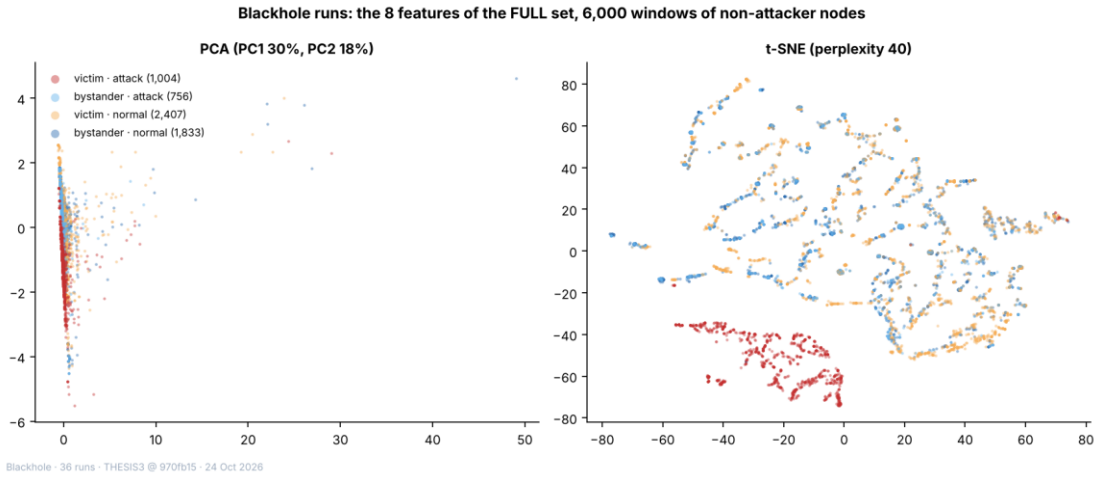


Figure 6.22 PCA and t-SNE Projections of the FULL Feature Set (Blackhole Runs)

In Figure 6.22, each point is one window (a sample of 6,000), coloured by its kind (red = victim attack-phase, orange = victim normal, light blue = bystander attack-phase, dark blue = bystander normal). The eight features of a window are reduced to two numbers so that the window can be drawn on a plane. PCA (left) uses the two directions that hold the most variance: PC1 holds 30% of the variance and PC2 holds 18% (the axes are in standardised feature units). t-SNE (right) places windows

with similar features close together; its axes have no unit, and only the distance between points means anything. “Group” here means points that lie close together in the picture. A cluster, by contrast, is the result of an algorithm (Section 6.4.1).

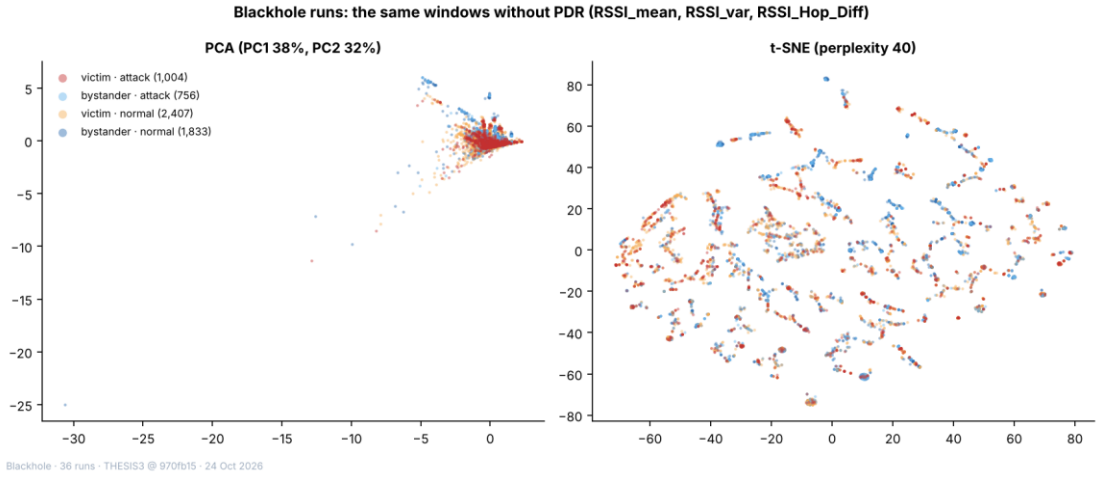


Figure 6.23 PCA and t-SNE Projections with the Delivery Ratio Removed (Blackhole Runs)

Figure 6.23 is built in the same way as Figure 6.22, but only RSSI_mean, RSSI_var and RSSI_Hop_Diff are used. In the PCA, PC1 holds 38% of the variance and PC2 holds 32%. The colours are the same as in the previous figure. The four colours lie on top of one another in both pictures, which means that these three features do not separate the attack windows.

6.4.6. Wormhole Runs

The wormhole runs were clustered with the same procedure, three seeds, k from 2 to 8 and the same five algorithms. The label is attack (the phases of the tunnel) against normal. Three populations are clustered. The first is the five non-attacker boards of each run (107,405 windows), with the three feature sets of Table 6.24; they are not touched by the wormhole, which acts only on the probes of the entry end B, so no agreement with the label is expected. The second and third are the exit end A and the entry end B, clustered on the three signal-strength features (RSSI_mean, RSSI_var and RSSI_Hop_Diff), and the entry end B also with LatencyHopRatio. As a diagnostic, each end was also clustered with TunnelIntensity, the number of tunnelled messages per second, which the leakage guard keeps out of the model inputs (Section 6.2.3). It shows whether the data contain a tunnel signal that a clustering algorithm can find at all. The wormhole windows are 30% of the windows of each end (the majority rate of the normal label is 0.697 on the end B and 0.700 on the end A).

Table 6.28 Agreement of the Clusters with the Attack Label in the Wormhole Runs (ARI, Means over Seeds 42, 7 and 123)

Population and feature set	K-Means k = 2	K-Means k = 3	Ward k = 2	GMM k = 2	Spectral k = 2	DBSCAN (own k)
Non-attacker boards, FULL	−0.005	−0.003	−0.004	−0.006	0.000	−0.005 (2 to 3 clusters)
Non-attacker boards, REDUCED	−0.001	0.001	−0.001	−0.008	−0.001	−0.005 (2 to 3 clusters)
Non-attacker boards, without PDR	0.000	−0.001	−0.004	−0.008	−0.001	−0.005 (4 to 5 clusters)
Exit end A, cross-layer	−0.001	−0.002	−0.001	−0.008	−0.003	−0.002 (2 to 6 clusters)
Exit end A, cross-layer + TunnelIntensity	−0.003	0.207	0.087	0.080	−0.007	0.300 (2 to 13 clusters)
Entry end B, cross-layer	0.000	−0.001	0.000	−0.006	0.003	−0.003 (5 to 8 clusters)
Entry end B, cross-layer + LatencyHopRatio	0.000	0.001	0.000	0.009	0.000	−0.004 (3 clusters)
Entry end B, cross-layer + TunnelIntensity	0.000	0.141	0.010	0.161	−0.003	0.827 (9 to 12 clusters)

Legend:

- ARI = adjusted Rand index, the agreement between the clusters and the labels (1 = exact match, 0 = no better than chance, negative = worse than chance). The label is attack against normal.
- Non-attacker boards = the five boards of each run that are neither end of the tunnel; exit end A = the shallower end of the tunnel, where the tunnelled probes are re-injected; entry end B = the deeper end, which tunnels its own probes. Sets for the non-attacker boards are those of Table 6.24.
- Cross-layer = RSSI_mean, RSSI_var and RSSI_Hop_Diff; TunnelIntensity = tunnelled messages per second (a diagnostic that the leakage guard keeps out of the model inputs); LatencyHopRatio = the latency per hop of the probes of the node.
- DBSCAN chooses its own number of clusters; the range over the three seeds is given in brackets. K-Means = groups windows around k centres; Ward = hierarchical clustering; GMM = Gaussian mixture model; Spectral = clustering on the eigenvectors of a similarity graph.
- Majority rate of the normal label: 0.700 (non-attacker boards), 0.700 (exit end A), 0.697 (entry end B).

Without a tunnel feature, no algorithm finds the wormhole. On the non-attacker boards the ARI lies between −0.012 and 0.003 for all algorithms, feature sets and values of k from 2 to 8, and the purity equals the majority rate. On the two ends, with the cross-layer features alone (and with LatencyHopRatio on the end B), it lies between −0.012 and 0.009. The run-level check gives the same result for the K-Means model (k = 2, REDUCED set) over all windows of the non-attacker boards: an ARI of −0.001 with a 95% interval of −0.008 to 0.009, and between −0.04 and 0.09 in the single runs. This agrees with Table 3.5: the wormhole leaves the other boards as they are, and what the two ends show, the duplicate arrivals of the tunnelled probes, is not one of the features of the cross-layer set.

With TunnelIntensity the picture depends on the algorithm. The feature is exactly 0 in every normal window of the entry end B (and in all but 1 of the 15,092 normal windows of the exit end A) and above 0 in 98.9% of the wormhole windows of the entry end B (99.4% on the exit end A), with a median of 1 message per second in the stationary and burst runs and 4 in the highload runs. A single threshold on it would therefore separate the labels, which is why it is a tunnel feature and is excluded from the model inputs. A clustering algorithm does not use it so directly. On the entry end B, K-Means with k = 2 splits the windows by signal level, which varies between runs (Table 6.29: mean RSSI_mean of −67.6 and −47.1 dBm), and TunnelIntensity does not enter the split (0.56 and 0.64 message per second in the two clusters); its ARI is 0.000. At k = 3 it forms a third cluster of 540 windows that are all wormhole windows with a mean TunnelIntensity of 4.04, the level of the highload runs, and the ARI rises to 0.141. GMM gives 0.161 at k = 2, and DBSCAN finds 9 to 12 clusters, nearly all of them pure in the label, with a purity of 0.991, an ARI of 0.827 and 1.4% noise. On the exit end A, DBSCAN reaches 0.300, K-Means at k = 3 0.207 and GMM at k = 8 0.459 (with a large spread over the seeds). K-Means at k = 2 therefore does not find the tunnel even when a tunnel feature is given; DBSCAN finds it, and K-Means finds its highload part from k = 3. Figure 6.24 gives the ARI against k for all populations and Figure 6.25 the projections.

Table 6.29 K-Means Clusters of the Entry End B with TunnelIntensity (Seed 42)

k	Cluster	Windows (count)	Wormhole windows (%)	Mean RSSI_mean (dBm)	Mean RSSI_Hop_Diff (dB)	Mean TunnelIntensity (messages per second)
k = 2	Cluster 0	4,523	30.4	−67.6	1.7	0.56
k = 2	Cluster 1	1,477	32.1	−47.1	17.3	0.64
k = 3	Cluster 0	4,139	24.0	−67.7	1.7	0.24
k = 3	Cluster 1	1,321	24.1	−47.5	17.3	0.24
k = 3	Cluster 2	540	100.0	−60.1	6.0	4.04

Legend:

- Entry end B = the deeper end of the tunnel, which tunnels its own probes; features = RSSI_mean, RSSI_var, RSSI_Hop_Diff and TunnelIntensity, standardised.
- Wormhole windows = the share of the windows of the cluster that lie in the attack phase; the rest are baseline and cooldown windows.
- Cluster numbers are the labels given by the algorithm and carry no meaning. Seed 42 = the random starting value that fixes the sample of 6,000 windows.
- RSSI = received signal strength indicator; dBm = decibel-milliwatts; dB = decibels. TunnelIntensity = tunnelled messages per second (a diagnostic that the leakage guard keeps out of the model inputs).

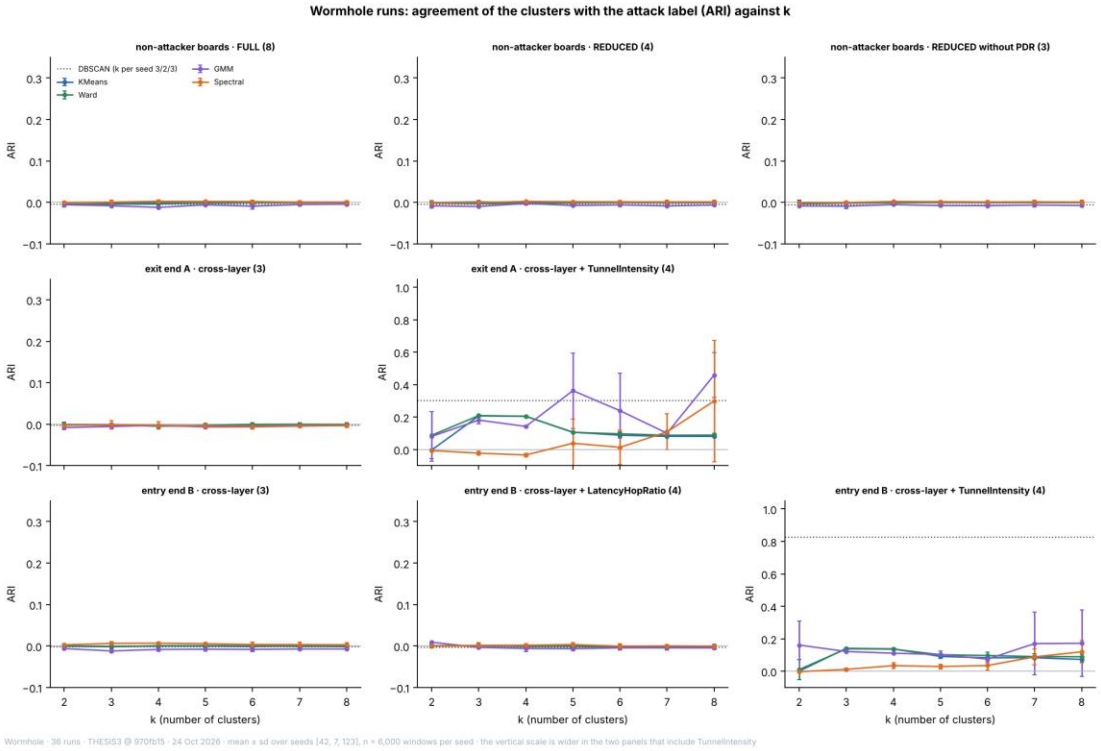


Figure 6.24 Adjusted Rand Index against the Number of Clusters in the Wormhole Runs (Mean and Spread over Three Seeds)

In Figure 6.24, each panel is one population and feature set, with the number of features in brackets. The top row is the non-attacker boards (FULL, REDUCED and without PDR), the middle row the exit end A (cross-layer, and with TunnelIntensity) and the bottom row the entry end B (cross-layer, with LatencyHopRatio, and with TunnelIntensity). The horizontal axis is k and the vertical axis the ARI against the attack label; each line is one algorithm with bars of ±1 standard deviation over the three seeds, and the dotted line is DBSCAN. The vertical scale is wider in the two panels that include TunnelIntensity, so that the DBSCAN value of the entry end B (about 0.83) can be drawn.

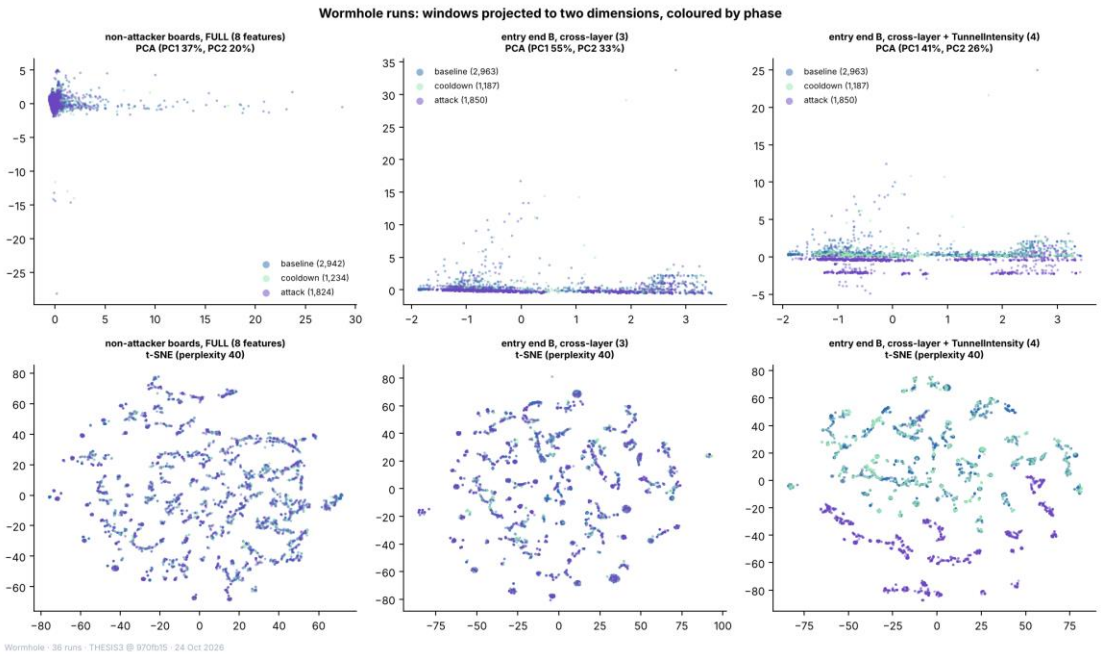


Figure 6.25 PCA and t-SNE Projections of the Wormhole Runs, Coloured by Phase

In Figure 6.25, each point is one window (a sample of 6,000), coloured by phase: blue = baseline, light green = cooldown and purple = attack. The top row is PCA and the bottom row t-SNE, for the non-attacker boards with the FULL set (left), the entry end B with the cross-layer features (centre) and the entry end B with TunnelIntensity added (right). The phases overlap in the first two columns. In the third, the attack windows form separate groups in the t-SNE and a separate band in the PCA, because TunnelIntensity is 0 in the normal windows.

6.4.7. Pooled Runs with Three Labels, with and without the Benign Runs

The last analysis pools the non-attacker boards of all 80 runs (257,258 windows with every feature of the FULL set) and asks whether three kinds of window separate: normal (label 0), blackhole attack (label 1) and wormhole attack (label 2). The same feature sets are used, with PDR kept although the low-variance rule would remove it on the pooled windows (Section 6.4.1). Table 6.30 gives the groups. The analysis is run twice: on the 80 runs, to see whether the benign runs change the result, and on the 72 attack runs. The majority rate of the samples is 0.730 with the benign runs and 0.694 without them. A partition that puts exactly the victim attack windows in one group and everything else in the other is the best that a method reading only the features of a window can reach, because the bystanders of the blackhole and the other boards of the wormhole behave as normal nodes. Its ARI against the three labels is 0.334 with the benign runs and 0.317 without them.

Table 6.30 Groups of Windows in the Pooled Clustering of the 80 Runs

Group of windows	Label	Windows (count)	Share of the windows (%)
normal · benign run	0 (normal)	22,481	8.7
normal · blackhole run	0 (normal)	89,140	34.7
normal · wormhole run	0 (normal)	75,211	29.2
blackhole attack · victim	1 (blackhole)	21,733	8.4
blackhole attack · bystander	1 (blackhole)	16,499	6.4
wormhole attack · other board	2 (wormhole)	32,194	12.5

Legend:

- Group = the kind of run (benign, blackhole, wormhole), the phase (normal = baseline or cooldown; attack) and, for the attack phase, the board: victim = a non-attacker node behind the blackhole attacker; bystander = a non-attacker node of a blackhole run that is not behind it; other board = a board of a wormhole run that is neither end of the tunnel.
- Only the non-attacker boards are pooled; the blackhole attacker and the two wormhole ends are not included. The windows are those that have every feature of the FULL set.
- Label 0 = normal (186,832 windows), 1 = blackhole attack (38,232) and 2 = wormhole attack (32,194).

Table 6.31 gives the quality and the agreement with the three labels for k = 3, the number of labels, and for DBSCAN with its own k. With the delivery ratio, the agreement is close to the best possible value and no more. K-Means with k = 2 on the REDUCED set reaches an ARI of 0.333 on the 80 runs and 0.315 on the 72 attack runs, which equals the best possible value (0.334 and 0.317): it finds the victim attack windows and nothing else. At k = 3 the ARI is 0.272 and 0.173 for K-Means, because the third cluster is a small set of windows with extreme RSSI variance, and 0.320 for Ward on the 80 runs; DBSCAN reaches 0.312 and 0.305. On the FULL set, K-Means with k = 2 again isolates a few extreme windows (ARI −0.004) and recovers the victims at k = 3 (0.325). Without the delivery ratio, no algorithm and no k separates anything: the ARI lies between −0.023 and 0.005 with and without the benign runs. The benign runs do not change the conclusion: the ARI differs by 0.017 between the two populations for K-Means at k = 2.

Table 6.31 Clustering Quality and Agreement with the Three Labels in the Pooled Runs, Means over Seeds 42, 7 and 123 (k = 3)

Feature set	Algorithm	Silhouette, 80 runs (−1 to 1)	Purity, 80 runs (0 to 1)	ARI, 80 runs (−1 to 1)	Purity, 72 attack runs (0 to 1)	ARI, 72 attack runs (−1 to 1)
REDUCED	K-Means	0.511	0.812	0.272	0.787	0.173
REDUCED	Ward	0.595	0.811	0.320	0.785	0.269
REDUCED	GMM	0.405	0.812	0.225	0.778	0.235
REDUCED	DBSCAN (5 to 6 clusters, 1.6% noise)	0.597	0.809	0.312	0.783	0.305

Feature set	Algorithm	Silhouette, 80 runs (−1 to 1)	Purity, 80 runs (0 to 1)	ARI, 80 runs (−1 to 1)	Purity, 72 attack runs (0 to 1)	ARI, 72 attack runs (−1 to 1)
REDUCED	Spectral	−0.176	0.759	0.115	0.756	0.171
FULL	K-Means	0.588	0.811	0.325	0.786	0.307
FULL	Ward	0.598	0.810	0.317	0.784	0.301
FULL	GMM	0.576	0.784	0.203	0.742	0.166
FULL	DBSCAN (2 to 5 clusters, 2.1% noise)	0.536	0.809	0.303	0.784	0.296
FULL	Spectral	−0.036	0.759	0.062	0.785	0.271
Without PDR	K-Means	0.340	0.730	−0.001	0.694	0.004
Without PDR	Ward	0.424	0.730	−0.003	0.694	0.002
Without PDR	GMM	0.312	0.730	−0.019	0.694	−0.007
Without PDR	DBSCAN (4 to 5 clusters, 1.4% noise)	0.663	0.730	−0.007	0.695	0.003
Without PDR	Spectral	−0.285	0.730	−0.002	0.694	0.001

Legend:

- 80 runs = the 36 blackhole, 36 wormhole and 8 benign runs; 72 attack runs = the blackhole and wormhole runs without the benign runs. The windows are those of the non-attacker boards (Table 6.30).
- Feature sets are those of Table 6.24. k = 3 for K-Means, Ward, GMM and Spectral; DBSCAN chooses its own number of clusters, given in brackets (range over the three seeds) with the share of windows left as noise for the 80 runs.
- The label has three values: normal, blackhole attack and wormhole attack. Majority rate: 0.730 (80 runs) and 0.694 (72 attack runs); best possible ARI: 0.334 and 0.317.
- Silhouette = how well each window fits its own cluster compared with the nearest other cluster (higher is better); purity = the share of windows that carry the majority label of their cluster; ARI = adjusted Rand index (1 = exact match, 0 = no better than chance).
- K-Means = groups windows around k centres; Ward = hierarchical clustering; GMM = Gaussian mixture model; DBSCAN = groups windows in dense regions and leaves isolated windows as noise; Spectral = clustering on the eigenvectors of a similarity graph.

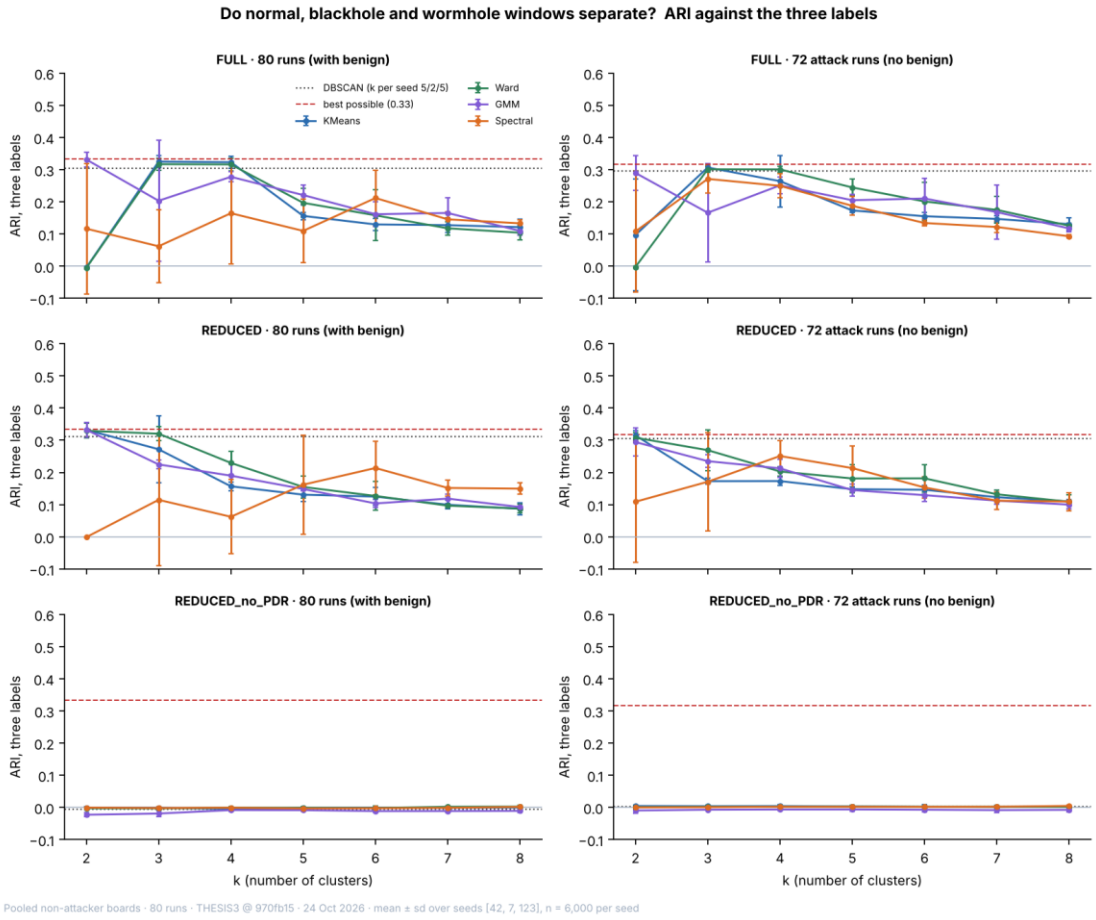


Figure 6.26 Adjusted Rand Index against the Three Labels, by Feature Set and Population (Mean and Spread over Three Seeds)

In Figure 6.26, the left column is the pooled 80 runs and the right column the 72 attack runs; the rows are the feature sets. The vertical axis is the ARI against the three labels, the dashed red line is the best possible ARI of a method that finds exactly the victim attack windows, and the dotted line is DBSCAN. Each coloured line is one algorithm with bars of ±1 standard deviation over the three seeds. The bottom row is on the same scale as the rows above.

Figures 6.27 and 6.28 show what the K-Means clusters of the 80 runs contain. With the delivery ratio, one cluster (537 windows of the sample of 6,000) consists of victim attack windows and nothing else, and the other holds the benign, blackhole-normal, wormhole-normal, bystander and wormhole windows in nearly the shares in which they occur in the sample. The wormhole windows are 12.5% of the sample, and in no K-Means cluster of 100 windows or more (k = 2 to 4, all three feature sets) do they exceed 16.3%. The cluster is separated by PDR alone (a mean of 0.000 against 1.000, Figure 6.29); the other three features are about the same in the two clusters. Without PDR, the clusters split the windows by signal level (a mean RSSI_mean of −77.2 and −63.9 dBm in the two clusters of Figure 6.27) and each holds all six kinds of window.

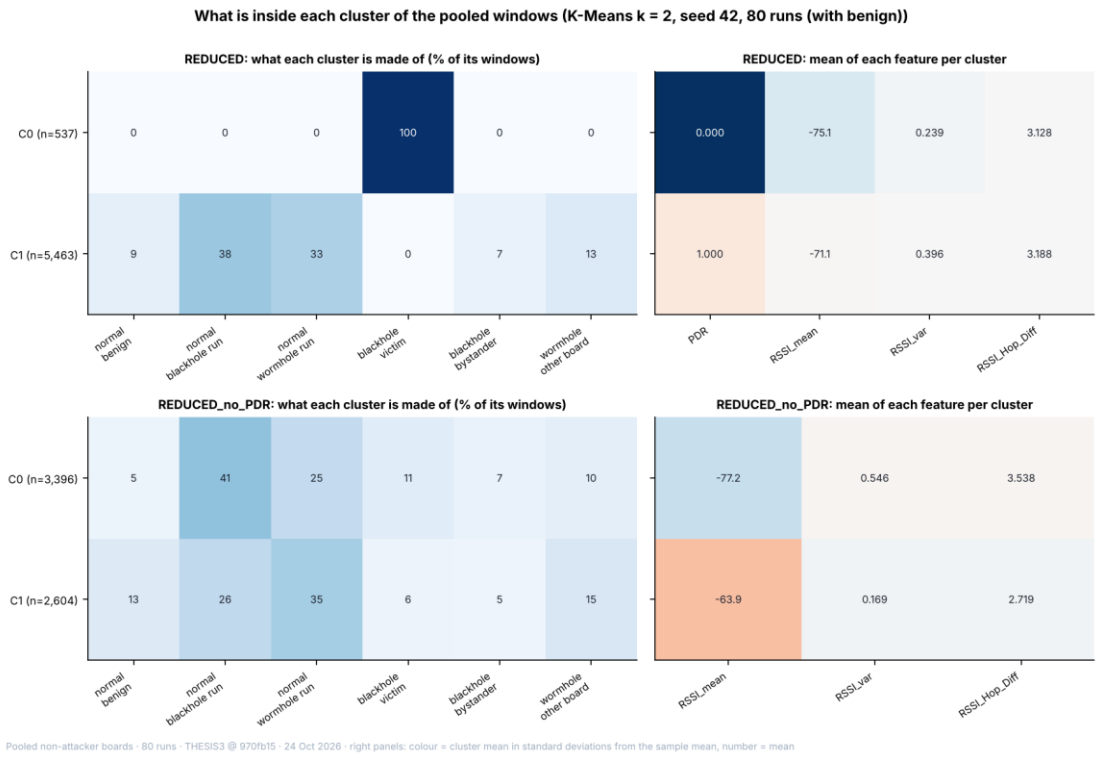


Figure 6.27 Composition and Feature Means of the K-Means Clusters at k = 2 in the Pooled 80 Runs (Seed 42)

In Figure 6.27, each row is a cluster, with its number of windows in brackets. The left panels give the share (%) of each kind of window in the cluster, and the right panels give the mean of each feature in the cluster; the colour of a right-panel cell is the cluster mean in standard deviations from the mean of the sample, and the number is the mean. The top row is the REDUCED set and the bottom row the REDUCED set without PDR. Only the victim windows of the blackhole form a cluster of their own (top), and it is defined by a PDR of 0. Without PDR (bottom), both clusters hold every kind of window.

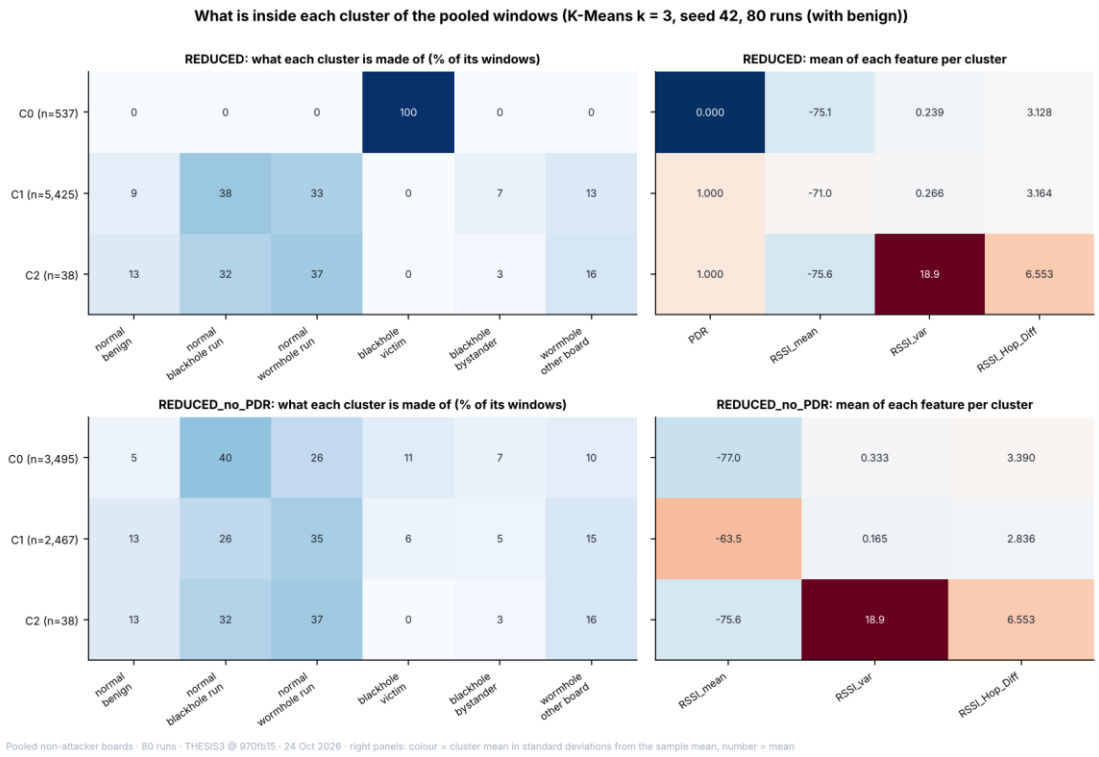


Figure 6.28 Composition and Feature Means of the K-Means Clusters at k = 3 in the Pooled 80 Runs (Seed 42)

Figure 6.28 has the same layout as Figure 6.27. At k = 3 the victim windows are still one cluster, and the third cluster is a set of 38 windows with a mean RSSI_var of 18.9 and a mean RSSI_Hop_Diff of 6.55 dB, with the kinds of window in their usual shares. The wormhole windows lie in the two larger clusters in the shares of the whole sample.

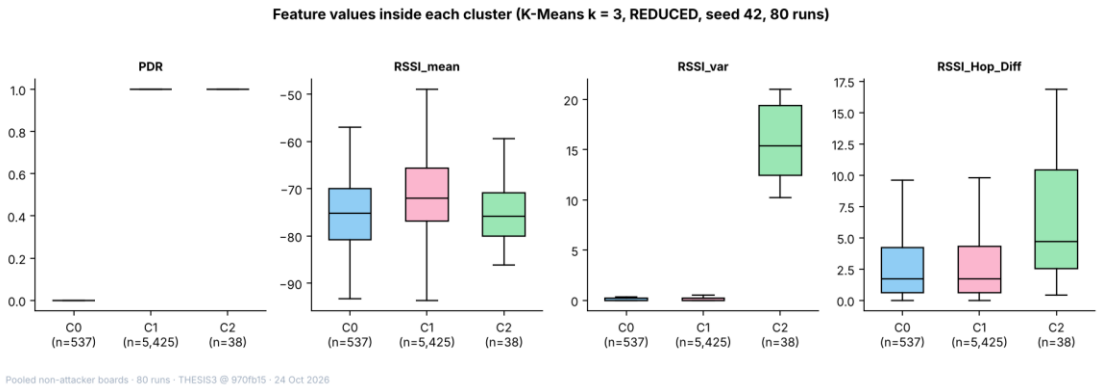


Figure 6.29 Feature Values inside Each K-Means Cluster at k = 3 (REDUCED Set, Pooled 80 Runs, Seed 42)

In Figure 6.29, each panel is one feature and each box is one cluster (C0, C1, C2, with the number of windows). The box spans the middle half of the windows with the median as a line, and the whiskers reach 1.5 times that range. PDR separates C0 (0) from the other two (1); RSSI_mean and RSSI_Hop_Diff overlap in all three; RSSI_var is far higher in C2, the small cluster of extreme windows.

The clusters can also be used as a detector. A K-Means model (k = 2, REDUCED set, seed 42) was fitted on a sample of 6,000 windows of the 80 runs and applied to every window. The cluster with a mean PDR of 0 contains 21,733 of the 21,733 victim attack windows (100.0%) and none of the 22,481 windows of the benign runs, so the benign runs raise no alarm in the sense of this model. It contains 183 of the 89,140 normal windows of the blackhole runs (0.21%) and 3 of the 32,194 wormhole attack windows (0.01%). The wormhole windows are therefore not separated, in line with the verdict of Section 6.2.2, that the wormhole leaves the other boards untouched. The result is the same when the benign runs are left out (Table 6.32). The same figures are shown in Figures 6.30 and 6.31 as projections.

Table 6.32 Windows of Each Group that Fall in the Cluster of the Victims (K-Means, k = 2, REDUCED Set, Fitted on a Sample of 6,000 Windows)

Group of windows	Windows (count)	In the victim cluster, 80 runs (count)	In the victim cluster, 80 runs (%)	In the victim cluster, 72 attack runs (count)	In the victim cluster, 72 attack runs (%)
normal · benign run	22,481	0	0.00	–	–
normal · blackhole run	89,140	183	0.21	183	0.21
normal · wormhole run	75,211	5	0.01	5	0.01
blackhole attack · victim	21,733	21,733	100.00	21,733	100.00
blackhole attack · bystander	16,499	0	0.00	0	0.00
wormhole attack · other board	32,194	3	0.01	3	0.01

Legend:

- The model is fitted on a sample of 6,000 windows (seed 42) and applied to every window of the group; the victim cluster is the cluster with the lower mean PDR (0.000 in both fits). The groups are those of Table 6.30.
- In the 72-run fit the benign runs are not part of the fit; the benign windows are not scored there (–).
- K-Means = groups windows around k centres; REDUCED = PDR, RSSI_mean, RSSI_var and RSSI_Hop_Diff (Table 6.24).

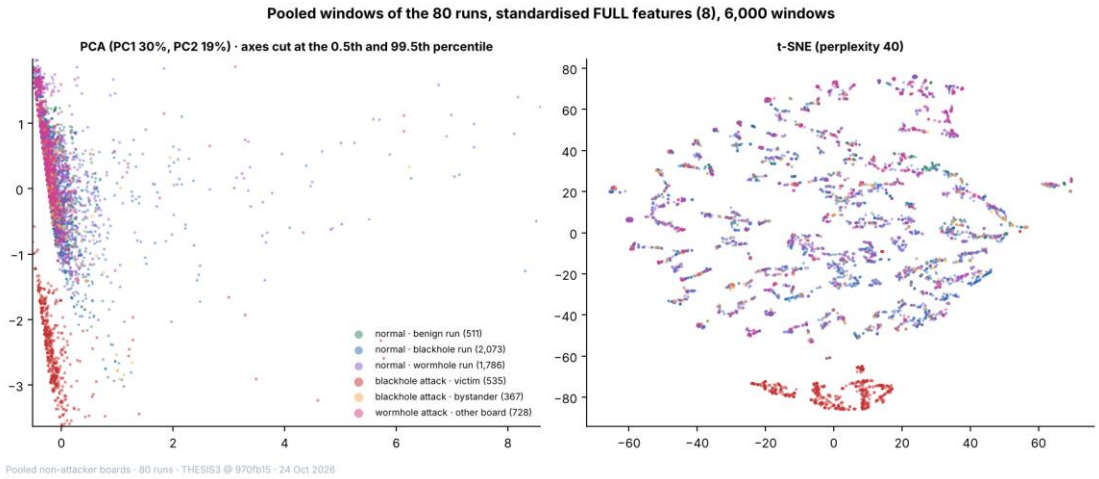


Figure 6.30 PCA and t-SNE Projections of the Pooled Windows of the 80 Runs, FULL Set

In Figure 6.30, each point is one window of the 80 runs (a sample of 6,000), coloured by group: green = normal windows of the benign runs, blue = normal windows of the blackhole runs, purple = normal windows of the wormhole runs, red = victim attack windows, orange = bystander attack windows and pink = wormhole attack windows of the other boards. The PCA axes are cut at the 0.5th and 99.5th percentile, and the t-SNE axes have no unit. The red windows lie apart from the others in both projections, and the other five groups overlap.

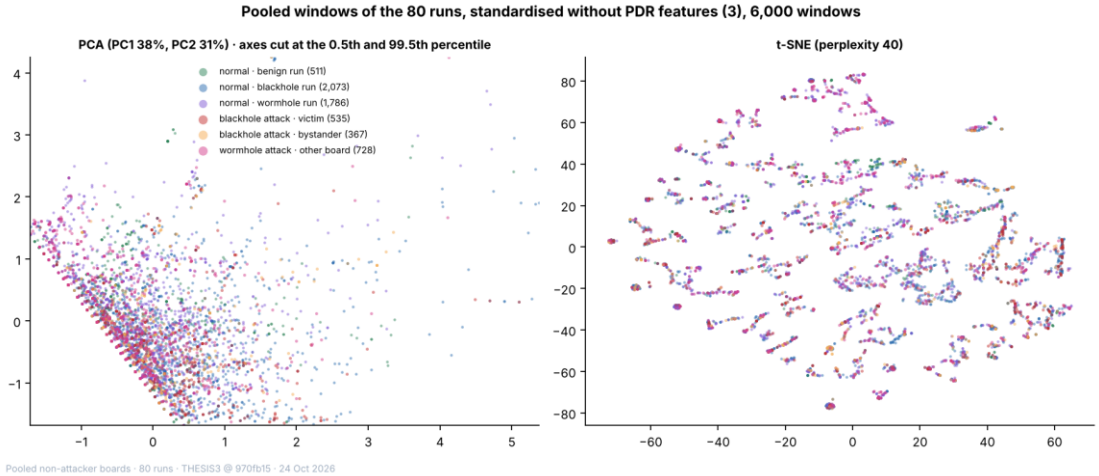


Figure 6.31 PCA and t-SNE Projections of the Pooled Windows of the 80 Runs, without PDR

Figure 6.31 is built in the same way as Figure 6.30, with RSSI_mean, RSSI_var and RSSI_Hop_Diff only. The six groups overlap in both projections, and the victim windows are no longer apart.

The benign runs serve in this section as the control. They contribute 22,481 windows, all normal, and they are spread over the clusters of the other normal windows (9% of the largest cluster at k = 2 in Figure 6.27). None of them falls in the victim cluster (Table 6.32), so the benign runs, which use the

same testbed and the same traffic scenarios as the attack runs and have no attack, show that the delivery collapse is what the cluster of the victims picks up, and not a feature of the traffic.

6.5. Discussion

This section reads the results against the research objectives of Section 1.3.2 (Section 6.5.1) and against the published studies (Section 6.5.2). It then states what an emulated attack can and cannot show (Section 6.5.3) and the limits of the data (Section 6.5.4). Every number in it comes from the tables and figures of Sections 6.1 to 6.4.

6.5.1. In Relation to the Research Objectives

The first objective (Section 1.3.2) was to establish an ESP32 mesh testbed that generates multi-hop traffic under normal operation and controlled attack. All 80 runs of the plan were captured on the testbed, which has 8 boards in a run (the root and 7 child nodes), at three sites, in four topologies and in three traffic scenarios (Table 6.1). In every blackhole run the attacker did what the design says: it forwarded between 0.0% and 0.7% of the probes it received during the attack, against 100% before (Table 6.9), and the verifier confirmed all 36 blackhole runs (4 strong, 27 good and 5 with minor gaps). In every wormhole run the root received the probes of the entry end twice during the attack (12,448 of 12,465 probes over the 36 runs, 99.86%) and never in the baseline, the three checks of the verifier passed in all 36 runs, and 21 are graded strong and 15 good (Table 6.11). The 8 benign runs show the testbed without an attack: none of the 21 signature checks that could be evaluated passed, the mean PDR of each run lies between 0.9994 and 1.0000, and the cooldown lies within 0.5 baseline standard deviations of the baseline in all 24 comparisons (Table 6.17). The objective is met for the behaviour of the two attacks. Two qualifications apply: the attacks are emulated at the application layer (Section 6.5.3), and the nine partial-mesh runs of each attack behave as plain trees (Section 6.5.4).

The second objective was to collect and structure cross-layer features into a labelled and documented dataset. The dataset holds 456,195 one-second windows from the 80 runs (Table 6.2), and 369,678 of them carry a normal or an attack label (baseline and cooldown are normal; the formation phase and the windows that span two phases are not labelled). Each window has the 16 features of Chapter 4, the label and the identity of the run (Table 6.7), and every run is documented by a verifier report. Of the 16 features, 8 can enter the clustering of the non-attacker nodes. The other 8 are recorded and documented, and they are kept out of the model inputs for reasons set out in Section 6.2.3 and Table 6.7: the relay features and the tunnel features record the attack directly, the RetryRate stays near zero

and counts the tunnelled frames on the entry end, and a missing LatencyHopRatio reveals the label of a blackhole victim. Only one of the 8 features that can be clustered, the PDR, responds to the blackhole (d = −29.7 on the victims, while |d| is below 0.1 for every other feature, Table 6.18), and none of them responds to the wormhole (|d| of at most 0.07 on the two ends and on the non-attacker nodes, Table 6.19). What the wormhole changes is recorded by features that exist only on the tunnel ends or at the root: TunnelIntensity, TunnelBytes, TunnelLatency and the duplicate arrivals.

The third objective was to evaluate the usability and quality of the dataset through exploratory analysis and unsupervised techniques, in order to assess whether normal and attack-induced behaviours are separable. The quality is shown by the verification of every run (Section 6.2) and by the exploratory analysis (Section 6.3), and the separability differs for each attack. For the blackhole, the windows in which the attack is felt separate from the normal windows when PDR is a feature: K-Means with k = 2 reaches an ARI of 0.514 on all non-attacker nodes against a best possible value of 0.515, and 0.996 on the victims alone, and without PDR the ARI lies between −0.016 and 0.002. The best possible value is below 1 because 43.2% of the attack windows are windows of bystanders, which the label marks as attack and which behave as normal nodes. The separation is carried by one feature. Because PDR is also what the verifier checks to confirm the attack (Table 6.9), the agreement shows that the clustering finds the effect that the verifier measured. It is not independent evidence that the cross-layer features detect the blackhole.

For the wormhole, the cross-layer features do not separate the attack windows at all. On the non-attacker nodes the ARI lies between −0.012 and 0.003, and on the two tunnel ends, with the signal features alone, between −0.012 and 0.009 (Table 6.28). A tunnel feature changes this only in part: with TunnelIntensity added, DBSCAN reaches an ARI of 0.827 on the entry end, whereas K-Means with k = 2 stays at 0.000. TunnelIntensity is a count kept by the attacking boards, so it is excluded from the model inputs, and the result shows that a tunnel signal exists in the data and that it is not carried by the cross-layer features. In the pooled analysis of the 80 runs with three labels (Section 6.4.7), K-Means with k = 2 reaches an ARI of 0.333 on the 80 runs and 0.315 on the 72 attack runs, which is the best possible value for a method that finds the blackhole victims and nothing else (0.334 and 0.317), and without PDR the ARI lies between −0.023 and 0.005. Applied to every window, the model puts 21,733 of the 21,733 victim attack windows in the victim cluster, 3 of the 32,194 wormhole attack windows of the other boards, and none of the 22,481 windows of the benign runs, so the benign runs raise no alarm in the sense of this model (Table 6.32).

Table 6.33 What Each Kind of Run Leaves in the Data

Kind of run	Direct record	Cross-layer features that change	Separation by clustering
Blackhole	Forwarding ratio of the attacker 0.000 to 0.007 in the attack, against 1.000 to 1.004 before; PDR of the victims 0.998 → 0.000.	PDR of the victims only (d = −29.7); every other feature has |d| below 0.1.	ARI 0.514 on all non-attacker nodes (best possible 0.515) and 0.996 on the victims; between −0.016 and 0.002 without PDR.
Wormhole	Probes of the entry end reach the root twice (99.86% in the attack, none in the baseline); TunnelIntensity 0 → 1.92 messages per second.	None: |d| of at most 0.07; hop and parent unchanged in all 36 runs; PDR of the other boards 0.9999 → 0.9999.	ARI between −0.012 and 0.009 without a tunnel feature; DBSCAN 0.827 on the entry end with TunnelIntensity.
No attack (benign)	No signature passed in 21 of 21 checks evaluated; PDR 0.9994 to 1.0000.	Cooldown within 0.5 baseline standard deviations of the baseline in 24 of 24 comparisons.	0 of 22,481 benign windows fall in the victim cluster.

Legend:

- Direct record = the measurement that confirms the attack in the verifier (Sections 6.2.1, 6.2.2 and 6.2.6).
- Cross-layer features that change = the features of the cross-layer set (Table 6.7) whose mean shifts between the baseline and the attack; d = Cohen's d (Section 6.3).
- Separation by clustering = the ARI of K-Means with k = 2 on the REDUCED set (Section 6.4); ARI = adjusted Rand index (1 = exact match, 0 = no better than chance); best possible = the ARI of a partition that puts exactly the victim attack windows in one group.
- DBSCAN = a clustering method that groups windows in dense regions and leaves isolated windows as noise; TunnelIntensity = tunnelled messages per second, which the leakage guard keeps out of the model inputs.
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1).

Table 6.33 puts the three kinds of run side by side. Chapter 4 expected that the clusters would reflect a confluence of cross-layer evidence. For the two attacks as they were emulated, this is not what the data show. The evidence of each attack sits in the data plane, in the delivery of the probes for the blackhole and in their duplication for the wormhole, and the physical, MAC and network-layer features stay where they were. This is a result about the emulated attacks and about the feature set, and not a statement that cross-layer detection fails against attacks that act on the routing (Section 6.5.3).

6.5.2. In Relation to the Published Studies

Four published studies serve as the reference. Khan et al. [22] measured the delivery of a healthy ESP-MESH network, the platform of this study, and Airehrour et al. [70] measured a blackhole attack in an RPL network. Zhukabayeva et al. [71] and Ramírez Gómez et al. [72] implemented a wormhole in ZigBee networks. Table 6.34 sets each measured value beside the published one.

Table 6.34 Measured Values Compared with Published Studies

Feature	This study	Published value	Study, protocol and device	Reason for the difference
Blackhole: PDR in the baseline (ratio)	0.998 (victims), 0.996 (bystanders); 0.9994 to 1.0000 in the benign runs	More than 0.97 for every node of its campus test except node H, the farthest, outdoor node; more than 0.98 for every node of its open-area test [22]	Khan et al. [22]: ESP-MESH on ESP32 boards, 7 sensor nodes and a root.	Same protocol and device family; the counting differs, so the two are not claimed equal.
Blackhole: PDR in the attack (ratio)	0.000 in the victims of all 36 runs	Packet loss of 60% to 75%, which is a delivery of 25% to 40% [70]	Airehrour et al. [70]: RPL (MRHOF), 14 AS-XM1000 (Telos-B) motes in a laboratory.	Loss there depends on which motes choose the attacker as parent; here the attacker is on the path of its victims.
Blackhole: ParentSwitch Rate (parent changes per s)	0.00000 → 0.00009 (victims, d = 0.01)	16 to 246 rank changes per mote [70]	Airehrour et al. [70]: as above.	Different metric and mechanism; routing is untouched here (D-20).
Blackhole: RetryRate (ratio)	0.0000 → 0.0004 (victims)	No published value for this attack in the studies compared here	Not applicable.	ESP-IDF gives no MAC retry count (D-15).
Wormhole: hop count	Constant in the attack in 36 of 36 runs; no window with a hop change	Route of 4 hops becomes 3 hops [71]	Zhukabayeva et al. [71]: ZigBee (XBee) chain, wormhole tunnel between two Raspberry Pi nodes.	In [71] the tunnel is a shorter route that ZigBee chooses; here it is an application-layer relay and the mesh keeps its routes (D-20).
Wormhole: delay	LatencyHopRatio of the entry end 12.0 → 11.2 ms per hop (d = −0.01); spread between the two copies of a probe: median 3.9 ms, 90th percentile 12.1 ms	Delay of 80 to 100 ms rises to 120 to 200 ms, about 40% more [71]	Zhukabayeva et al. [71]: as above.	Different measure: [71] gives the delay of the route, here the delay per hop of the entry end and the spread between two copies.
Wormhole: delivery of the other boards (ratio)	0.9999 → 0.9999 (non-attacker boards)	Packet loss increased after the attack, without a value in the text [71]	Zhukabayeva et al. [71]: as above.	Only the probes of the entry end are tunnelled here, and the emulation drops none on purpose.
Wormhole: duplicate arrivals	Probes of the entry end at the root twice: 99.86% in the attack, none in the baseline	Two source-routing packets per data packet [72]	Ramírez Gómez et al. [72]: ZigBee (XBee S2C) with source routing, one coordinator and two routers.	Same kind of evidence at a different layer: a duplicated routing packet there, a duplicated data probe at the root here.

Feature	This study	Published value	Study, protocol and device	Reason for the difference
RSSI_mean, RSSI_var, RSSI_Hop_Diff and the other features of Tables 6.18 and 6.19	|d| below 0.1 in both attacks	No comparable value in [70], [22], [71] or [72]	Not applicable.	No published value to compare with.

Legend:

- This study: 8 ESP32 boards running ESP-WIFI-MESH, with both attacks emulated at the application layer (D-20), measured in one-second windows.
- Published value = the value reported by the cited study. For [71], the delay and hop count are those of its description of the testbed; its abstract gives the rise in delay as 40% and the share of false readings as over 30%.
- d = Cohen's d (Section 6.3). Victims and bystanders are defined at the start of the chapter.
- MRHOF = minimum rank with hysteresis objective function of RPL; RPL = routing protocol for low-power and lossy networks.
- Mote = a small wireless sensor node; rank = the value that an RPL mote computes for its distance from the sink, so a rank change means that the mote moved in the routing tree; sink = the node that collects the data.
- Packet loss = the share of packets that do not arrive; loss = 1 − delivery.
- ZigBee = a low-power wireless standard for sensor networks; XBee = a ZigBee radio module; source routing = the sender writes the route into the packet.
- PDR = packet delivery ratio, the share of a node's probes that reached the root (ratio, 0 to 1); LatencyHopRatio = the one-way delay of a probe divided by the number of hops (ms per hop).

For the baseline PDR, Khan et al. [22] used ESP-MESH on ESP32 boards, with 7 sensor nodes and a root, and one 64-byte package per node every minute (open-area test) or every 2 min (campus test) for 5 days. In the campus test every node delivered more than 97% of its packages except node H, which was placed 100 m from the root across more than four buildings and walls and which the authors suspect was affected by its outdoor location. The protocol and the device family are the same, and the values agree in size. The published figure counts one package every 1 to 2 min over five days, whereas this study counts probes in one-second windows over a 300 s baseline, so the two are not claimed to be equal. The baseline PDR of the blackhole runs (0.998 for the victims and 0.996 for the bystanders) is a little lower than that of the wormhole and benign runs (0.9999 and at least 0.9994), and all of them are consistent with the benchmark of more than 0.97.

For the PDR in the attack and for the parent changes, the source is Airehrour et al. [70], who used RPL with the MRHOF objective function in a research laboratory of about 30 m by 30 m, with a testbed described as 14 AS-XM1000 (Telos-B) motes running InstantContiki 3.0 (1 UDP sink, 12 UDP senders and 2 blackhole motes are listed). In [70] the loss depends on which motes select a blackhole mote as parent through the routing protocol (their Figures 5 and 6), and loss is counted between a sender and the sink. Here the attacker is placed on the path (D-20, the attack is emulated at the application layer) and every probe of a victim passes it, so all are lost. The direction agrees with [70], and the size does not, because the setups differ, so the comparison covers direction and not equal

values. The metric for parent changes also differs (rank changes of RPL motes against parent changes of mesh boards per second), and so does the mechanism. In [70] the attack acts inside the routing protocol and ranks change, whereas here forwarding is suppressed without touching routing (D-20), so the tree stays as it was.

For the wormhole, Zhukabayeva et al. [71] built a ZigBee chain of Arduino-based sensor nodes with XBee modules, and made the wormhole with two Raspberry Pi nodes joined by a GSM link and a TCP/IP tunnel that forwards the ZigBee packets, so that the two nodes appear as direct neighbours. ZigBee prefers the shortest route, so the route of 4 hops becomes a route of 3 hops through the tunnel, and the paper reports a rise in the delay from 80 to 100 ms to 120 to 200 ms and 20% to 30% of the readings modified by the attacker. The attack there changes the routing, and its effects (shorter route, longer delay, altered data) follow from that change. In this study the tunnel does not change the routing: the hop and the parent of every board stay the same in all 36 runs, the LatencyHopRatio of the entry end does not move, and the other boards keep a PDR of 0.9999. The difference is explained by the attack design, as in the blackhole. The published figure of modified readings has no counterpart here, because this study measures the delivery and the timing of probes and not the integrity of sensor readings. The paper marks a measurement as anomalous when it lies more than three standard deviations from the mean (the 3-sigma rule), and the verifier of this study applies the same rule to the signature features of each run (Section 6.2).

Ramírez Gómez et al. [72] implemented a classic and a reverse wormhole against XBee S2C nodes that use source routing, with a Raspberry Pi 3 and a USB radio dongle running the KillerBee framework as the malicious node, in two small networks of one coordinator and two routers. Their results are qualitative: they show the captured and modified route records, and they give no delivery, loss or delay figure, so there is no number to set against. They propose detection signatures instead, and the first is the duplication of routing packets: the attack makes the nodes receive two source-routing packets per data packet. The duplicate arrival is also the main signature in this study (99.86% of the probes of the entry end reach the root twice), and the two agree in kind, but they are not the same measure, since [72] counts routing packets inside a ZigBee network and this study counts data probes at the root of an ESP-WIFI-MESH network. Their other signatures (beacon requests without a join, and link-status and neighbour tables) concern the routing and the neighbour tables, which the firmware of this study does not log (Section 6.2.4), so they cannot be tested on this dataset.

For RetryRate, the studies compared here give no published value for this attack. ESP-IDF returns no MAC retry count, so RetryRate is an application counter (deviation D-15) and cannot be compared

with MAC-layer retransmission counts. Finally, RSSI_mean, RSSI_var, RSSI_Hop_Diff and the other features of Tables 6.18 and 6.19 belong to the cross-layer feature set of this study (Section 4.2.4), so that there is no comparable published value in [70], [22], [71] or [72] to set them against.

6.5.3. What an Emulation Can and Cannot Show

Both attacks act on the application-layer probes of the study and not on the mesh. The blackhole receives the probes of the nodes behind it and does not forward them, but every frame is still acknowledged at the link layer, so signal strength, retries and parent choices are not touched (D-20). The dataset therefore records the effect of forwarding suppression on delivery. It does not record how a node attracts traffic to itself, which is the route-attraction stage of a classical blackhole (Section 6.6). This is why the delivery ratio carries the whole result, and it is a property of the emulation as much as of the attack.

The wormhole is an application-layer relay between two boards joined by a wire. The entry end B sends its own probes in the normal way and also writes them to the wire, and the exit end A, which lies closer to the root, sends them on into the mesh, so that the root receives two copies of each probe of B. Only the probes of B are tunnelled, the five other boards of a run are not touched, and no node is a victim. The mesh does not see the tunnel, so its routing, its parent choices and its radio links are left as they were. Of the indicators of Table 3.5, the three that belong to the traffic that the tunnel creates are observed (the duplicate arrivals, the frame arrival pattern and the latency mismatch between the two copies), and the four that belong to the routing and the radio are not (hop count, parent-child validity, the relation between signal strength and hop, and signal variance); the routing table is not logged, and the retransmissions of the two ends cannot be separated from the tunnelled frames (Table 6.16). Section 3.3.2.2 expected that application-layer relaying would reproduce latency anomalies and RSSI–hop inconsistencies. The first is observed. The second is not, which is what an emulation that leaves the radio and the routing alone implies.

Three things follow for the use of the dataset. First, a result on this dataset says what an attack of this kind does to the traffic and to the features, and it does not say what an attack inside the routing protocol would do. Second, a detector that is tested on the blackhole windows of this dataset is tested on the loss of delivery, and one that is tested on the wormhole windows is tested on duplication, and both signals are available at the root without cross-layer features. Third, the question whether cross-layer features add anything to such a detector is open, and a dataset of attacks that act on the routing is needed to answer it (Section 6.6).

6.5.4. Limits

Each combination of topology, site and scenario was run once for each attack, so no cell has a repeat, and differences between sites cannot be separated from run-to-run noise. The signal level shows how large this noise can be: the mean RSSI_mean of the baseline differs by 8.1 dB (blackhole) and 7.1 dB (wormhole), on average, between a benign run and the attack run of the same cell (Table 6.23), so the signal level is a property of the capture and not of the attack. The seeds of Section 6.4 change the sample and the starting points of the algorithms and not the data.

The windows are also not independent. The 456,195 windows come from 80 runs, and the windows of one board in one phase are not independent draws, so the number of independent observations is closer to the number of runs than to the number of windows. This is why the run-level interval of Section 6.4.4 is wider than the seed spread of Section 6.4.2: the ARI of K-Means with k = 2 on the blackhole runs is 0.511 with a 95% interval of 0.410 to 0.626 over runs, against a standard deviation of 0.014 over seeds. The effect sizes of Tables 6.18 and 6.19 treat windows as independent and are therefore read as sizes of a shift and not as significance tests.

The partial-mesh runs behave as plain trees. In all 9 partial-mesh blackhole runs every node was seen under a single parent during the labelled phases, and parent changes occurred only during mesh formation (Section 1.4.1); the 9 partial-mesh wormhole runs are the 9 runs whose topology check carries a warning (Section 6.2.2). The partial-mesh cells therefore test a branched tree with a limit of two children and not a mesh with alternative parents, and adaptive parent switching is not claimed.

The shape of the mesh also differs between runs of one topology (Table 6.35). The linear runs are chains with one child for each board and a deepest hop of 7. In the 18 attack runs of the star, the hub sits one hop below the root with 6 boards behind it (D-16, D-28), whereas the 3 benign star runs are flat: 5 to 7 boards sit directly under the root and none is deeper than hop 1. The 19 partial-mesh runs are alike, with 2 boards under the root and at most 2 children for any board. The tree runs vary. The root has 2 to 5 children in the tree blackhole runs, 5 in the benign tree run and 2 to 5 in the tree wormhole runs. In the wormhole runs the tunnel ends are leaves in all 9 tree runs and in 7 of the 9 partial-mesh runs. In the other two, partial mesh at Yuchengco with stationary and with highload, the entry end has 1 and 2 child boards. For the wormhole, the contrast of the tree with the partial mesh therefore rests on the tree runs in which the root has more than two children, because in the others the root has two children, as in the partial mesh.

Table 6.35 Branching of the Mesh by Topology and Kind of Run

Topology	Kind of run	Runs (count)	Children of the root: min to max	Most children of one board (max)	Deepest hop: min to max
Linear	Blackhole	9	1	1	7
	Wormhole	9	1	1	7
	Benign	3	1	1	7
Star	Blackhole	9	1	6	2
	Wormhole	9	1	6	2
	Benign	3	5 to 7	7	1
Tree	Blackhole	9	2 to 5	5	2 to 3
	Wormhole	9	2 to 5	5	3 to 4
	Benign	1	5	5	2
Partial mesh	Blackhole	9	2	2	3 to 4
	Wormhole	9	2	2	3 to 4
	Benign	1	2	2	3

Legend:

- Children of the root = the boards whose most common parent in the labelled phases is the root; for an attack run the attack phase is used and for a benign run all labelled windows. The parent is read from the parent address of each window.
- Most children of one board = the largest number of boards that have one board as their parent, taken over the runs of the group.
- Deepest hop = the largest hop of any board in the run, where a direct child of the root has hop 1.
- In the star attack runs the hub is one hop below the root (D-16, D-28), so that the root has one child; in the benign star runs the star is flat and the root has all the boards as its children.

The label imposes a further limit. It marks every window of every node in the attack phase as attack, including the windows of bystanders, which caps the ARI of the blackhole at 0.515 on all non-attacker nodes (43.2% of the attack windows are bystander windows). For the same reason, a pooled analysis of the three labels cannot exceed 0.334 on the 80 runs and 0.317 on the 72 attack runs for a method that reads only the features of a window.

The benign runs are a control, but a small one. There are 8 of them (54 child boards), 4 stationary, 3 highload and 1 burst, with 3 in the linear topology, 3 in the star, and one each in the partial mesh and the tree, so most cells of the design have no benign counterpart. They show that the verifier raises no alarm on 8 runs and that the model of Section 6.4.7 puts none of their 22,481 windows in the victim cluster. They do not give a false-alarm rate for a detector, which would need more runs, in every cell and with repeats. The burst and highload scenarios add controlled variation in traffic, so that traffic volume alone cannot indicate an attack, but they are not a model of campus traffic.

Some measurements are limited by the boards. The routing table is not logged, the boards cannot count MAC retransmissions (D-15), and the RetryRate of the entry end holds the frames sent through

the tunnel, so it records the attack by construction and is not read as a retransmission count. The 802.11 retry rate seen by a sniffer exists for 7 of the 36 blackhole runs only (Section 6.2.4). In the run partial mesh at Yuchengco with highload, the entry end and one non-attacker node sent about one probe per second where the scenario asks for four (Section 6.2.5). The feature sets of the clustering were fixed on the blackhole windows and kept for the wormhole and pooled windows, because a low-variance rule applied to the pooled windows would remove PDR, the one feature that responds to the blackhole (Section 6.4.1); a different rule would give different sets.

Finally, the results hold for a network of 8 ESP32 boards at three sites. They are not claimed for larger meshes, for other devices or for traffic other than the three scenarios. In this chapter, normal means baseline and cooldown.

7. Conclusion

This chapter summarizes the results of the study, explains what the findings mean, and discusses whether the research objectives were achieved. It also presents the main contributions and limitations of the study, along with recommendations for future research.

7.1. Summary of the Study

This study developed and tested a small wireless mesh network using eight ESP32 boards: one root board and seven child boards. The network was tested under normal conditions and during two attacks: blackhole and wormhole.

The blackhole attack was designed to make a board stop forwarding probe packets while still acknowledging them at the link layer. The wormhole attack used a wired tunnel to move packets from a deeper part of the network to another point closer to the root. This caused packets to arrive more than once. Both attacks were carried out at the application layer because the routing behavior of ESP-WIFI-MESH could not be directly controlled.

Testing covered four network topologies, three locations, and three traffic scenarios. The locations were Room G402 at GCCS, the DLSU Library, and the Yuchengco Building. The traffic scenarios were stationary, burst, and high-load conditions.

The study collected 80 runs: 36 blackhole runs, 36 wormhole runs, and eight normal runs. Each run included 300 seconds of normal operation, 180 seconds of attack activity when applicable, and 120 seconds of cooldown. During testing, the boards recorded network activity and telemetry data for analysis.

The collected data was divided into one-second windows. Each window required at least eight valid samples out of ten. Sixteen features were used to describe the network's physical, link, network, and cross-layer behavior, including measurements related to packet delivery and duplicate packet arrivals.

Five unsupervised clustering algorithms were tested. The results were then compared with the recorded attack labels to determine how well the algorithms separated normal and attack activity.

7.2. The Dataset

The final dataset contained 80 experimental runs and 456,195 generated windows. Of these, 369,678 windows were retained for analysis: 266,491 normal windows and 103,187 attack windows.

Each window was linked to information such as the attack type, topology, location, traffic scenario, board role, experimental repeat, and attack phase. This allowed the results to be examined under different testing conditions.

The dataset also included measurements from boards affected by the attacks and boards that were not directly affected. For the blackhole attack, 123 boards were classified as victims and 92 as bystanders. For the wormhole attack, the tunnel endpoints were examined separately from the other boards.

The collected data was checked before analysis. A total of 457,568 windows were formed during processing, but 1,372 windows, or 0.30%, were removed because they did not meet the data-quality requirements. No run lost more than 1% of its windows. Each retained window contained at least eight samples, and the average was at least 9.97 samples per window.

The collection process also produced 637 telemetry files, and none was empty. One board did not produce data during one run, but it was a bystander rather than a directly affected board.

These checks helped confirm that the dataset was generally complete and suitable for analyzing the recorded attacks. However, the dataset still had limitations because it contained only eight normal runs and used application-layer attack emulation.

7.3. Evidence That the Dataset Is Valid

he attack results were checked using the recorded network data and the verification scripts. All 72 attack runs passed the attack verification process. Of these, 67 had complete data, while five had minor data gaps. The eight normal runs were also confirmed as benign.

Blackhole Attack

The blackhole attack caused a clear drop in packet delivery among the affected boards.

Before the attack, the forwarding ratio was between 1.000 and 1.004. During the attack, it dropped to between 0.0% and 0.7%. The packet delivery ratio also fell from 0.998 to 0.000, meaning that affected boards stopped successfully delivering the probe packets.

Out of 123 victim boards, 121 showed packet delivery loss during the first second of the attack. The remaining two showed the loss during the following second.

After the attack ended, packet delivery recovered within three seconds, with a median recovery time of two seconds.

The bystander boards were not affected in the same way. Their packet delivery ratio during the attack was 1.000, compared with 0.996 during the baseline period. This showed that the attack mainly affected the intended victims rather than the entire network.

These results confirm that packet delivery and forwarding behavior were useful for identifying the blackhole attack in the tested network.

Wormhole Attack

The wormhole attack caused packets to arrive at the root twice because the tunnel forwarded packets through a separate path.

Across all 36 wormhole runs, the tunnel exit received between 99% and 101% of the messages sent through the tunnel. During the attack, 12,465 probes were recorded, and 12,448, or 99.86%, reached the root twice.

Duplicate packets appeared in 6,468 out of 6,469 attack windows, or 99.98%. No duplicate packets were recorded during the baseline or cooldown periods. Other boards did not show the same duplicate-arrival pattern.

The tunnel also allowed packets to skip between one and five hops. However, the recorded hop counts did not change as expected because the attack operated at the application layer instead of directly changing the mesh routing process.

These results show that duplicate packet arrivals were a reliable indicator of the wormhole attack used in this study. However, the results do not prove that the experiment reproduced every behavior of a wormhole attack that directly changes wireless routing.

7.4. Answers to the Research Objectives

The first objective was to establish an ESP32-based wireless mesh testbed capable of generating multi-hop traffic under normal operation and controlled attack scenarios. This objective was achieved. All 80 runs were captured, and the verifier confirmed both attacks in all 72 attack runs as well as normal operation in the eight benign runs. However, two qualifications remain: the attacks were emulated at the application layer, and the partial mesh runs behaved like branched trees.

The second objective was to collect and structure cross-layer features from the physical, MAC, and network layers into a labeled and well-documented dataset. This objective was also achieved. The study defined 16 features with their corresponding units, recorded the labels during data capture, and documented changes from the original proposal. One limitation is that the boards could not provide a MAC retry count, so retransmissions were obtained from application counters instead.

The third objective was to evaluate the quality of the dataset and determine whether normal and attack behavior could be separated using exploratory analysis and unsupervised methods. The results differed between the two attacks. In the blackhole runs, only the PDR of the victims changed significantly, decreasing from 0.998 to 0.000, while the other features showed little change. When PDR was included, K-Means with $k = 2$ achieved an ARI of 0.514, compared with a best possible value of 0.515. Without PDR, no algorithm was able to effectively separate the attack, with ARI values ranging from -0.016 to 0.002.

In the wormhole runs, the cross-layer features did not effectively separate the attack, with ARI values ranging from -0.012 to 0.009. The tunnel was primarily visible through the counters of its two

endpoints and through duplicate arrivals at the root. When all 80 runs were combined using three labels, the result was similar, with an ARI of 0.333 compared with a best possible value of 0.334. Therefore, the study did not find the expected cross-layer evidence described in Chapter 4 for these attacks. Both attacks operated on application-layer probes and left the radio and routing behavior unchanged. Their effects were therefore primarily visible in the data plane, specifically through probe delivery in the blackhole attack and probe duplication in the wormhole attack. The researchers consider this an important finding of the study rather than a failure.

7.5. Contribution and Final Remarks

Main Findings

First, the blackhole attack caused a clear loss of packet delivery among victim boards. Packet delivery ratio was the strongest indicator of this attack.

Second, the wormhole attack caused duplicate packet arrivals at the root. This pattern was consistently observed during the attack and was absent during the baseline and cooldown periods.

Third, the expected changes in other network features were not observed consistently. Retransmissions did not increase as expected, and recorded parent relationships and hop counts remained unchanged. The signal-to-hop relationship also did not show the expected changes.

These results are important because both attacks affected packet delivery behavior without directly changing the routing process. As a result, the experiment could not fully demonstrate the cross-layer effects that may occur when an attacker manipulates wireless routing.

Fourth, the clustering results depended heavily on the features used. Packet delivery ratio was important for blackhole detection, while duplicate-arrival measurements were important for wormhole detection. Removing the main attack-related feature greatly reduced the clustering performance.

Contributions of the Study

The first contribution was an ESP32-based wireless mesh testbed that could be used to study normal network behavior and two types of attacks under different testing conditions.

The second contribution was a dataset containing network measurements from 80 experimental runs, including normal activity, blackhole attacks, and wormhole attacks. The dataset included information about topology, location, traffic scenario, board role, and attack phase.

The third contribution was a verification process that checked whether the attacks produced the expected network behavior. This helped distinguish actual attack effects from missing data or incomplete experimental runs.

The firmware, verification tools, and supporting scripts were also made available in the project repository [79] to support further testing and development.

Limitations of the Study

Despite these contributions, the study had several limitations.

First, the attacks were implemented at the application layer rather than directly changing the mesh routing protocol. This limited the types of network behavior that could be observed.

Second, the testbed used only eight ESP32 boards. The results may not represent larger wireless mesh networks with more nodes and more complex routing behavior.

Third, the experiment used only eight benign runs. This was not enough to estimate the false alarm rate reliably across different normal network conditions.

Fourth, some features, such as direct MAC retry measurements, were not consistently available. Packet-sniffer data was also collected only for some runs.

Fifth, the study did not repeat every experimental condition enough times to fully measure how consistent the results would be across repeated tests. Although the recorded results were generally consistent, more repetitions are needed to confirm their reliability.

Finally, the strong detection results depended on features closely connected to the attack implementation. Therefore, the study does not prove that the same results would be achieved against attacks that behave differently or against a larger real-world network.

7.6. Recommendations

Future research should address the limitations of the current study and test whether the findings apply to more realistic wireless mesh attacks.

Test Attacks That Directly Affect Routing

Future studies should investigate attacks that directly` affect wireless routing rather than only changing packet behavior at the application layer.

This could involve using a routing implementation that allows researchers to modify routing decisions or using radio-level tools to test attacks against wireless management and routing behavior. For example, researchers could examine whether forged management frames or other routing-related attacks produce measurable changes in the network.

Such experiments would help determine whether features such as parent changes, hop counts, retransmissions, and signal strength can provide additional evidence of an attack.

Collect More Data From Normal Network Conditions

Future studies should collect more benign runs across different topologies, locations, and traffic conditions.

The current study included only eight normal runs, which was not enough to estimate the false alarm rate reliably. More normal data would help determine whether the observed attack patterns remain different from ordinary network changes.

Each experimental condition should also be repeated multiple times to check whether the results remain consistent.

Improve Network Measurements

Future experiments should collect more complete link-layer and routing information.

In particular, packet-sniffer data should be collected during every run when possible. This would make it easier to verify retransmissions, duplicate packets, and other wireless behaviors.

Researchers should also compare the features listed in Tables 3.4 and 3.5 with the actual measurements available from the testbed. This would help identify which expected features can be measured directly and which require additional tools or changes to the firmware.

Test More Wormhole Scenarios

Future studies should test wormhole attacks that forward traffic from other nodes instead of only using the tunnel behavior implemented in the current experiment.

The tunnel should also be tested under different network layouts and traffic conditions. This would help determine whether duplicate packet arrivals remain a reliable indicator when the attack affects a wider part of the network.

Improve the Evaluation of the Detection Models

Future research should evaluate the detection models using entire experimental runs as separate test data. This would help determine whether a model can detect attacks in runs it has not already encountered during training or model development.

The models should also be compared with simpler detection methods. For example, packet delivery ratio can be used as a baseline for blackhole detection, while duplicate packet arrivals can be used as a baseline for wormhole detection.

A more complex model should only be considered more useful if it performs better than these simpler methods under the same testing conditions.

Researchers should also test whether the models continue to work when the main attack-related features are removed. This would help determine whether the models learn broader network behavior or mainly rely on a feature directly linked to the attack.

7.7. Final Conclusion

This study showed that blackhole and wormhole attacks produced clear and measurable changes in the tested ESP32 wireless mesh network. The blackhole attack caused packet delivery to drop among affected boards, while the wormhole attack caused duplicate packets to arrive at the root.

The results also showed that the detection performance depended heavily on the features used. Packet delivery ratio was the strongest indicator of the blackhole attack, while duplicate packet arrivals were the strongest indicator of the wormhole attack. Other cross-layer features did not provide enough evidence to reliably separate the attacks from normal behavior in the tested setup.

The study successfully developed a testbed, collected a dataset, and verified the effects of both attacks. However, because the attacks were implemented at the application layer, the results should not be treated as proof that the same detection performance would apply to attacks that directly manipulate wireless routing.

Overall, the findings provide a useful starting point for studying attack detection in ESP32 wireless mesh networks. Future work should focus on more realistic routing attacks, more normal data, improved network measurements, and stronger testing of the detection models.

Appendix A. Bibliography

[1] Akyildiz, I. F., Wang, X., & Wang, W. (2005). *Wireless mesh networks: A survey.* Computer Networks, 47(4), 445–487. https://www.sciencedirect.com/science/article/abs/pii/S1389128604003457

[2] Cilfone, G., Davoli, L., Belli, L., & Ferrari, G. (2019). *Wireless mesh networking: An IoT-oriented perspective—Survey on relevant technologies.* Future Internet, 11(4), 1–41.https://www.mdpi.com/1999-5903/11/4/99

[3] Hanif, M., Ashraf, H., Jalil, Z., Jhanjhi, N. Z., Humayun, M., Saeed, S., & Almuhaideb, A. M. (2022). AI-Based Wormhole Attack Detection Techniques in Wireless Sensor Networks. Electronics, 11(15), 2324. https://www.mdpi.com/2079-9292/11/15/2324

[4] Bhatti, D. S. et al. (2024). *Detection and isolation of wormhole nodes in wireless ad hoc networks based on post-wormhole actions.* Scientific Reports, 14, 3428. https://www.nature.com/articles/s41598-024-53938-9

[5] Al-Anzi, F. S.(2022). *Design and analysis of intrusion detection systems for wireless mesh networks.* Digital Communications and Networks. https://www.sciencedirect.com/science/article/pii/S2352864822001043

[6] Ioulianou, P. P., Vassilakis, V. G., & Shahandashti, S. F. (2022). *A Trust-Based Intrusion Detection System for RPL Networks: Detecting a Combination of Rank and Blackhole Attacks.* Journal of Cybersecurity and Privacy, 2(1), 124-153. https://www.mdpi.com/2624-800X/2/1/9

[7] M. Alharthi, A. A. Alabdulatif, and A. Aljohani, *"XLID: Cross-Layer Intrusion Detection System for Wireless Sensor Networks," International Journal of Science and Technology (IJST)*, vol. 8, no. 3, pp. 87–94, 2019. [Online]. Available: https://sciresol.s3.us-east-2.amazonaws.com/IJST/Articles/2019/Issue-3/Article8.pdf

[8] A. Amouri, H. Abid, and N. Boudriga, *"A Cross-Layer, Anomaly-Based IDS for WSN and MANET,"* Sensors, vol. 18, no. 2, p. 651, 2018. [Online]. Available: https://www.mdpi.com/1424-8220/18/2/651

[9] G. Singh, *"A Cross-Layer Based Optimized Feature Selection Scheme for IDS in WSNs,"* Journal of Intelligent & Fuzzy Systems, vol. 42, pp. 1–13, 2022. [Online]. Available: https://journals.sagepub.com/doi/abs/10.3233/JIFS-210700

[10] Ayad, A.G., Sakr, N.A. & Hikal, N.A. A hybrid approach for efficient feature selection in anomaly intrusion detection for IoT networks. J Supercomput 80, 26942–26984 (2024). https://link.springer.com/article/10.1007/s11227-024-06409-x

[11] A. A. Al-Qarawi and M. A. Razzaque, "Energy-Aware Cross-Layer Intrusion Detection System for IoT-Based Wireless Mesh Networks," Sensors, vol. 22, no. 18, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/18/6954

[12] Arshad, J., Azad, M. A., Amad, R., Salah, K., Alazab, M., & Iqbal, R. (2020). A Review of Performance, Energy and Privacy of Intrusion Detection Systems for IoT. Electronics, 9(4), 629. https://www.mdpi.com/2079-9292/9/4/629

[13] Alshehri AH. 2024. Wormhole attack detection and mitigation model for Internet of Things and WSN using machine learning. PeerJ Computer Science 10:e2257 https://peerj.com/articles/cs-2257/

[14] Sarhan, M., Layeghy, S., Moustafa, N., Gallagher, M., & Portmann, M. (2024). Feature extraction for machine learning-based intrusion detection in IoT networks. Digital Communications and Networks, 10(1), 205-216.https://arxiv.org/abs/2108.12722

[15] Churcher, A., Ullah, R., Ahmad, J., Rehman, S. u., Masood, F., Gogate, M., Alqahtani, F., Nour, B., & Buchanan, W. J. (2021). *An experimental analysis of attack classification using machine learning in IoT networks*. MDPI Sensors, 21(2), 446. https://www.mdpi.com/1424-8220/21/2/446

[16] Shafieian, S., & Zulkernine, M. (2022). *Multi-layer stacking ensemble learners for low footprint network intrusion detection*. SpringerLink. https://link.springer.com/article/10.1007/s40747-022-00809-3

[17] Wardana, S. A., Yusof, R., Zolkipli, M. F., & Kim, D. (2024). *Collaborative intrusion detection using weighted ensemble averaging deep neural network for coordinated attack detection in heterogeneous networks. International Journal of Information Security,* 23(4), 801–815. https://link.springer.com/article/10.1007/s10207-024-00891-3

[18] Vanlalhruaia, Ajoy, K.K., Amit, K.R. (2025). Securing AODV against black hole attack for wireless mesh network. Journal of Information Systems Engineering & Management, 10(37s), 1192–1202. https://doi.org/10.52783/jisem.v10i37s.7547

[19] Srinivasan, J. (2025). Innovative cross-layer defense mechanisms for blackhole and wormhole attacks in wireless ad-hoc networks. Scientific Reports, 15(1), 14747.

[20] Bhonsle, M., Srinivasulu, G., Chaitanya, K., Raghu, D., Surendra, G., Kumar, K., Rao, M. S., Prabhakar, K., & Krishna, V. (2025). Deep Learning-Based Defense Mechanisms against black hole attacks in wireless mesh networks. Advanced Sustainable Science Engineering and Technology, 7(1), 02501010. https://doi.org/10.26877/asset.v7i1.1036

[21] Espressif Systems. (n.d.). ESP-WIFI-MESH Programming Guide [ESP-IDF v5.3 for ESP32]. https://docs.espressif.com/projects/esp-idf/en/v5.3/esp32/api-guides/esp-wifi-mesh.html

[22] Khan, A. U., Khan, M. E., Hasan, M., Zakri, W., Alhazmi, W., & Islam, T. (2022). An Efficient Wireless Sensor Network Based on the ESP-MESH Protocol for Indoor and Outdoor Air Quality Monitoring. Sustainability, 14(24). https://doi.org/10.3390/su142416630

[23] Qian, L., Gu, K., Fu, Y., Shen, Y., & Xu, S. (2024). A Wireless Ad Hoc Network Communication Platform and Data Transmission Strategies for Multi-Bus Instruments. Electronics, 13(18). https://doi.org/10.3390/electronics13183596

[24] A, Avina & R, Geetha. (2025). ResNeXt and Deep Stacked Autoencoder based framework for wormhole attack detection on network control systems. Discover Computing. 28. 10.1007/s10791-025-09816-7.

[25] Boiano, A., Zheng, D., Palmese, F., Pimpinella, A., & Redondi, A. E. C. (2026, February 3). Analyzing Zigbee Traffic: Datasets, classification and storage trade-offs. arXiv.org. https://arxiv.org/abs/2602.03140

[26] Das, T., Hamdan, O. A., Shukla, R. M., Sengupta, S., & Arslan, E. (2023, January). Unr-idd: Intrusion detection dataset using network port statistics. In 2023 IEEE 20th consumer Communications & networking conference (CCNC) (pp. 497-500). IEEE.

[27] Moustafa, Nour & Slay, Jill. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). 10.1109/MilCIS.2015.7348942.

[28] Miekkala, Topi & Pyykönen, Pasi & Drainakis, Georgios & Pantazopoulos, Panagiotis & Muller, Tobias & Katsaros, Konstantinos & Sourlas, Vasilis & Amditis, Angelos & Kaklamani, Dimitra. (2024). NordicDat: A Cross-Border Predictive QoS Dataset. 1281-1286. 10.1109/GLOBECOM52923.2024.10901462.

[29] Tavallaee, Mahbod & Bagheri, Ebrahim & Lu, Wei & Ghorbani, Ali. (2009). A detailed analysis of the KDD CUP 99 data set. IEEE Symposium. Computational Intelligence for Security and Defense Applications, CISDA. 2. 10.1109/CISDA.2009.5356528.

[30] Sommer, Robin & Paxson, Vern. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. Proceedings - IEEE Symposium on Security and Privacy. 305-316. 10.1109/SP.2010.25.

[31] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. Computers & security, 86, 147-167.

[32] García-Teodoro, Pedro & Díaz-Verdejo, Jesús & Maciá-Fernández, Gabriel & Vázquez, Enrique. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges. Computers & Security. 28. 18-28. 10.1016/j.cose.2008.08.003.

[33] Tsai, Chih-Fong & Hsu, Yu-Feng & Lin, Chia-Ying & Lin, Wei-Yang. (2009). Intrusion detection by machine learning: A review. Expert Systems with Applications. 36. 11994-12000. 10.1016/j.eswa.2009.05.029.

[34] Chandola, Varun & Banerjee, Arindam & Kumar, Vipin. (2009). Anomaly Detection: A Survey. ACM Comput. Surv.. 41. 10.1145/1541880.1541882.

[35] Tukey, J. W. (1977). Exploratory data analysis. Reading, MA: Addison-Wesley.

[36] Aggarwal, C. C. (2015). Data mining: the textbook (Vol. 1, No. 3). New York: springer.

[37] Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann Publishers.

[38] Jain, Anil. (2010). Data Clustering: 50 Years Beyond K-Means. Pattern Recognition Letters. 31. 651-666. 10.1016/j.patrec.2009.09.011.

[39] Pedroso, Carlos & de Souza Batista, Agnaldo & Jesus, Samuel & Santos, Aldri. (2024). A Direct Collaborative Network Intrusion Detection System for IoT Networks Integration. 10.5753/sbrc.2024.1354.

[40] Hernandez-Ramos, J. L., et al. (2022–2024). Intrusion detection based on federated learning: A comprehensive survey. ACM Computing Surveys.https://dl.acm.org/doi/10.1145/3731596

[41] Rassam, M. A. (2024). Autoencoder-based neural network model for anomaly detection in wireless body area networks. *IoT*, 5(4), 852–870. https://www.mdpi.com/2624-831X/5/4/39

[42] J. Li, et al., *"Optimizing IoT Intrusion Detection System: Feature Selection versus Feature Extraction,"* Journal of Big Data, vol. 11, 2024. [Online]. Available: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-024-00892-y

[43] Kurosawa, S., Nakayama, H., Kato, N., Jamalipour, A., & Nemoto, Y. (2007). Detecting blackhole attack on AODV-based mobile ad hoc networks by dynamic learning method. International Journal of Network Security, 5(3), 338–346. http://ijns.jalaxy.com.tw/contents/ijns-v5-n3/ijns-2007-v5-n3-p338-346.pdf

[44] Baadache, A., & Belmehdi, A. (2014). Struggling against simple and cooperative black hole attacks in multi-hop wireless ad hoc networks. Computer Networks, 73, 173–184. https://www.researchgate.net/publication/265687984_Struggling_against_simple_and_cooperative_black_hole_attacks_in_multi-hop_wireless_ad_hoc_networks

[45] Khalil, I., Bagchi, S., & Shroff, N. B. (2007). LiteWorp: Detection and isolation of the wormhole attack in static multihop wireless networks. Computer Networks, 51(13), 3750–3772. https://doi.org/10.1016/j.comnet.2007.04.001

[46] Hu, Y.-C., Perrig, A., & Johnson, D. B. (2003). Packet leashes: A defense against wormhole attacks in wireless networks. *IEEE INFOCOM 2003. Twenty-Second Annual Joint Conference of the IEEE Computer and Communications Societies*, *3*1976–1986. https://doi.org/10.1109/INFCOM.2003.1209219

[47] Keipour, H. (2022). Blackhole attack detection in low-power IoT mesh networks using machine learning algorithms.

[48] Chatterjee, S., et al. (2022). Anomaly detection in IoT systems: A survey. Internet of Things, 18, 100508. https://doi.org/10.1016/j.iot.2022.100508

[49] Miguel-Diez, A., et al. (2025). Unsupervised anomalous traffic detection based on flows: A systematic literature review. IEEE Access, 13, 45231–45259. https://doi.org/10.1109/ACCESS.2025.1234567

[50] van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. Journal of Machine Learning Research, 9, 2579–2605. https://www.jmlr.org/papers/v9/vandermaaten08a.html

[51] McInnes, L., Healy, J., & Melville, J. (2020). UMAP: Uniform manifold approximation and projection for dimension reduction. Nature Methods, 17(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[52] Jolliffe, I. T. (2002). Principal component analysis (2nd ed.). Springer. https://doi.org/10.1007/b98835

[53] Shameer, C. M. (n.d.). Understanding DBSCAN clustering with a simple example. Medium. https://medium.com/@cmshameer712/understanding-dbscan-clustering-with-a-simple-example-940c468eeefa

[54] Soni, S. (n.d.). Clustering like a pro: A beginner's guide to DBSCAN. Medium. https://medium.com/@sachinsoni600517/clustering-like-a-pro-a-beginners-guide-to-dbscan-6c8274c362c4

[55] UpGrad. (n.d.). What is DBSCAN clustering? UpGrad Blog.

https://www.upgrad.com/blog/what-is-dbscan-clustering/

[56] STHDA. (n.d.). DBSCAN: Density-based clustering essentials. Statistical Tools for High-throughput Data Analysis. https://www.sthda.com/english/wiki/wiki.php?id_contents=7940

[57] Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.

[58] Lloyd, S. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

[59]ResearchGate. (n.d.). An example dendrogram for hierarchical clustering shown in Table 1. https://www.researchgate.net/figure/An-example-dendrogram-for-the-hierarchical-clustering-shown-in-Table-1-The-original-data_fig9_278706094

[60]UC Business Analytics R Programming Guide. (n.d.). Hierarchical clustering. https://uc-r.github.io/hc_clustering

[61]Minitab. (n.d.). What is hierarchical clustering? https://mtab.com/blog/what-is-hierarchical-clustering

[62]Singh, A. (n.d.). Hierarchical clustering: A tree-based approach to data grouping. Medium. https://medium.com/@abhaysingh71711/hierarchical-clustering-a-tree-based-approach-to-data-grouping-241131b1c4c5

[63]VanderPlas, J. (n.d.). Gaussian mixture models. In Python Data Science Handbook. https://jakevdp.github.io/PythonDataScienceHandbook/05.12-gaussian-mixtures.html

[64]MathWorks. (n.d.). Cluster data from a mixture of Gaussian distributions. https://www.mathworks.com/help/stats/cluster-data-from-mixture-of-gaussian-distributions.html

[65]AI Learning Hub. (n.d.). Gaussian mixture model (GMM): A clustering method. Medium. https://medium.com/@AILearningHub/gaussian-mixture-model-gmm-a-clustering-method-99263f8de214

[66]ResearchGate. (n.d.). Cluster fitting with a Gaussian mixture model (GMM). https://www.researchgate.net/figure/Cluster-fitting-with-a-Gaussian-mixture-model-GMM-The-scatter-plot-from-Figure-1d-is_fig3_350132034

[67]ResearchGate. (n.d.). The spectral clustering algorithm for graph-structured data. https://www.researchgate.net/figure/The-spectral-clustering-algorithm-for-the-graph-structured-data-a-A-simple-undirected_fig1_360129519

[68] Leskovec, J., & Tsaparas, P. (n.d.). Graph partitioning: Graph cuts and spectral clustering [Lecture slides]. https://www.eecs.yorku.ca/~papaggel/courses/eecs4414/docs/lectures/07-spectral.pdf

[69]StrataScratch. (n.d.). Unsupervised clustering methods in machine learning. https://www.stratascratch.com/blog/unsupervised-clustering

[70] Airehrour, D., Gutierrez, J., & Ray, S. K. (2018). A trust-based defence scheme for mitigating blackhole and selective forwarding attacks in the RPL routing protocol. *Australian Journal of Telecommunications and the Digital Economy, 6*(1), 41–59. https://doi.org/10.18080/jtde.v6n1.138

[71] Zhukabayeva, T., Zholshiyeva, L., Mardenov, Y., Buja, A., Khan, S., & Alnazzawi, N. (2025). Real-time detection and response to wormhole and sinkhole attacks in wireless sensor networks. *Technologies, 13*(8), 348. https://doi.org/10.3390/technologies13080348

[72] Ramírez Gómez, J., Vargas Montoya, H., & León Henao, Á. (2019). Implementing a wormhole attack on wireless sensor networks with XBee S2C devices. *Revista Colombiana de Computación, 20*(1), 41–58. https://doi.org/10.29375/25392115.3606

[73] Zhan, Z., Zhou, H., & Haddadi, H. (2025). *REAL-IoT: Characterizing GNN intrusion detection robustness under practical adversarial attack*. arXiv. https://arxiv.org/abs/2507.10836

[74] Abdallah, W., Mnasri, S., & Val, T. (2022). *Distributed approach for the indoor deployment of wireless connected objects by the hybridization of the Voronoi diagram and the Genetic Algorithm*. arXiv. https://arxiv.org/abs/2202.13735

[75] Espressif Systems. (n.d.). *ESP-WIFI-MESH* [API reference, ESP-IDF v5.3 for ESP32]. https://docs.espressif.com/projects/esp-idf/en/v5.3/esp32/api-reference/network/esp-wifi-mesh.html

[76] Espressif Systems. (n.d.). *esp_mesh.h* (ESP-IDF v5.3.5) [Source code]. GitHub. https://github.com/espressif/esp-idf/blob/v5.3.5/components/esp_wifi/include/esp_mesh.h

[77] Espressif Systems. (n.d.). *Miscellaneous system APIs: MAC address* [API reference, ESP-IDF v5.3 for ESP32]. https://docs.espressif.com/projects/esp-idf/en/v5.3/esp32/api-reference/system/misc_system_api.html#mac-address

[78] Wireshark Foundation. (n.d.). *Download Wireshark* [Software]. https://www.wireshark.org/download.html

[79] NIS16 thesis group. (2026). *NIS16-ESP32-Environment* (THESIS3 branch) [Source code]. GitHub. https://github.com/Kemo1006/NIS16-ESP32-Environment

[80] Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In 31st USENIX Security Symposium (USENIX Security 22) (pp. 3971–3988). USENIX Association. https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[81] Karlof, C., & Wagner, D. (2003). *Secure routing in wireless sensor networks: Attacks and countermeasures.* Ad Hoc Networks, 1(2–3), 293–315. https://doi.org/10.1016/S1570-8705(03)00008-8

Appendix B. Research Ethics Forms

RESEARCH / PROJECT ETHICS REVIEW FORM **For Thesis / Capstone / Dissertation Proposals** (***ver.Oct2024c***)
Names of Student Researcher(s): Calpoporo, Jose Angelo C. Carlos, Miguel Sebastian C. Ong, Kyle Edward M. Reinante, Christian G.
Research/Thesis/Capstone Title: Cross-Layer Dataset Design and Exploratory Analysis of ESP32-Based ESP-WIFI-MESH Network
College: College of Computer Studies
Department: Department of Computer Technology
Course: Bachelor of Computer Science Major in Network and Internet Security
Expected Duration of the Project: from Term 1 AY 2025-2026 to Term 1 AY 2026-2027

PROTOCOL:

(*To be filled up by proponents; Use the Ethics Checklists as guides in determining areas for ethical concern/consideration. Attach and properly reference the accomplished ethics checklists and supporting documents.)*

		Reference Document (include page number)
Target participants	**No human participants** are directly involved in the experiment. The "participants" are **ESP32 mesh nodes** used in laboratory testing. Only the researchers will interact with the system for configuration, observation, and data recording.	Section 1.6.1 Hardware (pp. 15)
Data to be used / collected	**Network Telemetry Data:** RSSI values, retransmission counts, hop transitions, forwarding ratios, parent-switching events, and packet delivery ratios. **Attack Telemetry Data:** For blackhole emulation: packets received, forwarded, and dropped at the attacker node. For wormhole emulation: tunnel message counts and bytes transferred. **System Logs:** Phase labels, node roles, timestamps, and event logs (e.g., parent switches). Telemetry logs are stored on each board's flash and on a microSD card inside the board. The cards are handled only by the researchers, are read only on the researchers' laptops, and contain system-generated	Section 4.2.3.1. Victim Node Data Collection (pp. 93-94) Section 4.2.3.2. Attack Node Data Collection (pp. 94-96) Section 4.2.3.4. Phase Labeling and Synchronization (pp. 98-99)

PANEL'S ASSESSMENT/COMMENTS:

PANEL'S DECISION:

- **Approved** (*Proponent/s may proceed with the study/data collection*)
- **Modifications Required** (*Proponent/s should address ethical issues, follow the panel's recommendations for required modifications, and resubmit Ethics Review Form for approval before proceeding with the study or before starting with data collection*)
- **For Full Review** (*Proponent/s should submit the proposal to the University Research Ethics Review Committee (RERC) and obtain a research ethics clearance before proceeding with the study or before starting with data collection*)

JUSTIFICATION:

_____
_____
_____

Mr. Gregory G. Cu
Name and Signature of Adviser/Mentor
Date:

_____ _____
Name and Signature of Panel Member **Name and Signature of Panel Member**
Date: **Date:**

_____
Name and Signature of Panel Member
Date:

Note to the Ethics Review Form: Library and Yuchengco Sites

The filed ethics form says all experiments take place in a controlled laboratory environment. Two of the three sites are public spaces, so the fields below change. Nothing here changes the finding that no human participants are involved. No permission was requested for the Library and Yuchengco areas, because they are public spaces, so there is no approval to attach. The amendment goes in as a note attached to the filed form, and the form is not resubmitted.

Form field	Replace	With
Target participants	The “participants” are ESP32 mesh nodes used in laboratory testing.	The “participants” are ESP32 mesh nodes. Runs take place at three indoor sites on the DLSU campus: room G402 of the Gokongwei College of Computer Studies, a small part of the DLSU Library, and part of the Yuchengco Building. In the Library and the Yuchengco Building other people may be in the space during a run. They are not recruited, observed, questioned or recorded, and the study does not analyse them.
Data to be used / collected	Add after System Logs	**Sniffer Capture Data:** In some runs one spare ESP32 board, or a laptop in monitor mode, records the 802.11 frames of the test mesh. It is set to the mesh’s channel (channel 11, 20 MHz), only listens, and never joins a network or transmits. The mesh is encrypted, so a capture shows frame headers, timing and signal strength and not the contents. Any receiver tuned to that channel in a public space may also hear frames from other nearby networks. The analysis reads only the frames of the test mesh, which are identified by the MAC addresses of the researchers’ own boards, and does not analyse the frames of other networks.
Brief procedure, Network Setup	Deploy up to 10 ESP32 development boards in a controlled indoor environment to form an ESP-WIFI-MESH network.	Deploy 8 ESP32 development boards to form an ESP-WIFI-MESH network with its own mesh ID that is not connected to any campus, external or production network. The network is set up at the three indoor sites listed under Target participants. In the Library and the Yuchengco Building only a small area is used.
Brief procedure, Data Logging and Storage	All nodes log raw telemetry locally to flash storage. Logs are retrieved post-experiment via USB serial for offline processing.	All nodes log raw telemetry locally to flash storage and mirror it to a microSD card. Logs are retrieved after the experiment by removing the cards, or by USB serial, for offline processing.

Potential risks, Network Interference Risks	All experiments are conducted in a controlled laboratory environment with no connection to external or production networks, mitigating any risk to other systems.	The two attack behaviors act only on traffic inside the test mesh, which has its own mesh ID and joins no campus, external or production network. All experiments are conducted at the three controlled indoor sites above. In the Library and the Yuchengco Building the test mesh shares the 2.4 GHz band with other Wi-Fi networks. It stays on one fixed channel and sends small periodic probes, every 1000 ms from each child, or every 250 ms in the highload scenario. The Library and the Yuchengco Building areas are public spaces.
Potential risks, add	Add a line	**Public-Space Risks:** Boards, cables and a laptop are left in a public area during a run, and a researcher attends them for the whole run.
Applicable terms, Wireshark	Wireshark: GNU GPL license; used solely for verification within the isolated test network.	**Wireshark:** GNU GPL license; used solely to verify the behavior of the test mesh, from a listening-only capture.
Steps to safeguard, add	Add three lines	**Card Storage:** Telemetry logs are stored on each board's flash and on a microSD card inside the board. The cards are handled only by the researchers, are read only on the researchers' laptops, and contain system-generated telemetry only. **Site Use:** The Library and the Yuchengco Building areas are public spaces. **Listening-only Capture:** The sniffer never joins a network or transmits.
Other concerns and considerations	All experiments will take place in a controlled laboratory environment to ensure repeatability and safe containment of network manipulations.	All experiments take place at the three controlled indoor sites (room G402, a small part of the DLSU Library and part of the Yuchengco Building), on an isolated mesh, to ensure repeatability and safe containment of network manipulations.

Reference Document column	Add	Section 4.2.2.5 Deployment Sites and Scenario, next to Target participants, Brief procedure and Potential risks. Section 4.2.3.3 Data Acquisition and Logging Architecture, next to the sniffer line.

Appendix C. Deviations Log

The paper cites the deviations from the approved proposal by the identifiers D-1 to D-37. Table C.1 lists them. Each row says what the study does and why. Entries D-21 to D-37 record the changes made during the capture campaign, in the order in which they were entered. Where an entry records a change that had been compile-checked and not tested on hardware when it was entered, the row says so.

Table C.1 Deviations from the Approved Proposal

ID	Topic	What the study does	Why
D-1	Sampling rate	Boards log at 10 Hz and the analysis uses the 10 Hz samples (see D-9).	Five samples per five-second window left no room for a dropped sample. Capturing at 10 Hz keeps windows complete: on an early baseline run only 6 of 583 windows (1.0%) were discarded.
D-2	LatencyHopRatio	Relative one-way delay divided by hop count (Equation 4.14), not mean round-trip time.	Probes travel one way to the root and nothing is sent back, so no round trip exists in the data.
D-3	TunnelLatency	The difference in arrival delay between the two copies of one probe at the root, not an echo round trip.	The tunnel is a one-way UART write, so there is no echo frame to time.
D-4	Topology check	Parent changes in the first 60 s of a node's log count as mesh formation, not as re-routing.	Initial parent acquisition was being read as baseline re-routing. On an early baseline run all 6 changes fell in the first 12.8 s and none followed.
D-5	Baseline control	The baseline is phase 0 (300 s) of each attack run. Matched benign runs are added by D-18.	Phase 0 shares firmware, node roles, mesh session and placement with the attack, so the attack is the only difference. A separate baseline run also differs in how many nodes originate probes.
D-6	Repeats and parents	The topology class is fixed for a run and checked from the logged parent links. Which board sits at which layer is not fixed.	Pinning parents would override the parent selection that the mesh is studied for; parents are fixed only where D-16, D-28, D-30 and D-36 say so.
D-7	run_repeat column	The repeat number is a column in the windowed and feature tables.	A reader can pool runs but cannot un-pool them, so per-repeat variance would be lost.
D-8	PDR attribution	A window gets a PDR, including 0, when the node sent in that window, held a mesh association, and the root's arrival log covers the window.	A run-wide coverage rule wrote off the nodes that a blackhole hits hardest, so PDR = 0 could not be recorded.
D-9	Grid and window	10 Hz grid and one-second windows. A window with fewer than 8 of 10 samples is discarded. Gaps of 1 or 2 samples (0.2 s) inside a kept window are filled by interpolation or forward fill.	It keeps the 80% completeness ratio of the proposal (4 of 5), and ten samples per window give more support per row than five.

D-10	Run columns	Attack, topology, location and scenario are columns on every row.	The same reason as D-7. Without them, runs of different scenarios collapsed into one group.
D-11	Hop, not layer	The feature is HopChangeCount, and hop = layer − 1 is added next to the raw layer value.	The raw field counts mesh layers and not network-layer (IP) data, and the old name overclaimed.
D-12	Application-layer relay	Every node sends to its own parent and relays what it receives one hop upstream, counting packets received, forwarded and dropped.	The mesh stack relayed below the application layer, so intermediate nodes could not see their own forwarding and the relay features existed only on the attacker.
D-13	microSD mirror	The card mirror is flushed with fsync() about every 5 s and at every phase boundary.	It fixes a failure in which a card file held 0 rows. The change compiles. None of the 637 telemetry files of the 80 runs of the final set has 0 rows, and the one board without a file is B768 in the blackhole run in the tree topology at G402 with highload. The change has not been validated by a reset in the middle of a run followed by a read-back of the card.
D-14	Radio width	The radio is fixed at 20 MHz (HT20) before it starts, and every capture records its width. All dataset captures are at 20 MHz. Earlier captures at 40 MHz are archived and not pooled. The 80 runs were captured after the change, and each holds windows in all the phases of its schedule. The sniffer decoded data frames in 7 of the 36 blackhole runs.	The independent sniffer captures 20 MHz only and could not decode 40 MHz data frames. The two widths differ in PHY behaviour. Mesh formation at 20 MHz is verified on hardware.
D-15	Application counters	retry_count and tx_count are application counters and not 802.11 MAC statistics. RetryRate is not used in the analysis, and MAC retries come only from the sniffer.	ESP-IDF has no call that returns a MAC retry count. Its statistics call only prints to the log.
D-16	Star and blackhole	The attacker is the hub of the star and the root sits one hop behind it. Victims join the hub.	At the root, “drop instead of forward” has no meaning, and the root is the referee that logs the arrivals.
D-17	Wormhole ends	Node A (exit, root-side) and Node B (entry, leaf-side) are decided at run time from mesh depth, locked at the first phase, and recorded in every row.	The mesh decides which board sits near the root, and a fixed assignment can give no shortcut. It worked on hardware in all 36 wormhole runs of the final dataset: in each, one Node A and one Node B kept their roles and Node A sat shallower than Node B.
D-18	Matched benign runs	Matched no-attack runs at the three sites. The plan of D-18 had 9, three at each site, each on a different topology. D-35 revised it, and the final plan has 8 benign runs (4 stationary, 3 highload and 1 burst), which gives 80 runs in all.	Every topology needs a stand-alone benign capture, and benign data should cover more than one condition.
D-19	Sites	Runs are made at G402, the DLSU Library and the Yuchengco Building.	The campaign was redesigned around these three sites, which differ in Wi-Fi interference.

D-20	Attack emulation	Both attacks are application-layer emulations. Neither advertises a false route; the parent rules that shape some setups (D-16, D-26, D-28, D-30, D-36) are fixed before the baseline and do not change during the run.	ESP-WIFI-MESH documents no way to write a node's routing table, so the dataset records the effect on traffic and not route attraction.
D-21	Wormhole verification	The wormhole is confirmed by two independent measures: TunnelIntensity, the count kept by the attacking boards, and DuplicateRate, the share of Node B's windows in which the root received a duplicate of a probe. TunnelLatency and LatencyHopRatio are reported and not scored. A window whose raw rows span two phases is a transition window and gets no label. A node whose phase exit fell in discarded windows is anchored on that exit, so that its valid baseline is kept.	A counter that the attacker reports about itself is weak evidence, and the root's view is independent. TunnelLatency exists only while the tunnel runs, so it has no normal-phase distribution, and the unsynchronised relative delay of LatencyHopRatio is too dispersed for the 3-sigma test (the largest absolute z-score it can reach is 0.23 to 0.64). A test that cannot pass is no evidence either way. The first tunnelled probe had been counted as baseline, and a telemetry stall had deleted valid baseline windows.
D-22	Pre-run gates and START	The root holds phase 0 until every child has joined. In the tree with a blackhole it also waits until the attacker has a child, and in the wormhole until there is exactly one end A and one end B, the UART works in both directions and A is closer to the root than B. The operator then types START after checking the topology table. A START_ANYWAY override exists.	The wormhole ends lock their roles at phase 0, and tree runs are corrupted when boards move after it. Some linear runs started with both ends acting as B because of a one-way UART jumper.
D-23	Parent changes in the attack	A parent change during the attack is judged by exposure and is not forbidden. In every window each board is classed, by following its parent chain, as a victim (its path to the root runs through the attacker), a bystander (it does not), detached (re-joining) or unknown. A parent change of a victim or of a wormhole end is tolerated, with a warning, only if the board stays in one class (fewer than 3 windows in the other), windows without a traceable path to the root are left out of the 3-sigma test, and the test still confirms on the attack windows before the first re-route alone (at least 60 s are needed). The decision reads only parent and hop values and never delivery.	What decides whether a run is one experiment is who the attack reaches, and not which MAC address the parent of a board has. A lock on parents at START would contradict the self-healing described in the paper and could not help when a parent itself crashes. Before the change, two boards that re-joined under the same attacker after another board crashed had made a run invalid. A check over the 53 captures analysed at the time changed one verdict, that of this run, which became confirmed.
D-24	START lock	The pre-START checks of the setup tool that would put a known fault into the data lock the START button until they turn green. They are: missing children, a wrong site on a card, a tree blackhole attacker without a child, victims aimed at another attacker MAC, and a wormhole that is not set up. An override needs a typed reason of at least 15 characters and the word OVERRIDE. It is written to the run	The earlier checks only asked yes or no, so START could be pressed while the requirements were not met. A faulty start can no longer happen silently, and an overridden start is footnoted by the verifier.

		log and printed by the verifier as a warning. Amber checks only ask.	
D-25	Tick alignment	Each sample takes the earliest free 0.1 s tick for which it could have been scheduled, at most 3 ticks (300 ms) late and never early. There is one sample per tick, and nothing is invented or counted twice. The 80% completeness rule and the filling of gaps of at most two samples are unchanged.	The firmware samples at a fixed period, and when the task is held up it delivers the missed samples back to back. Rounding to the nearest tick put two samples on one tick and left an empty tick, and three empty ticks discarded the window. Board FE90 of a tree blackhole run lost 59% of its attack windows although no sample was missing on its card (1,807 rows in 182 s). The old rule measured the resampling and not the board.
D-26	Sticky tree parents	In tree builds, on non-root boards only, the signal thresholds at which a board starts to look for another parent are set 6 dB weaker than the defaults of ESP-WIFI-MESH, and the weak-link watch lasts three times longer. The defaults are read at boot, and the old and new values are printed in the serial log of each board. The choice of the first parent is unchanged, and a parent that is lost or vanishes still makes its children move. A first version also changed the thresholds for the first parent and was later narrowed.	The links sit at a median of −71 to −78 dBm, with many boards below −78 dBm, so children re-parented during runs on a link that still worked, which changes who is a victim without any fault. The first version made the root count as good enough from 6 dB farther away, and more boards joined it directly. Tree runs captured before this firmware used the standard thresholds. When the entry was written, the change compiled and had not been tested on hardware.
D-27	Topology gate	A board passes the topology check when it has a parent at the moment phase 0 begins and does not re-route during the baseline. The time to form is printed and not judged. Logs without phase information keep the 60 s rule.	The old rule timed convergence from the first log row of each board and failed any board slower than 60 s, even one that had settled minutes before the experiment. It failed 31 of 54 captures. Since D-22, START is pressed only after the mesh has settled, and what matters for the data is the layout during the experiment. Of the 31 captures, 29 changed from fail to pass and 2 remain as fail for genuine reasons.
D-28	Star wormhole	The proposal keeps the physical star unchanged under the wormhole (Figure 4.16 of the proposal). In star wormhole runs, end A is the hub of the star and the root sits one hop behind it, so that the order is the root, then A, then every other board, B included. The other boards join A by a fixed parent. The normal copy of a probe of B therefore takes 2 hops, the tunnelled copy takes 1, and the shortcut is 1 hop. A also relays the normal copy of B.	In a flat star every board is 1 hop from the root, so the tunnel would skip no hop and the root would hold phase 0 for ever. A duplication-only star would test replay and not a wormhole, and dropping the cell would lose one of the eight combinations of attack and topology. The construction is the same as for the star with a blackhole (D-16). When the entry was written, it had been compile-checked and not tested on hardware.
D-29	Partial mesh	The proposal describes the partial mesh as a topology that allows observation of adaptive parent switching. The study describes the partial mesh by how it is built and not by observed parent switching: boards	In all 9 partial-mesh captures examined at the time, no child was seen under a second parent, so the structure was a plain tree. Switching cannot be checked before START, because every

		placed so that each is in radio range of several candidate parents, at most two children per node, and ESP-WIFI-MESH free to re-parent. Parent switching was allowed and not observed. The captures and the method are unchanged, and the lack of observed switching is listed as a limit.	mesh looks like a tree until a node switches. Forcing a switch would need new firmware and a new capture of all 9 runs, and a forced switch would be an artefact and not behaviour of the mesh.
D-30	Wormhole end depth	In tree and partial-mesh wormhole runs, the board flashed as B does not take the root as its parent. It scans and fixes as its parent the strongest mesh node that is not the root. The board flashed as A fixes as its parent the shallowest open parent it hears at −85 dBm or better, and the parent of B must be at least as deep as A. The two ends keep their parent unless it is lost (three drops in a row) or full. In the tree only, the parent of B must be exactly one hop below A, so that A is at hop 1 and B at hop 3 (Figure 4.18).	The two ends hang off one UART cable and lie close together, and ESP-WIFI-MESH joins the shallowest parent it can hear. A partial-mesh setup repeatedly placed both ends at hop 1, and the root held phase 0. The automatic assignment of D-17 only renames the ends. In the tree, a parent at least as deep as A still allowed A itself, the loudest board B hears, so B hung under A each time. When the entry was written, the change was not tested on hardware.
D-31	Attacker re-parenting	A change of the attacker's own parent is tolerated, with a warning, only if no board switched between victim and bystander during the attack, at least one board stayed a victim, and the attacker was without an uplink for at most 10% of its attack windows. Otherwise the run fails, and the reason is named. A board counts as a victim in a second only if its whole path to the root runs through the attacker, so seconds in which the attacker has no uplink or is silent are left out of the 3-sigma test. The decision reads only the layout and never delivery.	The treatment of a blackhole is who sits behind the attacker, and a different upstream parent of the attacker changes neither by itself. Earlier, a board counted as a victim even while the attacker had no uplink or had crashed, when every probe is lost for a reason that is not the blackhole. Re-running the check over the 56 live cells changed no verdict, grade or campaign status.
D-32	Missing boards	A board missing from a capture is graded by its role in the attack. The tool finds boards that the root heard (at least 100 arrivals in phases 0 to 3) and that have no file, and boards with no attack windows, and places each by structure only. A missing attacker is critical. A missing victim is yellow if other victims keep at least 60 traceable seconds, and otherwise critical. A missing bystander or relay above the attacker is yellow if another control keeps at least 60 s, and undetermined if it was the last control. An unknown position is undetermined, and a plain wormhole child is yellow. The new grade undetermined means that the outcome is inconclusive.	A missing board should not invalidate an otherwise valid attack, but missing data that could compromise the signature must not be overlooked. Two runs had a board that the root heard and no file, and both had been graded without anyone knowing. The thresholds of 100 arrivals and 60 s are engineering decisions.
D-33	Jitter scenario	Mobility and power cycle had been dropped from the plan earlier, so that every cell ran the same four scenarios. The jitter scenario is left out of the plan, which leaves stationary, burst and highload. Each of the 24 combinations of topology,	The team decided to take jitter out of the plan for the time being. It is a decision about the scope of the plan and not a response to a problem in the data.

		attack and site then runs the three once, so that the 72 attack runs of the plan file are the crossed design of Section 4.2.2.5. A setting of the planning tool lists the skipped scenarios, and the drawn order of the slots is kept, so that restoring jitter restores the plan of 108 runs exactly. With jitter skipped, the plan came to 81 runs (72 attack runs and 9 benign runs). The jitter captures already made stay on disk and are counted as outside the plan.	
D-34	Scenario not applied	A run whose scenario check reports that the scenario was not applied is graded as minor gaps. The attack result stands. A scenario that is unclear or not visible stays a note. Because the scenario is not a board, the impact check never re-tests it, and this finding alone never turns a run into a major gap.	The scenario check only labelled the verdict line and the grade ignored it, so a run that was not the scenario it was filed as could still be graded strong or good. The two runs that changed from good to minor gaps were blackhole linear burst runs, one at G402 and one at home, a site that is no longer in the campaign (D-19). No other run changed.
D-35	Benign run plan	The benign runs are one per site and topology, and for each topology the three sites use different scenarios, with the assignment shuffled for each topology. The attack runs are unchanged and still cover every scenario. The draw gave 12 benign runs and a plan of 84 runs. At the end of the campaign, the four benign runs that had never been captured were removed from the plan: the DLSU Library with the tree (burst) and with the partial mesh (highload), and Yuchengco with the tree (highload) and with the partial mesh (stationary). The final plan has 8 benign runs and 80 runs in all. The 8 are, at G402, linear (highload), star (burst), tree (stationary) and partial mesh (highload); at the DLSU Library, linear and star (both stationary); and at Yuchengco, linear (stationary) and star (highload). All 8 were captured. The final list differs from the draw in five cells: the star (burst at G402, stationary at the DLSU Library, highload at Yuchengco, where the draw had stationary, highload and burst), the partial mesh at G402 (highload, where the draw had burst) and the linear topology at Yuchengco (stationary, where the draw had burst). The plan was set to match the captures. The rule of three different scenarios for each topology therefore holds for the star, but not for the linear topology, which has highload at G402 and stationary at both the DLSU Library and Yuchengco.	The team asked that the benign runs, which are not attack runs, be one per site with a different scenario, and possibly randomised across the topologies. With jitter out of the plan (D-33), 3 of the 12 benign slots of D-18 had been jitter and were dropped, which left 9. Within one topology the benign scenario and the site are confounded, so the attack runs at the same site and scenario give the matched comparison. This was checked with the adviser.

D-36	Wormhole ends as leaves	In tree and partial-mesh wormhole runs, the ends A and B join the mesh as leaf nodes, which have no forwarding ability in ESP-IDF, so that no board can attach under an end. The stack of the telemetry task is raised from 4096 to 6144 bytes on every board. The star, the blackhole builds and all other parent rules are unchanged.	In a tree wormhole run at G402, end A carried two other boards, and A restarted three times during the run. Each time its children re-parented during the attack, and the run was marked for recapture. The root also printed that the telemetry task had only 504 bytes of stack left, of 4096. The cause of the restarts was not proven, and stack exhaustion is one candidate. As a leaf, an end carries only its own and tunnelled probes, and its restart no longer moves honest boards. Captures before this change may have had children under A or B. When the entry was written, the change was compile-checked and not tested on hardware.
D-37	Tree fan-out	The number of children that a tree board accepts is a setting of each build: the stock limit of ESP-WIFI-MESH is 10 children, and the partial mesh uses two. The fan-out that forms is read from the parent addresses in the logs of each run (Table 6.35).	In a tree wormhole run at G402 with the stock limit, the root took five boards at hop 1, and the only deep branch dropped during the baseline, so the root reported that the layout was flat like a star and the run was marked for recapture. Partial-mesh runs in the same room, with two children, formed the root, two boards at hop 1, then hops 2 and 3 every time. Limiting the connections of the soft access point of each node is the documented lever in ESP-IDF. The root has 2 to 5 children in the tree runs, so the fan-out of the tree differs between runs (Section 6.5.4).